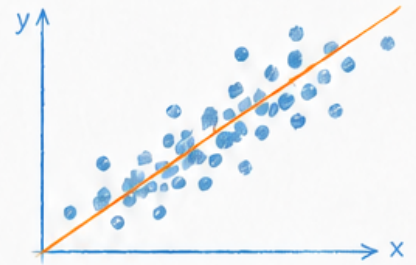
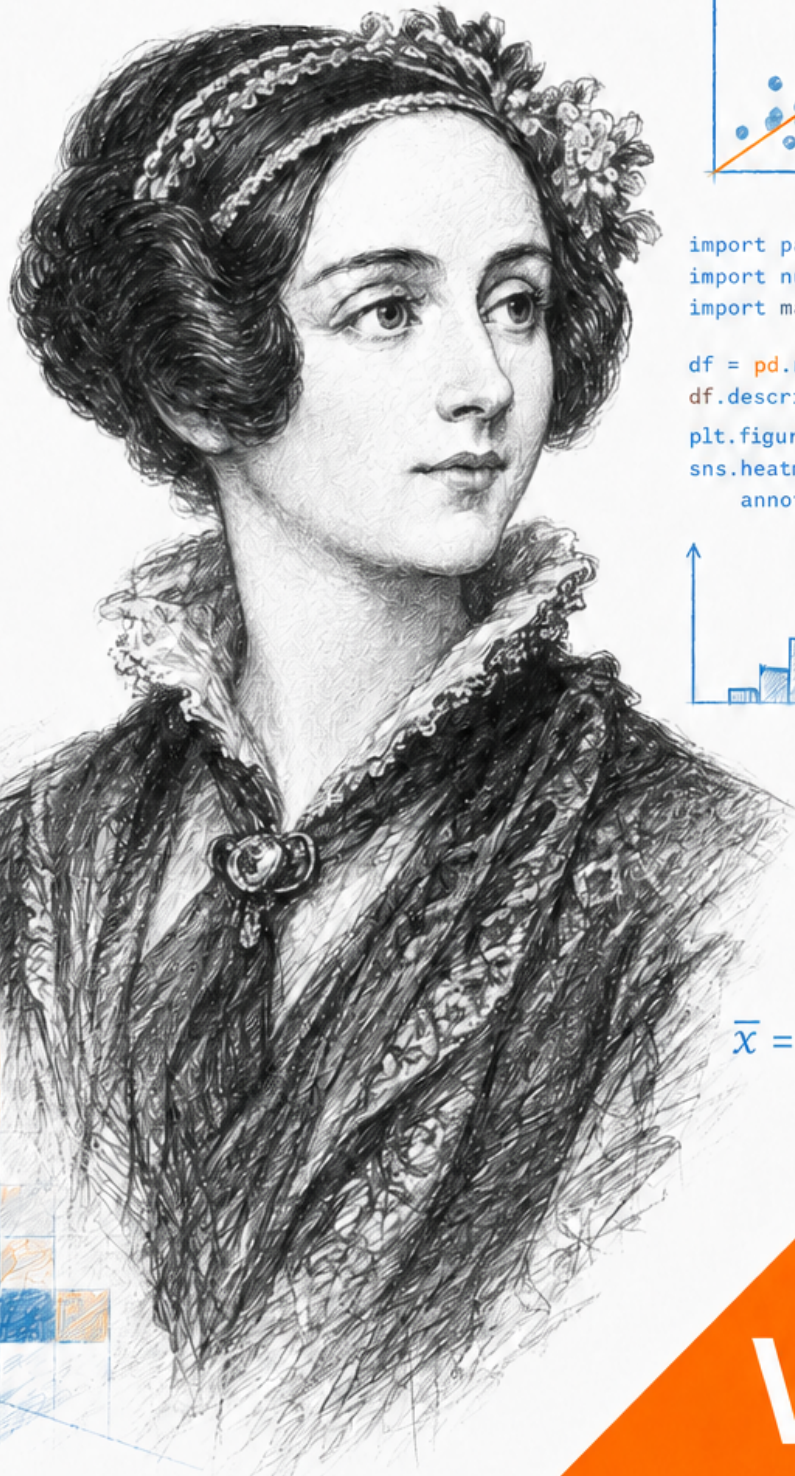


CIENCIA DE DATOS CON PYTHON

— DE LOS DATOS BRUTOS AL CONOCIMIENTO —



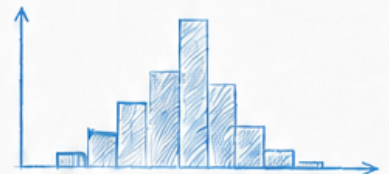
Ada Lovelace
(1815–1852)



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('datos.csv')
df.describe()

plt.figure(figsize=(8,5))
sns.heatmap(df.corr(),
            annot=True, cmap='coolwarm')
```



$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$



Vol. II

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>

Ada Lovelace

Ada Lovelace fue una matemática y escritora británica del siglo XIX, reconocida como una figura fundamental en la historia temprana de la computación. Nacida en Londres en 1815, trabajó sobre las ideas de Charles Babbage, especialmente sobre la Máquina Analítica, un dispositivo mecánico que Babbage imaginó como una máquina capaz de realizar cálculos generales.

Su aporte más recordado aparece en las notas que escribió al traducir un artículo sobre esa máquina. Allí incluyó una descripción detallada de cómo podría calcularse una secuencia de números de Bernoulli mediante instrucciones organizadas paso a paso. Por eso, suele considerarse a Ada Lovelace como la autora del primer algoritmo pensado para ser ejecutado por una máquina.

Su importancia para la ciencia de datos no está en haber trabajado con datos en el sentido moderno, sino en haber anticipado una idea decisiva: una máquina podía manipular símbolos siguiendo reglas, y no limitarse únicamente al cálculo aritmético. Lovelace imaginó que una máquina de este tipo podría operar con números, pero también con otros elementos representables mediante reglas formales, como música, texto o patrones.

Esa intuición resulta muy cercana a la base conceptual de la informática actual. En ciencia de datos, trabajamos constantemente con representaciones: convertimos información del mundo real en estructuras que una computadora puede procesar, transformar y analizar. En ese sentido, Lovelace ayudó a imaginar un futuro en el que las máquinas no solo calcularían resultados, sino que podrían participar en procesos más amplios de análisis, combinación y generación de información.

Por eso, Ada Lovelace ocupa un lugar central en la historia de la computación y en los antecedentes de la ciencia de datos. Su mirada permitió pensar a las máquinas como herramientas generales para procesar información, mucho antes de que existieran las computadoras modernas, los lenguajes de programación o los modelos de aprendizaje automático.

Créditos y uso del material

Ciencia de Datos con Python - Tomo 2

Análisis exploratorio y comunicación de resultados

Autor: **Ariel Palazzesi**

Año: **2026**

Versión actualizada de este documento:

<https://vintabytes.github.io/>

Este libro forma parte de una serie de materiales educativos sobre ciencia de datos con Python. La presente versión reúne en formato PDF el contenido desarrollado originalmente como cuadernos de trabajo para Google Colab.

Los cuadernos originales están disponibles de forma gratuita en el repositorio del proyecto:

Repositorio:

<https://github.com/VintaBytes/Ciencia-de-datos>

Desde allí, el lector puede acceder a los notebooks, abrirlos en Google Colab, ejecutar el código, modificar ejemplos y experimentar con los datos como parte del proceso de aprendizaje.

Derechos de autor

© 2026 Ariel Palazzesi. Todos los derechos reservados.

Este material se publica con fines educativos. Se permite su lectura, descarga y uso personal o educativo, así como la ejecución y modificación de los cuadernos en Google Colab con fines de aprendizaje.

No se permite la venta, redistribución, publicación de versiones modificadas, incorporación en cursos comerciales, reproducción total o parcial con fines comerciales, ni la creación de obras derivadas para su distribución pública sin autorización previa y por escrito del autor.

La disponibilidad pública de este libro o de sus cuadernos complementarios no implica cesión de derechos ni autorización para usos distintos de los indicados en esta página.

Para solicitar permisos de uso, adaptación o distribución, contactar al autor.

Uso educativo

Este material puede utilizarse como apoyo para el estudio personal, la consulta docente o el acompañamiento de cursos introductorios de programación, análisis de datos o ciencia de datos con Python, siempre respetando las condiciones de uso indicadas anteriormente.

Se recomienda acompañar la lectura del PDF con la ejecución de los cuadernos online, ya que buena parte del aprendizaje ocurre al probar instrucciones, observar resultados, modificar el código y revisar cómo cambian los datos después de cada operación.

Herramientas utilizadas

Los ejemplos y actividades de este tomo utilizan principalmente:

- Python;
- Pandas;
- Seaborn;
- Matplotlib;
- Google Colab;
- Jupyter Notebooks.

Las menciones a tecnologías, bibliotecas, plataformas o servicios de terceros se realizan con fines educativos y descriptivos. Cada herramienta conserva sus propias marcas, licencias y condiciones de uso.

Nota sobre el alcance del material

Este tomo continúa el trabajo iniciado en el primer volumen y se concentra en el análisis exploratorio de datos: formular preguntas, resumir información, comparar grupos, construir visualizaciones claras y comunicar conclusiones. No pretende cubrir todos los temas de ciencia de datos, sino fortalecer una base de análisis que permita avanzar luego hacia etapas más complejas.

Introducción

Después de aprender a cargar, inspeccionar, seleccionar, filtrar, ordenar y limpiar datos, aparece una nueva etapa del trabajo con Python: empezar a hacer preguntas más analíticas. Ya no alcanza con mirar si el dataset está ordenado o si una columna tiene el tipo correcto. Ahora necesitamos observar qué dicen los datos, qué patrones aparecen, qué diferencias existen entre grupos y qué conclusiones podemos sostener con evidencia.

Este segundo tomo continúa el recorrido iniciado en el primer volumen de **Ciencia de Datos con Python**. Si el primer tomo estuvo centrado en comprender y preparar datos tabulares, este tomo avanza hacia el análisis exploratorio: resumir variables, contar frecuencias, comparar categorías, agrupar datos, visualizar relaciones y comunicar resultados de manera clara.

El contenido de este libro proviene de una serie de cuadernos de trabajo preparados para Google Colab. En esta versión en PDF, esos cuadernos se presentan en formato de texto, para facilitar la lectura continua, la consulta y el estudio. Sin embargo, la experiencia más completa se obtiene al trabajar también con los cuadernos originales en línea: allí es posible ejecutar el código, modificarlo, observar los resultados y experimentar con los ejemplos.

Los cuadernos están disponibles de forma gratuita en un repositorio de GitHub. Si el lector lo desea, puede acceder a ellos desde allí, abrirlos en Google Colab y usarlos como material práctico complementario al libro. Esta forma de trabajo sigue siendo especialmente recomendable, porque el análisis de datos se aprende mirando resultados, comparando salidas, cambiando preguntas y volviendo sobre el dataset con más criterio.

Qué vas a encontrar en este tomo

Este tomo se concentra en las primeras herramientas de análisis exploratorio de datos. A lo largo de los capítulos se trabaja con preguntas iniciales sobre un dataset, resúmenes de variables numéricas, conteos de valores, frecuencias absolutas y relativas, porcentajes, tablas de resumen y comparaciones entre grupos.

También se introducen recursos fundamentales para analizar datos con Pandas, como `value_counts()`, `groupby()`, tablas agregadas, ordenamientos, rankings, comparación de categorías y búsqueda de valores extremos. Estas herramientas permiten dejar de mirar registros aislados y empezar a observar comportamientos generales dentro del conjunto de datos.

Una parte importante del recorrido está dedicada a la visualización. El objetivo no es hacer gráficos decorativos, sino aprender a elegir representaciones útiles para comparar distribuciones, observar relaciones entre variables numéricas, comunicar diferencias entre grupos y acompañar una explicación con evidencia visual.

Hacia el final del tomo se trabaja sobre la escritura de conclusiones y la construcción de un informe breve de análisis exploratorio. La idea central es que analizar datos no termina al obtener una tabla o un gráfico: también es necesario interpretar los resultados, expresar los hallazgos con cuidado y reconocer los límites de lo que los datos permiten afirmar.

Requisitos previos

Para aprovechar este libro conviene haber trabajado previamente con los contenidos del primer tomo o contar con conocimientos equivalentes. En particular, se recomienda conocer:

- nociones básicas de programación en Python;
- uso general de Google Colab o notebooks;
- estructura de un DataFrame de Pandas;
- selección de filas y columnas;
- filtros con condiciones;
- ordenamiento de datos;
- creación y modificación de columnas;
- detección básica de valores faltantes, duplicados y tipos de datos.

No es necesario tener experiencia previa en estadística avanzada o visualización profesional. Los conceptos se presentan de manera progresiva y se apoyan en ejemplos concretos, pero sí se espera cierta familiaridad con la manipulación inicial de datos tabulares.

A quién está dirigido

Este material está pensado para personas que ya dieron sus primeros pasos con Python y Pandas y quieren empezar a analizar datos con mayor profundidad. Puede ser útil para estudiantes, docentes, programadores, analistas, profesionales que trabajan con planillas o bases de datos, y cualquier persona que necesite transformar datos en preguntas, comparaciones, explicaciones y conclusiones.

El enfoque sigue siendo introductorio, pero busca construir hábitos de análisis sólidos: no quedarse con una única medida, comparar antes de concluir, mirar la

distribución de los datos, revisar grupos, detectar valores extremos y comunicar los resultados de manera comprensible.

Cómo trabajar con este libro y los cuadernos

Cada capítulo puede leerse directamente en este PDF, pero fue pensado originalmente como un cuaderno ejecutable. Por eso, la recomendación es acompañar la lectura con la versión online del cuaderno correspondiente siempre que sea posible.

En los cuadernos de Google Colab, conviene ejecutar cada celda, observar la salida y preguntarse qué aporta ese resultado. Muchas veces, una tabla, un conteo o un gráfico no tienen valor por sí solos: se vuelven útiles cuando los conectamos con una pregunta y con una interpretación razonable.

También es recomendable modificar algunos ejemplos: cambiar una columna, probar otro agrupamiento, ordenar de otra manera o comparar una categoría distinta. Ese tipo de exploración ayuda a comprender mejor las herramientas y a desarrollar autonomía para aplicarlas en otros datasets.

Los capítulos fueron diseñados como una secuencia progresiva. Algunas instrucciones aparecen varias veces en contextos distintos, porque en análisis de datos una misma herramienta puede servir para responder preguntas diferentes. Esa repetición controlada permite afianzar procedimientos y, al mismo tiempo, aprender a leer los resultados con más criterio.

Enfoque del libro

El objetivo principal de este tomo es pasar de la preparación de datos al análisis exploratorio. Esto implica aprender instrucciones nuevas, pero también construir una forma de trabajo: formular preguntas, producir resúmenes, comparar grupos, visualizar relaciones, interpretar resultados y comunicar conclusiones.

A lo largo del recorrido se priorizan tres ideas:

- analizar no es solo calcular, sino interpretar;
- una conclusión necesita apoyarse en datos observables;
- una buena visualización debe ayudar a comprender, no solo a decorar.

Con esa base, el lector queda mejor preparado para avanzar hacia etapas posteriores del trabajo con datos, donde las preguntas se vuelven más complejas y las herramientas permiten construir modelos predictivos.

Índice general - Tomo 2

Capítulo 24 - Primeras preguntas de análisis

- De preparar datos a analizarlos.
- Usar un dataset de práctica.
- Reconocer el tamaño del dataset.
- Formular primeras preguntas.
- Contar valores en una columna categórica.
- Contar registros por horario.
- Analizar el tamaño de las mesas.
- Calcular indicadores simples.

Capítulo 25 - Resumir columnas numéricas

- De las primeras preguntas al resumen de variables.
- Seleccionar las columnas numéricas.
- Resumir una columna numérica.
- Comparar promedio y mediana.
- Observar valores mínimos y máximos.
- Usar describe() para ver varios resúmenes juntos.
- Elegir qué indicadores mostrar.

Capítulo 26 - Contar valores y analizar frecuencias

- De resumir números a contar categorías.
- Identificar columnas categóricas.
- Contar valores con value_counts().
- Calcular frecuencias relativas.
- Convertir proporciones en porcentajes.
- Construir una tabla de frecuencias.
- Aplicar el mismo análisis a otra columna.
- Analizar otra variable categórica.

Capítulo 27 - Primeras tablas de resumen

- De resultados sueltos a tablas de resumen.
- Una primera tabla de indicadores generales.
- Una tabla de frecuencias.
- La forma de la tabla depende de la pregunta.
- Resumir características generales del dataset.
- Tablas para explorar y tablas para comunicar.
- Preparar una tabla más legible.
- Cuidar el formato de los valores.

Capítulo 28 - Agrupar datos con groupby()

- De las tablas generales a los resúmenes por grupo.
- Qué significa agrupar datos.
- Una primera pregunta por grupo.
- Leer la estructura de un groupby().
- Cambiar el indicador calculado.
- Calcular la cantidad de registros por grupo.
- Combinar varios resultados por grupo.
- Calcular varios indicadores con agg().

Capítulo 29 - Agrupaciones múltiples y varios indicadores con groupby()

- Profundizar el análisis por grupos.
- Agrupar por más de una columna.
- Convertir el resultado en una tabla.
- Calcular promedios por combinación de grupos.
- Presentar el resultado como tabla.
- Calcular varios indicadores por combinación de grupos.
- Ordenar una tabla agrupada.
- Cambiar el criterio de ordenamiento.

Capítulo 30 - Comparar categorías

- Comparar no es solo mirar qué valor es mayor.
- Construir una tabla para comparar días.

- Ordenar según distintos criterios.
- Comparar usando otro indicador.
- Comparar propinas promedio.
- Escribir una comparación con cuidado.
- Comparar horarios.
- De la tabla a una interpretación.

Capítulo 31 - Rankings, valores extremos y outliers

- Ordenar para detectar valores destacados.
- Construir una tabla base para rankings por día.
- Ranking por facturación total.
- Ranking por promedio de cuenta.
- Seleccionar los primeros lugares de un ranking.
- Seleccionar los últimos lugares de un ranking.
- Rankings de registros individuales.
- Observar las cuentas más bajas.

Capítulo 32 - Organizar resúmenes con tablas dinámicas

- De agrupar datos a cruzar variables.
- Recordar una agrupación doble con `groupby()`.
- Crear una tabla dinámica con `pivot_table()`.
- Entender los parámetros principales.
- Tabla dinámica de cantidad de cuentas.
- Reemplazar valores vacíos en una tabla de conteos.
- Tabla dinámica de total facturado.
- Tabla dinámica de promedio de propina.

Capítulo 33 - Análisis temporal básico

- Mirar los datos según momentos.
- Variables temporales simples.
- Primera pregunta: ¿cómo se distribuyen las cuentas según día y horario?.
- El orden temporal importa.
- Total facturado por día.
- Visualizar el total facturado por día.

- Cómo se construyó este gráfico.
- Total facturado y promedio de cuenta.

Capítulo 34 - Visualizar distribuciones

- Mirar cómo se reparten los valores.
- ¿Qué significa visualizar una distribución?.
- Empezar con una variable numérica.
- Primer gráfico de distribución: histograma.
- Leer la estructura del histograma.
- Cambiar la cantidad de intervalos.
- Marcar el promedio y la mediana.
- Cómo agregamos líneas de referencia.

Capítulo 35 - Comparar distribuciones por grupos

- Mirar una variable numérica según categorías.
- Cargar el dataset.
- Primera comparación: importe total según horario.
- Comparar distribuciones con boxplot.
- Cuidado al comparar grupos.
- Ver la cantidad de registros por grupo.
- Comparar importes según el día.
- Cantidad de registros por día.

Capítulo 36 - Visualizar relaciones entre variables numéricas

- Observar relaciones entre columnas.
- Cargar el dataset.
- Primer gráfico de relación: scatterplot.
- Leer un gráfico de dispersión.
- Medir la relación con correlación.
- Agregar una línea de tendencia.
- Cómo leer regplot.
- Agregar una variable categórica con color.

Capítulo 37 - Comunicar resultados con visualizaciones claras

- No alcanza con hacer un gráfico.
- Cargar el dataset.
- Preparar un resumen para graficar.
- Primer gráfico: correcto, pero mejorable.
- Mejorar título y etiquetas.
- Agregar valores sobre las barras.
- Dejar espacio para las etiquetas.
- Mejorar el estilo general.

Capítulo 38 - Integrar un análisis exploratorio completo

- Unir las piezas del análisis exploratorio.
- Cargar el dataset.
- Revisar la estructura general.
- Plantear preguntas de análisis.
- Resumir las variables numéricas.
- Contar las variables categóricas.
- Visualizar la distribución de las cuentas.
- Visualizar la distribución de las propinas.

Capítulo 39 - Escribir conclusiones a partir de los datos

- Pasar del resultado a la conclusión.
- Qué vamos a trabajar en este capítulo.
- Resultados de partida.
- Observación, interpretación y conclusión.
- Mejorar frases de análisis.
- Ejemplos de mejora de frases.
- Evitar conclusiones exageradas.
- Algunas expresiones útiles.

Capítulo 40 - Armar un informe breve de análisis exploratorio

- Del análisis al informe.

- Datos de referencia.
- Definir el propósito del informe.
- Seleccionar resultados relevantes.
- Interpretar qué resultados conviene usar.
- Elegir visualizaciones para el informe.
- Visualizar diferencias por día y horario.
- Visualizar la relación entre cuenta y propina.

Capítulo 41 - Cerrar el análisis exploratorio

- Qué aprendimos en esta etapa.
- Del resultado a la interpretación.
- Por qué este recorrido prepara el camino.
- El análisis exploratorio como hábito de trabajo.
- Síntesis final del Tomo II.

Capítulo 24 — Primeras preguntas de análisis

De preparar datos a analizarlos

En los capítulos anteriores trabajamos sobre la preparación de datos: revisamos estructuras, detectamos problemas, transformamos columnas, tratamos valores faltantes, convertimos tipos de datos y validamos el resultado final. A partir de ahora el foco cambia.

Ya no vamos a mirar un `DataFrame` principalmente para corregirlo, sino para empezar a hacer preguntas sobre los datos.

Analizar datos no significa aplicar muchos comandos de manera automática. Significa formular preguntas, buscar respuestas en el dataset, interpretar los resultados y decidir qué nuevas preguntas aparecen a partir de esas respuestas.

En este capítulo vamos a comenzar con preguntas simples:

- ¿Cuántos registros tenemos?
- ¿Qué variables contiene el dataset?
- ¿Qué valores son más frecuentes?
- ¿Cuáles son los valores mínimos, máximos o promedio?
- ¿Qué primeras ideas podemos obtener antes de hacer análisis más complejos?

Todavía no vamos a construir un análisis completo. El objetivo es dar el primer paso desde un dataset disponible hacia preguntas concretas de análisis.

Usar un dataset de práctica

Para comenzar esta nueva etapa vamos a usar un dataset de práctica incluido en una librería de Python.

Muchas librerías orientadas al análisis de datos, la visualización o el aprendizaje automático incluyen datasets pequeños y conocidos. Estos datasets se usan para aprender, probar herramientas, comparar resultados y practicar sin depender siempre de archivos externos.

Esto tiene una ventaja importante: podemos concentrarnos en las preguntas de análisis sin comenzar otra vez por la descarga, la carga manual o la limpieza profunda de los datos.

En este caso vamos a usar el dataset `tips`, disponible desde la librería Seaborn. Este dataset contiene información sobre cuentas y propinas en un restaurante. Cada fila representa una mesa atendida, e incluye datos como el total de la cuenta, la propina, el día, el horario y la cantidad de personas. Aunque es un dataset pequeño, resulta muy útil para practicar análisis exploratorio porque combina variables numéricas y categóricas.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Cada registro corresponde a una cuenta de restaurante. Algunas columnas contienen valores numéricos, como el total de la cuenta, la propina o la cantidad de personas. Otras columnas contienen categorías, como el día, el horario o si la mesa correspondía a una persona fumadora o no fumadora.

En esta etapa no vamos a limpiar el dataset. Lo vamos a usar como punto de partida para formular y responder primeras preguntas de análisis.

Reconocer el tamaño del dataset

Antes de formular preguntas de análisis, conviene saber con qué volumen de datos estamos trabajando.

Vamos a revisar cuántas filas y columnas tiene el dataset.

```
df.shape
```

```
(244, 7)
```

El dataset contiene **244 filas y 7 columnas**. Esto confirma que estamos trabajando con un conjunto de datos pequeño, adecuado para practicar análisis exploratorio sin que el volumen de información complique la lectura de los resultados.

Ahora vamos a revisar qué columnas están disponibles. No se trata solo de mirar nombres: cada columna representa una posible variable para analizar.

```
df.columns
```

```
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

Los nombres de las columnas nos muestran qué información está disponible para analizar.

Las columnas disponibles son:

- **total_bill**: total de la cuenta.
- **tip**: propina.
- **sex**: sexo de la persona registrada como pagadora o responsable de la cuenta.
- **smoker**: si la mesa correspondía a personas fumadoras o no fumadoras.
- **day**: día de la semana.
- **time**: momento del día.
- **size**: cantidad de personas en la mesa.

A partir de estas variables ya podemos formular primeras preguntas de análisis: cuánto se consumió, cuánto se dejó de propina, qué días aparecen en el dataset, qué horarios se registraron y qué tamaño tenían las mesas.

También conviene revisar cómo interpreta Pandas cada columna:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null   float64
1   tip         244 non-null   float64
2   sex         244 non-null   category
3   smoker      244 non-null   category
4   day         244 non-null   category
5   time        244 non-null   category
6   size        244 non-null   int64
```

```
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

La salida de `info()` confirma que el dataset tiene **244 registros** y que no aparecen valores faltantes en ninguna columna: todas tienen **244 valores no nulos**.

También vemos que Pandas interpreta las columnas de esta manera:

- `total_bill` y `tip` son columnas numéricas con decimales.
- `size` es una columna numérica entera.
- `sex`, `smoker`, `day` y `time` son columnas categóricas.

Esto es importante para el análisis porque no todas las preguntas se responden del mismo modo. Con las columnas numéricas podremos calcular totales, promedios, mínimos y máximos. Con las columnas categóricas podremos contar frecuencias y comparar grupos.

En otras palabras, el dataset ya nos ofrece dos caminos iniciales de análisis: resumir valores numéricos y contar categorías.

Formular primeras preguntas

Antes de calcular indicadores, conviene transformar la curiosidad general en preguntas concretas. En este dataset podríamos empezar con preguntas como:

- ¿Cuál fue el total promedio de las cuentas?
- ¿Cuál fue la propina promedio?
- ¿Cuál fue la cuenta más alta?
- ¿Qué días aparecen registrados?
- ¿Qué día aparece con más frecuencia?
- ¿Cuál es el tamaño de mesa más habitual?

Estas preguntas todavía son simples, pero marcan un cambio importante: ya no estamos revisando el dataset para prepararlo, sino para entender qué información contiene.

```
df[["total_bill", "tip", "size"]].describe()
```

índice	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100

índice	total_bill	tip	size
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

La tabla resume las tres columnas numéricas del dataset.

Podemos leer algunos primeros resultados:

- Hay **244 valores** en cada columna, lo que confirma que estas variables no tienen datos faltantes.
- El total promedio de las cuentas (`total_bill`) es de aproximadamente **19.79**.
- La propina promedio (`tip`) es de aproximadamente **3.00**.
- El tamaño promedio de las mesas (`size`) es de aproximadamente **2.57 personas**.

También podemos observar los valores extremos:

- La cuenta más baja fue de **3.07**.
- La cuenta más alta fue de **50.81**.
- La propina más baja fue de **1.00**.
- La propina más alta fue de **10.00**.
- El tamaño de las mesas va de **1 a 6** personas.

Esta salida ya nos permite obtener una primera imagen general del dataset. Sin embargo, todavía son resultados globales: sabemos cómo se comportan las columnas numéricas en conjunto, pero no sabemos si esos valores cambian según el día, el horario o el tipo de mesa.

Contar valores en una columna categórica

Hasta ahora observamos resúmenes generales de las columnas numéricas. Ahora vamos a mirar una columna categórica. Una primera pregunta posible es:

¿Qué días aparecen registrados en el dataset y cuántas cuentas hay en cada uno?

Para responderla podemos contar cuántas veces aparece cada valor en la columna `day`.

```
df["day"].value_counts()
```

day	count
Sat	87
Sun	76
Thur	62
Fri	19

```
dtype: int64
```

La columna `day` muestra registros para cuatro días: `Thur`, `Fri`, `Sat` y `Sun`.

El día con más cuentas registradas es **sábado**, con **87 cuentas**. Luego aparecen **domingo**, con **76**, y **jueves**, con **62**. El día con menos registros es **viernes**, con **19 cuentas**.

Este resultado no significa necesariamente que el sábado sea “mejor” o “más rentable”. Por ahora solo sabemos que en el dataset hay más cuentas registradas ese día. Para hablar de ingresos, promedios o comportamiento de consumo necesitaríamos hacer otros cálculos.

Contar registros por horario

También podemos preguntarnos en qué momento del día se registraron más cuentas.

La columna `time` indica si cada registro corresponde al almuerzo o a la cena. Contar sus valores nos permite ver cómo se distribuyen las cuentas entre esos dos momentos.

```
df["time"].value_counts()
```

time	count
Dinner	176
Lunch	68

```
dtype: int64
```

La mayor parte de las cuentas registradas corresponde a **Dinner**, con **176 registros**. En cambio, **Lunch** aparece con **68 registros**. Esto nos permite ver que el dataset tiene más observaciones de cenas que de almuerzos.

Esta salida solo describe la cantidad de registros por horario. Para analizar consumo, propinas o tamaño de mesa necesitaremos combinar esta información con columnas numéricas como `total_bill`, `tip` o `size`.

Analizar el tamaño de las mesas

Otra pregunta inicial es cuántas personas había en cada mesa.

La columna `size` indica el tamaño de la mesa. Aunque es una columna numérica, en este caso también puede analizarse como una categoría: mesa de 1 persona, de 2 personas, de 3 personas, y así sucesivamente. Contar estos valores nos permite saber qué tamaños de mesa aparecen con más frecuencia en el dataset.

```
df["size"].value_counts()
```

size	count
2	156
3	38
4	37
5	5
1	4
6	4

`dtype: int64`

El tamaño de mesa más frecuente es **2 personas**, con **156 registros**. Luego aparecen las mesas de **3 personas** y **4 personas**, con cantidades bastante similares: **38** y **37 registros** respectivamente.

Los tamaños menos frecuentes son las mesas de **1**, **5** y **6** personas. Este resultado nos muestra que el dataset está dominado por mesas pequeñas, especialmente de dos personas. Esta información puede ser importante más adelante, porque el tamaño de la mesa puede influir en el total de la cuenta y también en la propina.

Calcular indicadores simples

Además de contar registros, podemos calcular algunos indicadores numéricos básicos. Por ejemplo, podemos preguntarnos:

- ¿Cuál fue el total facturado en todas las cuentas?
- ¿Cuánto se dejó de propina en total?

- ¿Cuál fue el promedio del total de cuenta?
- ¿Cuál fue el promedio de propina?

Estas preguntas todavía no comparan grupos ni categorías. Son indicadores generales del dataset completo.

```
total_facturado = df["total_bill"].sum()
total_propinas = df["tip"].sum()
promedio_cuenta = df["total_bill"].mean()
promedio_propina = df["tip"].mean()

total_facturado, total_propinas, promedio_cuenta, promedio_propina
```

```
(np.float64(4827.77),
 np.float64(731.5799999999999),
 np.float64(19.78594262295082),
 np.float64(2.99827868852459))
```

Los resultados obtenidos corresponden a cuatro indicadores generales del dataset:

- El total facturado es **4827.77**.
- El total de propinas es **731.58**.
- El promedio de cuenta es aproximadamente **19.79**.
- El promedio de propina es aproximadamente **3.00**.

Estos valores resumen el conjunto completo de datos, sin distinguir todavía por día, horario, tamaño de mesa u otras categorías.

La salida es correcta, pero no es muy cómoda para leer: aparece como una tupla de valores y Pandas/NumPy muestra algunos números con demasiados decimales. Por eso conviene construir una tabla pequeña de indicadores.

Presentar indicadores en una tabla

Cuando calculamos varios indicadores, puede ser más claro organizarlos en una pequeña tabla. Esto no cambia los resultados, pero mejora la lectura y acerca el análisis a una forma más comunicable.

```
indicadores_generales = pd.DataFrame({
    "indicador": [
        "Total facturado",
        "Total de propinas",
        "Promedio de cuenta",
        "Promedio de propina"
    ],
    "valor": [
```

```

        total_facturado,
        total_propinas,
        promedio_cuenta,
        promedio_propina
    ]
})

indicadores_generales

```

índice	indicador	valor
0	Total facturado	4827.770000
1	Total de propinas	731.580000
2	Promedio de cuenta	19.785943
3	Promedio de propina	2.998279

Ahora los indicadores aparecen organizados en una tabla.

Esta forma de presentación es más clara que una salida con varios valores sueltos, porque cada número queda asociado a su significado. Sin embargo, los valores todavía aparecen con muchos decimales. Para comunicar mejor estos resultados, podemos redondearlos a dos decimales.

Redondear resultados

Cuando los indicadores tienen muchos decimales, la lectura puede volverse innecesariamente pesada. En este caso, redondear a dos decimales alcanza para comunicar los resultados de manera clara.

```

indicadores_generales["valor"] = indicadores_generales["valor"].round(2)

indicadores_generales

```

índice	indicador	valor
0	Total facturado	4827.77
1	Total de propinas	731.58
2	Promedio de cuenta	19.79
3	Promedio de propina	3.00

La tabla final presenta los indicadores generales de manera más clara. Podemos resumir esta primera lectura así:

- En el dataset se registró un total facturado de **4827.77**.

- El total de propinas fue de **731.58**.
- El promedio de cuenta fue de **19.79**.
- El promedio de propina fue de **3.00**.

Estos indicadores permiten tener una primera idea general del dataset. Sin embargo, todavía no muestran diferencias internas.

Por ejemplo, no sabemos si el promedio de cuenta cambia entre almuerzo y cena, si las propinas varían según el día o si las mesas más grandes generan cuentas más altas. Para responder ese tipo de preguntas vamos a necesitar comparar grupos.

Comparar una pregunta en dos subconjuntos

Hasta ahora calculamos indicadores para todo el dataset. Pero una pregunta de análisis muchas veces necesita comparar partes del conjunto de datos. Por ejemplo:

¿El promedio de cuenta es igual en el almuerzo y en la cena?

Todavía no necesitamos una herramienta nueva para responder esto. Podemos usar filtros, que ya conocemos, para separar las cuentas de almuerzo y cena, y luego calcular el promedio de `total_bill` en cada caso.

```
promedio_almuerzo = df[df["time"] == "Lunch"]["total_bill"].mean()
promedio_cena = df[df["time"] == "Dinner"]["total_bill"].mean()

promedio_almuerzo, promedio_cena
```

```
(np.float64(17.168676470588235), np.float64(20.79715909090909))
```

El promedio de cuenta durante el almuerzo es aproximadamente **17.17**. El promedio de cuenta durante la cena es aproximadamente **20.80**.

En este primer cálculo, las cuentas de la cena tienen un promedio más alto que las cuentas del almuerzo. De todos modos, la salida vuelve a aparecer como dos valores sueltos. Para comparar mejor, conviene organizar el resultado en una pequeña tabla.

Organizar la comparación

Cuando comparamos dos valores, una tabla simple puede ayudar a leer mejor el resultado.

Vamos a construir un pequeño `DataFrame` con el promedio de cuenta para cada horario.

```
comparacion_horario = pd.DataFrame({
    "horario": ["Lunch", "Dinner"],
    "promedio_total_cuenta": [promedio_almuerzo, promedio_cena]
})

comparacion_horario["promedio_total_cuenta"] =
comparacion_horario["promedio_total_cuenta"].round(2)

comparacion_horario
```

índice	horario	promedio_total_cuenta
0	Lunch	17.17
1	Dinner	20.80

La tabla permite comparar con más claridad el promedio de cuenta según el horario.

En este dataset, las cuentas registradas durante la cena (Dinner) tienen un promedio de **20.80**, mientras que las cuentas registradas durante el almuerzo (Lunch) tienen un promedio de **17.17**. Esto sugiere que, en estos datos, el consumo promedio es mayor durante la cena.

Esta comparación se resolvió con filtros, una herramienta que ya conocemos. Más adelante vamos a estudiar formas más directas de resumir datos por grupos, especialmente con `groupby()`.

Una respuesta puede abrir nuevas preguntas

Un análisis exploratorio no avanza en línea recta. Muchas veces, una respuesta genera nuevas preguntas. Por ejemplo, acabamos de observar que el promedio de cuenta es mayor durante la cena que durante el almuerzo. A partir de eso podríamos preguntarnos:

- ¿Hay muchas más cuentas de cena que de almuerzo?
- ¿La propina promedio también es mayor durante la cena?
- ¿El tamaño de las mesas cambia según el horario?

En este capítulo no vamos a responder todas esas preguntas. Pero sí podemos hacer una comparación adicional sencilla: revisar si la propina promedio también cambia entre almuerzo y cena.

```
promedio_propina_almuerzo = df[df["time"] == "Lunch"]["tip"].mean()
promedio_propina_cena = df[df["time"] == "Dinner"]["tip"].mean()

comparacion_propinas = pd.DataFrame({
    "horario": ["Lunch", "Dinner"],
```

```
"promedio_propina": [promedio_propina_almuerzo, promedio_propina_cena]
})

comparacion_propinas["promedio_propina"] =
comparacion_propinas["promedio_propina"].round(2)

comparacion_propinas
```

índice	horario	promedio_propina
0	Lunch	2.73
1	Dinner	3.10

La propina promedio también es mayor durante la cena. En el almuerzo (Lunch), la propina promedio es de **2.73**. En la cena (Dinner), la propina promedio es de **3.10**.

Este resultado acompaña lo que habíamos visto con el total de la cuenta: en este dataset, tanto el promedio de cuenta como el promedio de propina son más altos durante la cena. Este tipo de comparación empieza a mostrar una necesidad nueva: resumir información por grupos de manera más directa y ordenada.

Cierre del capítulo

En este capítulo comenzamos a trabajar con una idea central del análisis de datos: convertir un dataset en preguntas.

Usamos un dataset de práctica, revisamos su tamaño y sus variables principales, contamos valores en columnas categóricas y calculamos indicadores simples sobre columnas numéricas.

También vimos que una respuesta inicial puede abrir nuevas preguntas. Al observar que la cena tenía un promedio de cuenta más alto que el almuerzo, surgió naturalmente la necesidad de comparar otros indicadores, como la propina promedio.

Hasta ahora hicimos comparaciones sencillas usando filtros y cálculos directos. Este enfoque es útil para empezar, pero puede volverse incómodo cuando queremos comparar muchas categorías o calcular varios indicadores por grupo.

En los próximos capítulos vamos a profundizar en formas más ordenadas de resumir información, construir tablas de análisis y comparar grupos dentro de un dataset.

Capítulo 25 — Resumir columnas numéricas

De las primeras preguntas al resumen de variables

En el capítulo anterior comenzamos a formular preguntas de análisis sobre un dataset de práctica.

Contamos registros, revisamos valores frecuentes y calculamos algunos indicadores generales, como el total facturado, el total de propinas y los promedios de cuenta y propina.

En este capítulo vamos a concentrarnos en las columnas numéricas.

Una columna numérica puede resumirse de varias maneras. Podemos calcular su promedio, su mediana, su valor mínimo, su valor máximo o la cantidad de datos disponibles. Cada indicador muestra una parte distinta de la información.

El objetivo no es memorizar funciones, sino aprender a elegir qué resumen ayuda a responder una pregunta.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Esto nos permite mantener el mismo contexto del capítulo anterior y concentrarnos ahora en el análisis de columnas numéricas.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables del capítulo anterior: el total de la cuenta (`total_bill`), la propina (`tip`), algunas variables categóricas y el tamaño de la mesa (`size`).

En este capítulo nos vamos a concentrar especialmente en las columnas numéricas, porque queremos aprender a resumirlas con distintos indicadores.

Seleccionar las columnas numéricas

Para trabajar de manera ordenada, primero vamos a separar las columnas numéricas que nos interesa analizar.

En este dataset vamos a usar:

- `total_bill`: total de la cuenta.
- `tip`: propina.
- `size`: cantidad de personas en la mesa.

Aunque `size` representa una cantidad de personas y a veces puede analizarse como categoría, también es una variable numérica útil para calcular resúmenes.

```
columnas_numericas = ["total_bill", "tip", "size"]  
  
df[columnas_numericas].head()
```

índice	total_bill	tip	size
0	16.99	1.01	2
1	10.34	1.66	3
2	21.01	3.50	3
3	23.68	3.31	2
4	24.59	3.61	4

Ahora tenemos una vista reducida del dataset, solo con las columnas numéricas que vamos a resumir. Esto nos permite concentrarnos en los valores que pueden responder preguntas como:

- ¿cuál es el total promedio de una cuenta?
- ¿cuál es la propina típica?
- ¿cuántas personas suele haber por mesa?
- ¿cuáles son los valores más bajos y más altos?

A partir de acá vamos a trabajar primero con una sola columna, `total_bill`, para entender mejor qué aporta cada indicador.

Resumir una columna numérica

Vamos a comenzar con la columna `total_bill`, que representa el total de cada cuenta.

Una primera forma de resumir esta variable es calcular algunos indicadores básicos: cantidad de valores, promedio, mediana, mínimo y máximo.

```
cantidad_cuentas = df["total_bill"].count()
promedio_cuenta = df["total_bill"].mean()
mediana_cuenta = df["total_bill"].median()
minimo_cuenta = df["total_bill"].min()
maximo_cuenta = df["total_bill"].max()

cantidad_cuentas, promedio_cuenta, mediana_cuenta, minimo_cuenta, maximo_cuenta
```

```
(np.int64(244), np.float64(19.78594262295082), 17.795, 3.07, 50.81)
```

Los valores calculados resumen la columna `total_bill`.

La salida es correcta, pero no resulta demasiado cómoda para leer: los valores aparecen como una tupla y algunos se muestran con muchos decimales.

Como vimos en el capítulo anterior, cuando queremos comunicar varios indicadores conviene organizarlos en una tabla. En este caso vamos a construir directamente una tabla de resumen, redondeando los valores para facilitar la lectura.

```
resumen_total_cuenta = pd.DataFrame({
    "indicador": [
        "Cantidad de cuentas",
        "Promedio",
        "Mediana",
        "Mínimo",
        "Máximo"
    ],
    "valor": [
        cantidad_cuentas,
        promedio_cuenta,
        mediana_cuenta,
        minimo_cuenta,
        maximo_cuenta
    ]
})

resumen_total_cuenta["valor"] = resumen_total_cuenta["valor"].round(2)
```

```
resumen_total_cuenta
```

índice	indicador	valor
0	Cantidad de cuentas	244.00
1	Promedio	19.79
2	Mediana	17.80
3	Mínimo	3.07
4	Máximo	50.81

La tabla resume la columna `total_bill` de forma más clara. Podemos leerla así:

- El dataset contiene **244 cuentas**.
- El promedio del total de cuenta es **19.79**.
- La mediana es **17.80**, es decir, aproximadamente la mitad de las cuentas está por debajo de ese valor y la otra mitad por encima.
- La cuenta más baja es de **3.07**.
- La cuenta más alta es de **50.81**.

El promedio y la mediana no son iguales. El promedio es más alto que la mediana, lo que puede indicar que algunas cuentas de valor elevado empujan el promedio hacia arriba.

Esto no significa todavía que esas cuentas sean errores o valores problemáticos. Por ahora solo observamos que la columna tiene algunos valores bastante más altos que el valor central.

Comparar promedio y mediana

El promedio y la mediana son dos formas distintas de resumir una columna numérica.

El promedio reparte el total entre todos los registros. La mediana, en cambio, representa el valor que queda en el centro cuando los datos se ordenan.

Cuando ambos valores son parecidos, suelen contar una historia similar. Cuando se alejan, puede ser una señal de que hay valores altos o bajos que influyen sobre el promedio.

Vamos a comparar estos dos indicadores en las columnas `total_bill` y `tip`.

```
comparacion_promedio_mediana = pd.DataFrame({  
    "columna": ["total_bill", "tip"],
```

```

    "promedio": [
        df["total_bill"].mean(),
        df["tip"].mean()
    ],
    "mediana": [
        df["total_bill"].median(),
        df["tip"].median()
    ]
})

comparacion_promedio_mediana[["promedio", "mediana"]] =
comparacion_promedio_mediana[["promedio", "mediana"]].round(2)

comparacion_promedio_mediana

```

índice	columna	promedio	mediana
0	total_bill	19.79	17.8
1	tip	3.00	2.9

En las dos columnas comparadas, el promedio es mayor que la mediana.

En `total_bill`, el promedio es **19.79** y la mediana es **17.80**. La diferencia no es enorme, pero muestra que algunas cuentas más altas influyen sobre el promedio.

En `tip`, el promedio es **3.00** y la mediana es **2.90**. En este caso, ambos valores están muy cerca, por lo que resumen la columna de manera bastante similar.

Esta comparación muestra por qué no conviene mirar un solo indicador. El promedio es útil, pero la mediana puede ayudarnos a entender mejor qué valor está más cerca del centro de los datos.

Observar valores mínimos y máximos

Además de mirar valores centrales, también conviene revisar los extremos.

Los valores mínimos y máximos nos muestran el rango observado en una columna. No alcanzan por sí solos para explicar toda la distribución, pero ayudan a detectar qué tan separados están los valores más bajos y más altos.

```

extremos = pd.DataFrame({
    "columna": ["total_bill", "tip", "size"],
    "mínimo": [
        df["total_bill"].min(),
        df["tip"].min(),
        df["size"].min()
    ],
    "máximo": [

```

```

    df["total_bill"].max(),
    df["tip"].max(),
    df["size"].max()
]
})

extremos

```

índice	columna	mínimo	máximo
0	total_bill	3.07	50.81
1	tip	1.00	10.00
2	size	1.00	6.00

La tabla muestra los valores mínimos y máximos de las tres columnas numéricas.

En `total_bill`, las cuentas van desde **3.07** hasta **50.81**. Esto indica que hay una diferencia importante entre la cuenta más baja y la más alta.

En `tip`, las propinas van desde **1.00** hasta **10.00**.

En `size`, el tamaño de las mesas va desde **1** hasta **6** personas.

Estos valores extremos ayudan a entender el rango de cada variable. Sin embargo, mirar solo mínimos y máximos puede ser insuficiente, porque no nos dice dónde se concentra la mayoría de los datos.

Para eso podemos usar otros indicadores, como los cuartiles.

Usar `describe()` para ver varios resúmenes juntos

Pandas ofrece una forma rápida de obtener varios indicadores descriptivos de las columnas numéricas mediante `describe()`. Esta salida incluye la cantidad de valores, el promedio, el desvío estándar, el mínimo, los cuartiles y el máximo.

En este capítulo nos vamos a concentrar especialmente en el **mínimo**, el **máximo**, la **mediana** y los **cuartiles**. El desvío estándar aparecerá en la tabla, pero no lo vamos a desarrollar en detalle todavía.

```
df[columnas_numericas].describe()
```

índice	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672

índice	total_bill	tip	size
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

La salida de `describe()` reúne varios resúmenes numéricos en una sola tabla. Para `total_bill`, podemos observar que:

- el promedio es **19.79**;
- la mediana, indicada por `50%`, es **17.80**;
- el 25% de las cuentas tiene valores de hasta aproximadamente **13.35**;
- el 75% de las cuentas tiene valores de hasta aproximadamente **24.13**;
- el valor máximo es **50.81**.

Esto permite leer la columna con más detalle que usando solo promedio, mínimo y máximo. Por ejemplo, aunque la cuenta máxima es **50.81**, el 75% de las cuentas tiene valores de hasta aproximadamente **24.13**. Eso muestra que la cuenta más alta está bastante separada de la zona donde se concentra la mayoría de los valores.

En `tip` ocurre algo parecido, aunque con valores más pequeños: la mediana es **2.90**, el 75% de las propinas llega hasta aproximadamente **3.56**, y el valor máximo es **10.00**.

La columna `size` muestra que la mayoría de las mesas son pequeñas: la mediana es **2** y el 75% de los registros corresponde a mesas de hasta **3 personas**.

Elegir qué indicadores mostrar

`describe()` es útil para obtener una vista rápida, pero no siempre necesitamos mostrar todos sus indicadores. En un análisis propio, muchas veces conviene elegir solo las medidas que ayudan a responder la pregunta.

Vamos a construir una tabla más simple con cuatro indicadores para cada columna numérica: promedio, mediana, mínimo y máximo.

```
resumen_numerico = pd.DataFrame({
    "columna": columnas_numericas,
    "promedio": [
```

```

        df["total_bill"].mean(),
        df["tip"].mean(),
        df["size"].mean()
    ],
    "mediana": [
        df["total_bill"].median(),
        df["tip"].median(),
        df["size"].median()
    ],
    "mínimo": [
        df["total_bill"].min(),
        df["tip"].min(),
        df["size"].min()
    ],
    "máximo": [
        df["total_bill"].max(),
        df["tip"].max(),
        df["size"].max()
    ]
}

resumen_numerico[["promedio", "mediana", "mínimo", "máximo"]] =
resumen_numerico[["promedio", "mediana", "mínimo", "máximo"]].round(2)

resumen_numerico

```

índice	columna	promedio	mediana	mínimo	máximo
0	total_bill	19.79	17.8	3.07	50.81
1	tip	3.00	2.9	1.00	10.00
2	size	2.57	2.0	1.00	6.00

La tabla resume las tres columnas numéricas con los mismos indicadores. Esto permite comparar rápidamente qué representa cada variable:

- `total_bill` tiene un promedio de **19.79**, una mediana de **17.80**, un mínimo de **3.07** y un máximo de **50.81**.
- `tip` tiene un promedio de **3.00**, una mediana de **2.90**, un mínimo de **1.00** y un máximo de **10.00**.
- `size` tiene un promedio de **2.57**, una mediana de **2.00**, un mínimo de **1.00** y un máximo de **6.00**.

La tabla también muestra que el promedio y la mediana no siempre coinciden. En `total_bill`, el promedio queda por encima de la mediana; en `tip`, ambos valores están más cerca; y en `size`, la mediana confirma que el tamaño típico de mesa es de **2 personas**.

Este tipo de resumen es útil cuando queremos presentar solo los indicadores más relevantes, sin incluir toda la información que aparece en `describe()`.

Cierre del capítulo

En este capítulo nos concentramos en resumir columnas numéricas. Vimos que una misma columna puede describirse con distintos indicadores: cantidad de valores, promedio, mediana, mínimo, máximo y cuartiles. Cada uno aporta una mirada diferente.

El promedio permite obtener una medida general, pero puede verse influido por valores altos o bajos. La mediana ayuda a identificar un valor central menos sensible a esos extremos. Los mínimos y máximos muestran el rango observado, mientras que los cuartiles permiten ver mejor dónde se concentra buena parte de los datos.

También vimos que `describe()` ofrece una vista rápida de varios indicadores, pero que muchas veces conviene construir una tabla propia con las medidas que realmente queremos comunicar.

En el próximo capítulo vamos a cambiar el foco: dejaremos por un momento las columnas numéricas y vamos a trabajar con conteos, frecuencias y proporciones en columnas categóricas.

Capítulo 26 — Contar valores y analizar frecuencias

De resumir números a contar categorías

En el capítulo anterior trabajamos con columnas numéricas. Calculamos promedios, medianas, valores mínimos, valores máximos y cuartiles. Esos indicadores nos ayudaron a resumir variables como el total de la cuenta, la propina y el tamaño de la mesa.

Pero no todas las columnas se analizan de la misma manera. En muchas situaciones queremos responder preguntas como:

- ¿qué día aparece más veces en el dataset?
- ¿cuántos registros corresponden al almuerzo y cuántos a la cena?
- ¿qué proporción de las cuentas corresponde a cada categoría?
- ¿hay categorías muy frecuentes o muy poco frecuentes?

Para responder este tipo de preguntas necesitamos contar valores y analizar frecuencias.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Mantener el mismo dataset nos permite concentrarnos en una idea nueva: analizar columnas categóricas mediante conteos, proporciones y porcentajes.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset. Seguimos trabajando con las mismas variables de los capítulos anteriores, pero ahora el foco estará en las columnas categóricas.

Columnas como `sex`, `smoker`, `day` y `time` no se resumen calculando promedios o medianas. En estos casos suele ser más útil contar cuántas veces aparece cada categoría y qué proporción representa dentro del total.

Identificar columnas categóricas

Para organizar el análisis, vamos a separar las columnas categóricas que nos interesa revisar. En este dataset vamos a trabajar principalmente con:

- `sex`
- `smoker`
- `day`
- `time`

Cada una de estas columnas permite responder preguntas sobre la composición del dataset.

```
columnas_categoricas = ["sex", "smoker", "day", "time"]  
  
df[columnas_categoricas].head()
```

índice	sex	smoker	day	time
0	Female	No	Sun	Dinner
1	Male	No	Sun	Dinner
2	Male	No	Sun	Dinner
3	Male	No	Sun	Dinner
4	Female	No	Sun	Dinner

La vista reducida muestra solo las columnas categóricas que vamos a analizar. Cada una contiene valores que representan grupos o categorías. Por ejemplo, `day` indica el día registrado y `time` distingue entre almuerzo y cena.

En este capítulo vamos a empezar contando cuántas veces aparece cada valor en una columna. Para eso usaremos `value_counts()`, una herramienta que ya vimos en capítulos anteriores, pero que ahora vamos a estudiar con más detalle.

Contar valores con `value_counts()`

Cuando analizamos columnas categóricas, muchas veces queremos saber cuántas veces aparece cada categoría. A esa cantidad la llamamos **frecuencia**.

Por ejemplo, si el día `Sat` aparece 87 veces, decimos que su frecuencia es 87. Más adelante también vamos a calcular qué porcentaje del total representa esa cantidad. Una primera pregunta categórica puede ser:

¿Cuántas cuentas hay registradas para cada día?

Para responderla vamos a contar cuántas veces aparece cada valor en la columna `day`.

```
df["day"].value_counts()
```

day	count
Sat	87
Sun	76
Thur	62
Fri	19

```
dtype: int64
```

La salida muestra la cantidad de cuentas registradas para cada día. El día con más registros es **sábado (Sat)**, con **87 cuentas**. Luego aparecen **domingo (Sun)**, con **76**, y **jueves (Thur)**, con **62**. El día con menos registros es **viernes (Fri)**, con **19 cuentas**.

A estos valores los llamamos **frecuencias absolutas**, porque indican cuántas veces aparece cada categoría.

Por ahora solo estamos contando registros. Esto nos permite describir la composición del dataset, pero no alcanza para afirmar qué día tuvo mayor facturación o mejores propinas.

Calcular frecuencias relativas

Las frecuencias absolutas indican cuántas veces aparece cada categoría. Pero a veces no alcanza con saber la cantidad. También puede ser útil saber qué parte del total representa esa cantidad.

A esa relación la llamamos **frecuencia relativa**.

Por ejemplo, si una categoría aparece 87 veces en un dataset de 244 registros, no solo podemos decir que aparece 87 veces: también podemos calcular qué proporción del total representa.

En Pandas, `value_counts()` permite obtener frecuencias relativas usando `normalize=True`.

```
df["day"].value_counts(normalize=True)
```

day	proportion
Sat	0.356557
Sun	0.311475
Thur	0.254098
Fri	0.077869

`dtype: float64`

La salida muestra las **frecuencias relativas** de la columna `day`. En lugar de mostrar cantidades de registros, ahora cada valor indica qué parte del total representa cada día.

Por ejemplo:

- `Sat` representa aproximadamente **0.36** del total.
- `Sun` representa aproximadamente **0.31**.
- `Thur` representa aproximadamente **0.25**.
- `Fri` representa aproximadamente **0.08**.

Las frecuencias relativas son útiles porque permiten comparar categorías considerando el total de registros. Sin embargo, expresadas como decimales pueden ser menos intuitivas para leer. Por eso, muchas veces conviene convertirlas en porcentajes.

Convertir proporciones en porcentajes

Las frecuencias relativas pueden interpretarse como proporciones. Para hacerlas más fáciles de leer, podemos multiplicarlas por 100 y redondearlas. Así obtenemos el porcentaje de registros que corresponde a cada categoría.

```
df["day"].value_counts(normalize=True) * 100
```

day	proportion
Sat	35.655738
Sun	31.147541
Thur	25.409836
Fri	7.786885

dtype: float64

Ahora las frecuencias relativas aparecen expresadas como porcentajes. Podemos leer los resultados así:

- **Sat** representa aproximadamente el **35.66%** de los registros.
- **Sun** representa aproximadamente el **31.15%**.
- **Thur** representa aproximadamente el **25.41%**.
- **Fri** representa aproximadamente el **7.79%**.

La información es la misma que en la celda anterior, pero expresada en una escala más familiar. Aun así, la salida todavía puede mejorarse si redondeamos los valores y los organizamos en una tabla junto con las frecuencias absolutas.

Construir una tabla de frecuencias

Para comunicar mejor el resultado, podemos combinar en una misma tabla:

- la frecuencia absoluta, es decir, la cantidad de registros;
- el porcentaje, es decir, qué parte del total representa cada categoría.

Vamos a construir esa tabla para la columna `day`.

```
frecuencia_dias = pd.DataFrame({
    "frecuencia": df["day"].value_counts(),
    "porcentaje": df["day"].value_counts(normalize=True) * 100
})

frecuencia_dias["porcentaje"] = frecuencia_dias["porcentaje"].round(2)

frecuencia_dias
```

day	frecuencia	porcentaje
Sat	87	35.66

day	frecuencia	porcentaje
Sun	76	31.15
Thur	62	25.41
Fri	19	7.79

La tabla combina dos formas de leer la misma distribución. La columna `frecuencia` indica cuántas cuentas hay registradas para cada día. La columna `porcentaje` indica qué parte del total representa cada uno.

Podemos observar que:

- `Sat` concentra **87 registros**, equivalentes al **35.66%** del dataset.
- `Sun` concentra **76 registros**, equivalentes al **31.15%**.
- `Thur` concentra **62 registros**, equivalentes al **25.41%**.
- `Fri` concentra **19 registros**, equivalentes al **7.79%**.

Esta tabla es más informativa que el conteo aislado, porque permite ver tanto la cantidad de casos como el peso relativo de cada categoría.

Aplicar el mismo análisis a otra columna

El mismo procedimiento puede aplicarse a otras columnas categóricas.

Ahora vamos a analizar la columna `time`, que distingue entre almuerzo (`Lunch`) y cena (`Dinner`).

```
frecuencia_horario = pd.DataFrame({
    "frecuencia": df["time"].value_counts(),
    "porcentaje": df["time"].value_counts(normalize=True) * 100
})

frecuencia_horario["porcentaje"] = frecuencia_horario["porcentaje"].round(2)

frecuencia_horario
```

time	frecuencia	porcentaje
Dinner	176	72.13
Lunch	68	27.87

La tabla muestra que la mayoría de los registros corresponde a la cena.

En el dataset, `Dinner` aparece con **176 registros**, equivalentes al **72.13%** del total. En cambio, `Lunch` aparece con **68 registros**, equivalentes al **27.87%**.

Esta diferencia es importante para interpretar análisis posteriores. Si más adelante comparamos consumos o propinas por horario, debemos recordar que el dataset no tiene la misma cantidad de registros para almuerzo y cena.

Analizar otra variable categórica

También podemos revisar la columna `smoker`, que indica si la cuenta corresponde a una mesa registrada como fumadora o no fumadora. La pregunta ahora es:

¿Qué proporción de los registros corresponde a cada categoría?

```
frecuencia_fumadores = pd.DataFrame({
    "frecuencia": df["smoker"].value_counts(),
    "porcentaje": df["smoker"].value_counts(normalize=True) * 100
})

frecuencia_fumadores["porcentaje"] = frecuencia_fumadores["porcentaje"].round(2)

frecuencia_fumadores
```

smoker	frecuencia	porcentaje
No	151	61.89
Yes	93	38.11

La tabla muestra la distribución de la columna `smoker`.

En el dataset, la categoría `No` aparece con **151 registros**, equivalentes al **61.89%** del total. La categoría `Yes` aparece con **93 registros**, equivalentes al **38.11%**. Esto significa que hay más registros de mesas no fumadoras que de mesas fumadoras.

Como en los casos anteriores, esta tabla describe la composición del dataset. No indica todavía si un grupo consume más, deja más propina o tiene cuentas más altas. Para responder esas preguntas necesitaremos combinar categorías con variables numéricas.

Repetir el análisis en otra columna

Para completar esta primera revisión de frecuencias, vamos a aplicar el mismo procedimiento a la columna `sex`. La pregunta es la misma que antes:

¿Cuántos registros hay en cada categoría y qué porcentaje representa cada una?

```
frecuencia_sexo = pd.DataFrame({
    "frecuencia": df["sex"].value_counts(),
    "porcentaje": df["sex"].value_counts(normalize=True) * 100
})

frecuencia_sexo["porcentaje"] = frecuencia_sexo["porcentaje"].round(2)

frecuencia_sexo
```

sex	frecuencia	porcentaje
Male	157	64.34
Female	87	35.66

La tabla muestra la distribución de la columna `sex`.

En el dataset, la categoría `Male` aparece con **157 registros**, equivalentes al **64.34%** del total. La categoría `Female` aparece con **87 registros**, equivalentes al **35.66%**.

Esto indica que, dentro de este dataset, hay más registros asociados a `Male` que a `Female`.

Nuevamente, esta salida describe cantidades y proporciones. No permite concluir, por sí sola, que un grupo consuma más o deje más propina. Para eso habría que combinar esta variable categórica con columnas numéricas como `total_bill` o `tip`.

Cierre del capítulo

En este capítulo trabajamos con columnas categóricas. A diferencia de las columnas numéricas, que pueden resumirse con promedios, medianas, mínimos o máximos, las columnas categóricas suelen analizarse contando cuántas veces aparece cada valor.

Vimos tres formas de leer estas columnas:

- **frecuencias absolutas**, que indican la cantidad de registros por categoría;
- **frecuencias relativas**, que indican qué proporción del total representa cada categoría;
- **porcentajes**, que expresan esas proporciones de una manera más fácil de interpretar.

También construimos tablas de frecuencia que combinan cantidad de registros y porcentaje. Esta forma de presentación permite describir mejor la composición del dataset.

Sin embargo, contar categorías no alcanza para responder preguntas más analíticas, como qué día tiene mayor facturación promedio, qué horario deja mejores propinas o si un grupo tiene cuentas más altas que otro.

Para responder ese tipo de preguntas necesitaremos combinar categorías con variables numéricas. Ese será el próximo paso en el análisis.

Capítulo 27 — Primeras tablas de resumen

De resultados sueltos a tablas de resumen

En los capítulos anteriores calculamos distintos tipos de resúmenes.

Por un lado, trabajamos con columnas numéricas: promedios, medianas, mínimos, máximos y cuartiles. Por otro, analizamos columnas categóricas contando valores, calculando frecuencias relativas y expresando proporciones como porcentajes.

Ahora vamos a dar un paso más: organizar resultados en **tablas de resumen**.

Una tabla de resumen no busca mostrar todos los datos originales. Su objetivo es reunir algunos indicadores importantes para responder una pregunta, comparar información o comunicar una primera lectura del dataset.

En este capítulo vamos a construir tablas simples, pensadas para ordenar resultados y facilitar la interpretación.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. antener el mismo dataset nos permite concentrarnos en una idea nueva:** cómo organizar resultados en tablas de resumen claras y útiles.**

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables de los capítulos anteriores. En este capítulo no vamos a centrarnos en una columna aislada, sino en cómo reunir distintos resultados en tablas de resumen.

La idea es que una tabla de resumen ayude a leer mejor el dataset sin mostrar todos los registros originales.

Una primera tabla de indicadores generales

Para comenzar, vamos a construir una tabla con algunos indicadores generales del dataset. La pregunta que queremos responder es:

¿Qué información básica podemos resumir sobre las cuentas y las propinas?

Vamos a calcular la cantidad de cuentas, el total facturado, el total de propinas, el promedio de cuenta y el promedio de propina.

```
resumen_general = pd.DataFrame({
    "indicador": [
        "Cantidad de cuentas",
        "Total facturado",
        "Total de propinas",
        "Promedio de cuenta",
        "Promedio de propina"
    ],
    "valor": [
        df["total_bill"].count(),
        df["total_bill"].sum(),
        df["tip"].sum(),
        df["total_bill"].mean(),
        df["tip"].mean()
    ]
})

resumen_general["valor"] = resumen_general["valor"].round(2)

resumen_general
```

índice	indicador	valor
0	Cantidad de cuentas	244.00
1	Total facturado	4827.77
2	Total de propinas	731.58
3	Promedio de cuenta	19.79
4	Promedio de propina	3.00

La tabla reúne varios indicadores generales del dataset. En lugar de mostrar cada cálculo por separado, organiza los resultados en una forma más clara:

- hay **244 cuentas** registradas;
- el total facturado es **4827.77**;
- el total de propinas es **731.58**;
- el promedio de cuenta es **19.79**;
- el promedio de propina es **3.00**.

Esta tabla funciona como un primer resumen general. No muestra diferencias por día, horario o tipo de mesa, pero permite comunicar rápidamente algunos datos básicos del conjunto.

Una tabla de frecuencias

No todas las tablas de resumen se construyen con promedios o sumas. Cuando trabajamos con columnas categóricas, una tabla de resumen puede mostrar cuántas veces aparece cada categoría y qué porcentaje representa.

Vamos a construir una tabla de frecuencias para la columna `day`.

```
tabla_dias = pd.DataFrame({
    "frecuencia": df["day"].value_counts(),
    "porcentaje": df["day"].value_counts(normalize=True) * 100
})

tabla_dias["porcentaje"] = tabla_dias["porcentaje"].round(2)

tabla_dias
```

day	frecuencia	porcentaje
Sat	87	35.66
Sun	76	31.15
Thur	62	25.41
Fri	19	7.79

Esta tabla resume la distribución de la columna `day`.

A diferencia de la tabla anterior, acá cada fila no representa un indicador general, sino una categoría de la columna analizada. Podemos observar que:

- `Sat` concentra **87 registros**, equivalentes al **35.66%** del total.
- `Sun` concentra **76 registros**, equivalentes al **31.15%**.

- **Thur** concentra **62 registros**, equivalentes al **25.41%**.
- **Fri** concentra **19 registros**, equivalentes al **7.79%**.

Esta tabla permite describir cómo se distribuyen los registros según el día.

La forma de la tabla depende de la pregunta

Las dos tablas anteriores resumen información, pero no tienen la misma estructura.

En la primera tabla, cada fila era un indicador general del dataset: cantidad de cuentas, total facturado o promedio de propina.

En la segunda tabla, cada fila era una categoría de una columna: **Sat**, **Sun**, **Thur** o **Fri**.

Esto muestra una idea importante: no existe una única forma de construir tablas de resumen. La forma depende de qué pregunta queremos responder y de qué información queremos comunicar.

Resumir características generales del dataset

También podemos construir una tabla que no se limite solo a indicadores numéricos. Por ejemplo, podemos resumir cuántos registros tiene el dataset, cuántos días distintos aparecen, cuántos horarios distintos hay registrados y cuáles son los valores promedio de cuenta y propina.

Esta tabla no reemplaza el análisis detallado, pero puede servir como una presentación general del conjunto de datos.

```
resumen_dataset = pd.DataFrame({
    "aspecto": [
        "Cantidad de registros",
        "Cantidad de días distintos",
        "Cantidad de horarios distintos",
        "Promedio de cuenta",
        "Promedio de propina"
    ],
    "valor": [
        df.shape[0],
        df["day"].nunique(),
        df["time"].nunique(),
        df["total_bill"].mean(),
        df["tip"].mean()
    ]
})

resumen_dataset["valor"] = resumen_dataset["valor"].round(2)
```

índice	aspecto	valor
0	Cantidad de registros	244.00
1	Cantidad de días distintos	4.00
2	Cantidad de horarios distintos	2.00
3	Promedio de cuenta	19.79
4	Promedio de propina	3.00

Esta tabla reúne características generales del dataset en una sola vista. Podemos leer que el conjunto tiene **244 registros**, distribuidos en **4 días distintos** y **2 horarios distintos**. También recupera dos indicadores numéricos generales: el promedio de cuenta, **19.79**, y el promedio de propina, **3.00**.

Este tipo de tabla puede ser útil como presentación inicial de un dataset, porque combina información estructural con algunos indicadores básicos. Sin embargo, todavía no muestra cómo cambian esos indicadores entre categorías. Por ejemplo, no nos dice si el promedio de cuenta es distinto según el día o según el horario.

Tablas para explorar y tablas para comunicar

Cuando analizamos datos, no todas las tablas cumplen la misma función.

Algunas tablas sirven para explorar. Son tablas de trabajo: pueden tener muchos valores, nombres técnicos o detalles que usamos mientras investigamos el dataset. Otras están pensadas para comunicar. En ese caso conviene que sean más claras, más breves y que incluyan solo los datos necesarios para responder una pregunta.

La misma información puede organizarse de distintas maneras según el objetivo.

Preparar una tabla más legible

La tabla anterior es correcta, pero podemos hacerla un poco más clara para presentarla. Vamos a construir una nueva versión con nombres más descriptivos y algunos valores redondeados.

```
tabla_presentacion = pd.DataFrame({
    "Resumen": [
        "Registros analizados",
        "Días incluidos",
        "Horarios incluidos",
        "Cuenta promedio",
```

```

        "Propina promedio"
    ],
    "Valor": [
        df.shape[0],
        df["day"].nunique(),
        df["time"].nunique(),
        round(df["total_bill"].mean(), 2),
        round(df["tip"].mean(), 2)
    ]
})

tabla_presentacion

```

índice	Resumen	Valor
0	Registros analizados	244.00
1	Días incluidos	4.00
2	Horarios incluidos	2.00
3	Cuenta promedio	19.79
4	Propina promedio	3.00

La nueva tabla contiene la misma información general, pero con nombres más pensados para la lectura.

Los nombres de las columnas empiezan con mayúscula y los indicadores usan expresiones más claras, como **Registros analizados**, **Cuenta promedio** y **Propina promedio**.

Este tipo de ajuste puede parecer menor, pero es importante cuando una tabla deja de ser solo una salida de trabajo y empieza a funcionar como parte de una comunicación de resultados.

Cuidar el formato de los valores

Una tabla puede ser correcta desde el punto de vista del cálculo, pero todavía puede necesitar pequeños ajustes de presentación. En este caso, algunos valores representan cantidades enteras, como la cantidad de registros, días u horarios. No tiene mucho sentido mostrarlos con decimales.

Vamos a preparar una versión final mostrando las cantidades como enteros y los promedios con dos decimales.

```

tabla_presentacion = pd.DataFrame({
    "Resumen": [
        "Registros analizados",
        "Días incluidos",

```

```

    "Horarios incluidos",
    "Cuenta promedio",
    "Propina promedio"
],
"Valor": [
    df.shape[0],
    df["day"].nunique(),
    df["time"].nunique(),
    f'{df["total_bill"].mean():.2f}',
    f'{df["tip"].mean():.2f}'
]
})

tabla_presentacion

```

índice	Resumen	Valor
0	Registros analizados	244
1	Días incluidos	4
2	Horarios incluidos	2
3	Cuenta promedio	19.79
4	Propina promedio	3.00

Esta versión queda más cuidada para comunicar resultados.

Los valores que representan cantidades aparecen como números enteros: **244** registros, **4** días y **2** horarios. En cambio, los valores que representan promedios conservan dos decimales: **19.79** para la cuenta promedio y **3.00** para la propina promedio.

Este detalle mejora la lectura de la tabla. No se trata solo de calcular correctamente, sino también de presentar cada resultado con un formato adecuado.

Cierre del capítulo

En este capítulo trabajamos con tablas de resumen. Vimos que una tabla de resumen no tiene una única forma. Puede organizar indicadores generales, frecuencias por categoría o características generales de un dataset.

También distinguimos entre tablas de trabajo y tablas pensadas para comunicar resultados. En una tabla de trabajo puede ser suficiente mostrar la salida tal como la produce Pandas. En una tabla de presentación, en cambio, conviene cuidar los nombres, el orden de los indicadores y el formato de los valores.

Hasta ahora construimos resúmenes generales del dataset o tablas basadas en conteos de categorías. Pero todavía no respondimos preguntas como:

- ¿cuál es el promedio de cuenta por día?
- ¿cuál es la propina promedio según el horario?
- ¿cuántos registros hay en cada combinación de día y horario?

Para responder ese tipo de preguntas vamos a necesitar resumir datos por grupos. Ese será el próximo paso.

Capítulo 28 — Agrupar datos con `groupby()`

De las tablas generales a los resúmenes por grupo

En los capítulos anteriores construimos resúmenes generales del dataset.

Calculamos indicadores numéricos, analizamos frecuencias categóricas y organizamos resultados en tablas de resumen. Esas herramientas nos permitieron responder preguntas como:

- ¿cuál es el promedio general de las cuentas?
- ¿cuántos registros hay para cada día?
- ¿qué porcentaje del dataset representa cada categoría?
- ¿cómo presentar indicadores de forma más clara?

Ahora vamos a avanzar hacia una pregunta más analítica:

¿Cómo cambian esos indicadores según un grupo?

Por ejemplo, no solo queremos saber cuál es el promedio general de las cuentas. También podríamos preguntarnos:

¿Cuál es el promedio de cuenta para cada día?

Para responder ese tipo de preguntas necesitamos agrupar datos.

Qué significa agrupar datos

Agrupar datos significa separar un dataset en partes según los valores de una columna y luego calcular algún resumen dentro de cada parte.

Por ejemplo, si agrupamos por la columna `day`, Pandas separa los registros según el día:

- registros de `Thur`;
- registros de `Fri`;
- registros de `Sat`;
- registros de `Sun`.

Luego podemos calcular un indicador para cada grupo, como el promedio de cuenta, el total facturado o la cantidad de registros.

En Pandas, una de las herramientas principales para hacer esto es `groupby()`.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Esto nos permite continuar con el mismo contexto y concentrarnos en una herramienta nueva: `groupby()`.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables de los capítulos anteriores. En este capítulo vamos a usar especialmente:

- `day`, para formar grupos según el día;
- `time`, para formar grupos según el horario;
- `total_bill`, para calcular resúmenes sobre el total de las cuentas;
- `tip`, para calcular resúmenes sobre las propinas.

La idea central será combinar una columna categórica con una columna numérica.

Una primera pregunta por grupo

Hasta ahora sabemos calcular el promedio general de una columna. Por ejemplo, podemos calcular el promedio general de `total_bill`. Pero ahora queremos hacer una pregunta más específica:

¿Cuál es el promedio de cuenta para cada día?

Para responderla, necesitamos agrupar los datos por `day` y calcular el promedio de `total_bill` dentro de cada grupo.

```
df.groupby("day", observed=True)["total_bill"].mean()
```

day	total_bill
Thur	17.682742
Fri	17.151579
Sat	20.441379
Sun	21.410000

`dtype: float64`

El resultado muestra el promedio de `total_bill` para cada día.

Ahora ya no tenemos un único promedio general, sino un promedio separado por grupo:

- `Thur` : aproximadamente **17.68**.
- `Fri` : aproximadamente **17.15**.
- `Sat` : aproximadamente **20.44**.
- `Sun` : aproximadamente **21.41**.

Según estos datos, el promedio de cuenta más alto aparece el domingo (`Sun`), seguido por el sábado (`Sat`). Los promedios más bajos aparecen jueves (`Thur`) y viernes (`Fri`).

Este es el cambio central que introduce `groupby()` : nos permite calcular indicadores separados para cada categoría.

En esta celda usamos `observed=True` porque `day` es una columna categórica. Este parámetro le indica a Pandas que trabaje con las categorías observadas en los datos. En este caso no cambia el resultado, pero evita un aviso relacionado con cambios futuros en Pandas.

```
promedio_cuenta_por_dia = df.groupby("day", observed=True)["total_bill"].mean()

promedio_cuenta_por_dia = promedio_cuenta_por_dia.round(2)

promedio_cuenta_por_dia
```

day	total_bill
Thur	17.68
Fri	17.15
Sat	20.44
Sun	21.41

`dtype: float64`

Ahora el resultado es más fácil de leer. El promedio de cuenta por día queda así:

- **Thur : 17.68**
- **Fri : 17.15**
- **Sat : 20.44**
- **Sun : 21.41**

Esta tabla muestra que el promedio de cuenta es mayor durante el domingo y el sábado que durante el jueves y el viernes.

A diferencia de los resúmenes generales, este resultado ya compara grupos dentro del dataset. La columna `day` define los grupos y la columna `total_bill` aporta los valores numéricos que se resumen.

Leer la estructura de un `groupby()`

La expresión que usamos puede leerse en tres partes:

```
df.groupby("day", observed=True)["total_bill"].mean()
```

Primero, `groupby("day")` agrupa las filas según el valor de la columna `day`.

Luego, `["total_bill"]` indica qué columna numérica queremos resumir.

Finalmente, `.mean()` calcula el promedio dentro de cada grupo.

En palabras simples:** agrupamos por día y calculamos el promedio de cuenta para cada día**.

Cambiar el indicador calculado

Una vez que tenemos clara la idea de agrupar, podemos cambiar el indicador. En lugar de calcular el promedio de cuenta por día, podemos calcular el total facturado por día.

La agrupación será la misma: `day`. Lo que cambia es el cálculo final.

```
total_facturado_por_dia = df.groupby("day", observed=True)["total_bill"].sum()

total_facturado_por_dia = total_facturado_por_dia.round(2)

total_facturado_por_dia
```

day	total_bill
Thur	1096.33
Fri	325.88
Sat	1778.40
Sun	1627.16

`dtype: float64`

Ahora calculamos el **total facturado por día**.

El resultado muestra que:

- `Sat` tiene el total facturado más alto: **1778.40**.
- `Sun` aparece en segundo lugar: **1627.16**.
- `Thur` suma **1096.33**.
- `Fri` tiene el total más bajo: **325.88**.

Este resultado no debe leerse igual que el promedio anterior.

El promedio de cuenta nos dice cuánto se consume, en promedio, por registro. El total facturado depende tanto del valor de las cuentas como de la cantidad de registros que hay para cada día.

Por eso, un día puede tener mayor facturación total no solo porque sus cuentas sean más altas, sino también porque tiene más cuentas registradas.

Calcular la cantidad de registros por grupo

Para interpretar mejor un total, también conviene saber cuántos registros hay en cada grupo.

Vamos a contar cuántas cuentas hay registradas para cada día usando `groupby()`.

```
cantidad_cuentas_por_dia = df.groupby("day", observed=True)["total_bill"].count()

cantidad_cuentas_por_dia
```

day	total_bill
Thur	62
Fri	19
Sat	87
Sun	76

dtype: int64

La tabla muestra cuántas cuentas hay registradas para cada día:

- **Thur** : **62** cuentas.
- **Fri** : **19** cuentas.
- **Sat** : **87** cuentas.
- **Sun** : **76** cuentas.

Esta información ayuda a interpretar el total facturado por día.

Por ejemplo, **Sat** tiene el total facturado más alto, pero también es el día con más cuentas registradas. En cambio, **Sun** tiene un promedio de cuenta más alto que **Sat**, aunque su total facturado es menor porque tiene menos registros.

Esto muestra por qué conviene mirar más de un indicador antes de sacar conclusiones.

Combinar varios resultados por grupo

Hasta ahora calculamos tres resultados separados por día:

- promedio de cuenta;
- total facturado;
- cantidad de cuentas.

Para analizar mejor los grupos, puede ser útil reunir esos resultados en una sola tabla.

```
resumen_por_dia = pd.DataFrame({
    "cantidad_cuentas": cantidad_cuentas_por_dia,
    "total_facturado": total_facturado_por_dia,
    "promedio_cuenta": promedio_cuenta_por_dia
})
```

```
})
```

```
resumen_por_dia
```

day	cantidad_cuentas	total_facturado	promedio_cuenta
Thur	62	1096.33	17.68
Fri	19	325.88	17.15
Sat	87	1778.40	20.44
Sun	76	1627.16	21.41

Ahora tenemos una tabla de resumen por día.

Cada fila representa un grupo definido por la columna `day`, y cada columna muestra un indicador distinto:

- `cantidad_cuentas`: cantidad de registros para ese día.
- `total_facturado`: suma de `total_bill` para ese día.
- `promedio_cuenta`: promedio de `total_bill` para ese día.

Esta tabla permite leer mejor las diferencias entre grupos.

Por ejemplo, `Sat` tiene la mayor cantidad de cuentas y el mayor total facturado. Sin embargo, `Sun` tiene el promedio de cuenta más alto. Esto muestra que el total facturado y el promedio de cuenta no responden exactamente la misma pregunta.

Calcular varios indicadores con `agg()`

La tabla anterior fue construida combinando varios resultados separados. Pandas también permite calcular varios indicadores dentro de una misma agrupación usando `.agg()`.

La palabra `agg` viene de “aggregate”, que podemos entender como “agregar” o “resumir”. En este contexto significa aplicar una o varias funciones de resumen sobre cada grupo.

```
resumen_por_dia_agg = df.groupby("day", observed=True)["total_bill"].agg(  
    cantidad_cuentas="count",  
    total_facturado="sum",  
    promedio_cuenta="mean"  
)  
  
resumen_por_dia_agg = resumen_por_dia_agg.round(2)  
  
resumen_por_dia_agg
```

day	cantidad_cuentas	total_facturado	promedio_cuenta
Thur	62	1096.33	17.68
Fri	19	325.88	17.15
Sat	87	1778.40	20.44
Sun	76	1627.16	21.41

El resultado es el mismo que habíamos construido manualmente, pero ahora lo obtuvimos con una sola agrupación y una sola llamada a `.agg()`.

Esto tiene una ventaja importante: el código queda más compacto y expresa mejor la intención del análisis. La instrucción puede leerse así:

- agrupar por `day`;
- tomar la columna `total_bill`;
- calcular cantidad, suma y promedio para cada grupo;
- asignar nombres claros a cada indicador.

Cuando necesitamos calcular varios resúmenes sobre una misma columna, `.agg()` suele ser una forma más ordenada de hacerlo.

Convertir el índice en una columna

Después de usar `groupby()`, la columna usada para agrupar suele quedar como índice del resultado. En este caso, `day` aparece como índice de la tabla.

Muchas veces conviene convertir ese índice nuevamente en una columna común, especialmente si queremos seguir trabajando con el resultado como un `DataFrame`.

```
resumen_por_dia_agg = resumen_por_dia_agg.reset_index()

resumen_por_dia_agg
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta
0	Thur	62	1096.33	17.68
1	Fri	19	325.88	17.15
2	Sat	87	1778.40	20.44
3	Sun	76	1627.16	21.41

Ahora `day` vuelve a aparecer como una columna común del `DataFrame`.

Esto puede resultar más cómodo para leer, ordenar, filtrar o usar la tabla en pasos posteriores. El contenido no cambió: seguimos teniendo los mismos grupos y los mismos indicadores. Lo que cambió es la forma de presentar la tabla.

Agrupar por otra columna

La misma lógica puede aplicarse a otra variable categórica.

Ahora vamos a agrupar por `time`, que distingue entre almuerzo (`Lunch`) y cena (`Dinner`), y vamos a calcular los mismos indicadores sobre `total_bill`.

```
resumen_por_horario = df.groupby("time", observed=True)["total_bill"].agg(  
    cantidad_cuentas="count",  
    total_facturado="sum",  
    promedio_cuenta="mean"  
)  
  
resumen_por_horario = resumen_por_horario.round(2).reset_index()  
  
resumen_por_horario
```

índice	time	cantidad_cuentas	total_facturado	promedio_cuenta
0	Lunch	68	1167.47	17.17
1	Dinner	176	3660.30	20.80

La tabla resume las cuentas según el horario.

Podemos observar que:

- `Lunch` tiene **68 cuentas**, un total facturado de **1167.47** y un promedio de cuenta de **17.17**.
- `Dinner` tiene **176 cuentas**, un total facturado de **3660.30** y un promedio de cuenta de **20.80**.

En este dataset, la cena concentra más registros, más facturación total y también un promedio de cuenta más alto.

Esta tabla muestra cómo `groupby()` permite responder preguntas más específicas que un resumen general. Ya no estamos mirando solo el total del dataset, sino comparando indicadores entre grupos.

Agrupar por más de una columna

Hasta ahora agrupamos por una sola columna: primero por `day` y luego por `time`. Pero también podemos agrupar usando más de una columna. Esto permite responder preguntas más detalladas. Por ejemplo:

¿Cuántas cuentas hay para cada combinación de día y horario?

Para responder esta pregunta, podemos agrupar por `day` y `time` al mismo tiempo.

```
cuentas_por_dia_y_horario = df.groupby(["day", "time"], observed=True)
["total_bill"].count()

cuentas_por_dia_y_horario
```

day		total_bill
Thur	Lunch	61
Dinner	1	
Fri	Lunch	7
Dinner	12	
Sat	Dinner	87
Sun	Dinner	76

`dtype: int64`

El resultado muestra la cantidad de cuentas para cada combinación de `day` y `time`.

Ahora los grupos no se forman solo por día, sino por la combinación entre día y horario. Podemos leer algunos resultados:

- `Thur` tiene **61 cuentas** en `Lunch` y **1 cuenta** en `Dinner`.
- `Fri` tiene **7 cuentas** en `Lunch` y **12 cuentas** en `Dinner`.
- `Sat` tiene **87 cuentas** en `Dinner`.
- `Sun` tiene **76 cuentas** en `Dinner`.

Esto permite responder una pregunta más detallada que antes. No solo sabemos cuántas cuentas hay por día o por horario, sino cuántas hay para cada combinación de ambas variables.

También aparece una diferencia en la forma de la salida: ahora el resultado tiene dos niveles de índice, porque agrupamos por dos columnas.

Convertir una agrupación doble en tabla

Cuando agrupamos por más de una columna, Pandas puede devolver un resultado con varios niveles de índice.

Para leerlo de forma más parecida a una tabla común, podemos usar `reset_index()`.

```
cuentas_por_dia_y_horario = cuentas_por_dia_y_horario.reset_index()

cuentas_por_dia_y_horario = cuentas_por_dia_y_horario.rename(
    columns={"total_bill": "cantidad_cuentas"}
)

cuentas_por_dia_y_horario
```

índice	day	time	cantidad_cuentas
0	Thur	Lunch	61
1	Thur	Dinner	1
2	Fri	Lunch	7
3	Fri	Dinner	12
4	Sat	Dinner	87
5	Sun	Dinner	76

Ahora el resultado tiene forma de tabla común.

Las columnas `day` y `time` indican el grupo al que pertenece cada fila. La columna `cantidad_cuentas` contiene la cantidad de cuentas en esa combinación de día y horario. La información es la misma que antes, pero esta presentación suele ser más cómoda para leer y para seguir trabajando.

Por ejemplo, podemos ver rápidamente que:

- `Thur` aparece principalmente en `Lunch`;
- `Fri` tiene registros tanto en `Lunch` como en `Dinner`;
- `Sat` y `Sun` aparecen solo con registros de `Dinner`.

Esta primera agrupación por más de una columna muestra que `groupby()` puede responder preguntas cada vez más específicas.

Cierre del capítulo

En este capítulo comenzamos a trabajar con `groupby()`, una de las herramientas más importantes de Pandas para el análisis de datos.

La idea central fue agrupar filas según una o más columnas y calcular resúmenes dentro de cada grupo. Vimos que podemos usar `groupby()` para responder preguntas como:

- ¿cuál es el promedio de cuenta por día?
- ¿cuál es el total facturado por día?
- ¿cuántas cuentas hay en cada grupo?
- ¿cómo cambian los indicadores según el horario?
- ¿cuántas cuentas hay para cada combinación de día y horario?

También vimos que no todos los indicadores responden la misma pregunta. El total facturado depende de los valores de las cuentas y de la cantidad de registros, mientras que el promedio de cuenta describe el valor promedio dentro de cada grupo.

Finalmente, usamos `.agg()` para calcular varios indicadores en una misma agrupación y `reset_index()` para convertir el resultado en una tabla más cómoda de leer.

En los próximos capítulos vamos a seguir usando agrupaciones para comparar categorías, construir tablas más completas y preparar resultados para visualizaciones.

Capítulo 29 — Agrupaciones múltiples y varios indicadores con `groupby()`

Profundizar el análisis por grupos

En el capítulo anterior presentamos el uso de `groupby()`.

Vimos que agrupar datos permite calcular indicadores separados para cada categoría. Por ejemplo, calculamos el promedio de cuenta por día, el total facturado por día y la cantidad de cuentas por grupo.

En este capítulo vamos a avanzar un paso más.

Ahora vamos a usar `groupby()` para responder preguntas más detalladas, combinando más de una columna de agrupación y calculando varios indicadores al mismo tiempo.

El objetivo no es escribir consultas más largas, sino construir resúmenes que permitan comparar mejor los datos.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn.

Como ya conocemos sus variables principales, podemos concentrarnos directamente en las agrupaciones y en la interpretación de los resultados.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables, pero en este capítulo vamos a prestar especial atención a las combinaciones entre columnas categóricas y variables numéricas.

Por ejemplo, ya no solo nos interesa saber el promedio de cuenta por día. También podemos preguntarnos cómo cambia ese promedio si además distinguimos entre almuerzo y cena.

Agrupar por más de una columna

En el capítulo anterior vimos una primera agrupación por dos columnas. Ahora vamos a retomarla con más intención analítica. La pregunta será:

¿Cuántas cuentas hay para cada combinación de día y horario?

Para responderla vamos a agrupar por `day` y `time` al mismo tiempo.

```
cuentas_por_dia_y_horario = df.groupby(
    ["day", "time"],
    observed=True
)["total_bill"].count()

cuentas_por_dia_y_horario
```

day		total_bill
Thur	Lunch	61
Dinner	1	
Fri	Lunch	7
Dinner	12	
Sat	Dinner	87
Sun	Dinner	76

`dtype: int64`

El resultado muestra la cantidad de cuentas para cada combinación de `day` y `time`.

Ahora cada grupo está definido por dos valores: un día y un horario. Por ejemplo, `Thur` y `Lunch` forman un grupo, mientras que `Fri` y `Dinner` forman otro. Podemos observar que:

- `Thur` aparece casi completamente asociado a `Lunch`, con **61 cuentas** en almuerzo y solo **1** en cena.
- `Fri` tiene registros tanto en `Lunch` como en `Dinner`, con **7** y **12 cuentas** respectivamente.
- `Sat` y `Sun` aparecen únicamente asociados a `Dinner`, con **87** y **76 cuentas**.

Esta salida nos permite ver una estructura que no aparecía al analizar `day` y `time` por separado: algunos días están asociados principalmente, o exclusivamente, a un horario.

Convertir el resultado en una tabla

El resultado anterior tiene un índice de dos niveles: `day` y `time`. Eso es normal cuando agrupamos por más de una columna, pero puede ser más cómodo convertir esos niveles en columnas comunes usando `reset_index()`.

También vamos a cambiar el nombre de la columna `total_bill`, porque en este caso no contiene importes, sino una cantidad de cuentas.

```
cuentas_por_dia_y_horario = cuentas_por_dia_y_horario.reset_index()

cuentas_por_dia_y_horario = cuentas_por_dia_y_horario.rename(
    columns={"total_bill": "cantidad_cuentas"}
)

cuentas_por_dia_y_horario
```

índice	day	time	cantidad_cuentas
0	Thur	Lunch	61
1	Thur	Dinner	1
2	Fri	Lunch	7
3	Fri	Dinner	12
4	Sat	Dinner	87
5	Sun	Dinner	76

Ahora el resultado tiene una forma más parecida a una tabla común.

Las columnas `day` y `time` indican la combinación que define cada grupo. La columna `cantidad_cuentas` indica cuántos registros hay en cada combinación. Este formato suele ser más cómodo para leer y también para seguir trabajando con el resultado en pasos posteriores.

Además, el cambio de nombre de la columna ayuda a evitar una confusión importante: aunque usamos `total_bill` para contar registros, el resultado ya no representa importes de cuentas, sino cantidades.

Calcular promedios por combinación de grupos

Contar registros por combinación de categorías es útil, pero también podemos calcular indicadores numéricos. Ahora vamos a responder esta pregunta:

¿Cuál es el promedio de cuenta para cada combinación de día y horario?

Para eso vamos a agrupar nuevamente por `day` y `time`, pero esta vez vamos a calcular el promedio de `total_bill`.

```
promedio_cuenta_por_dia_y_horario = df.groupby(
    ["day", "time"],
    observed=True
)["total_bill"].mean()

promedio_cuenta_por_dia_y_horario
```

day		total_bill
Thur	Lunch	17.664754
Dinner	18.780000	
Fri	Lunch	12.845714
Dinner	19.663333	
Sat	Dinner	20.441379
Sun	Dinner	21.410000

`dtype: float64`

El resultado muestra el promedio de `total_bill` para cada combinación de `day` y `time`.

Ahora no estamos calculando un promedio por día solamente, ni un promedio por horario solamente, sino un promedio para cada combinación observada. Podemos leer algunos resultados:

- **Thur** tiene un promedio de **17.66** en **Lunch** y **18.78** en **Dinner**.
- **Fri** tiene un promedio de **12.85** en **Lunch** y **19.66** en **Dinner**.
- **Sat** tiene un promedio de **20.44** en **Dinner**.
- **Sun** tiene un promedio de **21.41** en **Dinner**.

Esta salida permite comparar con más detalle, pero también requiere cuidado. Por ejemplo, **Thur** en **Dinner** aparece con un solo registro, como vimos en el conteo anterior. Por eso, aunque su promedio sea **18.78**, no conviene interpretarlo con la misma fuerza que un grupo con muchos registros.

Presentar el resultado como tabla

Para leer mejor el resultado, vamos a convertir el índice en columnas comunes y redondear el promedio a dos decimales.

```
promedio_cuenta_por_dia_y_horario =  
promedio_cuenta_por_dia_y_horario.round(2).reset_index()  
  
promedio_cuenta_por_dia_y_horario = promedio_cuenta_por_dia_y_horario.rename(  
    columns={"total_bill": "promedio_cuenta"}  
)  
  
promedio_cuenta_por_dia_y_horario
```

índice	day	time	promedio_cuenta
0	Thur	Lunch	17.66
1	Thur	Dinner	18.78
2	Fri	Lunch	12.85
3	Fri	Dinner	19.66
4	Sat	Dinner	20.44
5	Sun	Dinner	21.41

Ahora la tabla es más fácil de leer.

Cada fila representa una combinación de día y horario, y la columna **promedio_cuenta** muestra el promedio de **total_bill** para ese grupo. Esta forma de presentación evita el índice de dos niveles y deja explícito qué variables forman cada grupo.

También permite detectar rápidamente que los promedios más altos aparecen en Sun - Dinner y Sat - Dinner, mientras que Fri - Lunch tiene el promedio más bajo.

De todos modos, esta lectura debe combinarse con la cantidad de registros de cada grupo. Un promedio calculado sobre pocos casos puede ser menos representativo que uno calculado sobre muchos registros.

Calcular varios indicadores por combinación de grupos

Para interpretar mejor una agrupación doble, conviene mirar más de un indicador. Vamos a calcular, para cada combinación de day y time:

- cantidad de cuentas;
- total facturado;
- promedio de cuenta;
- promedio de propina.

Esto nos permitirá comparar grupos con más información.

```
resumen_dia_horario = df.groupby(
    ["day", "time"],
    observed=True
).agg(
    cantidad_cuentas=("total_bill", "count"),
    total_facturado=("total_bill", "sum"),
    promedio_cuenta=("total_bill", "mean"),
    promedio_propina=("tip", "mean")
)

resumen_dia_horario = resumen_dia_horario.round(2).reset_index()

resumen_dia_horario
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Thur	Lunch	61	1077.55	17.66	2.77
1	Thur	Dinner	1	18.78	18.78	3.00
2	Fri	Lunch	7	89.92	12.85	2.38
3	Fri	Dinner	12	235.96	19.66	2.94
4	Sat	Dinner	87	1778.40	20.44	2.99
5	Sun	Dinner	76	1627.16	21.41	3.26

La tabla resume cada combinación de day y time usando varios indicadores.

Ahora podemos leer cada grupo con más contexto:

- **Thur - Lunch** tiene **61 cuentas**, un total facturado de **1077.55**, un promedio de cuenta de **17.66** y una propina promedio de **2.77**.
- **Thur - Dinner** tiene solo **1 cuenta**, por eso sus promedios deben interpretarse con mucha cautela.
- **Fri - Lunch** tiene **7 cuentas** y el promedio de cuenta más bajo: **12.85**.
- **Fri - Dinner** tiene **12 cuentas** y un promedio de cuenta de **19.66**.
- **Sat - Dinner** concentra **87 cuentas** y el total facturado más alto: **1778.40**.
- **Sun - Dinner** tiene **76 cuentas** y el promedio de cuenta más alto: **21.41**.

Esta tabla muestra por qué conviene combinar varios indicadores. El grupo con mayor total facturado no siempre coincide con el grupo de mayor promedio, y los grupos con pocos registros deben leerse con cuidado.

Ordenar una tabla agrupada

Una vez que tenemos una tabla de resumen, podemos ordenarla según el indicador que nos interese analizar. Por ejemplo, si queremos saber qué combinación de día y horario concentra mayor facturación total, podemos ordenar la tabla por `total_facturado`.

```
resumen_dia_horario.sort_values(  
    by="total_facturado",  
    ascending=False  
)
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
4	Sat	Dinner	87	1778.40	20.44	2.99
5	Sun	Dinner	76	1627.16	21.41	3.26
0	Thur	Lunch	61	1077.55	17.66	2.77
3	Fri	Dinner	12	235.96	19.66	2.94
2	Fri	Lunch	7	89.92	12.85	2.38
1	Thur	Dinner	1	18.78	18.78	3.00

La tabla quedó ordenada de mayor a menor según `total_facturado`.

Ahora podemos ver rápidamente qué combinaciones concentran más facturación total:

- **Sat - Dinner** aparece en primer lugar, con **1778.40**.

- **Sun - Dinner** aparece en segundo lugar, con **1627.16**.
- **Thur - Lunch** aparece en tercer lugar, con **1077.55**.

El ordenamiento ayuda a responder una pregunta concreta: qué combinación de día y horario aporta más facturación total.

Pero también confirma algo importante: los grupos con mayor facturación total son, en este caso, grupos con muchos registros. Por eso conviene leer `total_facturado` junto con `cantidad_cuentas` y `promedio_cuenta`.

Cambiar el criterio de ordenamiento

Si ordenamos por otro indicador, la interpretación puede cambiar. Ahora vamos a ordenar la misma tabla por `promedio_cuenta`, para ver qué combinación tiene el mayor valor promedio por cuenta.

```
resumen_dia_horario.sort_values(
    by="promedio_cuenta",
    ascending=False
)
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
5	Sun	Dinner	76	1627.16	21.41	3.26
4	Sat	Dinner	87	1778.40	20.44	2.99
3	Fri	Dinner	12	235.96	19.66	2.94
1	Thur	Dinner	1	18.78	18.78	3.00
0	Thur	Lunch	61	1077.55	17.66	2.77
2	Fri	Lunch	7	89.92	12.85	2.38

Ahora la tabla está ordenada por `promedio_cuenta`.

La lectura cambia respecto del ordenamiento anterior:

- **Sun - Dinner** aparece en primer lugar, con un promedio de cuenta de **21.41**.
- **Sat - Dinner** queda en segundo lugar, con **20.44**.
- **Fri - Dinner** aparece en tercer lugar, con **19.66**.

En cambio, cuando ordenamos por `total_facturado`, el primer lugar era **Sat - Dinner**.

Esto muestra una diferencia importante:

- `total_facturado` ayuda a identificar dónde se acumuló más dinero en total.
- `promedio_cuenta` ayuda a identificar dónde el valor promedio de cada cuenta fue mayor.

Son preguntas relacionadas, pero no equivalentes.

Filtrar grupos con pocos registros

Cuando analizamos promedios por grupo, conviene revisar cuántos registros tiene cada grupo. Un promedio calculado sobre un solo registro puede ser correcto, pero no representa una tendencia general. En nuestra tabla, por ejemplo, `Thur-Dinner` tiene solo **1 cuenta**.

Para evitar interpretaciones débiles, podemos filtrar los grupos que tengan una cantidad mínima de registros.

```
resumen_grupos_relevantes = resumen_dia_horario[
    resumen_dia_horario["cantidad_cuentas"] >= 10
]

resumen_grupos_relevantes
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Thur	Lunch	61	1077.55	17.66	2.77
3	Fri	Dinner	12	235.96	19.66	2.94
4	Sat	Dinner	87	1778.40	20.44	2.99
5	Sun	Dinner	76	1627.16	21.41	3.26

La tabla conserva solo los grupos con **10 o más cuentas**.

Al aplicar ese criterio, quedaron cuatro combinaciones:

- `Thur-Lunch`, con **61 cuentas**.
- `Fri-Dinner`, con **12 cuentas**.
- `Sat-Dinner`, con **87 cuentas**.
- `Sun-Dinner`, con **76 cuentas**.

Quedaron fuera `Thur-Dinner`, porque tenía solo **1 cuenta**, y `Fri-Lunch`, porque tenía **7 cuentas**.

Este filtro no significa que esos grupos sean incorrectos o que deban eliminarse del dataset. Simplemente evita dar demasiado peso analítico a promedios calculados sobre muy pocos registros.

Ordenar los grupos relevantes

Después de filtrar grupos con pocos registros, podemos ordenar nuevamente por `promedio_cuenta`. Así comparamos los grupos que tienen una cantidad mínima de casos.

```
resumen_grupos_relevantes.sort_values(  
    by="promedio_cuenta",  
    ascending=False  
)
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
5	Sun	Dinner	76	1627.16	21.41	3.26
4	Sat	Dinner	87	1778.40	20.44	2.99
3	Fri	Dinner	12	235.96	19.66	2.94
0	Thur	Lunch	61	1077.55	17.66	2.77

Ahora la tabla filtrada quedó ordenada por `promedio_cuenta`.

Entre los grupos con al menos **10 cuentas**, el mayor promedio de cuenta aparece en `Sun - Dinner`, con **21.41**. Luego aparecen `Sat - Dinner`, con **20.44**, y `Fri - Dinner`, con **19.66**.

El grupo `Thur - Lunch` tiene una cantidad importante de registros, **61 cuentas**, pero su promedio de cuenta es menor: **17.66**.

Esta lectura es más prudente que la anterior porque dejamos fuera combinaciones con muy pocos registros.

Cierre del capítulo

En este capítulo profundizamos el uso de `groupby()`.

Partimos de agrupaciones por más de una columna y vimos que esto permite responder preguntas más específicas. En lugar de analizar solo por día o solo por horario, pudimos estudiar combinaciones como `Sun - Dinner`, `Sat - Dinner` o `Thur - Lunch`.

También calculamos varios indicadores para cada grupo: cantidad de cuentas, total facturado, promedio de cuenta y promedio de propina. Esto nos permitió ver que no todos los indicadores cuentan la misma historia.

El total facturado ayuda a identificar dónde se acumuló más dinero. El promedio de cuenta permite comparar el valor medio de cada registro. La cantidad de cuentas ayuda a interpretar qué tan sólido puede ser cada promedio.

Además, ordenamos tablas agrupadas según distintos criterios y filtramos grupos con pocos registros para evitar interpretaciones demasiado débiles.

A partir de ahora, `groupby()` será una herramienta central para construir análisis más completos. En los próximos capítulos vamos a usar estas agrupaciones para comparar categorías, construir rankings y preparar resultados para visualizaciones.

Capítulo 30 — Comparar categorías

Comparar no es solo mirar qué valor es mayor

En los capítulos anteriores aprendimos a resumir datos por grupos usando `groupby()` y a construir tablas con varios indicadores.

Eso nos permite construir tablas con indicadores por categoría: cantidad de registros, totales, promedios y otros resúmenes.

Ahora vamos a concentrarnos en la comparación. Comparar categorías no significa solamente buscar el número más alto. Primero tenemos que saber qué estamos comparando. Por ejemplo, no es lo mismo preguntar:

- ¿qué día tiene más cuentas registradas?
- ¿qué día tiene mayor facturación total?
- ¿qué día tiene mayor promedio de cuenta?
- ¿qué día tiene mayor propina promedio?

Todas esas preguntas comparan días, pero no comparan lo mismo.

En este capítulo vamos a construir y leer comparaciones entre categorías, prestando atención a qué indicador usamos y qué conclusiones podemos sacar.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Como ya conocemos sus columnas principales, podemos concentrarnos en comparar categorías a partir de distintos indicadores.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3

índice	total_bill	tip	sex	smoker	day	time	size
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables de los capítulos anteriores. En este capítulo vamos a usar esas variables para comparar categorías. Nos interesará especialmente observar cómo cambian algunos indicadores según columnas como `day` y `time`.

La idea no será solamente calcular valores, sino aprender a leer qué significa cada comparación.

Construir una tabla para comparar días

Vamos a comenzar comparando los días registrados en el dataset. La pregunta general será:

¿Cómo se comportan las cuentas y las propinas según el día?

Para responderla vamos a construir una tabla con varios indicadores por día:

- cantidad de cuentas;
- total facturado;
- promedio de cuenta;
- promedio de propina.

```
comparacion_dias = df.groupby(
    "day",
    observed=True
).agg(
    cantidad_cuentas=("total_bill", "count"),
    total_facturado=("total_bill", "sum"),
    promedio_cuenta=("total_bill", "mean"),
    promedio_propina=("tip", "mean")
)

comparacion_dias = comparacion_dias.round(2).reset_index()

comparacion_dias
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Thur	62	1096.33	17.68	2.77

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
1	Fri	19	325.88	17.15	2.73
2	Sat	87	1778.40	20.44	2.99
3	Sun	76	1627.16	21.41	3.26

La tabla permite comparar los días usando varios indicadores.

Podemos leer algunos resultados:

- **Sat** tiene la mayor cantidad de cuentas: **87**.
- **Sat** también tiene el mayor total facturado: **1778.40**.
- **Sun** tiene el promedio de cuenta más alto: **21.41**.
- **Sun** también tiene la propina promedio más alta: **3.26**.
- **Fri** tiene la menor cantidad de cuentas y el menor total facturado.

Esta tabla muestra una idea importante: el resultado de una comparación depende del indicador que usemos.

Si comparamos por facturación total, destaca **Sat**. Si comparamos por promedio de cuenta o promedio de propina, destaca **Sun**.

Ordenar según distintos criterios

Para comparar categorías, muchas veces conviene ordenar la tabla según el indicador que queremos analizar.

Primero vamos a ordenar los días por **total_facturado**, de mayor a menor.

```
comparacion_dias.sort_values(
    by="total_facturado",
    ascending=False
)
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
2	Sat	87	1778.40	20.44	2.99
3	Sun	76	1627.16	21.41	3.26
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

Al ordenar por **total_facturado**, el primer lugar queda para **Sat**, con **1778.40**.

Luego aparecen:

- Sun , con **1627.16**;
- Thur , con **1096.33**;
- Fri , con **325.88**.

Este orden responde una pregunta concreta: **qué día acumuló más facturación total**.

Pero no necesariamente responde cuál fue el día con cuentas más altas en promedio. Para eso necesitamos ordenar por otro indicador.

Comparar usando otro indicador

Ahora vamos a ordenar la misma tabla por `promedio_cuenta`. Esto permite responder otra pregunta:

¿Qué día tuvo el mayor promedio de cuenta?

```
comparacion_dias.sort_values(  
    by="promedio_cuenta",  
    ascending=False  
)
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
3	Sun	76	1627.16	21.41	3.26
2	Sat	87	1778.40	20.44	2.99
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

Al ordenar por `promedio_cuenta`, el primer lugar cambia.

Ahora aparece Sun , con un promedio de cuenta de **21.41**. Luego aparecen:

- Sat , con **20.44**;
- Thur , con **17.68**;
- Fri , con **17.15**.

Esto muestra que Sat acumuló más facturación total, pero Sun tuvo cuentas más altas en promedio.

La diferencia es importante:

- `total_facturado` depende de cuánto se consume y de cuántas cuentas hay.
- `promedio_cuenta` resume el valor medio de cada cuenta.

Por eso, antes de decir que una categoría “rinde más” o “es mejor”, tenemos que aclarar qué indicador estamos usando.

Comparar propinas promedio

También podemos ordenar los días según la propina promedio. Esta comparación responde otra pregunta:

¿Qué día tuvo la mayor propina promedio?

```
comparacion_dias.sort_values(  
    by="promedio_propina",  
    ascending=False  
)
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
3	Sun	76	1627.16	21.41	3.26
2	Sat	87	1778.40	20.44	2.99
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

Al ordenar por `promedio_propina`, el orden queda igual que al ordenar por `promedio_cuenta`.

`Sun` aparece en primer lugar, con una propina promedio de **3.26**. Luego aparecen:

- `Sat`, con **2.99**;
- `Thur`, con **2.77**;
- `Fri`, con **2.73**.

En este caso, el día con mayor promedio de cuenta también tiene la mayor propina promedio.

Esto puede sugerir una relación entre cuentas más altas y propinas más altas, pero todavía no alcanza para afirmarlo con seguridad. Por ahora solo estamos comparando promedios por día.

Escribir una comparación con cuidado

Una tabla puede mostrar diferencias entre categorías, pero la interpretación escrita debe ser prudente. No es lo mismo decir:

“El domingo es el mejor día.”

que decir:

“En este dataset, el domingo tiene el mayor promedio de cuenta y la mayor propina promedio.”

La segunda frase es más precisa, porque indica:

- que estamos hablando de este dataset;
- qué categoría estamos observando;
- qué indicadores usamos para comparar;
- qué resultado concreto encontramos.

Cuando comunicamos un análisis, conviene evitar conclusiones demasiado generales si los datos solo permiten una comparación específica.

Comparar horarios

Ahora vamos a comparar otra categoría: el horario. La pregunta será:

¿Cómo cambian las cuentas y las propinas entre almuerzo y cena?

Vamos a construir una tabla similar a la anterior, pero agrupando por `time`.

```
comparacion_horarios = df.groupby(
    "time",
    observed=True
).agg(
    cantidad_cuentas=("total_bill", "count"),
    total_facturado=("total_bill", "sum"),
    promedio_cuenta=("total_bill", "mean"),
    promedio_propina=("tip", "mean")
)

comparacion_horarios = comparacion_horarios.round(2).reset_index()

comparacion_horarios
```

índice	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Lunch	68	1167.47	17.17	2.73

índice	time	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
1	Dinner	176	3660.30	20.80	3.10

La tabla compara los dos horarios registrados en el dataset.

Podemos observar que:

- **Dinner** tiene **176 cuentas**, mientras que **Lunch** tiene **68**.
- **Dinner** acumula una facturación total de **3660.30**, frente a **1167.47** en **Lunch**.
- El promedio de cuenta también es mayor en **Dinner**: **20.80** contra **17.17**.
- La propina promedio es mayor en **Dinner**: **3.10** contra **2.73**.

En este caso, **Dinner** aparece por encima de **Lunch** en todos los indicadores calculados.

De todos modos, la interpretación debe mencionar qué se está comparando. Podemos decir que, en este dataset, la cena concentra más registros, más facturación total y mayores promedios de cuenta y propina.

De la tabla a una interpretación

Una tabla de comparación no debe leerse solo buscando el número más grande. Conviene preguntarse:

- ¿qué indicador estoy comparando?
- ¿los grupos tienen cantidades de registros parecidas?
- ¿el resultado habla de totales o de promedios?
- ¿la conclusión está limitada al dataset analizado?

Por ejemplo, a partir de las tablas anteriores podemos escribir:

En este dataset, el sábado concentra la mayor facturación total por día, mientras que el domingo presenta el mayor promedio de cuenta y la mayor propina promedio. Al comparar horarios, la cena supera al almuerzo tanto en cantidad de registros como en facturación total, promedio de cuenta y promedio de propina.

Esta interpretación es más clara y más prudente que decir simplemente “sábado es mejor” o “la cena es mejor”, porque indica qué se comparó y con qué indicadores.

Cierre del capítulo

En este capítulo usamos tablas agrupadas para comparar categorías.

Vimos que comparar no significa solamente buscar el valor más alto. Primero hay que definir qué indicador estamos usando: cantidad de registros, total facturado, promedio de cuenta o promedio de propina.

También vimos que distintos indicadores pueden llevar a lecturas distintas. Un grupo puede tener mayor facturación total porque tiene más registros, mientras que otro puede tener un promedio más alto por cuenta.

Por eso, al comparar categorías, conviene leer varios indicadores juntos y escribir conclusiones prudentes, limitadas al dataset analizado.

En los próximos capítulos vamos a seguir trabajando con comparaciones, rankings y valores extremos, para detectar categorías destacadas y ordenar resultados de manera más precisa.

Capítulo 31 — Rankings, valores extremos y outliers

Ordenar para detectar valores destacados

En los capítulos anteriores usamos tablas agrupadas para comparar categorías. Vimos que una misma categoría puede destacarse según distintos indicadores: cantidad de registros, total facturado, promedio de cuenta o promedio de propina.

Ahora vamos a trabajar con una idea relacionada: construir rankings. Un ranking es una lista ordenada según un criterio. Ese criterio puede ser, por ejemplo:

- mayor facturación total;
- mayor promedio de cuenta;
- mayor propina;
- menor cantidad de registros;
- cuentas más altas o más bajas.

Ordenar los datos nos ayuda a detectar rápidamente valores destacados. Pero también exige interpretar con cuidado: un valor alto, bajo o inusual no siempre significa lo mismo.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Como ya conocemos sus variables principales, podemos concentrarnos en ordenar resultados, construir rankings y observar valores extremos.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2

índice	total_bill	tip	sex	smoker	day	time	size
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las mismas variables de los capítulos anteriores. En este capítulo vamos a ordenar datos para detectar valores destacados. Algunas veces vamos a ordenar tablas de resumen, donde cada fila representa una categoría o grupo. Otras veces vamos a ordenar registros individuales del dataset.

En ambos casos, lo importante será aclarar siempre cuál es el criterio de ordenamiento.

Construir una tabla base para rankings por día

Para empezar, vamos a construir una tabla de resumen por día. Esta tabla será parecida a las que usamos en capítulos anteriores, pero ahora la vamos a usar como base para construir rankings. Calcularemos:

- cantidad de cuentas;
- total facturado;
- promedio de cuenta;
- promedio de propina.

```
resumen_dias = df.groupby(
    "day",
    observed=True
).agg(
    cantidad_cuentas=("total_bill", "count"),
    total_facturado=("total_bill", "sum"),
    promedio_cuenta=("total_bill", "mean"),
    promedio_propina=("tip", "mean")
)

resumen_dias = resumen_dias.round(2).reset_index()

resumen_dias
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73
2	Sat	87	1778.40	20.44	2.99
3	Sun	76	1627.16	21.41	3.26

La tabla resume los principales indicadores por día. Cada fila representa un día y cada columna muestra un indicador distinto. Esta tabla será nuestra base para construir rankings. Según el criterio que elijamos, el orden de los días puede cambiar.

Por ejemplo, podemos ordenar por `total_facturado` si queremos saber qué día acumuló más dinero, o por `promedio_cuenta` si queremos saber qué día tuvo cuentas más altas en promedio.

Ranking por facturación total

Un primer ranking posible es ordenar los días según el total facturado. La pregunta que queremos responder es:

¿Qué día acumuló más facturación total?

Para eso vamos a ordenar la tabla `resumen_dias` por la columna `total_facturado`, de mayor a menor.

```
ranking_facturacion = resumen_dias.sort_values(
    by="total_facturado",
    ascending=False
)

ranking_facturacion
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
2	Sat	87	1778.40	20.44	2.99
3	Sun	76	1627.16	21.41	3.26
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

El ranking por `total_facturado` muestra que:

- `Sat` ocupa el primer lugar, con **1778.40**.
- `Sun` ocupa el segundo lugar, con **1627.16**.
- `Thur` ocupa el tercer lugar, con **1096.33**.
- `Fri` queda en último lugar, con **325.88**.

Este ranking responde una pregunta específica: qué día acumuló más facturación total.

También conviene mirar la columna `cantidad_cuentas`. `Sat` no solo tiene la mayor facturación total, sino también la mayor cantidad de cuentas registradas. Esto ayuda a interpretar por qué aparece en primer lugar.

Ranking por promedio de cuenta

Ahora vamos a ordenar los días según otro criterio: el promedio de cuenta. La pregunta cambia:

¿Qué día tuvo el mayor valor promedio por cuenta?

El ranking puede cambiar porque el promedio no mide lo mismo que el total facturado.

```
ranking_promedio_cuenta = resumen_dias.sort_values(
    by="promedio_cuenta",
    ascending=False
)

ranking_promedio_cuenta
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
3	Sun	76	1627.16	21.41	3.26
2	Sat	87	1778.40	20.44	2.99
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

El ranking por `promedio_cuenta` cambia el orden. Ahora el primer lugar corresponde a `Sun`, con un promedio de cuenta de **21.41**. Luego aparecen:

- `Sat`, con **20.44**;
- `Thur`, con **17.68**;
- `Fri`, con **17.15**.

Este ranking no responde qué día facturó más en total, sino qué día tuvo cuentas más altas en promedio. La diferencia es importante: `Sat` acumula más facturación total, pero `Sun` tiene el promedio de cuenta más alto.

Seleccionar los primeros lugares de un ranking

A veces no necesitamos ver el ranking completo. Si solo queremos observar los primeros lugares, podemos ordenar la tabla y luego usar `head()`.

Por ejemplo, vamos a quedarnos con los **2 días con mayor facturación total**.

```
ranking_facturacion.head(2)
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
2	Sat	87	1778.40	20.44	2.99
3	Sun	76	1627.16	21.41	3.26

La salida muestra los **2 primeros lugares** del ranking por facturación total. Los días con mayor facturación total son:

- **Sat**, con **1778.40**;
- **Sun**, con **1627.16**.

Esta selección es útil cuando queremos concentrarnos solo en las categorías más destacadas según un criterio. En este caso, el criterio es `total_facturado`. Si cambiamos el criterio de ordenamiento, también puede cambiar el ranking.

Seleccionar los últimos lugares de un ranking

También podemos mirar el otro extremo del ranking. Si una tabla ya está ordenada de mayor a menor, `tail()` permite ver los últimos registros.

Ahora vamos a observar los **2 días con menor facturación total**.

```
ranking_facturacion.tail(2)
```

índice	day	cantidad_cuentas	total_facturado	promedio_cuenta	promedio_propina
0	Thur	62	1096.33	17.68	2.77
1	Fri	19	325.88	17.15	2.73

La salida muestra los **2 últimos lugares** del ranking por facturación total. En este caso aparecen:

- **Thur**, con **1096.33**;
- **Fri**, con **325.88**.

Esto permite observar el extremo inferior del ranking. Sin embargo, también conviene mirar la cantidad de cuentas. **Fri** tiene mucha menos facturación total, pero también tiene solo **19 cuentas** registradas. Por eso, el total facturado debe interpretarse junto con la cantidad de registros.

Rankings de registros individuales

Hasta ahora ordenamos una tabla de resumen por día. También podemos ordenar el dataset original para observar registros individuales destacados. Por ejemplo, podemos preguntarnos:

¿Cuáles fueron las cuentas más altas del dataset?

Para responderlo vamos a ordenar el `DataFrame` original por `total_bill`, de mayor a menor.

```
cuentas_mas_altas = df.sort_values(
    by="total_bill",
    ascending=False
)

cuentas_mas_altas.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
170	50.81	10.00	Male	Yes	Sat	Dinner	3
212	48.33	9.00	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
156	48.17	5.00	Male	No	Sun	Dinner	6
182	45.35	3.50	Male	Yes	Sun	Dinner	3

La tabla muestra las **5 cuentas más altas** del dataset. La cuenta más alta tiene un `total_bill` de **50.81** y una propina de **10.00**. Luego aparecen otras cuentas cercanas a **48** y **45**.

También podemos observar que estas cuentas altas aparecen en `Dinner`, principalmente durante `Sat` y `Sun`. Estos registros son valores destacados dentro del dataset. No sabemos todavía si son valores extremos en sentido estricto, pero sí sabemos que ocupan los primeros lugares cuando ordenamos por `total_bill`.

Este tipo de ranking puede servir para detectar casos que conviene mirar con más atención.

Observar las cuentas más bajas

También podemos ordenar en el sentido contrario para observar las cuentas más bajas del dataset. Esto permite revisar el otro extremo de la distribución.

```
cuentas_mas_bajas = df.sort_values(
    by="total_bill",
    ascending=True
)

cuentas_mas_bajas.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
67	3.07	1.00	Female	Yes	Sat	Dinner	1
92	5.75	1.00	Female	Yes	Fri	Dinner	2
111	7.25	1.00	Female	No	Sat	Dinner	1
172	7.25	5.15	Male	Yes	Sun	Dinner	2
149	7.51	2.00	Male	No	Thur	Lunch	2

La tabla muestra las **5 cuentas más bajas** del dataset. La cuenta más baja tiene un `total_bill` de **3.07**. Luego aparecen cuentas de **5.75**, **7.25**, **7.25** y **7.51**.

Estos registros se encuentran en distintos días y horarios, aunque todos corresponden a mesas pequeñas, de **1** o **2** personas. Como ocurría con las cuentas más altas, estos valores no deben interpretarse automáticamente como errores. Por ahora solo sabemos que son los valores más bajos cuando ordenamos la columna `total_bill`.

El ordenamiento nos ayuda a detectar casos destacados, pero todavía necesitamos contexto para decidir si son valores esperables, raros o problemáticos.

Valores extremos y outliers

Cuando ordenamos los datos, los valores extremos aparecen con facilidad. Un valor extremo es un dato que está muy por encima o muy por debajo de la mayoría de los valores observados.

En análisis de datos también suele usarse el término **outlier** para referirse a valores que se apartan de manera marcada del comportamiento general del dataset. Pero detectar un valor extremo no significa que debamos eliminarlo.

Un outlier puede ser:

- un error de carga o medición;
- un caso real poco frecuente;
- una situación especial que merece análisis;
- un dato válido, pero muy diferente del resto.

Por eso, antes de modificar o eliminar un valor extremo, conviene observarlo con más atención.

Observar un valor extremo con más contexto

Vamos a revisar el registro con la cuenta más alta del dataset. La idea no es modificarlo ni eliminarlo, sino observar qué información lo acompaña.

```
df.loc[df["total_bill"].idxmax()]
```

índice	170
total_bill	50.81
tip	10.0
sex	Male
smoker	Yes
day	Sat
time	Dinner
size	3

dtype: object

El registro con la cuenta más alta tiene un `total_bill` de **50.81**. Al observar el resto de las columnas, vemos que corresponde a una mesa de **3 personas**, durante `Dinner`, un sábado (`Sat`). También registra una propina de **10.00**.

Esta información ayuda a contextualizar el valor. No estamos mirando solo un número aislado, sino el registro completo. Por ahora no observamos ningún elemento que indique automáticamente que este dato sea un error.

Observar el valor mínimo

También podemos revisar el registro con la cuenta más baja. Esto nos permite mirar el otro extremo de la columna `total_bill` y analizarlo con el mismo criterio: observar antes de decidir.

```
df.loc[df["total_bill"].idxmin()]
```

índice	67
total_bill	3.07

índice	67
tip	1.0
sex	Female
smoker	Yes
day	Sat
time	Dinner
size	1

`dtype: object`

El registro con la cuenta más baja tiene un `total_bill` de **3.07**. Al observar el resto de las columnas, vemos que corresponde a una mesa de **1 persona**, durante `Dinner`, un sábado (`Sat`). La propina registrada es de **1.00**.

Este valor es bajo en comparación con el resto del dataset, pero el contexto ayuda a interpretarlo: una cuenta pequeña puede ser razonable si corresponde a una sola persona.

Nuevamente, no alcanza con ver que un valor está en un extremo para decidir que es un error. Primero debemos observarlo dentro del contexto de las demás variables.

Qué hacer con un outlier

Detectar un valor extremo no significa modificarlo automáticamente. Antes de decidir qué hacer con un outlier, conviene hacerse algunas preguntas:

- ¿el valor es posible dentro del contexto?
- ¿hay otras columnas que ayuden a explicarlo?
- ¿podría ser un error de carga o medición?
- ¿afecta mucho los promedios u otros indicadores?
- ¿el objetivo del análisis requiere conservarlo, marcarlo o tratarlo de alguna manera?

En algunos casos, un outlier puede ser un error y habrá que corregirlo o excluirlo. En otros casos, puede ser un dato real y valioso para el análisis. Por eso, en esta etapa no vamos a eliminar estos valores. Los vamos a interpretar como señales para observar con más atención.

Cierre del capítulo

En este capítulo trabajamos con rankings, valores extremos y outliers. Vimos que un ranking es una lista ordenada según un criterio. Ese criterio debe estar claro: facturación total, promedio de cuenta, propina, cantidad de registros u otra variable.

También vimos que podemos construir rankings de categorías, a partir de tablas agrupadas, o rankings de registros individuales, ordenando directamente el dataset original. Ordenar los datos nos ayuda a detectar valores destacados: los más altos, los más bajos o los que se apartan del comportamiento general.

A esos valores que se alejan mucho del resto solemos llamarlos **valores extremos** u **outliers**. Pero un outlier no es automáticamente un error. Puede ser un dato incorrecto, un caso raro, una situación especial o simplemente un valor válido pero poco frecuente. Por eso, antes de eliminar o modificar un valor extremo, conviene observarlo, contextualizarlo y decidir según el objetivo del análisis.

En los próximos capítulos vamos a seguir construyendo herramientas para resumir, comparar y comunicar resultados de manera más clara.

Capítulo 32 — Organizar resúmenes con tablas dinámicas

De agrupar datos a cruzar variables

En los capítulos anteriores usamos `groupby()` para resumir datos por grupos.

Agrupamos por día, por horario y también por más de una variable a la vez. Eso nos permitió responder preguntas como:

- ¿qué día tuvo mayor facturación total?
- ¿qué horario tuvo mayor promedio de cuenta?
- ¿qué combinaciones de día y horario aparecen en el dataset?

Ahora vamos a trabajar con otra forma de organizar resúmenes: las **tablas dinámicas**.

Una tabla dinámica permite cruzar variables. Por ejemplo, podemos colocar los días en las filas, los horarios en las columnas y dentro de la tabla mostrar un indicador, como el promedio de cuenta o la cantidad de registros.

Esto no reemplaza a `groupby()`. Es otra forma de resumir datos, especialmente útil cuando queremos comparar categorías en forma de tabla.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Como ya conocemos sus columnas principales, podemos concentrarnos en la forma de organizar los resúmenes.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3

índice	total_bill	tip	sex	smoker	day	time	size
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La tabla muestra las primeras filas del dataset.

Seguimos trabajando con las columnas que ya conocemos: `total_bill`, `tip`, `day`, `time`, `size` y otras variables categóricas.

En este capítulo nos van a interesar especialmente las columnas `day` y `time`, porque nos permiten cruzar dos categorías: el día y el horario. También vamos a usar columnas numéricas como `total_bill` y `tip`, que serán los valores que vamos a resumir dentro de las tablas dinámicas.

Recordar una agrupación doble con `groupby()`

Antes de crear una tabla dinámica, vamos a recordar cómo se veía una agrupación doble con `groupby()`. La pregunta será:

¿Cuál es el promedio de cuenta según el día y el horario?

Primero responderemos esta pregunta con `groupby()`.

```
promedio_groupby = df.groupby(
    ["day", "time"],
    observed=True
)["total_bill"].mean().round(2)

promedio_groupby
```

day		total_bill
Thur	Lunch	17.66
Dinner	18.78	
Fri	Lunch	12.85
Dinner	19.66	
Sat	Dinner	20.44
Sun	Dinner	21.41

`dtype: float64`

La salida muestra el promedio de cuenta para cada combinación de `day` y `time`.

Por ejemplo:

- **Thur** en **Lunch** tiene un promedio de cuenta de **17.66**.
- **Thur** en **Dinner** tiene un promedio de cuenta de **18.78**.
- **Fri** en **Lunch** tiene un promedio de cuenta de **12.85**.
- **Fri** en **Dinner** tiene un promedio de cuenta de **19.66**.

Esta salida es correcta y útil, pero tiene forma de lista agrupada.

Para comparar visualmente días y horarios, puede ser más cómodo reorganizar el resultado en forma de tabla, colocando los días como filas y los horarios como columnas.

Crear una tabla dinámica con `pivot_table()`

Vamos a responder la misma pregunta, pero usando una tabla dinámica. La pregunta sigue siendo:

¿Cuál es el promedio de cuenta según el día y el horario?

En esta tabla vamos a colocar:

- los días en las filas;
- los horarios en las columnas;
- el promedio de cuenta dentro de la tabla.

```
tabla_promedio_cuenta = df.pivot_table(  
    values="total_bill",  
    index="day",  
    columns="time",  
    aggfunc="mean",  
    observed=True  
).round(2)
```

tabla_promedio_cuenta

time	Lunch	Dinner
day		
Thur	17.66	18.78
Fri	12.85	19.66
Sat	NaN	20.44
Sun	NaN	21.41

La tabla dinámica muestra la misma información que obtuvimos con `groupby()`, pero organizada de otra manera. Ahora cada fila representa un día y cada columna representa un horario.

Esto facilita algunas comparaciones. Por ejemplo, podemos ver rápidamente que:

- `Thur` tiene datos tanto para `Lunch` como para `Dinner`;
- `Fri` también tiene datos para ambos horarios;
- `Sat` y `Sun` solo muestran datos para `Dinner`.

También aparecen algunos valores `NaN`. Por ahora podemos interpretarlos como combinaciones sin datos, y más adelante los vamos a revisar con más detalle.

Entender los parámetros principales

En la tabla dinámica anterior usamos cuatro parámetros importantes:

- `values`: indica qué columna numérica queremos resumir.
- `index`: indica qué variable se ubicará en las filas.
- `columns`: indica qué variable se ubicará en las columnas.
- `aggfunc`: indica qué cálculo se aplicará sobre los datos.

En nuestro ejemplo:

- `values="total_bill"` significa que resumimos la columna de cuentas;
- `index="day"` coloca los días en las filas;
- `columns="time"` coloca los horarios en las columnas;
- `aggfunc="mean"` calcula el promedio.

También usamos `observed=True` porque algunas columnas categóricas, como `day` y `time`, están guardadas como categorías.

En este caso, este parámetro ayuda a trabajar con las combinaciones observadas en los datos y evita avisos de Pandas relacionados con cambios futuros.

Tabla dinámica de cantidad de cuentas

Una tabla dinámica no sirve solamente para calcular promedios. También podemos contar cuántos registros hay en cada combinación de categorías. Ahora vamos a responder esta pregunta:

¿Cuántas cuentas hay para cada combinación de día y horario?

```

tabla_cantidad = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",
    aggfunc="count",
    observed=True
)

tabla_cantidad

```

time	Lunch	Dinner
day		
Thur	61.0	1.0
Fri	7.0	12.0
Sat	NaN	87.0
Sun	NaN	76.0

La tabla muestra cuántas cuentas hay para cada combinación de día y horario.

Por ejemplo:

- **Thur** tiene **61** cuentas en **Lunch** y **1** cuenta en **Dinner**.
- **Fri** tiene **7** cuentas en **Lunch** y **12** en **Dinner**.
- **Sat** tiene **87** cuentas en **Dinner**.
- **Sun** tiene **76** cuentas en **Dinner**.

También aparecen valores **NaN** en algunas combinaciones. En este caso, esos valores indican que no hay cuentas registradas para esa combinación de día y horario.

Como estamos contando registros, esos **NaN** pueden interpretarse como ausencia de datos para esa combinación.

Además, al aparecer valores **NaN**, Pandas muestra los conteos con decimales, como **61.0** o **1.0**. Eso no cambia el significado: siguen siendo cantidades de cuentas.

Reemplazar valores vacíos en una tabla de conteos

En una tabla de conteos, puede ser útil reemplazar los valores vacíos por **0**. Si no hay registros para una combinación, podemos mostrarlo como cero cuentas.

Para eso, `pivot_table()` permite usar el parámetro `fill_value`.

```

tabla_cantidad = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",
    aggfunc="count",
    observed=True,
    fill_value=0
)

tabla_cantidad

```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

Ahora la tabla muestra **0** en las combinaciones donde no había registros.

Por ejemplo:

- **Sat** en **Lunch** aparece como **0**;
- **Sun** en **Lunch** aparece como **0**.

En una tabla de conteos, esto resulta razonable: si no hay cuentas para una combinación, podemos mostrar cero cuentas.

Pero este criterio no debe aplicarse automáticamente a todos los casos. En una tabla de promedios, reemplazar un valor vacío por **0** podría ser confuso, porque podría parecer que el promedio real fue cero, cuando en realidad no había datos para calcularlo.

Tabla dinámica de total facturado

Además de contar registros, también podemos sumar valores. Ahora vamos a calcular el total facturado para cada combinación de día y horario. La pregunta será:

¿Cuánto se facturó en total según el día y el horario?

```

tabla_total_facturado = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",

```

```
aggfunc="sum",
observed=True
).round(2)

tabla_total_facturado
```

time	Lunch	Dinner
day		
Thur	1077.55	18.78
Fri	89.92	235.96
Sat	NaN	1778.40
Sun	NaN	1627.16

La tabla muestra el total facturado para cada combinación de día y horario.

Podemos observar, por ejemplo, que:

- **Thur** en **Lunch** acumuló **1077.55**;
- **Fri** en **Dinner** acumuló **235.96**;
- **Sat** en **Dinner** acumuló **1778.40**;
- **Sun** en **Dinner** acumuló **1627.16**.

En este caso, el total facturado depende mucho de la cantidad de cuentas registradas en cada combinación.

Por eso, al interpretar esta tabla conviene recordar la tabla anterior de conteos. Una combinación puede facturar más no necesariamente porque sus cuentas sean más altas, sino porque tiene más registros.

Tabla dinámica de promedio de propina

También podemos usar otra columna numérica como valor de análisis. Ahora vamos a calcular el promedio de propina para cada combinación de día y horario. La pregunta será:

¿Cuál fue la propina promedio según el día y el horario?

```
tabla_promedio_propina = df.pivot_table(
    values="tip",
    index="day",
    columns="time",
    aggfunc="mean",
    observed=True
).round(2)
```

tabla_promedio_propina

time	Lunch	Dinner
day		
Thur	2.77	3.00
Fri	2.38	2.94
Sat	NaN	2.99
Sun	NaN	3.26

La tabla muestra el promedio de propina para cada combinación de día y horario.

Podemos observar, por ejemplo, que:

- **Thur** en **Lunch** tiene una propina promedio de **2.77**;
- **Fri** en **Dinner** tiene una propina promedio de **2.94**;
- **Sat** en **Dinner** tiene una propina promedio de **2.99**;
- **Sun** en **Dinner** tiene una propina promedio de **3.26**.

En esta tabla estamos usando la columna **tip** como valor de análisis, pero mantenemos la misma organización: días en las filas y horarios en las columnas.

Esto muestra una ventaja de las tablas dinámicas: podemos conservar la misma estructura y cambiar el indicador que queremos analizar.

Combinaciones sin datos

En varias tablas dinámicas aparecieron valores **NaN**. Ahora vamos a comprobar de dónde salen esos valores.

En este dataset no aparecen cuentas de **Sat** en **Lunch** ni de **Sun** en **Lunch**. Para verificarlo, vamos a filtrar el dataset original.

```
df[
    (df["day"].isin(["Sat", "Sun"])) &
    (df["time"] == "Lunch")
]
```

índice	total_bill	tip	sex	smoker	day	time	size
--------	------------	-----	-----	--------	-----	------	------

La salida muestra solamente los nombres de las columnas, pero no contiene filas.

Esto significa que el filtro no encontró registros que cumplan ambas condiciones:

- que el día sea `Sat` o `Sun`;
- que el horario sea `Lunch`.

Por eso, en las tablas dinámicas aparecían valores `NaN` para esas combinaciones.

Este ejemplo muestra algo importante: una celda vacía en una tabla dinámica puede representar una combinación que no existe en los datos, no necesariamente un problema del dataset.

Agregar totales a una tabla dinámica

Las tablas dinámicas también pueden incluir totales. Con el parámetro `margins=True`, Pandas agrega una fila y una columna adicional con resúmenes generales.

Vamos a calcular nuevamente el total facturado por día y horario, pero agregando totales.

```
tabla_total_con_totales = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",
    aggfunc="sum",
    observed=True,
    margins=True
).round(2)
```

tabla_total_con_totales

time	Lunch	Dinner	All
day			
Thur	1077.55	18.78	1096.33
Fri	89.92	235.96	325.88
Sat	NaN	1778.40	1778.40
Sun	NaN	1627.16	1627.16
All	1167.47	3660.30	4827.77

La tabla ahora incluye una fila y una columna llamadas `All`.

La columna `All` muestra el total facturado por cada día, sumando `Lunch` y `Dinner`.

Por ejemplo:

- **Thur** acumuló un total de **1096.33**;
- **Fri** acumuló **325.88**;
- **Sat** acumuló **1778.40**;
- **Sun** acumuló **1627.16**.

La fila **All** muestra el total facturado por cada horario:

- **Lunch** acumuló **1167.47**;
- **Dinner** acumuló **3660.30**.

La celda final, donde se cruza **All** con **All**, muestra el total general del dataset: **4827.77**.

Es importante recordar que **All** no es una categoría original del dataset. Es una fila y una columna agregadas por Pandas para mostrar totales.

Tablas dinámicas con más de un valor

Hasta ahora usamos una sola columna como valor de análisis. Pero `pivot_table()` también permite resumir más de una columna numérica al mismo tiempo.

Por ejemplo, podemos calcular el promedio de `total_bill` y el promedio de `tip` en una misma tabla dinámica.

```
tabla_dos_valores = df.pivot_table(
    values=["total_bill", "tip"],
    index="day",
    columns="time",
    aggfunc="mean",
    observed=True
).round(2)
```

tabla_dos_valores

índice	tip	total_bill		
time	Lunch	Dinner	Lunch	Dinner
day				
Thur	2.77	3.00	17.66	18.78
Fri	2.38	2.94	12.85	19.66
Sat	NaN	2.99	NaN	20.44
Sun	NaN	3.26	NaN	21.41

La tabla dinámica ahora resume dos columnas numéricas: `tip` y `total_bill`.

Para cada una de esas columnas, Pandas muestra los valores separados por horario. Por eso la tabla tiene dos niveles de encabezado:

- primero aparece la variable resumida: `tip` o `total_bill`;
- debajo aparece el horario: `Lunch` o `Dinner`.

Esta forma de tabla permite reunir más información en una sola salida, pero también puede volverse más difícil de leer.

Por eso, cuando una tabla dinámica incluye varios valores, conviene preguntarse si realmente mejora la comparación o si sería más claro mostrar tablas separadas.

Tablas dinámicas con más de una función

También podemos aplicar más de una función sobre una misma columna. Por ejemplo, podemos resumir `total_bill` calculando al mismo tiempo:

- cantidad de cuentas;
- promedio de cuenta;
- total facturado.

Para eso, en `aggfunc` podemos pasar una lista de funciones.

```
tabla_varias_funciones = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",
    aggfunc=["count", "mean", "sum"],
    observed=True
).round(2)
```

tabla_varias_funciones

índice	count	mean	sum			
time	Lunch	Dinner	Lunch	Dinner	Lunch	Dinner
day						
Thur	61.0	1.0	17.66	18.78	1077.55	18.78
Fri	7.0	12.0	12.85	19.66	89.92	235.96
Sat	NaN	87.0	NaN	20.44	NaN	1778.40
Sun	NaN	76.0	NaN	21.41	NaN	1627.16

La tabla dinámica ahora aplica tres funciones sobre la columna `total_bill`:

- `count`: cuenta cuántos registros hay;
- `mean`: calcula el promedio;
- `sum`: calcula el total.

Para cada función, Pandas vuelve a separar los resultados por horario: `Lunch` y `Dinner`.

Esta tabla concentra mucha información en una sola salida. Por ejemplo, para `Thur` en `Lunch` podemos leer:

- cantidad de cuentas: **61**;
- promedio de cuenta: **17.66**;
- total facturado: **1077.55**.

Este tipo de tabla puede ser muy útil, pero también puede volverse más difícil de leer si agregamos demasiadas variables o demasiadas funciones.

Por eso, al construir tablas dinámicas conviene equilibrar dos necesidades: resumir bastante información, pero mantener una tabla que pueda interpretarse con claridad.

`groupby()` y `pivot_table()`

Tanto `groupby()` como `pivot_table()` permiten resumir datos por grupos. La diferencia principal está en la forma de organizar el resultado.

Con `groupby()`, solemos obtener una tabla larga o una serie agrupada. Esa forma es muy útil para calcular, ordenar, filtrar y seguir trabajando con los datos.

Con `pivot_table()`, el resultado se organiza como una tabla cruzada: una variable queda en las filas, otra en las columnas y los valores resumidos quedan dentro de la tabla.

No son herramientas rivales. Son dos formas distintas de resumir información. En general:

- `groupby()` resulta cómodo para calcular resúmenes por grupos;
- `pivot_table()` resulta cómodo para comparar categorías cruzadas en forma de tabla.

Una tabla dinámica para comunicar

Para terminar, vamos a construir una tabla dinámica simple y fácil de leer. Mostraremos la cantidad de cuentas por día y horario, reemplazando las combinaciones sin datos por 0.

Esta tabla resume de forma clara cómo se distribuyen los registros del dataset.

```
tabla_final = df.pivot_table(  
    values="total_bill",  
    index="day",  
    columns="time",  
    aggfunc="count",  
    observed=True,  
    fill_value=0  
)  
  
tabla_final
```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

La tabla final resume la cantidad de cuentas por día y horario.

Es una tabla simple y clara para comunicar la distribución de los registros:

- **Thur** concentra la mayoría de sus cuentas en **Lunch**;
- **Fri** tiene registros tanto en **Lunch** como en **Dinner**;
- **Sat** y **Sun** solo tienen registros en **Dinner**;
- las combinaciones sin registros aparecen como 0.

En este caso, usar `fill_value=0` ayuda a leer mejor la tabla, porque estamos trabajando con cantidades de registros.

Cierre del capítulo

En este capítulo trabajamos con tablas dinámicas usando `pivot_table()`.

Vimos que una tabla dinámica permite cruzar variables categóricas y resumir una columna numérica dentro de esa combinación.

Usamos los parámetros principales:

- `values`, para indicar qué columna queremos resumir;
- `index`, para definir las filas;
- `columns`, para definir las columnas;
- `aggfunc`, para elegir el cálculo que queremos aplicar.

También vimos que una tabla dinámica puede calcular distintos tipos de resumen: promedios, conteos, sumas y otros indicadores.

Aparecieron valores `NaN` en combinaciones sin datos. Aprendimos que esos valores no siempre indican un error: muchas veces muestran que no hay registros para una combinación de categorías.

También usamos `fill_value=0`, pero con cuidado. En tablas de conteo puede ser útil reemplazar valores vacíos por cero. En cambio, en tablas de promedios podría ser engañoso.

Finalmente, vimos que `pivot_table()` puede incluir totales con `margins=True`, trabajar con más de una columna de valores y aplicar más de una función de agregación.

`groupby()` y `pivot_table()` no son herramientas rivales. Ambas sirven para resumir datos, pero organizan el resultado de manera diferente.

En el próximo capítulo vamos a empezar a trabajar con gráficos desde cero. Para eso usaremos Matplotlib, una de las bibliotecas más importantes de Python para visualización de datos.

Capítulo 33 — Análisis temporal básico

Mirar los datos según momentos

En los capítulos anteriores trabajamos con distintas formas de resumir datos.

Usamos frecuencias, indicadores numéricos, agrupaciones con `groupby()`, rankings, valores extremos y tablas dinámicas con `pivot_table()`.

Ahora vamos a dar un paso más: empezar a analizar los datos según una dimensión temporal simple.

En este capítulo vamos a observar cómo cambian algunos indicadores según el día y el horario. Para eso seguiremos usando el dataset `tips`, que ya conocemos.

La idea no será todavía trabajar con fechas completas ni con series temporales avanzadas. En cambio, vamos a usar columnas como `day` y `time` para responder preguntas como:

- ¿en qué días se registraron más cuentas?
- ¿qué días acumularon mayor facturación?
- ¿cómo se distribuyen las cuentas entre `Lunch` y `Dinner`?
- ¿qué diferencias aparecen cuando miramos cantidades, totales y promedios?

También vamos a comenzar a usar gráficos simples para acompañar el análisis. Un gráfico no reemplaza a una tabla, pero puede ayudarnos a ver comparaciones de manera más clara.

Cargar el dataset de trabajo

Vamos a seguir usando el dataset `tips` de Seaborn. Como ya conocemos sus columnas principales, podemos concentrarnos en el nuevo objetivo del capítulo: analizar la información según días y horarios.

También vamos a importar `matplotlib.pyplot`, porque más adelante usaremos algunos gráficos simples para visualizar los resultados.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")
```

```
df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset.

Cada fila representa una cuenta registrada en un restaurante. Entre las columnas disponibles aparecen:

- `total_bill`: importe total de la cuenta.
- `tip`: propina.
- `sex`: sexo de la persona registrada en la cuenta.
- `smoker`: indica si la mesa corresponde a fumadores o no fumadores.
- `day`: día en que se registró la cuenta.
- `time`: momento del día, por ejemplo `Lunch` o `Dinner`.
- `size`: cantidad de personas en la mesa.

Para este capítulo nos van a interesar especialmente las columnas `day` y `time`.

Aunque no son fechas completas, nos permiten hacer un primer análisis temporal básico: observar cómo cambian las cuentas según el día y el horario.

Variables temporales simples

En análisis de datos, no siempre trabajamos desde el comienzo con fechas completas como día, mes y año. A veces, una primera aproximación temporal puede hacerse con variables más simples.

En este dataset vamos a usar:

- `day`, que indica el día de la semana.
- `time`, que indica si la cuenta corresponde a `Lunch` o `Dinner`.

Estas columnas nos permiten empezar a comparar momentos.

```
df[["day", "time", "total_bill", "tip", "size"]].head()
```

índice	day	time	total_bill	tip	size
0	Sun	Dinner	16.99	1.01	2
1	Sun	Dinner	10.34	1.66	3
2	Sun	Dinner	21.01	3.50	3
3	Sun	Dinner	23.68	3.31	2
4	Sun	Dinner	24.59	3.61	4

En esta vista reducida aparecen solo las columnas que vamos a usar con más frecuencia en este capítulo.

Las primeras filas corresponden al día `Sun` y al horario `Dinner`. Cada registro incluye el importe total de la cuenta, la propina y el tamaño de la mesa.

A partir de estas columnas podemos empezar a formular preguntas temporales simples.

Por ejemplo, podemos preguntarnos si todas las combinaciones de día y horario tienen la misma cantidad de cuentas, o si algunos momentos aparecen mucho más representados que otros.

Primera pregunta: ¿cómo se distribuyen las cuentas según día y horario?

Antes de calcular importes totales o promedios, conviene mirar cuántos registros tenemos para cada momento. Esto es importante porque no todos los días y horarios tienen necesariamente la misma cantidad de datos.

Vamos a construir una tabla que muestre la cantidad de cuentas registradas para cada combinación de `day` y `time`.

```
tabla_cuentas_dia_horario = df.pivot_table(
    values="total_bill",
    index="day",
    columns="time",
    aggfunc="count",
    observed=True,
    fill_value=0
)

tabla_cuentas_dia_horario
```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

La tabla muestra cuántas cuentas se registraron para cada combinación de día y horario.

Podemos observar que los registros no están distribuidos de manera pareja:

- **Thur** concentra la mayoría de sus cuentas en **Lunch**: 61 cuentas.
- **Fri** tiene registros en ambos horarios: 7 en **Lunch** y 12 en **Dinner**.
- **Sat** aparece solo en **Dinner**, con 87 cuentas.
- **Sun** también aparece solo en **Dinner**, con 76 cuentas.

Esto nos muestra una primera característica temporal del dataset: algunos días están asociados principalmente a un horario.

Antes de comparar importes o promedios, es importante conocer esta distribución de registros. Si un día tiene muchas más cuentas que otro, eso puede influir en los totales que calculemos después.

El orden temporal importa

En capítulos anteriores usamos ordenamientos para construir rankings. Por ejemplo, podíamos ordenar de mayor a menor para encontrar los días con mayor facturación o las cuentas más altas.

Pero cuando analizamos datos según una dimensión temporal, no siempre conviene ordenar por el valor más grande. Muchas veces necesitamos respetar el orden lógico de los momentos. En este dataset, los días aparecen como categorías. Para analizarlos temporalmente, vamos a definir un orden:

Thur, **Fri**, **Sat**, **Sun**

Ese orden nos va a ayudar a leer los resultados como una secuencia de días y no solo como una lista de categorías.

```
orden_dias = ["Thur", "Fri", "Sat", "Sun"]
```

```
tabla_cuentas_dia_horario = tabla_cuentas_dia_horario.reindex(orden_dias)

tabla_cuentas_dia_horario
```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

La tabla mantiene el mismo aspecto que antes, pero ahora hicimos algo importante: definimos de manera explícita el orden de los días. Esto puede parecer un detalle menor, pero es importante en análisis temporal.

Si estuviéramos construyendo un ranking, podríamos ordenar de mayor a menor según la cantidad de cuentas. Pero en este caso queremos leer los días como una secuencia:

Thur → Fri → Sat → Sun

De esta manera, las comparaciones respetan el orden lógico de los momentos que estamos analizando.

Total facturado por día

Ya vimos cuántas cuentas se registraron en cada combinación de día y horario. Ahora podemos avanzar hacia otra pregunta:

¿cuánto se facturó en total cada día?

Para responderla, vamos a agrupar los datos por `day` y calcular tres indicadores:

- cantidad de cuentas;
- total facturado;
- promedio de cuenta.

Estos tres valores nos permiten comparar los días desde perspectivas distintas.

```
resumen_dia = df.groupby("day", observed=True).agg(
    cantidad_cuentas=("total_bill", "count"),
    total_facturado=("total_bill", "sum"),
    promedio_cuenta=("total_bill", "mean")
).round(2)
```

```
resumen_dia = resumen_dia.reindex(orden_dias)
```

```
resumen_dia
```

day	cantidad_cuentas	total_facturado	promedio_cuenta
Thur	62	1096.33	17.68
Fri	19	325.88	17.15
Sat	87	1778.40	20.44
Sun	76	1627.16	21.41

La tabla resume tres indicadores por día.

Podemos observar que:

- **Sat** tiene la mayor cantidad de cuentas, con 87 registros.
- **Sun** tiene 76 cuentas, menos que **Sat**, pero presenta el promedio de cuenta más alto: 21.41.
- **Thur** tiene 62 cuentas y un total facturado de 1096.33.
- **Fri** tiene pocos registros en comparación con los demás días: solo 19 cuentas.

Si miramos el total facturado, los valores más altos aparecen en **Sat** y **Sun**. Pero si miramos el promedio de cuenta, **Sun** supera a **Sat**.

Esta diferencia es importante: un día puede facturar mucho porque tiene muchas cuentas, porque sus cuentas son más altas, o por una combinación de ambas cosas.

Visualizar el total facturado por día

Ahora vamos a representar el total facturado por día con un gráfico de barras. El gráfico no agrega información nueva por sí mismo: usa los mismos datos que ya calculamos en la tabla anterior. Sin embargo, puede ayudarnos a comparar los días de manera más directa.

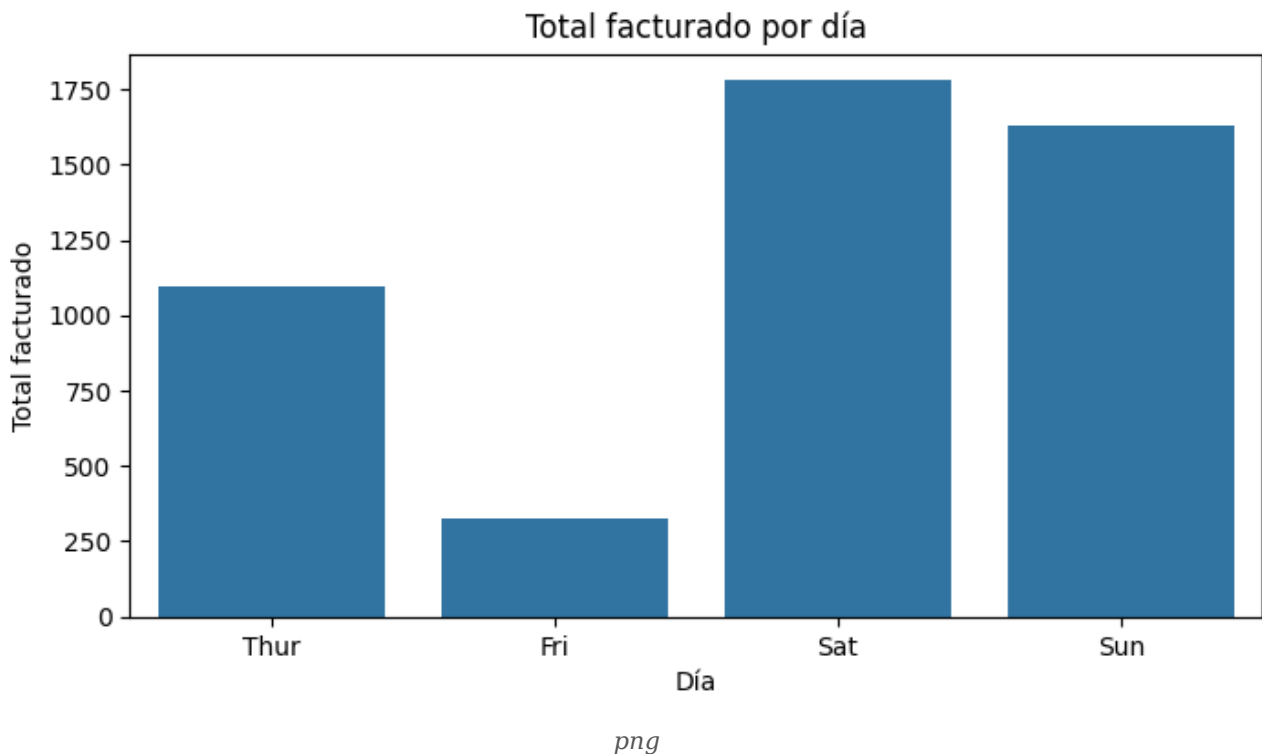
En este caso vamos a usar el orden temporal que definimos antes:

Thur, **Fri**, **Sat**, **Sun**

```
plt.figure(figsize=(8, 4))

sns.barplot(
    data=resumen_dia.reset_index(),
    x="day",
    y="total_facturado"
```

```
)  
  
plt.title("Total facturado por día")  
plt.xlabel("Día")  
plt.ylabel("Total facturado")  
  
plt.show()
```



El gráfico muestra el total facturado para cada día.

Cada barra representa un día del dataset, y la altura de la barra indica cuánto se facturó en total ese día. Visualmente se observa que **Sat** y **Sun** tienen los valores más altos. **Thur** aparece en un nivel intermedio y **Fri** queda bastante por debajo del resto.

Esto coincide con la tabla anterior:

- **Sat** : 1778.40
- **Sun** : 1627.16
- **Thur** : 1096.33
- **Fri** : 325.88

El gráfico ayuda a ver rápidamente la diferencia entre los días, especialmente la distancia entre **Fri** y los días de fin de semana.

Cómo se construyó este gráfico

En la celda anterior usamos dos herramientas:

`matplotlib.pyplot`, que importamos como `plt`, y `seaborn`, que importamos como `sns`.

La línea:

```
plt.figure(figsize=(8, 4))
```

crea el espacio donde se va a dibujar el gráfico.

El argumento `figsize=(8, 4)` indica el tamaño de la figura: 8 unidades de ancho y 4 de alto.

Luego usamos:

```
sns.barplot(  
    data=resumen_dia.reset_index(),  
    x="day",  
    y="total_facturado"  
)
```

Esta instrucción construye un gráfico de barras.

Le indicamos tres cosas principales:

- `data`: la tabla que contiene los datos que vamos a graficar.
- `x`: la columna que se ubicará en el eje horizontal.
- `y`: la columna que se ubicará en el eje vertical.

Usamos `resumen_dia.reset_index()` porque en la tabla `resumen_dia` el día estaba como índice. Para graficarlo con Seaborn, necesitamos que `day` vuelva a estar disponible como una columna común.

Después agregamos un título y etiquetas para los ejes:

```
plt.title("Total facturado por día")  
plt.xlabel("Día")  
plt.ylabel("Total facturado")
```

`plt.title()` agrega el título del gráfico.

`plt.xlabel()` define el nombre del eje horizontal.

`plt.ylabel()` define el nombre del eje vertical.

Por último:

```
plt.show()
```

muestra el gráfico en pantalla.

En general, esta estructura se puede leer así:

1. crear el espacio del gráfico;
2. indicar qué datos se van a graficar;
3. definir qué va en cada eje;
4. agregar título y etiquetas;
5. mostrar el resultado.

Total facturado y promedio de cuenta

El total facturado es un indicador importante, pero no alcanza por sí solo para interpretar completamente lo que ocurre en cada día. Un día puede tener mucha facturación porque recibió muchas cuentas, aunque el promedio de cada cuenta no sea el más alto. Por eso, ahora vamos a mirar el promedio de cuenta por día. La pregunta cambia:

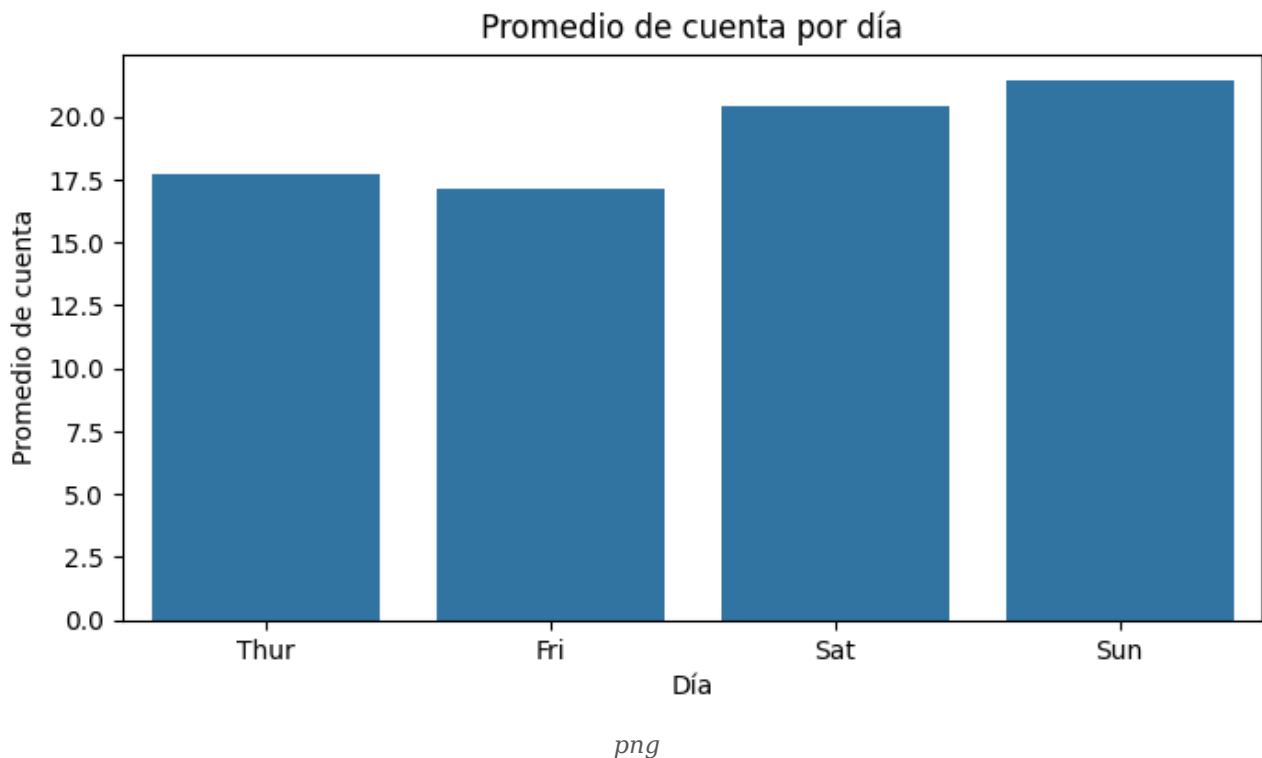
ya no queremos saber cuánto se facturó en total, sino **cuánto fue, en promedio, el importe de cada cuenta en cada día.**

```
plt.figure(figsize=(8, 4))

sns.barplot(
    data=resumen_dia.reset_index(),
    x="day",
    y="promedio_cuenta"
)

plt.title("Promedio de cuenta por día")
plt.xlabel("Día")
plt.ylabel("Promedio de cuenta")

plt.show()
```



El gráfico muestra el promedio de cuenta para cada día. A diferencia del gráfico anterior, ahora no estamos mirando cuánto se facturó en total, sino cuánto fue el importe promedio de cada cuenta.

Visualmente se observa que **Sun** tiene el promedio más alto, seguido por **Sat**.

En cambio, **Thur** y **Fri** tienen promedios más bajos y bastante cercanos entre sí.

Esto nos ayuda a distinguir dos ideas:

- **Sat** fue el día con mayor facturación total.
- **Sun** fue el día con mayor promedio de cuenta.

Por eso, en un análisis exploratorio conviene mirar más de un indicador. Si solo observamos el total facturado, podemos perder de vista que otro día tuvo cuentas más altas en promedio.

Comparar días y horarios al mismo tiempo

Hasta ahora analizamos los días de manera general. Pero el dataset también tiene la columna **time**, que distingue entre **Lunch** y **Dinner**.

Esto nos permite hacer una pregunta más específica:

¿cómo cambian la cantidad de cuentas, el total facturado y el promedio de cuenta según el día y el horario?

Para responderla, vamos a agrupar los datos usando dos columnas:

- `day`
- `time`

Este tipo de análisis nos permite observar no solo qué pasa en cada día, sino también en qué momento del día se concentran los registros.

```
resumen_dia_horario = df.groupby(["day", "time"], observed=True).agg(  
    cantidad_cuentas=("total_bill", "count"),  
    total_facturado=("total_bill", "sum"),  
    promedio_cuenta=("total_bill", "mean")  
)  
.round(2)  
  
resumen_dia_horario
```

day		cantidad_cuentas	total_facturado	promedio_cuenta
Thur	Lunch	61	1077.55	17.66
Dinner	1	18.78	18.78	
Fri	Lunch	7	89.92	12.85
Dinner	12	235.96	19.66	
Sat	Dinner	87	1778.40	20.44
Sun	Dinner	76	1627.16	21.41

La tabla muestra los indicadores agrupados por día y horario.

Podemos observar que la distribución no es pareja:

`Thur` tiene casi todos sus registros en `Lunch`: 61 cuentas. En `Dinner` aparece solo 1 cuenta.

`Fri` tiene registros en ambos horarios: 7 cuentas en `Lunch` y 12 en `Dinner`.

`Sat` y `Sun` aparecen solamente en `Dinner`.

También podemos comparar los importes:

- En `Thur`, el total facturado de `Lunch` es 1077.55.
- En `Fri`, `Dinner` factura más que `Lunch`: 235.96 frente a 89.92.
- En `Sat` y `Sun`, toda la facturación corresponde a `Dinner`.

Hay que tener cuidado con el promedio de `Thur` en `Dinner`. Aunque el promedio es 18.78, ese valor se calcula con una sola cuenta. Por eso no conviene darle la misma importancia que a promedios calculados con muchos registros.

Este es un punto importante del análisis exploratorio: antes de interpretar un promedio, siempre conviene mirar cuántos datos lo sostienen.

Preparar la tabla para graficar

La tabla anterior tiene dos niveles de índice: `day` y `time`. Eso ocurre porque agrupamos usando dos columnas:

```
df.groupby(["day", "time"])
```

Este formato es útil para leer la tabla, pero para graficar con Seaborn suele ser más cómodo volver a convertir esos índices en columnas comunes. Para eso vamos a usar `reset_index()`.

De esta manera, `day` y `time` vuelven a quedar como columnas de la tabla, junto con `cantidad_cuentas`, `total_facturado` y `promedio_cuenta`.

```
resumen_dia_horario_grafico = resumen_dia_horario.reset_index()

resumen_dia_horario_grafico
```

índice	day	time	cantidad_cuentas	total_facturado	promedio_cuenta
0	Thur	Lunch	61	1077.55	17.66
1	Thur	Dinner	1	18.78	18.78
2	Fri	Lunch	7	89.92	12.85
3	Fri	Dinner	12	235.96	19.66
4	Sat	Dinner	87	1778.40	20.44
5	Sun	Dinner	76	1627.16	21.41

La tabla conserva la misma información que antes, pero ahora tiene una estructura más simple para graficar.

Antes, `day` y `time` formaban parte del índice de la tabla.

Ahora aparecen como columnas:

- `day`
- `time`

- `cantidad_cuentas`
- `total_facturado`
- `promedio_cuenta`

Este formato nos permite indicar con más facilidad qué variable queremos ubicar en el eje horizontal, cuál en el eje vertical y cuál usar para separar los grupos.

Graficar días y horarios

Ahora vamos a construir un gráfico de barras para comparar el total facturado según el día y el horario.

En este gráfico:

- el eje horizontal mostrará los días;
- el eje vertical mostrará el total facturado;
- el color de las barras separará los horarios `Lunch` y `Dinner`.

Para lograr esa separación vamos a usar el argumento `hue`.

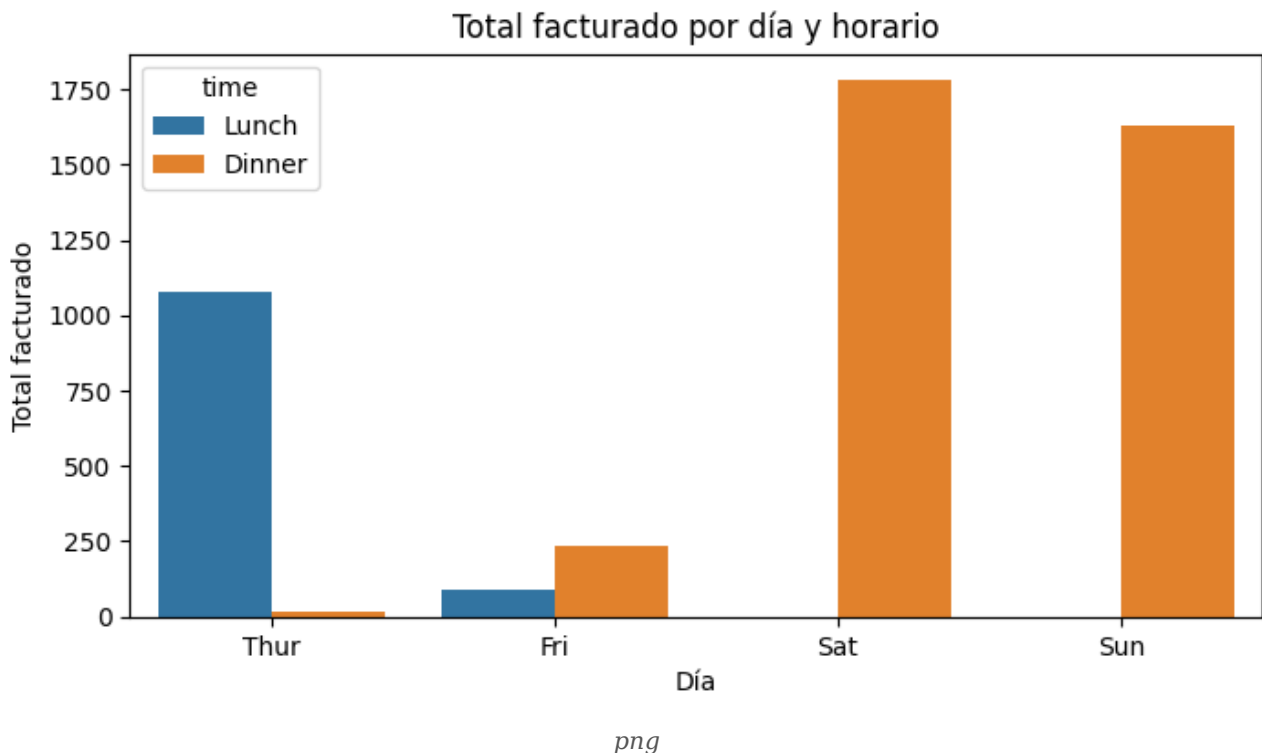
En Seaborn, `hue` permite dividir las barras según una segunda variable categórica.

```
plt.figure(figsize=(8, 4))

sns.barplot(
    data=resumen_dia_horario_grafico,
    x="day",
    y="total_facturado",
    hue="time",
    order=orden_dias
)

plt.title("Total facturado por día y horario")
plt.xlabel("Día")
plt.ylabel("Total facturado")

plt.show()
```



El gráfico muestra el total facturado según el día y el horario.

La separación por color permite distinguir entre `Lunch` y `Dinner`.

Podemos observar que:

- `Thur` concentra casi toda su facturación en `Lunch`.
- `Fri` tiene facturación en ambos horarios, aunque `Dinner` es mayor que `Lunch`.
- `Sat` y `Sun` aparecen solo en `Dinner`.
- Los valores más altos corresponden a `Sat` y `Sun` en horario de cena.

Este gráfico refuerza algo que ya habíamos visto en la tabla: la facturación no se distribuye de manera pareja entre días y horarios.

También conviene prestar atención a las barras muy pequeñas o ausentes. Por ejemplo, `Thur` tiene una barra mínima en `Dinner` porque solo hay una cuenta registrada en ese horario. En `Sat` y `Sun` no aparece barra para `Lunch` porque no hay registros de almuerzo para esos días en este dataset.

Leer el argumento `hue`

En el gráfico anterior usamos el argumento `hue`.

La línea principal fue:

```
sns.barplot(
    data=resumen_dia_horario_grafico,
    x="day",
    y="total_facturado",
    hue="time",
    order=orden_dias
)
```

El argumento `x="day"` indica que los días se ubican en el eje horizontal.

El argumento `y="total_facturado"` indica que la altura de las barras representa el total facturado.

El argumento `hue="time"` agrega una separación por horario. Por eso, dentro de cada día, Seaborn intenta mostrar barras diferentes para `Lunch` y `Dinner`.

Finalmente, `order=orden_dias` fuerza el orden de los días que definimos antes:

`Thur`, `Fri`, `Sat`, `Sun`

Esto es importante porque queremos leer el gráfico respetando el orden temporal básico, no como un ranking.

Visualizar la cantidad de cuentas por día y horario

El total facturado nos muestra cuánto dinero se acumuló en cada combinación de día y horario. Pero, como vimos antes, ese total depende mucho de la cantidad de cuentas registradas. Por eso vamos a construir un gráfico similar, pero ahora usando `cantidad_cuentas`. La pregunta será:

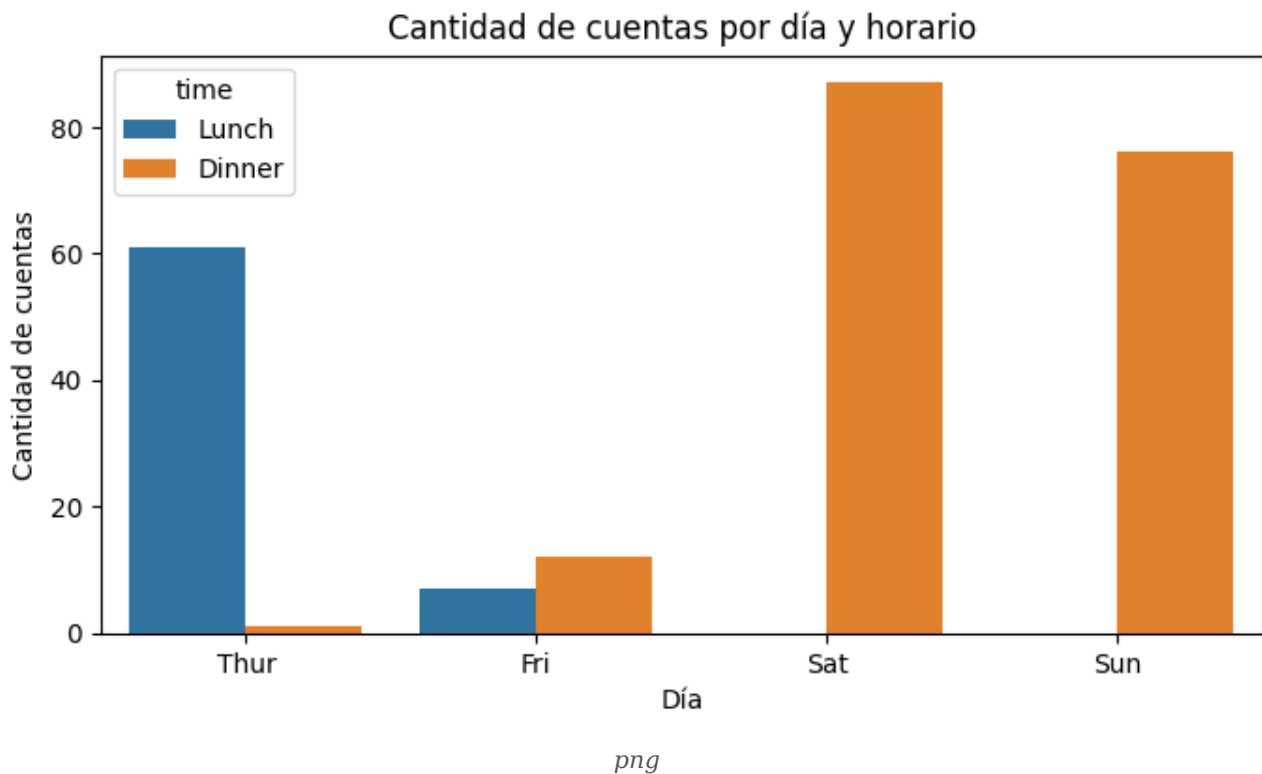
¿cuántas cuentas se registraron en cada día y horario?

```
plt.figure(figsize=(8, 4))

sns.barplot(
    data=resumen_dia_horario_grafico,
    x="day",
    y="cantidad_cuentas",
    hue="time",
    order=orden_dias
)

plt.title("Cantidad de cuentas por día y horario")
plt.xlabel("Día")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El gráfico muestra la cantidad de cuentas registradas según día y horario.

La lectura visual es bastante clara:

- `Thur` concentra la mayoría de sus cuentas en `Lunch`.
- `Fri` tiene pocas cuentas en ambos horarios, aunque hay más registros en `Dinner`.
- `Sat` y `Sun` concentran sus cuentas en `Dinner`.
- No aparecen barras de `Lunch` para `Sat` y `Sun` porque no hay registros de almuerzo para esos días en el dataset.

Este gráfico ayuda a entender mejor el gráfico anterior de facturación.

Por ejemplo, `Sat` y `Sun` tenían una facturación total alta, pero también son los días con más cuentas en horario de cena. Es decir, el total facturado no depende solamente del importe de cada cuenta, sino también de cuántas cuentas se registraron.

Por eso, cuando hacemos un análisis temporal básico, conviene mirar al menos dos cosas: la cantidad de registros disponibles para cada momento y el valor del indicador que queremos comparar.

Integrar la lectura

Hasta ahora analizamos los datos desde varias perspectivas:

Primero observamos la cantidad de cuentas por día y horario.

Después calculamos el total facturado y el promedio de cuenta por día.

Luego usamos gráficos de barras para visualizar esas comparaciones.

Lo importante no es elegir una única tabla o un único gráfico, sino combinar varias miradas. Por ejemplo:

`Sat` tiene la mayor cantidad de cuentas y también el mayor total facturado.

`Sun` tiene menos cuentas que `Sat`, pero alcanza el promedio de cuenta más alto.

`Thur` aparece principalmente asociado al almuerzo.

`Fri` tiene pocos registros en comparación con los demás días, por lo que sus resultados deben interpretarse con cuidado.

Esta lectura integrada es parte del análisis exploratorio: mirar los datos desde distintas preguntas antes de sacar conclusiones.

Cuidado: esto no es una serie temporal completa

Aunque en este capítulo hablamos de análisis temporal básico, es importante hacer una aclaración. Las columnas `day` y `time` nos permiten comparar momentos, pero no tenemos fechas completas. No sabemos a qué semana, mes o año corresponde cada registro. Tampoco sabemos si los datos representan una semana completa, varias semanas o una muestra parcial.

Por eso, con este dataset no podemos hablar de una tendencia histórica. No sería correcto decir, por ejemplo, que "la facturación aumenta con el paso de los días" como si estuviéramos observando una evolución real en el tiempo. Lo que sí podemos decir es más prudente: "en este dataset, las cuentas se distribuyen de manera diferente según el día y el horario"; "`Sat` y `Sun` concentran más facturación en `Dinner`"; "`Thur` concentra más registros en `Lunch`"; y "algunos promedios deben interpretarse teniendo en cuenta cuántas cuentas los sostienen".

Cierre del capítulo

En este capítulo empezamos a analizar datos según una dimensión temporal simple.

Trabajamos con las columnas `day` y `time` para observar cómo cambian las cantidades, los totales y los promedios según el momento.

También incorporamos gráficos de barras con Seaborn para visualizar algunos resultados.

Vimos que un gráfico puede ayudar a comparar, pero no reemplaza a la interpretación. Para leer correctamente un gráfico necesitamos saber qué variable aparece en cada eje, qué representa cada barra y cuántos datos hay detrás de cada valor.

Además, distinguimos entre ordenar datos como ranking y respetar un orden temporal. En este capítulo usamos el orden:

`Thur`, `Fri`, `Sat`, `Sun`

Esa decisión nos permitió leer los días como una secuencia y no solo como categorías independientes.

En el próximo capítulo vamos a avanzar hacia otro tipo de visualización: las distribuciones.

Ya no nos vamos a preguntar solamente cómo cambian los datos según un día o un horario, sino cómo se distribuyen los valores de una variable numérica.

Capítulo 34 — Visualizar distribuciones

Mirar cómo se reparten los valores

En el capítulo anterior analizamos los datos según una dimensión temporal simple.

Usamos las columnas `day` y `time` para comparar cantidades, totales y promedios según el día y el horario. Ahora vamos a cambiar el foco.

En lugar de preguntarnos cómo cambian los datos entre momentos, vamos a observar cómo se distribuyen los valores de una variable numérica. Por ejemplo:

¿la mayoría de las cuentas tienen importes bajos, medios o altos?

¿hay muchas cuentas parecidas entre sí?

¿aparecen algunos valores mucho más grandes que el resto?

Para responder este tipo de preguntas vamos a empezar a usar gráficos de distribución.

En este capítulo trabajaremos principalmente con la columna `total_bill`, que representa el importe total de cada cuenta.

¿Qué significa visualizar una distribución?

Cuando analizamos una columna numérica, no siempre alcanza con calcular un promedio. El promedio resume muchos datos en un solo número, pero no nos muestra cómo están repartidos los valores.

Dos columnas pueden tener promedios parecidos y, sin embargo, distribuciones muy diferentes. Visualizar una distribución significa observar cómo se reparten los valores de una variable:

- qué valores aparecen con más frecuencia;
- si los datos están concentrados o dispersos;
- si hay valores muy pequeños o muy grandes;
- si aparecen valores extremos.

Para empezar, vamos a cargar nuevamente el dataset `tips`.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset `tips`.

Cada fila representa una cuenta registrada en un restaurante. En este capítulo vamos a concentrarnos especialmente en la columna `total_bill`, que indica el importe total de cada cuenta. Esta columna es numérica, por lo que podemos analizarla de varias maneras:

- podemos calcular medidas de resumen, como el promedio o la mediana;
- podemos buscar valores mínimos y máximos;
- y también podemos visualizar cómo se reparten sus valores.

Ese último punto será el eje principal del capítulo.

Empezar con una variable numérica

Antes de hacer un gráfico, conviene mirar algunos valores de resumen. Esto nos ayuda a tener una primera idea de la variable que vamos a visualizar. Vamos a calcular un resumen estadístico de la columna `total_bill`.

Este resumen nos mostrará valores como:

- la cantidad de datos;
- el promedio;
- el valor mínimo;
- los cuartiles;
- el valor máximo.

Después vamos a comparar esta información con lo que se ve en un gráfico de distribución.

```
df["total_bill"].describe().round(2)
```

índice	total_bill
count	244.00
mean	19.79
std	8.90
min	3.07
25%	13.35
50%	17.80
75%	24.13
max	50.81

`dtype: float64`

El resumen estadístico de `total_bill` nos da una primera idea de cómo son los importes de las cuentas. En el dataset hay 244 cuentas registradas.

El importe promedio de una cuenta es 19.79, mientras que la mediana es 17.80. Recordemos que la mediana corresponde al valor central: aproximadamente la mitad de las cuentas tienen un importe menor o igual a 17.80, y la otra mitad tiene un importe mayor o igual.

También podemos observar que:

- el valor mínimo es 3.07;
- el 25% de las cuentas tiene un importe menor o igual a 13.35;
- el 75% de las cuentas tiene un importe menor o igual a 24.13;
- el valor máximo es 50.81.

Estos números son útiles, pero todavía no nos muestran con claridad cómo se reparten los valores.

Para eso vamos a construir nuestro primer gráfico de distribución.

Primer gráfico de distribución: histograma

Un histograma sirve para visualizar cómo se distribuyen los valores de una variable numérica. La idea es dividir el rango de valores en intervalos y contar cuántos datos caen dentro de cada intervalo.

En nuestro caso, vamos a usar un histograma para observar cómo se reparten los importes de la columna `total_bill`. Este gráfico nos puede ayudar a responder preguntas como:

¿la mayoría de las cuentas se concentra en valores bajos, medios o altos?

¿hay muchos valores grandes?

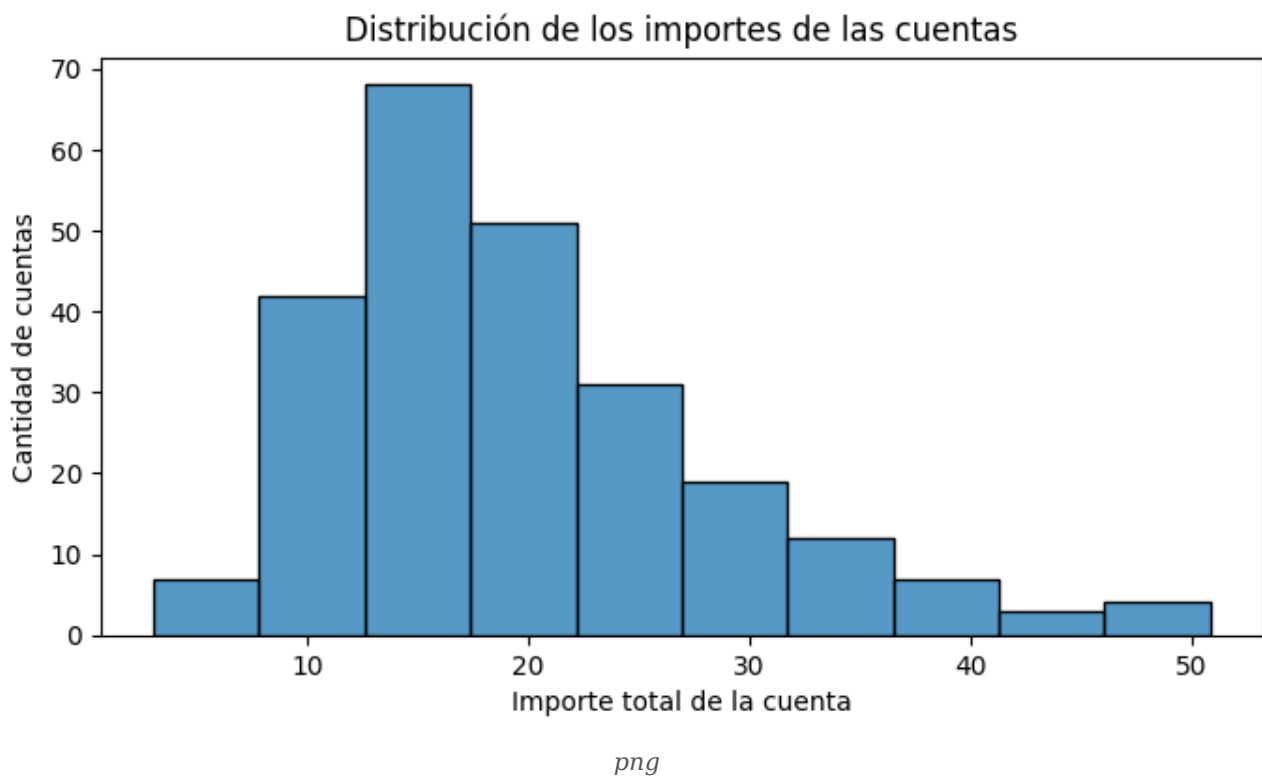
¿los valores están repartidos de manera pareja o se concentran en una zona?

```
plt.figure(figsize=(8, 4))

sns.histplot(
    data=df,
    x="total_bill",
    bins=10
)

plt.title("Distribución de los importes de las cuentas")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El histograma muestra cómo se distribuyen los importes de las cuentas.

En el eje horizontal aparecen los valores de `total_bill`, es decir, los importes totales de las cuentas.

En el eje vertical aparece la cantidad de cuentas que caen dentro de cada intervalo.

La mayor concentración de cuentas se encuentra aproximadamente entre 10 y 25. Esto significa que la mayoría de los importes registrados están en esa zona.

También se observa que hay menos cuentas a medida que los importes son más altos. Hacia la derecha del gráfico aparecen algunas cuentas con valores mayores, cercanos a 40 o 50, pero son poco frecuentes.

Esta forma nos sugiere que la distribución no es simétrica: hay muchas cuentas de valores bajos o medios, y unas pocas cuentas bastante más altas.

Leer la estructura del histograma

En la celda anterior usamos `sns.histplot()`.

La instrucción principal fue:

```
sns.histplot(data=df, x="total_bill", bins=10)
```

El argumento `data=df` indica qué DataFrame vamos a usar.

El argumento `x="total_bill"` indica qué columna queremos visualizar.

El argumento `bins=10` indica en cuántos intervalos se divide el rango de valores.

En un histograma, cada barra representa un intervalo de valores, no un valor individual.

Por ejemplo, una barra puede representar cuentas entre cierto importe mínimo y cierto importe máximo. La altura de esa barra indica cuántas cuentas caen dentro de ese intervalo.

Por eso, el histograma no se lee igual que un gráfico de barras categórico. En un gráfico de barras categórico, cada barra representa una categoría. En un histograma, cada barra representa un rango de valores numéricos.

Cambiar la cantidad de intervalos

El argumento `bins` puede modificar bastante la apariencia del histograma. Si usamos pocos intervalos, el gráfico queda más resumido. Si usamos más intervalos, aparecen más detalles, pero también puede ser más difícil leer una forma general.

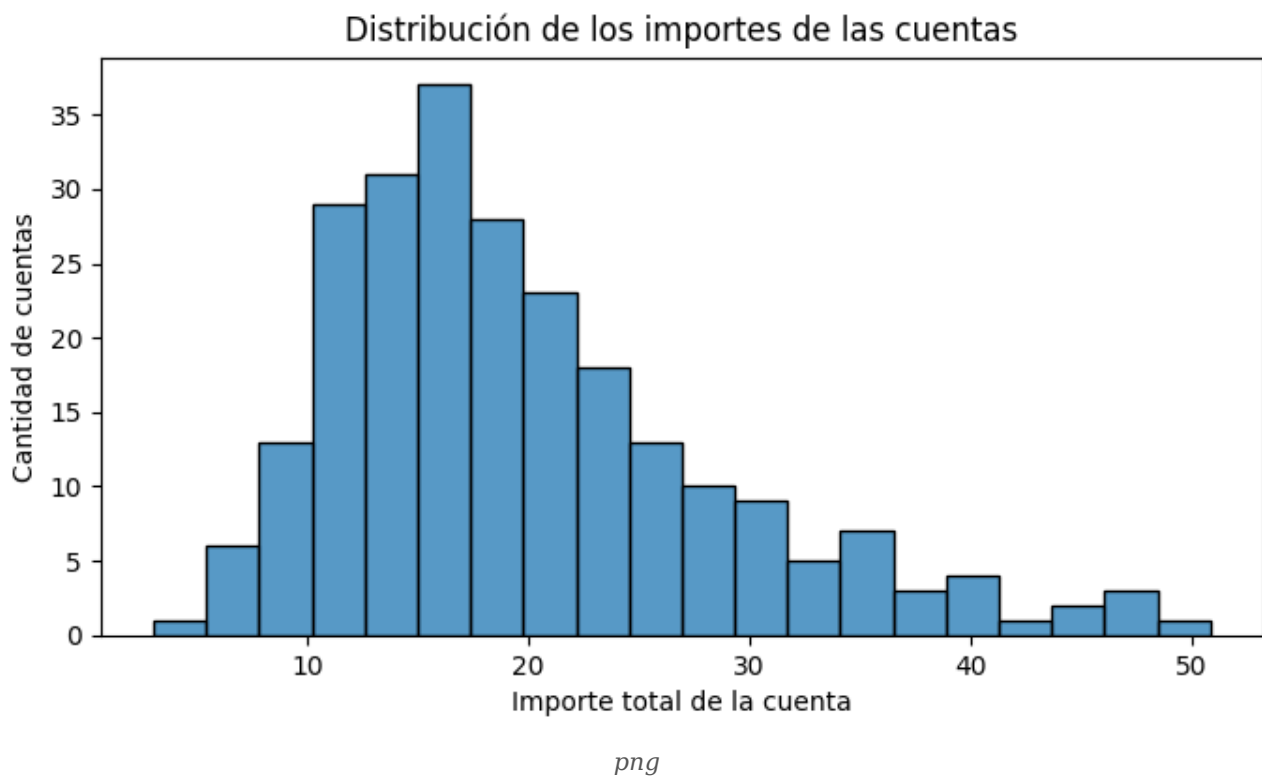
Vamos a repetir el histograma usando más intervalos para observar qué cambia.

```
plt.figure(figsize=(8, 4))

sns.histplot(
    data=df,
    x="total_bill",
    bins=20
)

plt.title("Distribución de los importes de las cuentas")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



Al aumentar la cantidad de intervalos, el histograma muestra más detalle. La forma general se mantiene: la mayor concentración de cuentas sigue apareciendo aproximadamente entre 10 y 25; hay menos cuentas a medida que los importes son más altos; y aparecen algunos valores grandes hacia la derecha del gráfico.

Este segundo histograma permite ver mejor que la distribución tiene una cola hacia la derecha. Esto significa que la mayoría de los valores se concentra en importes bajos o medios, mientras que unos pocos valores más altos se extienden hacia la derecha del gráfico.

Por eso, en el resumen estadístico anterior, el promedio era 19.79 y la mediana era 17.80.

Cuando hay algunos valores altos, el promedio puede quedar un poco más elevado que la mediana.

Marcar el promedio y la mediana

Una forma útil de conectar una tabla de resumen con un gráfico es marcar algunos valores importantes sobre la visualización.

Vamos a volver a graficar la distribución de `total_bill`, pero esta vez vamos a agregar dos líneas verticales:

- una línea para el promedio;
- otra línea para la mediana.

Esto nos va a permitir observar dónde quedan ubicados esos valores dentro de la distribución.

```
promedio_total = df["total_bill"].mean()
mediana_total = df["total_bill"].median()

plt.figure(figsize=(8, 4))

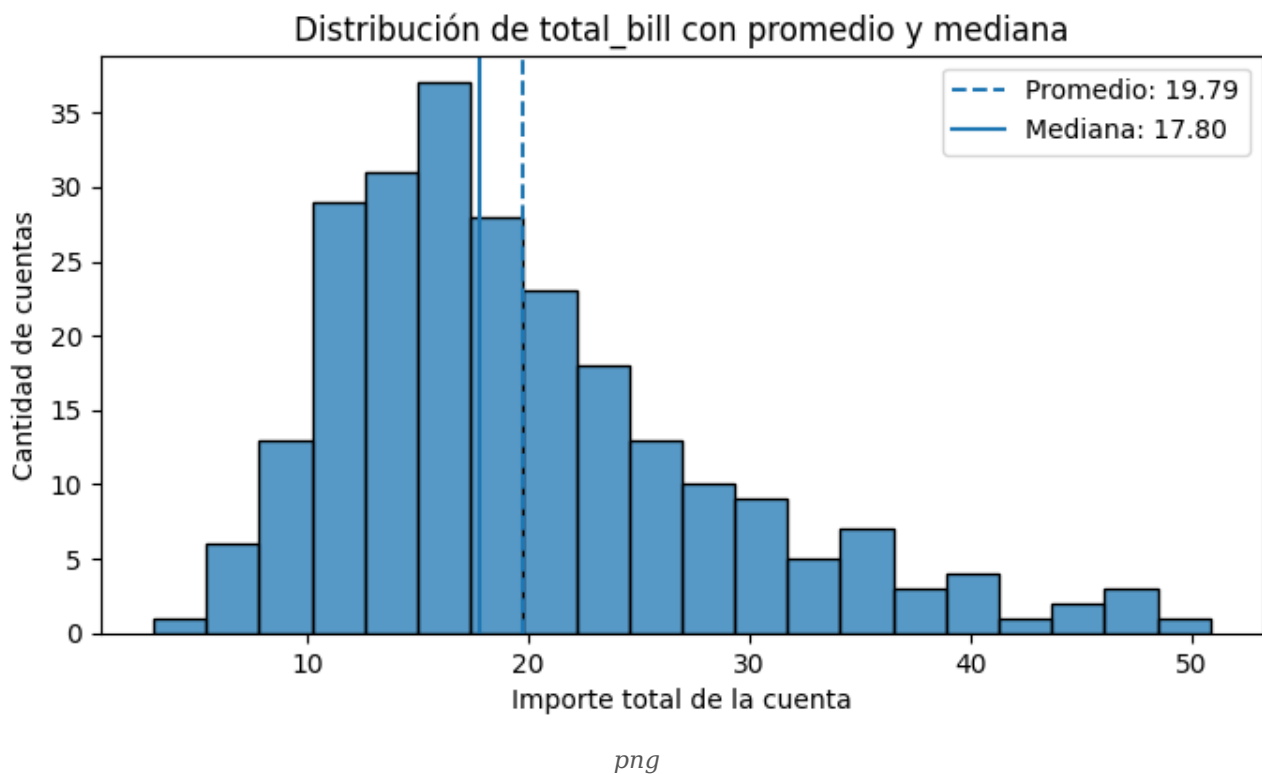
sns.histplot(
    data=df,
    x="total_bill",
    bins=20
)

plt.axvline(
    promedio_total,
    linestyle="--",
    label=f"Promedio: {promedio_total:.2f}"
)

plt.axvline(
    mediana_total,
    linestyle="-",
    label=f"Mediana: {mediana_total:.2f}"
)

plt.title("Distribución de total_bill con promedio y mediana")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Cantidad de cuentas")
plt.legend()

plt.show()
```



El histograma muestra nuevamente la distribución de `total_bill`, pero ahora agregamos dos líneas verticales.

- La línea punteada marca el promedio: 19.79.
- La línea continua marca la mediana: 17.80.

Se puede observar que el promedio queda un poco más a la derecha que la mediana. Esto coincide con lo que vimos antes: la distribución tiene varios valores altos hacia la derecha. Esos valores altos empujan el promedio hacia arriba.

La mediana, en cambio, representa el valor central de los datos y suele verse menos afectada por algunos valores grandes.

Por eso, cuando una distribución tiene una cola hacia la derecha, es común que el promedio sea mayor que la mediana.

Cómo agregamos líneas de referencia

En la celda anterior usamos algunas instrucciones nuevas.

Primero calculamos dos valores:

```
promedio_total = df["total_bill"].mean()
```

```
mediana_total = df["total_bill"].median()
```

La primera línea calcula el promedio de la columna `total_bill`.

La segunda línea calcula la mediana de esa misma columna.

Después usamos `plt.axvline()` para dibujar líneas verticales sobre el gráfico. Por ejemplo:

```
plt.axvline(promedio_total, linestyle="--", label=f"Promedio: {promedio_total:.2f}")
```

Esta instrucción agrega una línea vertical en la posición del promedio.

El argumento `linestyle="--"` indica que la línea se dibuja con estilo punteado.

El argumento `label` define el texto que aparecerá en la leyenda.

La parte `f"Promedio: {promedio_total:.2f}"` permite construir un texto que incluye el valor del promedio con dos decimales.

También usamos:

```
plt.legend()
```

Esta instrucción muestra la leyenda del gráfico. La leyenda sirve para identificar qué representa cada línea.

Otra forma de visualizar la distribución: boxplot

El histograma nos ayuda a ver la forma general de una distribución. Ahora vamos a usar otro gráfico muy común para analizar variables numéricas: el boxplot, también llamado diagrama de caja.

Un boxplot permite observar de manera compacta:

- la mediana;
- el rango donde se concentra la mitad central de los datos;
- los bigotes, que muestran hasta dónde llegan los valores que no se consideran extremos según el criterio del gráfico;
- posibles valores extremos.

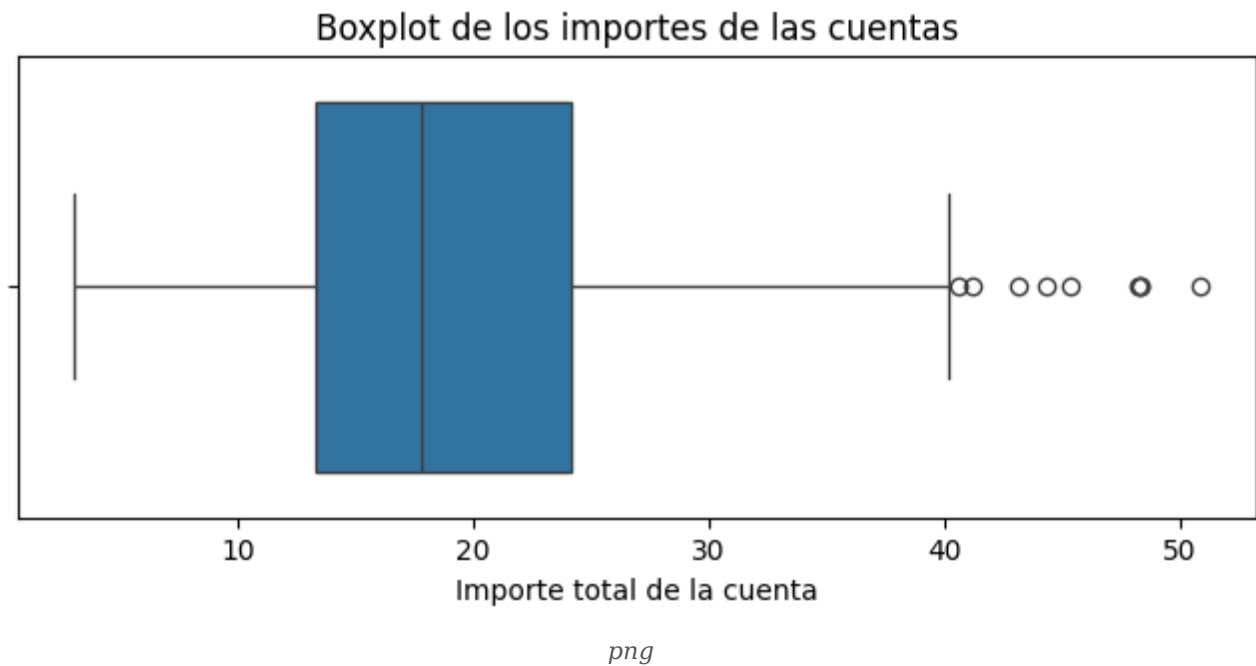
Este tipo de gráfico es especialmente útil cuando queremos detectar si hay valores alejados del resto.

```
plt.figure(figsize=(8, 3))
```

```
sns.boxplot(
    data=df,
    x="total_bill"
)

plt.title("Boxplot de los importes de las cuentas")
plt.xlabel("Importe total de la cuenta")

plt.show()
```



El boxplot muestra la distribución de `total_bill` de una manera más resumida que el histograma.

La caja representa la zona donde se concentra la mitad central de los datos. En este caso, la caja se extiende aproximadamente desde 13.35 hasta 24.13. Esto coincide con los valores del primer y tercer cuartil que vimos en el resumen estadístico. La línea dentro de la caja marca la mediana, que era 17.80.

También se observa que el lado derecho del gráfico se extiende más que el izquierdo. Esto vuelve a mostrar que hay algunos importes bastante más altos que la mayoría de las cuentas.

Los puntos que aparecen hacia la derecha, aproximadamente por encima de 40, representan posibles valores extremos. No significa necesariamente que sean errores, sino que son cuentas más alejadas del comportamiento general del dataset.

Cómo leer un boxplot

Un boxplot resume una variable numérica usando algunas referencias importantes.

- La caja muestra el rango donde se encuentra la mitad central de los datos.
- El borde izquierdo de la caja representa el primer cuartil. En este caso, corresponde aproximadamente a 13.35.
- El borde derecho de la caja representa el tercer cuartil. En este caso, corresponde aproximadamente a 24.13.
- La línea dentro de la caja representa la mediana. En este caso, la mediana es 17.80.
- Las líneas que salen de la caja se llaman bigotes. Sirven para mostrar hasta dónde llegan los valores que todavía se consideran dentro del rango esperado según el criterio del gráfico.
- Los puntos que aparecen fuera de los bigotes son posibles valores extremos.

Este tipo de gráfico es útil porque permite ver rápidamente la concentración central de los datos, la posición de la mediana y la presencia de valores alejados.

Conectar el boxplot con los valores numéricos

Para interpretar mejor el boxplot, podemos recuperar algunos valores que ya aparecieron en el resumen estadístico.

Vamos a calcular:

- primer cuartil;
- mediana;
- tercer cuartil;
- rango intercuartílico.

El rango intercuartílico mide la distancia entre el tercer cuartil y el primer cuartil.

Nos ayuda a describir cuán extendida está la mitad central de los datos.

```
q1 = df["total_bill"].quantile(0.25)
mediana = df["total_bill"].median()
q3 = df["total_bill"].quantile(0.75)
rango_intercuartilico = q3 - q1

resumen_boxplot = pd.DataFrame({
    "indicador": [
        "Primer cuartil",
        "Mediana",
        "Tercer cuartil",
```

```

        "Rango intercuartílico"
    ],
    "valor": [
        q1,
        mediana,
        q3,
        rango_intercuartilico
    ]
})

resumen_boxplot["valor"] = resumen_boxplot["valor"].round(2)

resumen_boxplot

```

índice	indicador	valor
0	Primer cuartil	13.35
1	Mediana	17.80
2	Tercer cuartil	24.13
3	Rango intercuartílico	10.78

La tabla muestra algunos valores clave para interpretar el boxplot.

El primer cuartil es 13.35. Esto significa que el 25% de las cuentas tiene un importe menor o igual a 13.35.

La mediana es 17.80. Esto indica que la mitad de las cuentas tiene un importe menor o igual a 17.80, y la otra mitad tiene un importe mayor o igual.

El tercer cuartil es 24.13. Esto significa que el 75% de las cuentas tiene un importe menor o igual a 24.13.

El rango intercuartílico es 10.78. Este valor surge de restar el primer cuartil al tercer cuartil:

$$24.13 - 13.35 = 10.78$$

Ese rango describe la amplitud de la mitad central de los datos. En otras palabras, la mitad central de las cuentas se encuentra aproximadamente entre 13.35 y 24.13.

Histograma y boxplot: dos miradas complementarias

El histograma y el boxplot muestran la misma variable, pero de formas distintas. El histograma permite ver la forma general de la distribución. Nos muestra en qué zonas se concentran más valores y si hay una cola hacia alguno de los lados.

El boxplot resume esa distribución en pocos elementos: cuartiles, mediana, bigotes y posibles valores extremos.

En este caso, ambos gráficos cuentan una historia parecida:

- la mayoría de las cuentas se concentra en valores bajos o medios;
- la mitad central de los datos está aproximadamente entre 13.35 y 24.13;
- la mediana está en 17.80;
- y hay algunos importes altos que aparecen alejados del grupo principal.

Usar ambos gráficos juntos ayuda a interpretar mejor la variable. El histograma muestra la forma; el boxplot resume la posición, la dispersión y los posibles valores extremos.

Aplicar la misma idea a otra variable: propinas

Hasta ahora analizamos la distribución de `total_bill`. Para reforzar la idea, vamos a mirar otra variable numérica del dataset: `tip`.

La columna `tip` representa el monto de la propina.

Vamos a seguir el mismo camino: primero calculamos un resumen estadístico; después visualizamos la distribución con un histograma; y luego usamos un boxplot para observar la mediana, los cuartiles y posibles valores extremos.

```
df["tip"].describe().round(2)
```

índice	tip
count	244.00
mean	3.00
std	1.38
min	1.00
25%	2.00
50%	2.90
75%	3.56
max	10.00

`dtype: float64`

El resumen estadístico de `tip` muestra cómo se comportan las propinas registradas en el dataset. Hay 244 propinas, una por cada cuenta.

El promedio de propina es 3.00, mientras que la mediana es 2.90. En este caso, ambos valores están bastante cerca.

También podemos observar que:

- la propina mínima es 1.00;
- el 25% de las propinas es menor o igual a 2.00;
- el 50% de las propinas es menor o igual a 2.90;
- el 75% de las propinas es menor o igual a 3.56;
- la propina máxima es 10.00.

La mayor parte de las propinas parece ubicarse en valores relativamente bajos, entre 2 y 4 aproximadamente.

Pero para observar mejor cómo se reparten esos valores, conviene construir un histograma.

Histograma de las propinas

Vamos a visualizar ahora la distribución de la columna `tip`. La pregunta que queremos responder es:

¿cómo se reparten los valores de las propinas?

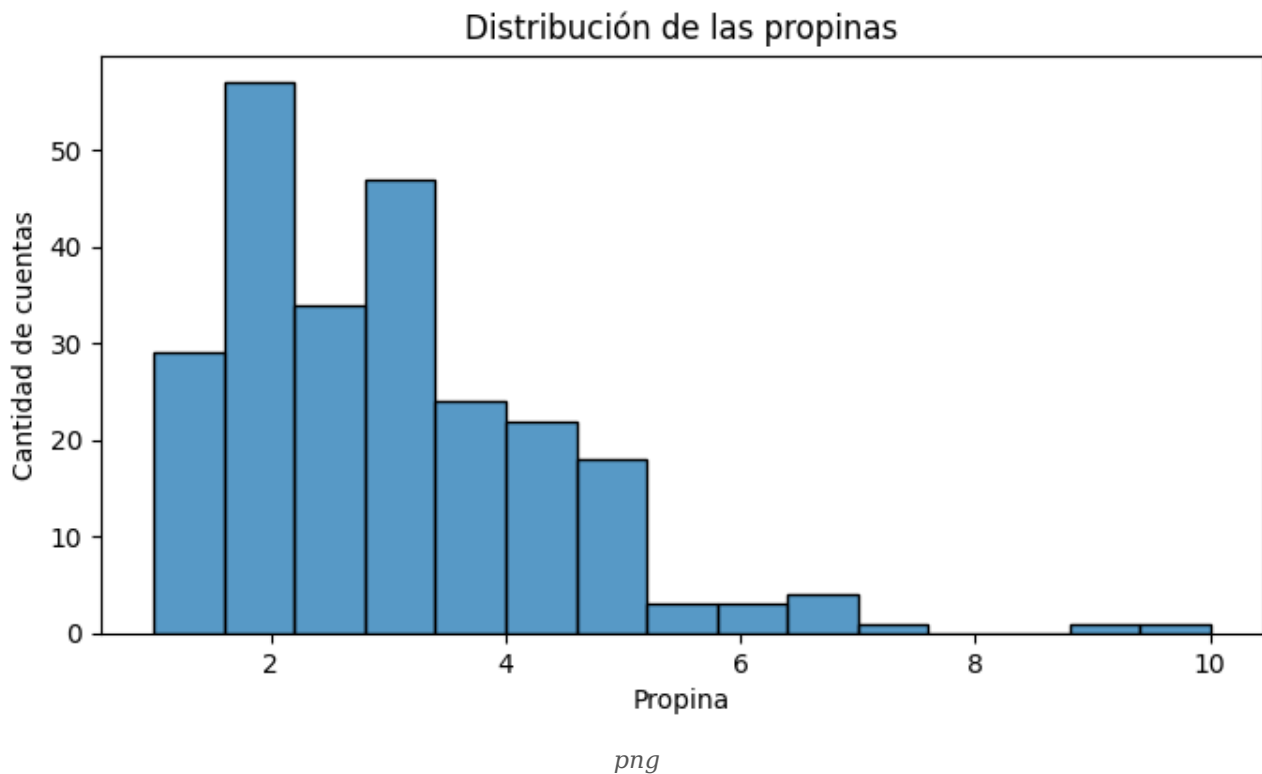
Este gráfico nos permitirá observar si la mayoría de las propinas se concentra en una zona, si hay valores altos poco frecuentes y si la distribución tiene una forma parecida o diferente a la de `total_bill`.

```
plt.figure(figsize=(8, 4))

sns.histplot(
    data=df,
    x="tip",
    bins=15
)

plt.title("Distribución de las propinas")
plt.xlabel("Propina")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El histograma muestra cómo se distribuyen los valores de `tip`.

La mayoría de las propinas se concentra aproximadamente entre 1 y 5. También se observa que las propinas altas son menos frecuentes. Hay pocos valores por encima de 6, y aparecen algunos casos aislados cercanos a 9 o 10.

Esto indica que la distribución tiene una cola hacia la derecha: muchos valores se agrupan en la zona baja o media, mientras que unos pocos valores más altos se alejan del grupo principal.

Esta lectura coincide con el resumen estadístico anterior:

- el promedio de propina es 3.00;
- la mediana es 2.90;
- el 75% de las propinas es menor o igual a 3.56;
- el valor máximo llega a 10.00.

Aunque el promedio y la mediana están bastante cerca, los valores altos siguen siendo importantes para entender la forma completa de la distribución.

Boxplot de las propinas

Ahora vamos a visualizar la misma variable con un boxplot. El objetivo es observar de forma más compacta:

- dónde se ubica la mediana;
- entre qué valores se concentra la mitad central de las propinas;
- si aparecen posibles valores extremos.

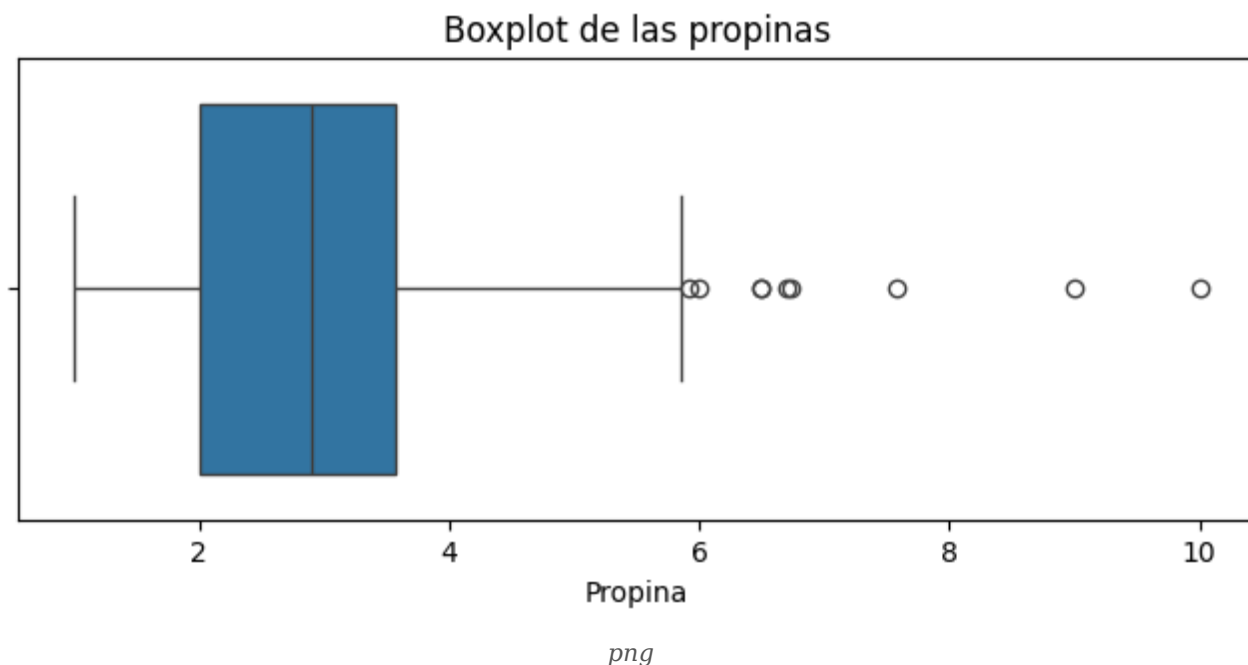
Como ya vimos con `total_bill`, el boxplot no reemplaza al histograma. Nos ofrece otra mirada sobre la misma distribución.

```
plt.figure(figsize=(8, 3))

sns.boxplot(
    data=df,
    x="tip"
)

plt.title("Boxplot de las propinas")
plt.xlabel("Propina")

plt.show()
```



El boxplot de `tip` resume la distribución de las propinas.

La caja se concentra aproximadamente entre 2.00 y 3.56, que son el primer y tercer cuartil que vimos en el resumen estadístico.

La línea dentro de la caja marca la mediana, que es 2.90.

Esto significa que la mitad central de las propinas se encuentra en un rango bastante acotado: entre 2.00 y 3.56.

También aparecen varios puntos hacia la derecha, por encima de los valores más habituales. Esos puntos representan posibles valores extremos.

En este caso, no necesariamente indican errores. Pueden ser propinas reales, pero más altas que la mayoría de las observaciones del dataset.

El boxplot confirma lo que ya mostraba el histograma: la mayoría de las propinas se concentra en valores bajos o medios, y hay unos pocos valores altos alejados del grupo principal.

Comparar las distribuciones observadas

En este capítulo analizamos dos variables numéricas:

- `total_bill`, que representa el importe total de la cuenta;
- y `tip`, que representa la propina.

En ambos casos usamos el mismo recorrido:

- primero calculamos un resumen estadístico;
- después construimos un histograma;
- y finalmente usamos un boxplot.

Esto nos permitió ver que ambas variables tienen una distribución con cola hacia la derecha.

En otras palabras, hay muchos valores bajos o medios y pocos valores altos.

También vimos que los valores altos pueden influir en el promedio. Por eso, al analizar una distribución, conviene mirar el promedio junto con la mediana, los cuartiles y los posibles valores extremos.

Qué aprendimos sobre histogramas y boxplots

Un histograma muestra la forma general de una distribución. Nos ayuda a ver dónde se concentran los valores, si hay zonas con muchos o pocos datos, y si la distribución se extiende más hacia un lado que hacia otro.

Un boxplot, en cambio, resume la distribución de manera más compacta. Nos permite identificar rápidamente la mediana, la mitad central de los datos y posibles valores extremos.

Estos dos gráficos no compiten entre sí. Se complementan. El histograma muestra mejor la forma de la distribución. El boxplot muestra mejor la síntesis de la distribución.

Cuidado al interpretar valores extremos

En los boxplots vimos algunos puntos alejados del resto. Es importante interpretarlos con cuidado. Un valor extremo no significa automáticamente que haya un error en los datos. Puede ser:

- un dato real pero poco frecuente;
- una cuenta especialmente alta;
- una propina especialmente alta;
- o, en otros contextos, un posible error de carga o medición.

En análisis exploratorio, los valores extremos no se eliminan automáticamente. Primero se observan, se describen y, si es necesario, se investigan con más detalle.

Cierre del capítulo

En este capítulo aprendimos a visualizar distribuciones de variables numéricas. Trabajamos con `total_bill` y `tip`, y usamos dos gráficos principales:

`histplot()`, para construir histogramas;

y `boxplot()`, para construir diagramas de caja.

También conectamos los gráficos con medidas de resumen como el promedio, la mediana, los cuartiles y el rango intercuartílico.

La idea central del capítulo fue que una variable numérica no se entiende completamente mirando solo un promedio.

Para analizarla mejor, necesitamos observar cómo se distribuyen sus valores.

En el próximo capítulo vamos a avanzar hacia otro tipo de pregunta: cómo visualizar comparaciones entre grupos o categorías.

Capítulo 35 — Comparar distribuciones por grupos

Mirar una variable numérica según categorías

En el capítulo anterior aprendimos a visualizar distribuciones de variables numéricas. Trabajamos con `total_bill` y `tip`, usando histogramas y boxplots para observar cómo se reparten sus valores. Ahora vamos a avanzar un paso más.

En lugar de mirar una variable numérica de manera general, vamos a compararla según distintos grupos o categorías.

Por ejemplo:

¿los importes de las cuentas son parecidos en `Lunch` y en `Dinner`?

¿hay diferencias entre los días registrados en el dataset?

¿la distribución de las propinas cambia según alguna categoría?

Para responder este tipo de preguntas vamos a combinar variables numéricas con variables categóricas.

En este capítulo vamos a usar principalmente:

- `total_bill`, como variable numérica;
- `time`, `day` y otras columnas categóricas para formar grupos.

Cargar el dataset

Como en los capítulos anteriores, vamos a trabajar con el dataset `tips`. Primero importamos las librerías necesarias y cargamos los datos.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2

índice	total_bill	tip	sex	smoker	day	time	size
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset `tips`. Cada fila representa una cuenta registrada en un restaurante.

En capítulos anteriores ya analizamos algunas columnas por separado. Ahora vamos a combinar dos tipos de variables: una variable numérica, como `total_bill` y una variable categórica, como `time`, `day`, `sex` o `smoker`.

Esto nos permitirá comparar distribuciones entre grupos. Por ejemplo, podemos mirar si los importes de las cuentas se distribuyen de la misma manera en `Lunch` y en `Dinner`.

Primera comparación: importe total según horario

Vamos a empezar comparando la columna `total_bill` según el horario de la cuenta.

Para eso usaremos la columna `time`, que distingue entre:

- `Lunch`;
- `Dinner`.

Antes de graficar, conviene calcular un resumen numérico por grupo.

Así podremos observar cuántas cuentas hay en cada horario y cuáles son los valores principales de `total_bill` para cada uno.

```
resumen_time = df.groupby("time", observed=True)["total_bill"].agg(
    cantidad_cuentas="count",
    promedio="mean",
    mediana="median",
    minimo="min",
    maximo="max"
).round(2)

resumen_time
```

time	cantidad_cuentas	promedio	mediana	minimo	maximo
Lunch	68	17.17	15.96	7.51	43.11

time	cantidad_cuentas	promedio	mediana	minimo	maximo
Dinner	176	20.80	18.39	3.07	50.81

La tabla resume los importes de las cuentas según el horario. En `Lunch` hay 68 cuentas registradas, mientras que en `Dinner` hay 176. Esto indica que el dataset tiene muchos más registros de cena que de almuerzo.

También se observan diferencias en los importes:

- el promedio de `total_bill` en `Lunch` es 17.17;
- el promedio de `total_bill` en `Dinner` es 20.80;
- la mediana en `Lunch` es 15.96;
- la mediana en `Dinner` es 18.39.

En este dataset, las cuentas de `Dinner` tienden a tener importes más altos que las cuentas de `Lunch`.

Sin embargo, esta tabla todavía no nos muestra cómo se distribuyen los valores dentro de cada horario. Para eso vamos a usar un boxplot.

Comparar distribuciones con boxplot

Un boxplot permite comparar una variable numérica entre distintos grupos.

En este caso vamos a usar:

- `time` en el eje horizontal;
- `total_bill` en el eje vertical.

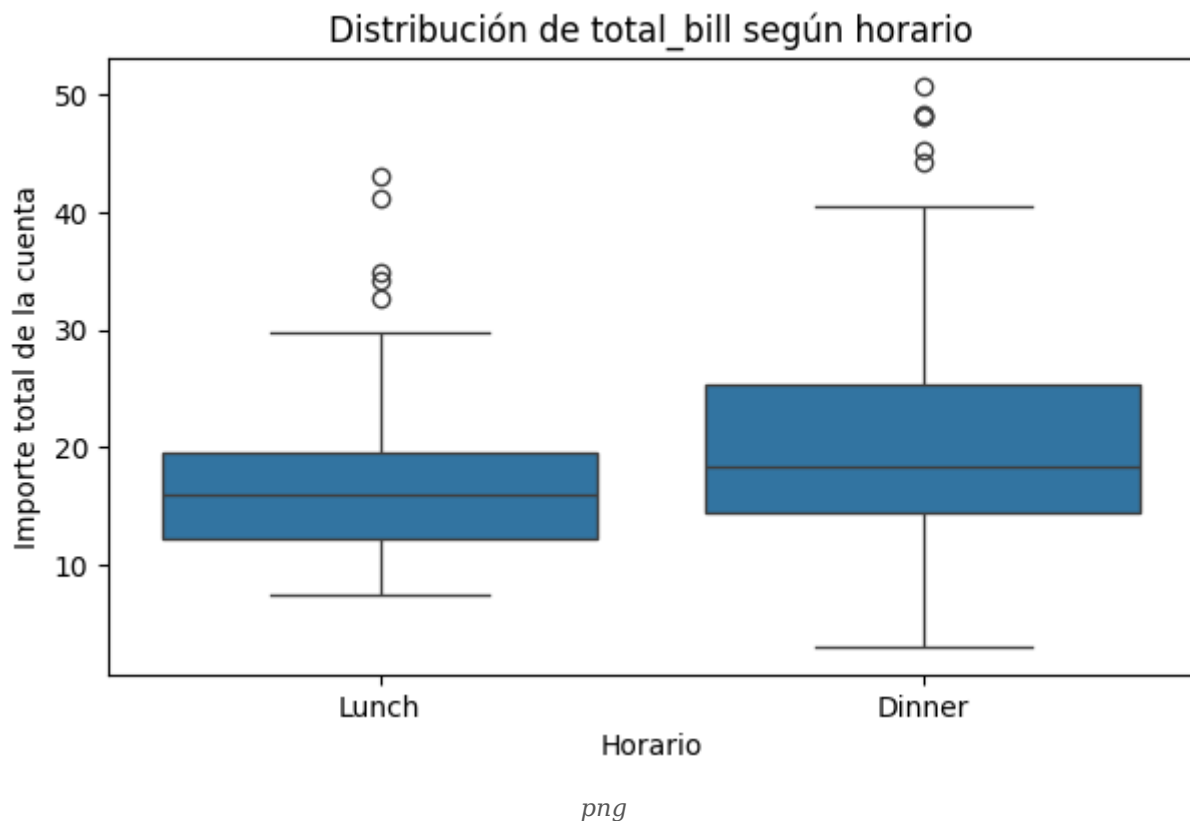
Así podremos comparar la distribución de los importes entre `Lunch` y `Dinner`.

```
plt.figure(figsize=(7, 4))

sns.boxplot(
    data=df,
    x="time",
    y="total_bill"
)

plt.title("Distribución de total_bill según horario")
plt.xlabel("Horario")
plt.ylabel("Importe total de la cuenta")

plt.show()
```



El boxplot compara la distribución de los importes de las cuentas según el horario.

- En `Lunch`, la mediana se ubica alrededor de 15.96.
- En `Dinner`, la mediana es más alta, alrededor de 18.39.

También se observa que la caja de `Dinner` está algo más extendida. Esto indica que la mitad central de las cuentas de cena tiene una mayor dispersión que la mitad central de las cuentas de almuerzo.

Además, en ambos horarios aparecen posibles valores extremos hacia importes altos.

- En `Lunch` hay algunas cuentas por encima de 30 y 40.
- En `Dinner` aparecen varios valores altos, incluso cercanos a 50.

Esta comparación refuerza lo que vimos en la tabla: en este dataset, las cuentas de `Dinner` tienden a ser más altas que las de `Lunch`.

Cuidado al comparar grupos

Cuando comparamos distribuciones entre grupos, no alcanza con mirar solamente qué grupo tiene valores más altos. También tenemos que observar cuántos datos hay en cada grupo.

En este caso, `Dinner` tiene 176 cuentas y `Lunch` tiene 68.

Eso significa que la comparación no está basada en la misma cantidad de registros. La diferencia no invalida el análisis, pero nos recuerda que siempre conviene mirar el tamaño de cada grupo antes de sacar conclusiones.

Ver la cantidad de registros por grupo

Para reforzar esta idea, vamos a contar cuántas cuentas hay en cada horario. Ya vimos ese dato en la tabla de resumen, pero ahora lo vamos a visualizar con un gráfico de barras. La pregunta será:

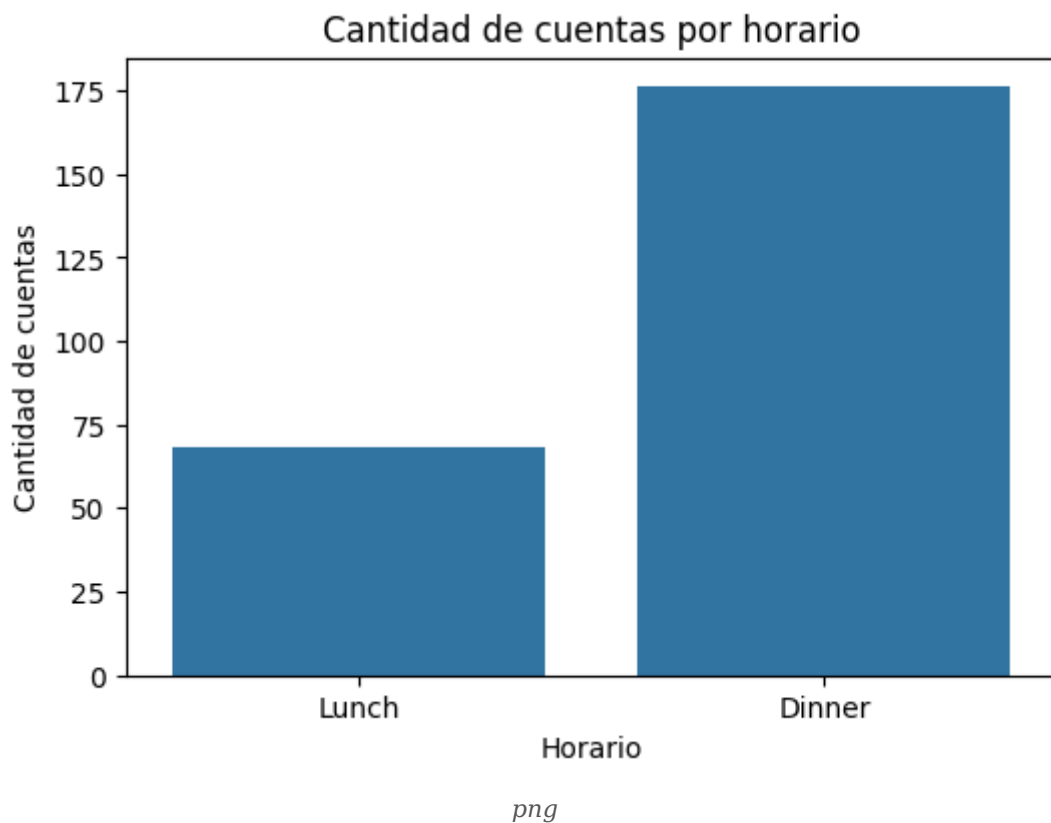
¿cuántas cuentas hay registradas en cada horario?

```
plt.figure(figsize=(6, 4))

sns.countplot(
    data=df,
    x="time"
)

plt.title("Cantidad de cuentas por horario")
plt.xlabel("Horario")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El gráfico muestra la cantidad de cuentas registradas en cada horario. Se observa que `Dinner` tiene muchas más cuentas que `Lunch`.

Esto coincide con la tabla anterior:

- `Lunch`: 68 cuentas;
- `Dinner`: 176 cuentas.

Este dato es importante porque nos ayuda a interpretar mejor la comparación de distribuciones.

Por ejemplo, en el boxplot de `Dinner` aparecen más valores altos y más posibles valores extremos. Eso puede estar relacionado, en parte, con que hay más registros de cena que de almuerzo.

Cuando un grupo tiene muchos más datos, también aumentan las chances de observar valores más variados o extremos. Por eso, al comparar grupos, conviene mirar siempre dos cosas:

- la forma de la distribución; y
- la cantidad de datos que sostiene cada grupo.

Comparar importes según el día

Ahora vamos a hacer una comparación similar, pero usando la columna `day`.

La pregunta será:

¿cómo se distribuyen los importes de las cuentas según el día?

Para eso vamos a usar nuevamente un boxplot.

Como los días registrados pueden leerse siguiendo un orden temporal básico, vamos a definir el orden:

`Thur`, `Fri`, `Sat`, `Sun`

Esto nos permitirá leer el gráfico de manera más clara.

```
orden_dias = ["Thur", "Fri", "Sat", "Sun"]

plt.figure(figsize=(8, 4))

sns.boxplot(
    data=df,
    x="day",
```

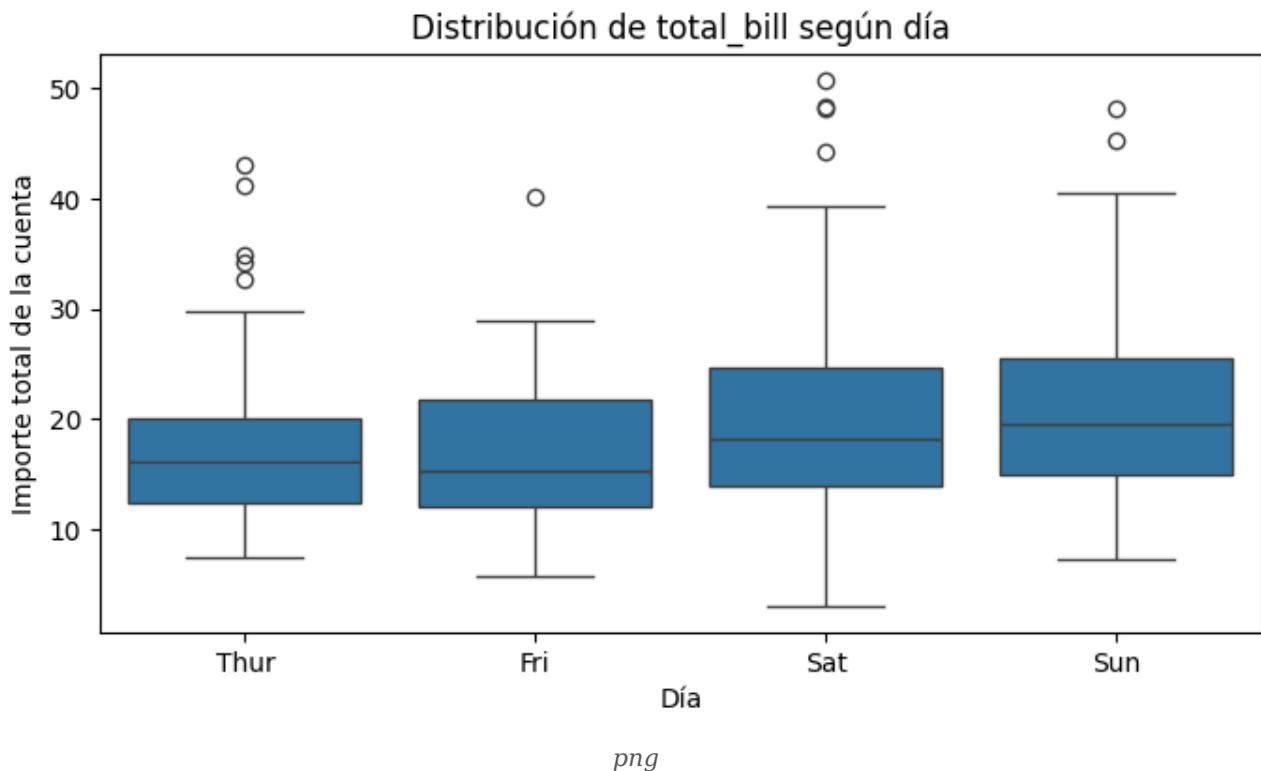
```

    y="total_bill",
    order=orden_dias
)

plt.title("Distribución de total_bill según día")
plt.xlabel("Día")
plt.ylabel("Importe total de la cuenta")

plt.show()

```



El boxplot muestra la distribución de `total_bill` según el día. Podemos observar que las medianas no son iguales en todos los días.

`Thur` y `Fri` tienen medianas más bajas, mientras que `Sat` y `Sun` muestran medianas algo más altas.

También se ve que `Sat` y `Sun` tienen cajas más extendidas. Esto indica una mayor dispersión en la mitad central de los importes.

En varios días aparecen posibles valores extremos hacia importes altos.

Por ejemplo, `Thur` tiene algunos valores por encima de 30 y 40, `Sat` tiene valores cercanos a 50, y `Sun` también muestra algunos importes altos.

La lectura general sugiere que, en este dataset, las cuentas de fin de semana tienden a tener importes más altos que las de `Thur` y `Fri`. Pero antes de sacar conclusiones, conviene mirar cuántas cuentas hay registradas en cada día.

Cantidad de registros por día

Como vimos con `Lunch` y `Dinner`, la cantidad de registros puede influir en la interpretación.

Si un día tiene muchas más cuentas que otro, es más probable que aparezcan valores variados o extremos.

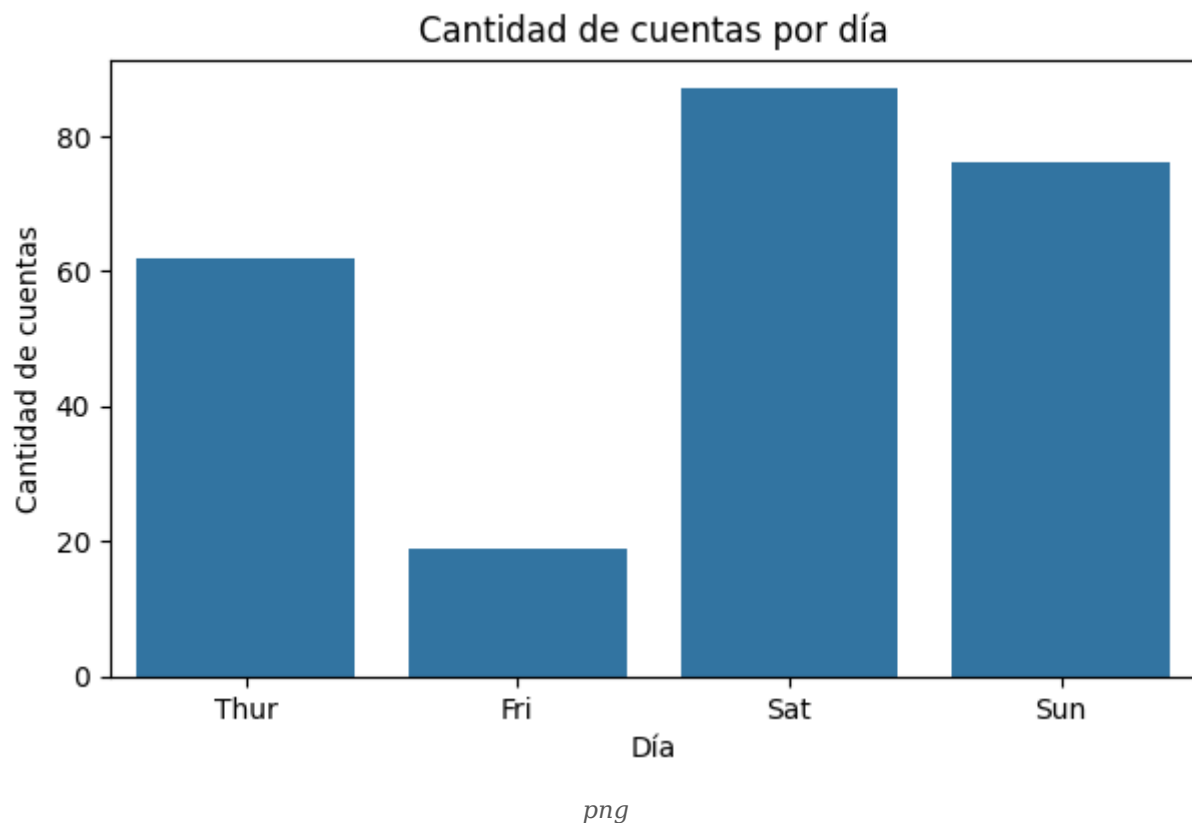
Por eso vamos a visualizar cuántas cuentas hay en cada día.

```
plt.figure(figsize=(7, 4))

sns.countplot(
    data=df,
    x="day",
    order=orden_dias
)

plt.title("Cantidad de cuentas por día")
plt.xlabel("Día")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El gráfico muestra cuántas cuentas hay registradas en cada día.

La cantidad de registros no es pareja:

- **Fri** tiene muy pocas cuentas en comparación con los demás días;
- **Thur** tiene una cantidad intermedia;
- **Sat** es el día con más cuentas registradas;
- **Sun** también tiene una cantidad alta de registros.

Esto es importante para interpretar el boxplot anterior. Por ejemplo, **Sat** y **Sun** muestran más variación y algunos valores altos, pero también son los días con más cuentas registradas.

En cambio, **Fri** tiene pocos datos. Por eso, cualquier conclusión sobre ese día debe tomarse con más cuidado.

Cuando comparamos distribuciones por grupos, siempre conviene mirar juntos:

- la forma de la distribución;
- la posición de la mediana;
- la dispersión de los valores;
- y la cantidad de registros de cada grupo.

Resumir numéricamente la comparación por día

El boxplot nos dio una lectura visual. Ahora vamos a construir una tabla de resumen para comparar esos mismos grupos con algunos indicadores numéricos.

Para cada día vamos a calcular:

- cantidad de cuentas;
- promedio de `total_bill`;
- mediana de `total_bill`;
- valor mínimo;
- valor máximo.

Esto nos permitirá conectar la lectura del gráfico con valores concretos.

```
resumen_dia_total = df.groupby("day", observed=True)["total_bill"].agg(  
    cantidad_cuentas="count",  
    promedio="mean",  
    mediana="median",  
    minimo="min",  
    maximo="max"  
) .round(2)  
  
resumen_dia_total = resumen_dia_total.loc[orden_dias]  
  
resumen_dia_total
```

day	cantidad_cuentas	promedio	mediana	minimo	maximo
Thur	62	17.68	16.20	7.51	43.11
Fri	19	17.15	15.38	5.75	40.17
Sat	87	20.44	18.24	3.07	50.81
Sun	76	21.41	19.63	7.25	48.17

La tabla resume los importes de las cuentas según el día. Podemos observar que `Sat` y `Sun` tienen promedios más altos que `Thur` y `Fri`.

También ocurre algo parecido con la mediana:

- `Thur`: mediana de 16.20;
- `Fri`: mediana de 15.38;
- `Sat`: mediana de 18.24;
- `Sun`: mediana de 19.63.

En este dataset, **Sun** tiene el promedio y la mediana más altos. Sin embargo, también tenemos que mirar la cantidad de cuentas.

Fri tiene solo 19 registros, bastante menos que los demás días. Por eso, sus valores deben interpretarse con más cuidado. En cambio, **Sat** y **Sun** tienen más registros, lo que hace que la comparación sea más estable.

Esta tabla refuerza una idea importante: al comparar distribuciones por grupos, conviene combinar gráficos con resúmenes numéricos.

Comparar otra variable numérica: propinas según horario

Hasta ahora comparamos **total_bill** según distintas categorías. Ahora vamos a aplicar la misma lógica a otra variable numérica: **tip**. La pregunta será:

¿las propinas se distribuyen de la misma manera en **Lunch y en **Dinner**?**

Primero vamos a calcular una tabla de resumen por horario.

```
resumen_time_tip = df.groupby("time", observed=True)["tip"].agg(  
    cantidad_cuentas="count",  
    promedio="mean",  
    mediana="median",  
    minimo="min",  
    maximo="max"  
) .round(2)  
  
resumen_time_tip
```

time	cantidad_cuentas	promedio	mediana	minimo	maximo
Lunch	68	2.73	2.25	1.25	6.7
Dinner	176	3.10	3.00	1.00	10.0

La tabla resume las propinas según el horario.

- En **Lunch**, el promedio de propina es 2.73 y la mediana es 2.25.
- En **Dinner**, el promedio de propina es 3.10 y la mediana es 3.00.

En este dataset, las propinas de **Dinner** tienden a ser algo más altas que las propinas de **Lunch**.

También se observa que el valor máximo en **Dinner** llega a 10.00, mientras que en **Lunch** llega a 6.70.

De todos modos, como ya vimos antes, `Dinner` tiene muchos más registros que `Lunch`. Por eso, además de comparar los valores centrales, conviene observar la distribución completa con un gráfico.

Visualizar propinas según horario

Vamos a usar un boxplot para comparar la distribución de `tip` entre `Lunch` y `Dinner`.

Este gráfico nos permitirá observar:

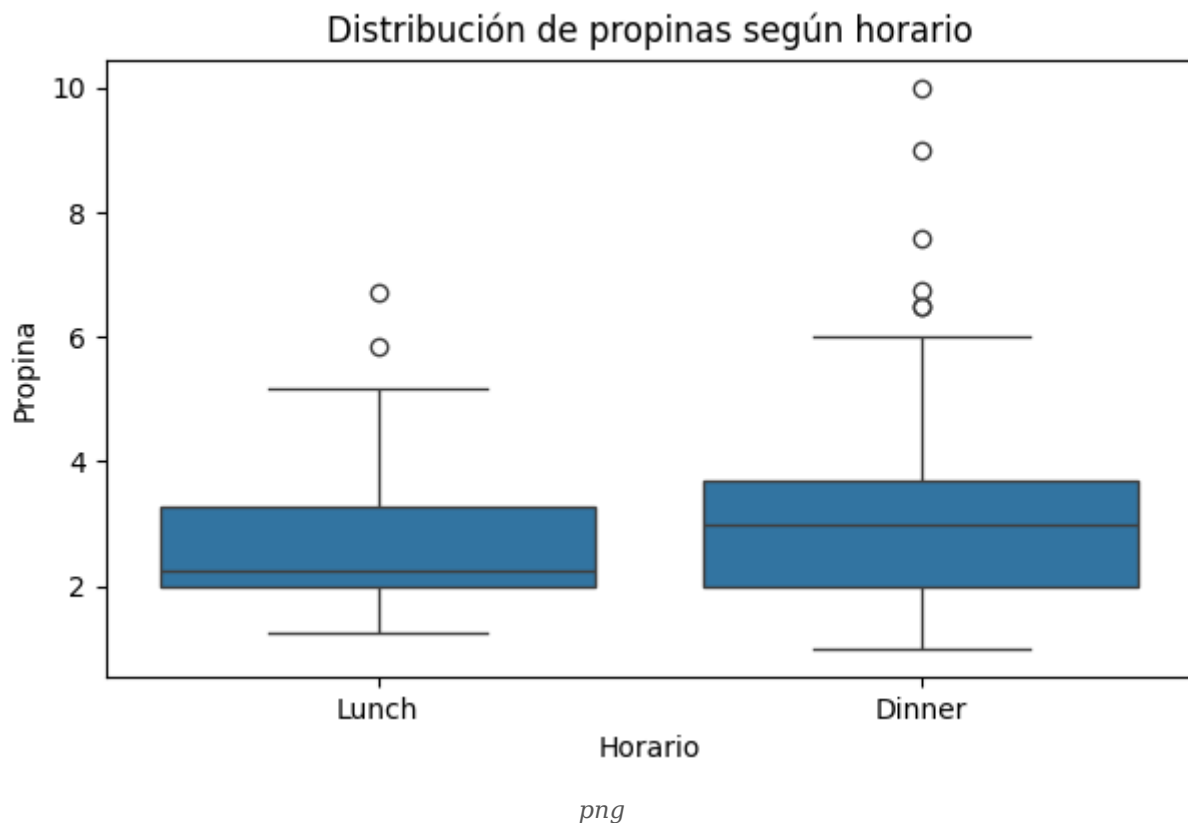
- la mediana de las propinas en cada horario;
- la dispersión de la mitad central de los datos;
- la presencia de posibles valores extremos.

```
plt.figure(figsize=(7, 4))

sns.boxplot(
    data=df,
    x="time",
    y="tip"
)

plt.title("Distribución de propinas según horario")
plt.xlabel("Horario")
plt.ylabel("Propina")

plt.show()
```



El boxplot compara la distribución de las propinas entre `Lunch` y `Dinner`. La mediana de `Dinner` aparece más alta que la mediana de `Lunch`. También se observa que la caja de `Dinner` está algo más extendida, lo que indica mayor dispersión en la mitad central de las propinas. En ambos horarios aparecen posibles valores extremos hacia valores altos.

- En `Lunch` hay algunos valores por encima de 5.
- En `Dinner` aparecen más valores altos, incluyendo propinas cercanas a 9 y 10.

Esta comparación sugiere que, en este dataset, las propinas de `Dinner` tienden a ser un poco más altas y más variables que las propinas de `Lunch`.

De todos modos, esta lectura debe hacerse recordando que `Dinner` tiene muchos más registros que `Lunch`.

Comparar con más de una categoría

Hasta ahora comparamos una variable numérica usando una sola variable categórica.

Por ejemplo:

```
total_bill según time;
```

```
total_bill según day;
```

`tip` según `time`.

Ahora vamos a preparar una comparación un poco más detallada. Queremos observar la distribución de `total_bill` según el día, pero separando también por horario.

Antes de graficar, conviene revisar cuántos registros hay en cada combinación de día y horario.

```
cuentas_dia_horario = pd.crosstab(
    df["day"],
    df["time"]
)

cuentas_dia_horario = cuentas_dia_horario.loc[orden_dias]

cuentas_dia_horario
```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

La tabla muestra cuántas cuentas hay en cada combinación de día y horario. La distribución no es pareja:

- `Thur` tiene casi todos sus registros en `Lunch` y solo 1 cuenta en `Dinner`;
- `Fri` tiene pocos registros en ambos horarios, con 7 en `Lunch` y 12 en `Dinner`;
- `Sat` tiene registros solo en `Dinner`;
- `Sun` tiene registros solo en `Dinner`.

Esto es muy importante.

Si hacemos un gráfico separando por día y horario, no todos los grupos estarán igualmente representados. Por ejemplo, no tendría sentido comparar seriamente el `Dinner` de `Thur` con otros grupos, porque tiene una sola cuenta. Tampoco aparecerán valores de `Lunch` para `Sat` y `Sun`, porque no hay registros en esas combinaciones.

Esta tabla nos recuerda que un gráfico puede mostrar diferencias, pero siempre tenemos que revisar cuántos datos hay detrás de cada comparación.

Boxplot con día y horario

Ahora vamos a construir un boxplot de `total_bill` según el día, separando además por horario. Para eso vamos a usar el argumento `hue`.

En este caso:

- `x="day"` ubica los días en el eje horizontal;
- `y="total_bill"` muestra el importe total de la cuenta;
- `hue="time"` separa cada día según `Lunch` y `Dinner`.

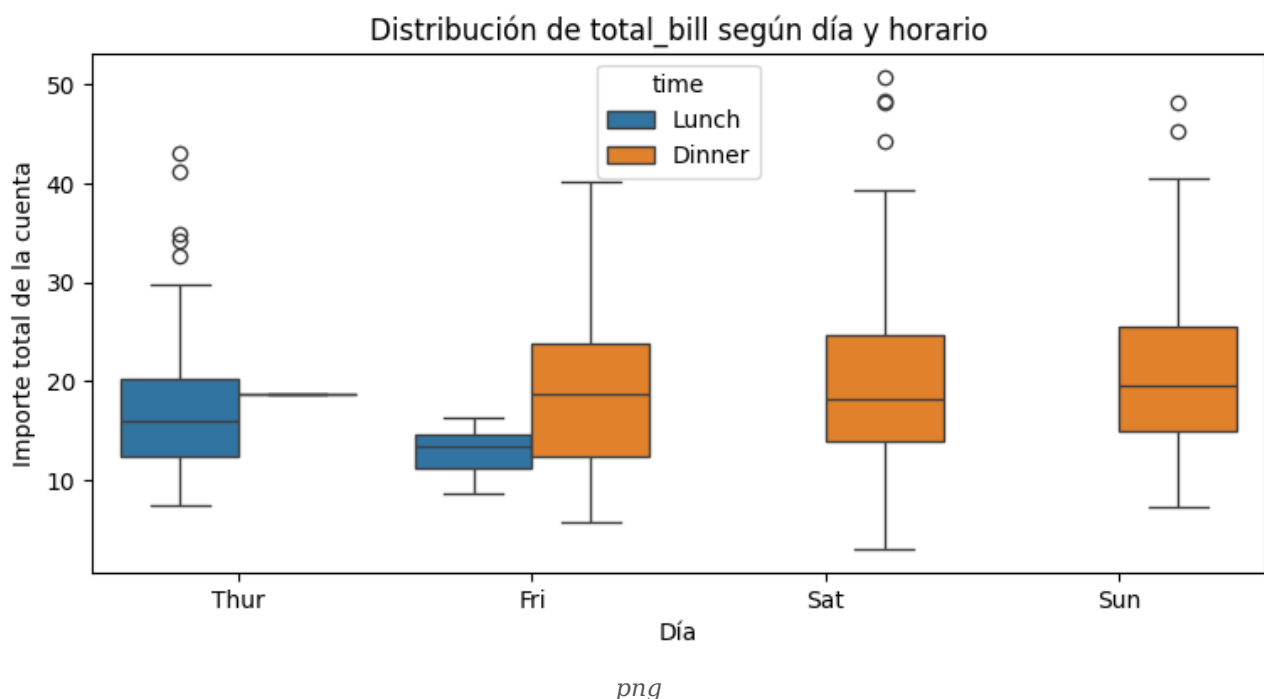
Este gráfico nos va a permitir comparar más de una categoría al mismo tiempo, pero debemos leerlo junto con la tabla anterior.

```
plt.figure(figsize=(9, 4))

sns.boxplot(
    data=df,
    x="day",
    y="total_bill",
    hue="time",
    order=orden_dias
)

plt.title("Distribución de total_bill según día y horario")
plt.xlabel("Día")
plt.ylabel("Importe total de la cuenta")

plt.show()
```



El boxplot compara `total_bill` según el día y, además, separa cada día por horario. La lectura debe hacerse junto con la tabla anterior, porque no todas las combinaciones tienen la misma cantidad de datos.

En `Thur`, casi todos los registros corresponden a `Lunch`. La caja azul representa bien ese grupo porque tiene 61 cuentas. En cambio, el valor de `Dinner` para `Thur` corresponde a una sola cuenta, por eso no conviene interpretarlo como una distribución.

En `Fri`, aparecen datos tanto para `Lunch` como para `Dinner`, pero ambos grupos tienen pocos registros. La comparación puede observarse, pero con cautela.

En `Sat` y `Sun`, solo aparecen registros de `Dinner`. Por eso no hay cajas de `Lunch` para esos días.

El gráfico muestra que las cuentas de `Dinner` en `Sat` y `Sun` tienden a ubicarse en importes más altos que los almuerzos de `Thur` y `Fri`.

Pero la conclusión más importante no es solo esa diferencia, sino la forma de leer el gráfico: cuando usamos más de una categoría, siempre tenemos que revisar cuántos datos hay en cada combinación.

Leer hue en un boxplot

En este gráfico volvimos a usar el argumento `hue`.

La instrucción principal fue:

```
sns.boxplot(  
    data=df,  
    x="day",  
    y="total_bill",  
    hue="time",  
    order=orden_dias  
)
```

El argumento `x="day"` ubica los días en el eje horizontal.

El argumento `y="total_bill"` indica que queremos comparar los importes de las cuentas.

El argumento `hue="time"` divide cada día según el horario: `Lunch` y `Dinner`.

Por eso, dentro de cada día, Seaborn intenta mostrar una distribución para cada horario.

Si una combinación no tiene registros, no aparece una caja.

Si una combinación tiene muy pocos registros, el gráfico puede mostrar algo, pero la interpretación debe ser cuidadosa.

Ver los datos individuales

El boxplot resume una distribución, pero no muestra cada dato de manera individual. Cuando algunos grupos tienen pocos registros, puede ser útil ver los puntos originales. Para eso podemos usar un gráfico de puntos con `stripplot`.

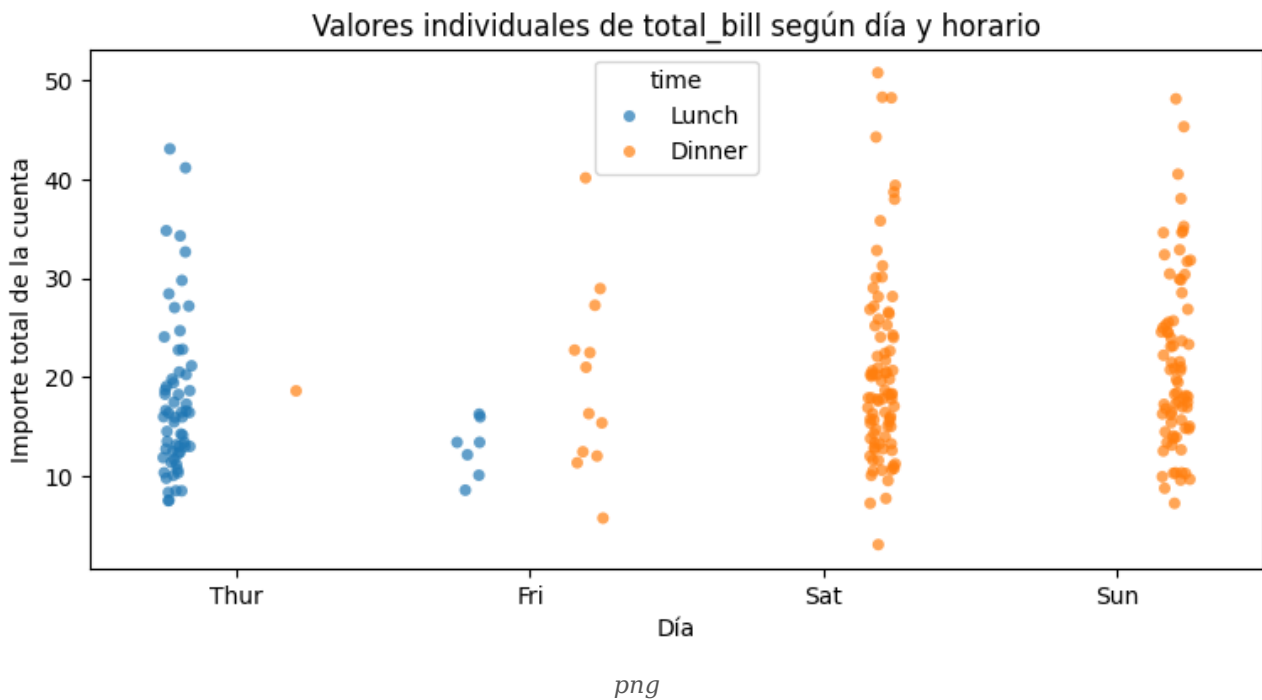
Este gráfico no reemplaza al boxplot, pero nos ayuda a ver cuántas observaciones hay en cada grupo y cómo se ubican.

```
plt.figure(figsize=(9, 4))

sns.stripplot(
    data=df,
    x="day",
    y="total_bill",
    hue="time",
    order=orden_dias,
    dodge=True,
    jitter=True,
    alpha=0.7
)

plt.title("Valores individuales de total_bill según día y horario")
plt.xlabel("Día")
plt.ylabel("Importe total de la cuenta")

plt.show()
```



El gráfico de puntos muestra los valores individuales de `total_bill` según día y horario. Cada punto representa una cuenta.

Esta visualización permite ver con más claridad cuántos datos hay detrás de cada grupo.

Por ejemplo:

- en `Thur` se observa una gran concentración de puntos azules, correspondientes a `Lunch`;
- en `Thur` aparece un solo punto naranja, correspondiente a `Dinner`;
- en `Fri` hay pocos puntos tanto en `Lunch` como en `Dinner`;
- en `Sat` y `Sun` aparecen solamente puntos de `Dinner`;
- no hay puntos de `Lunch` en `Sat` ni en `Sun`, porque no existen registros para esas combinaciones.

Este gráfico refuerza una idea importante: cuando usamos un boxplot, vemos un resumen de la distribución, pero no vemos directamente la cantidad de observaciones. El `stripplot` nos ayuda a recuperar esa información visualmente.

Por eso, cuando los grupos tienen pocos datos o cantidades muy diferentes, puede ser útil combinar gráficos de resumen con gráficos que muestren los datos individuales.

Cómo leer `stripplot`

En la celda anterior usamos `sns.stripplot()`.

La instrucción principal fue:

```
sns.stripplot(  
    data=df,  
    x="day",  
    y="total_bill",  
    hue="time",  
    order=orden_dias,  
    dodge=True,  
    jitter=True,  
    alpha=0.7  
)
```

El argumento `x="day"` ubica los días en el eje horizontal.

El argumento `y="total_bill"` ubica el importe de cada cuenta en el eje vertical.

El argumento `hue="time"` colorea los puntos según el horario.

El argumento `dodge=True` separa visualmente los puntos de `Lunch` y `Dinner` dentro de cada día.

El argumento `jitter=True` agrega una pequeña separación horizontal entre puntos para que no queden todos encimados.

El argumento `alpha=0.7` hace que los puntos sean un poco transparentes. Esto ayuda a leer zonas donde hay muchos puntos cercanos.

Cierre del capítulo

En este capítulo comparamos distribuciones de variables numéricas entre grupos:

- Primero analizamos `total_bill` según `time`.
- Después comparamos `total_bill` según `day`.
- Luego analizamos `tip` según `time`.
- Finalmente combinamos dos categorías: `day` y `time`.

En todos los casos seguimos una idea central: no alcanza con mirar si un grupo tiene valores más altos que otro. También necesitamos observar:

- la mediana;
- la dispersión;
- los posibles valores extremos;
- y la cantidad de datos disponible en cada grupo.

Esto es especialmente importante cuando algunos grupos tienen muchos registros y otros tienen muy pocos.

Usamos principalmente boxplots para comparar variables numéricas según categorías. También usamos gráficos de barras para revisar la cantidad de registros por grupo, y un `stripplot` para visualizar los datos individuales.

La idea más importante fue que una comparación entre grupos debe leerse con cuidado. Un gráfico puede sugerir diferencias, pero esas diferencias deben interpretarse teniendo en cuenta cuántos datos hay detrás de cada grupo.

En el próximo capítulo vamos a seguir trabajando con visualizaciones, pero avanzando hacia gráficos que permiten observar relaciones entre variables numéricas.

Capítulo 36 — Visualizar relaciones entre variables numéricas

Observar relaciones entre columnas

En el capítulo anterior comparamos distribuciones de variables numéricas entre distintos grupos. Por ejemplo, analizamos cómo cambiaban `total_bill` y `tip` según el horario o el día.

Ahora vamos a mirar otro tipo de relación. En lugar de comparar una variable numérica según una categoría, vamos a observar dos variables numéricas al mismo tiempo. La pregunta principal será:

¿hay alguna relación entre el importe total de la cuenta y la propina?

Para responderla vamos a trabajar con dos columnas del dataset `tips`:

- `total_bill`, que representa el importe total de la cuenta;
- `tip`, que representa el monto de la propina.

La idea será construir gráficos que nos permitan ver si, cuando una variable aumenta, la otra también tiende a aumentar.

Cargar el dataset

Como en los capítulos anteriores, vamos a trabajar con el dataset `tips`. Primero importamos las librerías necesarias y cargamos los datos.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2

índice	total_bill	tip	sex	smoker	day	time	size
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset `tips`. Cada fila representa una cuenta registrada en un restaurante. En este capítulo vamos a observar especialmente dos columnas numéricas:

- `total_bill`, que indica el importe total de la cuenta;
- y `tip`, que indica el monto de la propina.

La pregunta inicial será:

¿las cuentas más altas suelen estar asociadas con propinas más altas?

Para empezar a responder, vamos a construir un gráfico de dispersión.

Primer gráfico de relación: scatterplot

Un gráfico de dispersión, o `scatterplot`, permite visualizar la relación entre dos variables numéricas. Cada punto del gráfico representa una fila del dataset.

En nuestro caso:

- el eje horizontal mostrará `total_bill`;
- el eje vertical mostrará `tip`;
- cada punto representará una cuenta.

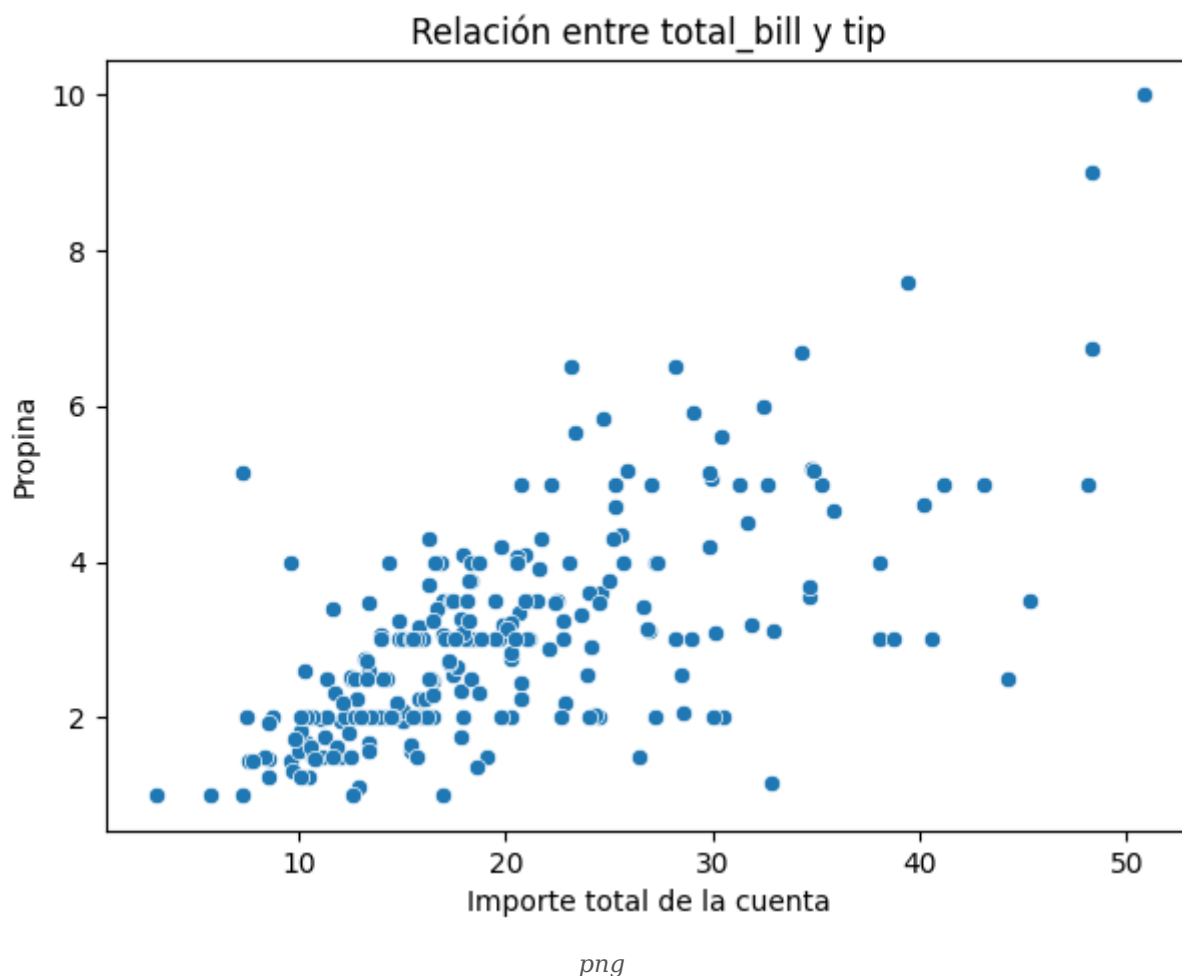
Si los puntos tienden a subir a medida que avanzan hacia la derecha, eso puede sugerir una relación positiva entre las variables.

```
plt.figure(figsize=(7, 5))

sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip"
)

plt.title("Relación entre total_bill y tip")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Propina")

plt.show()
```



El gráfico de dispersión muestra la relación entre el importe total de la cuenta y la propina. Cada punto representa una cuenta del dataset.

En el eje horizontal aparece `total_bill`, es decir, el importe total de la cuenta.

En el eje vertical aparece `tip`, es decir, el monto de la propina.

La nube de puntos sugiere una relación positiva: en general, cuando el importe de la cuenta aumenta, la propina también tiende a aumentar. Sin embargo, la relación no es perfecta.

Para importes similares, pueden aparecer propinas diferentes. Por ejemplo, hay cuentas cercanas a 20 con propinas que van aproximadamente desde 2 hasta 5.

Esto nos recuerda que una relación entre variables no significa que todos los puntos sigan exactamente el mismo patrón. En datos reales, suele haber variación.

Leer un gráfico de dispersión

En un `scatterplot`, cada punto representa una observación.

La posición horizontal del punto depende del valor de una variable.

La posición vertical depende del valor de otra variable.

En este caso:

- si un punto está más a la derecha, representa una cuenta con mayor `total_bill`;
- si un punto está más arriba, representa una propina mayor;
- si muchos puntos forman una dirección ascendente, puede haber una relación positiva entre las variables.

Una relación positiva significa que, cuando una variable aumenta, la otra tiende a aumentar también.

Pero todavía estamos haciendo una lectura visual. Para complementar esa lectura, podemos calcular una medida numérica de relación.

Medir la relación con correlación

Una forma de resumir la relación lineal entre dos variables numéricas es calcular la correlación. La correlación puede tomar valores entre -1 y 1.

De manera general:

- valores cercanos a 1 indican una relación positiva fuerte;
- valores cercanos a -1 indican una relación negativa fuerte;
- valores cercanos a 0 indican poca relación lineal.

Vamos a calcular la correlación entre `total_bill` y `tip`.

```
correlacion_total_tip = df["total_bill"].corr(df["tip"])

round(float(correlacion_total_tip), 2)
```

```
0.68
```

La correlación entre `total_bill` y `tip` es aproximadamente 0.68. Este valor es positivo, por lo que coincide con lo que vimos en el gráfico: cuando el importe total de la cuenta aumenta, la propina tiende a aumentar. No es una correlación perfecta, porque no vale 1.

Esto también coincide con el gráfico de dispersión: los puntos muestran una tendencia ascendente, pero no están ubicados exactamente sobre una línea.

En términos prácticos, podemos decir que en este dataset hay una relación positiva moderada entre el importe total de la cuenta y la propina.

Es importante recordar que la correlación resume una relación lineal. Puede ser útil, pero no reemplaza la visualización.

Agregar una línea de tendencia

Para observar mejor la dirección general de la relación, podemos agregar una línea de tendencia.

Seaborn tiene una función llamada `regplot()` que combina:

- un gráfico de dispersión;
- una línea de tendencia.

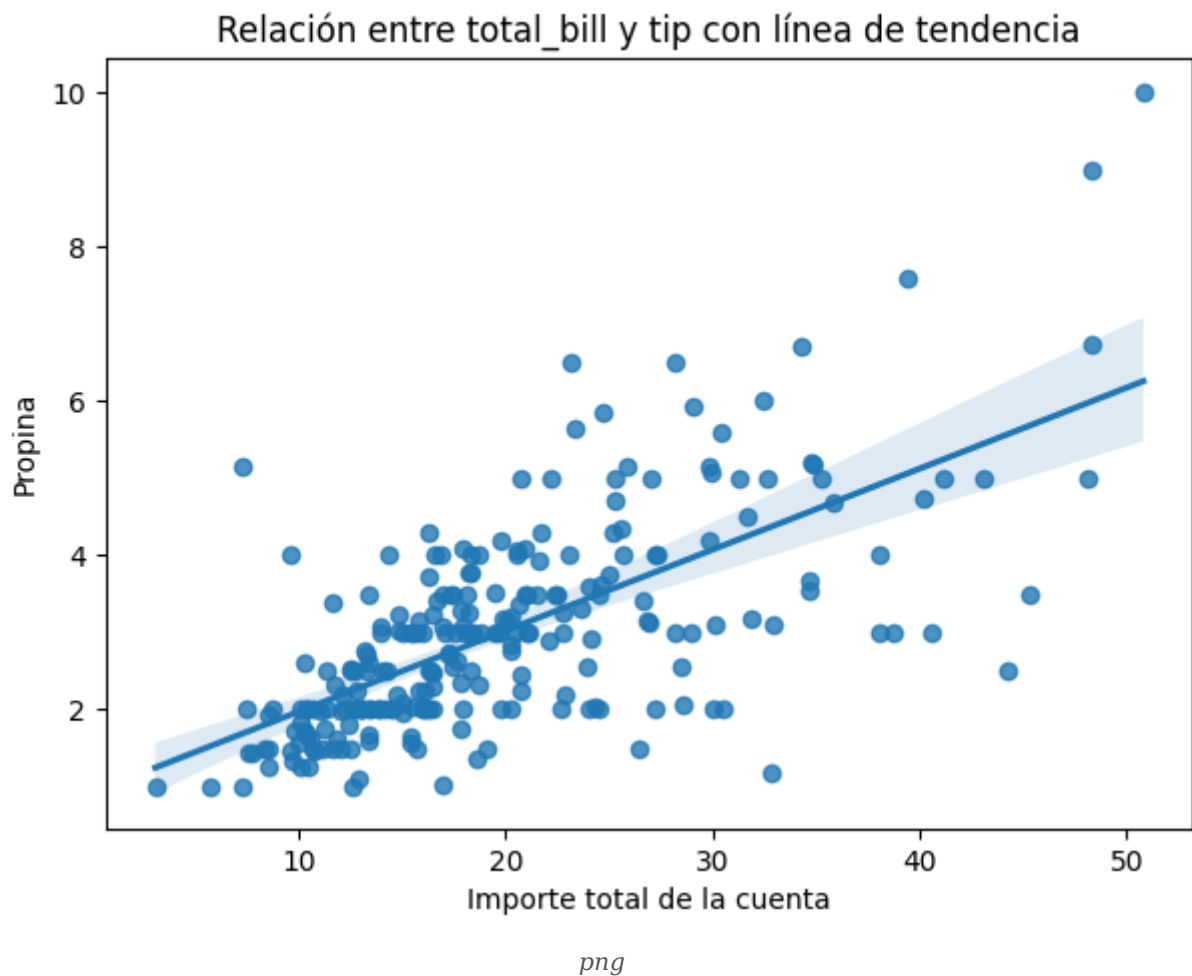
Esto nos ayuda a ver si la relación general entre las variables es ascendente, descendente o casi plana.

```
plt.figure(figsize=(7, 5))

sns.regplot(
    data=df,
    x="total_bill",
    y="tip"
)

plt.title("Relación entre total_bill y tip con línea de tendencia")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Propina")

plt.show()
```



El gráfico vuelve a mostrar la relación entre `total_bill` y `tip`, pero ahora agrega una línea de tendencia. La línea tiene una inclinación ascendente.

Esto refuerza la idea de que existe una relación positiva entre ambas variables: a medida que aumenta el importe total de la cuenta, la propina tiende a aumentar.

También se observa una banda clara alrededor de la línea. Esa banda representa una zona de incertidumbre o variación alrededor de la tendencia estimada. Es importante notar que la línea no pasa exactamente por todos los puntos. Esto ocurre porque la relación no es perfecta. Hay cuentas con importes parecidos que tienen propinas diferentes, y también hay algunos puntos alejados del patrón general.

La línea de tendencia resume la dirección general, pero no elimina la variabilidad de los datos.

Cómo leer `regplot`

En la celda anterior usamos `sns.regplot()`.

La instrucción principal fue:

```
sns.regplot(  
    data=df,  
    x="total_bill",  
    y="tip"  
)
```

Esta función combina dos elementos: un gráfico de dispersión y una línea de tendencia.

El argumento `data=df` indica qué DataFrame vamos a usar.

El argumento `x="total_bill"` ubica el importe total de la cuenta en el eje horizontal.

El argumento `y="tip"` ubica la propina en el eje vertical.

Cuando la línea sube hacia la derecha, la relación general es positiva.

Cuando la línea baja hacia la derecha, la relación general es negativa.

Cuando la línea queda casi horizontal, la relación lineal es débil o casi inexistente.

Agregar una variable categórica con color

Hasta ahora analizamos la relación entre `total_bill` y `tip` usando solo dos variables numéricas. Ahora vamos a sumar una variable categórica para enriquecer la lectura. Podemos usar el color de los puntos para distinguir grupos.

Por ejemplo, vamos a colorear los puntos según el horario de la cuenta:

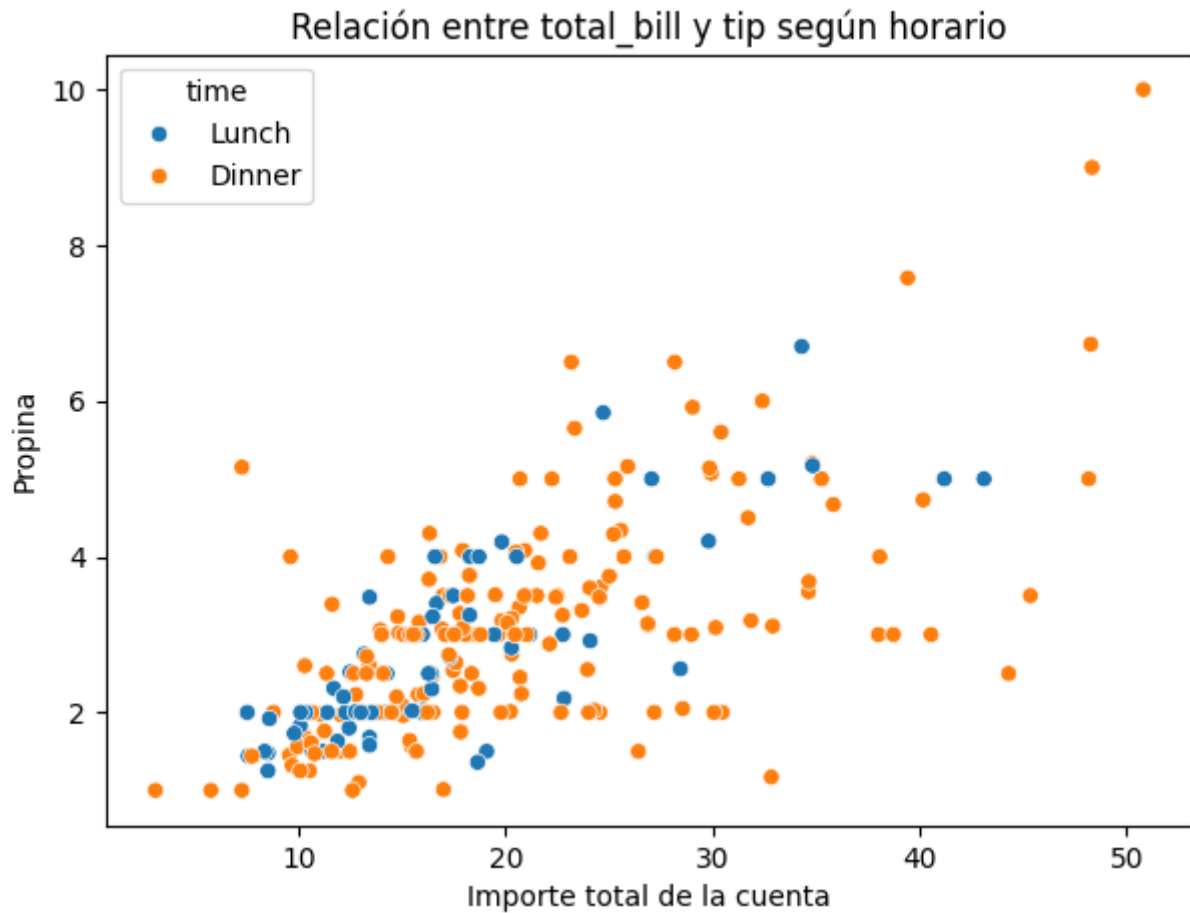
- `Lunch`;
- `Dinner`.

La pregunta será:

¿la relación entre importe total y propina se ve parecida en almuerzo y cena?

```
plt.figure(figsize=(7, 5))  
  
sns.scatterplot(  
    data=df,  
    x="total_bill",  
    y="tip",  
    hue="time"  
)  
  
plt.title("Relación entre total_bill y tip según horario")  
plt.xlabel("Importe total de la cuenta")  
plt.ylabel("Propina")
```

```
plt.show()
```



png

El gráfico muestra la relación entre `total_bill` y `tip`, separando los puntos por horario.

- Los puntos azules corresponden a `Lunch`.
- Los puntos naranjas corresponden a `Dinner`.

La relación positiva general se mantiene: en ambos horarios, las cuentas más altas tienden a asociarse con propinas más altas.

También se observa que hay muchos más puntos de `Dinner` que de `Lunch`, algo que ya habíamos visto en capítulos anteriores.

Además, varios de los valores más altos de `total_bill` y `tip` aparecen en `Dinner`.

Esto no significa que todas las cenas tengan importes o propinas altos, sino que en este dataset los valores más grandes se observan principalmente en ese horario.

El color nos ayuda a ver si la relación general se comporta de manera parecida o diferente entre grupos.

Leer `hue` en un `scatterplot`

En la celda anterior usamos nuevamente el argumento `hue`.

La instrucción principal fue:

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="time"
)
```

El argumento `x="total_bill"` ubica el importe total de la cuenta en el eje horizontal.

El argumento `y="tip"` ubica la propina en el eje vertical.

El argumento `hue="time"` colorea los puntos según el horario.

Esto permite sumar una tercera variable al gráfico. Ya no miramos solamente la relación entre dos variables numéricas, sino que también podemos observar si esa relación cambia entre categorías.

Calcular la correlación por horario

Como agregamos la variable `time`, podemos preguntarnos si la relación entre `total_bill` y `tip` es igual en `Lunch` y en `Dinner`. Para explorarlo, vamos a calcular la correlación por separado para cada horario. Esto nos permitirá comparar la relación numérica dentro de cada grupo.

```
correlacion_lunch = df.loc[df["time"] == "Lunch", "total_bill"].corr(
    df.loc[df["time"] == "Lunch", "tip"]
)

correlacion_dinner = df.loc[df["time"] == "Dinner", "total_bill"].corr(
    df.loc[df["time"] == "Dinner", "tip"]
)

correlacion_por_horario = pd.DataFrame({
    "horario": ["Lunch", "Dinner"],
    "correlacion_total_bill_tip": [correlacion_lunch, correlacion_dinner]
})

correlacion_por_horario["correlacion_total_bill_tip"] =
```

```
correlacion_por_horario["correlacion_total_bill_tip"].round(2)

correlacion_por_horario
```

índice	horario	correlacion_total_bill_tip
0	Lunch	0.81
1	Dinner	0.63

La tabla muestra la correlación entre `total_bill` y `tip` calculada por separado para cada horario.

- En `Lunch`, la correlación es 0.81.
- En `Dinner`, la correlación es 0.63.

En ambos casos la correlación es positiva. Esto significa que, tanto en almuerzo como en cena, las cuentas más altas tienden a estar asociadas con propinas más altas.

Sin embargo, la correlación es más alta en `Lunch` que en `Dinner`.

Esto sugiere que, dentro de los almuerzos, la relación lineal entre importe total y propina es más fuerte que dentro de las cenas.

De todos modos, debemos interpretar esta comparación con cuidado, porque no hay la misma cantidad de datos en ambos grupos: `Lunch` tiene menos registros que `Dinner`.

Cuidado: correlación no significa causalidad

La correlación nos ayuda a describir una relación entre dos variables numéricas. Pero es importante hacer una aclaración: **correlación no significa causalidad**.

En este caso, vemos que `total_bill` y `tip` tienen una relación positiva. Eso no alcanza para afirmar, de manera general, que una variable sea la única causa de la otra. Lo que sí podemos decir, de forma prudente, es que en este dataset las cuentas de mayor importe suelen estar asociadas con propinas más altas.

La correlación resume una relación, pero no explica por sí sola todos los motivos detrás de esa relación.

Otra relación: importe total y tamaño del grupo

Además de `total_bill` y `tip`, el dataset tiene otra columna numérica interesante: `size`. La columna `size` indica la cantidad de personas asociadas a la cuenta. Podemos preguntarnos:

¿las cuentas de grupos más grandes tienden a tener importes más altos?

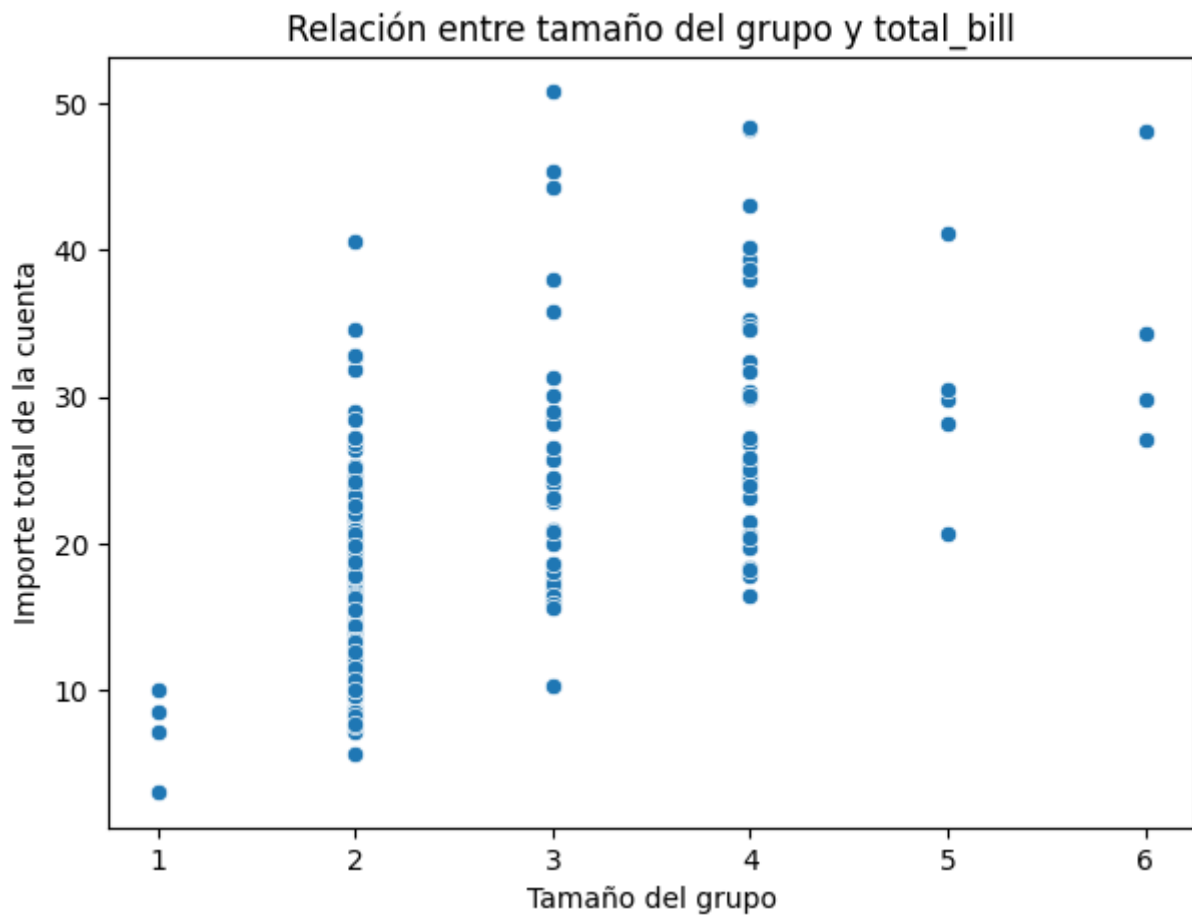
Para explorar esta relación vamos a construir otro gráfico de dispersión.

```
plt.figure(figsize=(7, 5))

sns.scatterplot(
    data=df,
    x="size",
    y="total_bill"
)

plt.title("Relación entre tamaño del grupo y total_bill")
plt.xlabel("Tamaño del grupo")
plt.ylabel("Importe total de la cuenta")

plt.show()
```



png

El gráfico muestra la relación entre el tamaño del grupo y el importe total de la cuenta.

En el eje horizontal aparece `size`, que indica la cantidad de personas asociadas a la cuenta.

En el eje vertical aparece `total_bill`, el importe total de la cuenta.

A diferencia de `total_bill` y `tip`, la variable `size` solo toma algunos valores enteros: 1, 2, 3, 4, 5 y 6. Por eso los puntos aparecen agrupados en columnas verticales.

La lectura general sugiere que los grupos más grandes tienden a tener cuentas más altas. Sin embargo, también se observa bastante variación dentro de cada tamaño de grupo. Por ejemplo, para grupos de 2 personas hay cuentas de importes muy distintos. Lo mismo ocurre con grupos de 3 y 4 personas.

Esto muestra que el tamaño del grupo puede estar relacionado con el importe total, pero no lo explica completamente.

Cuando una variable numérica tiene pocos valores

Aunque `size` es una variable numérica, no se comporta igual que `total_bill` o `tip`. `total_bill` y `tip` pueden tomar muchos valores diferentes. En cambio, `size` representa una cantidad de personas y solo toma algunos valores enteros.

Cuando usamos un scatterplot con una variable de este tipo, muchos puntos pueden quedar encimados en las mismas posiciones del eje horizontal. Por eso, antes de seguir interpretando, conviene mirar cuántos registros hay para cada tamaño de grupo.

```
conteo_size = df["size"].value_counts().sort_index()

conteo_size
```

size	count
1	4
2	156
3	38
4	37
5	5
6	4

`dtype: int64`

La tabla muestra cuántas cuentas hay para cada valor de `size`. La mayoría de las cuentas corresponde a grupos de 2 personas: hay 156 registros.

Luego aparecen grupos de 3 y 4 personas, con cantidades bastante menores. En cambio, los grupos de 1, 5 y 6 personas tienen muy pocos registros. Esto es importante para interpretar el gráfico anterior. Aunque parece haber una relación positiva entre `size` y `total_bill`, no todos los tamaños de grupo tienen la misma cantidad de datos.

Por ejemplo, hay muy pocos casos con `size` igual a 5 o 6. Por eso, las conclusiones sobre esos tamaños deben tomarse con cuidado.

Una vez más, la cantidad de registros por grupo nos ayuda a leer mejor la visualización.

Calcular la correlación entre `size` y `total_bill`

Ahora vamos a calcular la correlación entre `size` y `total_bill`. Esto nos permitirá resumir numéricamente la relación que vimos en el gráfico.

De todos modos, debemos recordar algo importante: como `size` tiene pocos valores posibles, la correlación debe interpretarse junto con el gráfico y con el conteo de registros por tamaño de grupo.

```
correlacion_size_total = df["size"].corr(df["total_bill"])

round(float(correlacion_size_total), 2)
```

0.6

La correlación entre `size` y `total_bill` es aproximadamente 0.60. Este valor es positivo. Eso significa que, en este dataset, los grupos más grandes tienden a estar asociados con cuentas de mayor importe. Esta lectura coincide con el gráfico de dispersión anterior.

Sin embargo, la relación no es perfecta. Dentro de un mismo tamaño de grupo hay cuentas con importes muy diferentes. Por ejemplo, entre los grupos de 2 personas aparecen cuentas bajas, medias y algunas bastante altas.

Además, algunos tamaños de grupo tienen muy pocos registros, como 1, 5 y 6. Por eso, esta correlación es útil como resumen, pero debe interpretarse junto con la visualización y con el conteo de registros.

Mejorar la visualización con jitter

En el gráfico anterior, muchos puntos quedaron alineados verticalmente porque `size` tiene pocos valores posibles. Podemos mejorar la visualización agregando una pequeña variación horizontal a los puntos. A esa pequeña variación se la suele llamar `jitter`.

El `jitter` no cambia los datos reales, solo separa un poco los puntos en el gráfico para que se vean mejor.

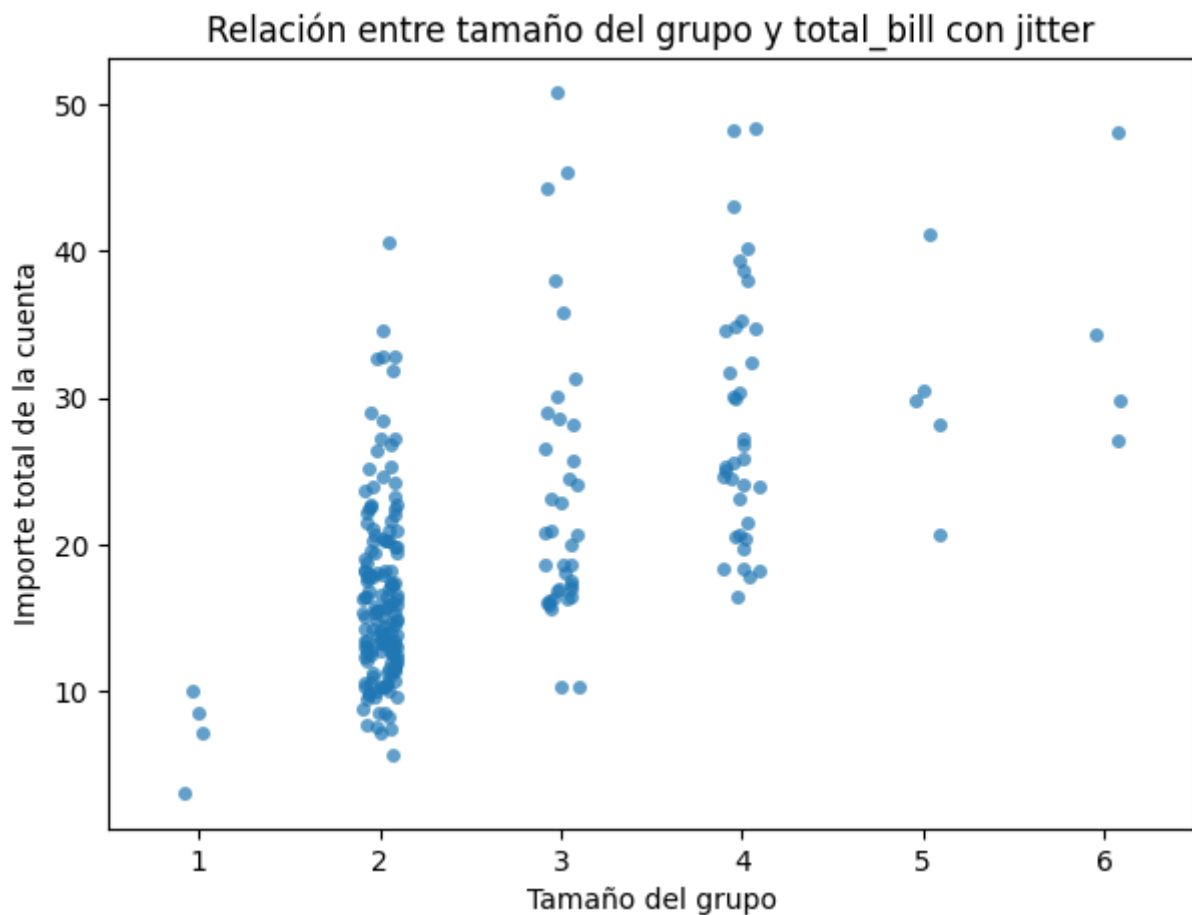
Para eso vamos a usar `stripplot`, que ya usamos en el capítulo anterior.

```
plt.figure(figsize=(7, 5))

sns.stripplot(
    data=df,
    x="size",
    y="total_bill",
    jitter=True,
    alpha=0.7
)

plt.title("Relación entre tamaño del grupo y total_bill con jitter")
plt.xlabel("Tamaño del grupo")
plt.ylabel("Importe total de la cuenta")

plt.show()
```



png

El gráfico con `jitter` muestra la relación entre `size` y `total_bill` de una manera más legible. Cada punto sigue representando una cuenta.

La diferencia es que ahora los puntos tienen una pequeña separación horizontal dentro de cada valor de `size`. Esto permite ver mejor cuántos registros hay en cada tamaño de grupo. Por ejemplo, se nota claramente que hay muchos registros con `size = 2`.

También se observa que los tamaños 1, 5 y 6 tienen pocos puntos, tal como mostraba la tabla de conteo.

La lectura general se mantiene: los grupos más grandes tienden a tener cuentas más altas, pero hay bastante variación dentro de cada tamaño.

El `jitter` no cambia los valores reales de `size`. Solo ayuda a evitar que muchos puntos queden completamente encimados en el gráfico.

Comparar varias correlaciones

Hasta ahora calculamos correlaciones entre pares de variables:

`total_bill` y `tip`;

`size` y `total_bill`.

Pero también podemos calcular una matriz de correlación entre todas las variables numéricas del dataset.

En este caso, vamos a seleccionar:

- `total_bill`;
- `tip`;
- `size`.

La matriz de correlación nos permitirá ver, en una sola tabla, cómo se relacionan estas variables entre sí.

```
columnas_numericas = ["total_bill", "tip", "size"]  
  
matriz_correlacion = df[columnas_numericas].corr().round(2)  
  
matriz_correlacion
```

índice	total_bill	tip	size
total_bill	1.00	0.68	0.60
tip	0.68	1.00	0.49
size	0.60	0.49	1.00

La matriz de correlación muestra la relación entre cada par de variables numéricas. La diagonal principal tiene valores iguales a 1.00 porque cada variable tiene correlación perfecta consigo misma.

Fuera de la diagonal podemos leer las relaciones entre variables distintas:

- `total_bill` y `tip` tienen una correlación de 0.68;
- `total_bill` y `size` tienen una correlación de 0.60;
- `tip` y `size` tienen una correlación de 0.49.

Las tres correlaciones son positivas.

Esto indica que, en este dataset:

- las cuentas más altas tienden a asociarse con propinas más altas;
- los grupos más grandes tienden a asociarse con cuentas más altas;

y los grupos más grandes también tienden a asociarse con propinas más altas, aunque esta relación es más débil que las anteriores.

Como siempre, la correlación resume una relación lineal, pero no reemplaza la interpretación visual ni el conocimiento del contexto.

Visualizar la matriz de correlación

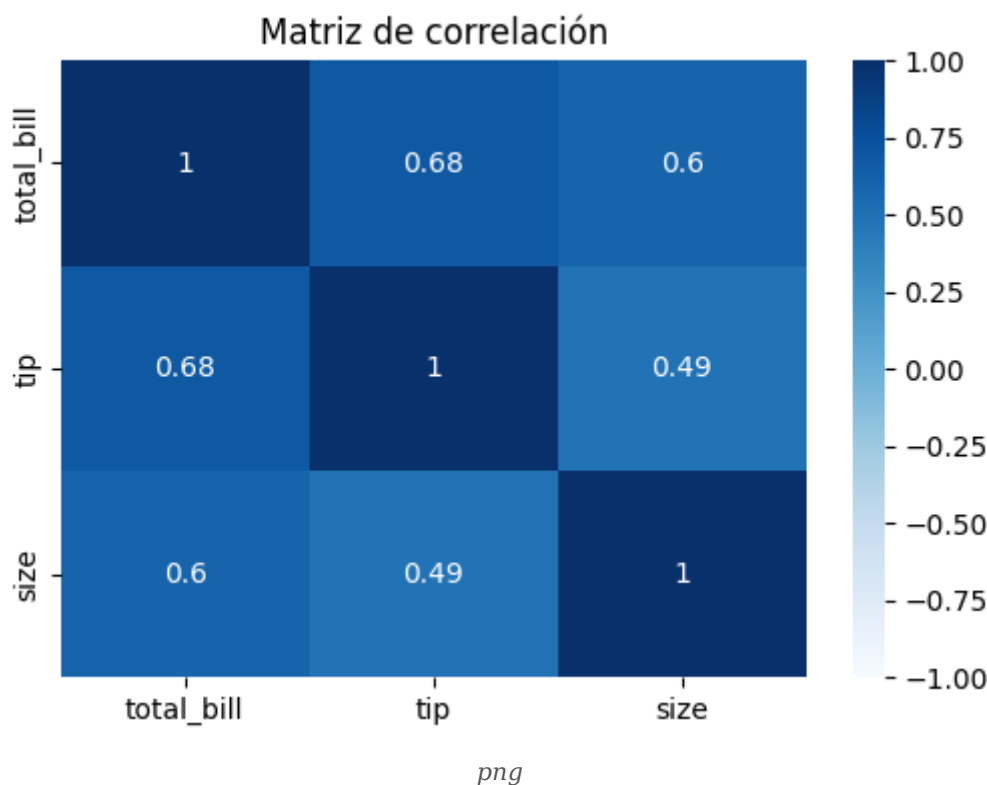
Una matriz de correlación también puede visualizarse con un mapa de calor, o `heatmap`. En un mapa de calor, los valores se representan usando colores. Esto permite identificar rápidamente qué relaciones son más fuertes o más débiles.

Vamos a construir un `heatmap` con las correlaciones entre `total_bill`, `tip` y `size`.

```
plt.figure(figsize=(6, 4))

sns.heatmap(
    matriz_correlacion,
    annot=True,
    cmap="Blues",
    vmin=-1,
    vmax=1
)

plt.title("Matriz de correlación")
plt.show()
```



El mapa de calor representa visualmente la matriz de correlación. Cada celda muestra la correlación entre dos variables. Los valores también aparecen escritos dentro de cada celda porque usamos `annot=True`.

La diagonal principal tiene valores iguales a 1, ya que cada variable se compara consigo misma. En este caso, todas las correlaciones son positivas.

La relación más alta entre variables distintas es la de `total_bill` y `tip`, con una correlación de 0.68. Luego aparece la relación entre `total_bill` y `size`, con una correlación de 0.60. La relación más débil de las tres es la de `tip` y `size`, con una correlación de 0.49.

El color ayuda a identificar rápidamente qué relaciones son más fuertes, pero siempre conviene leer también los valores numéricos.

Cómo leer heatmap

En la celda anterior usamos `sns.heatmap()`.

La instrucción principal fue:

```
sns.heatmap(
    matriz_correlacion,
    annot=True,
    cmap="Blues",
    vmin=-1,
    vmax=1
)
```

El argumento `matriz_correlacion` indica qué tabla queremos visualizar.

El argumento `annot=True` muestra los valores dentro de cada celda.

El argumento `cmap="Blues"` define la paleta de colores.

Los argumentos `vmin=-1` y `vmax=1` fijan la escala de la correlación entre -1 y 1.

Esto es útil porque la correlación siempre se interpreta dentro de ese rango.

Si usamos la misma escala para distintos mapas de calor, las comparaciones visuales son más consistentes.

Cierre del capítulo

En este capítulo aprendimos a visualizar relaciones entre variables numéricas.

Usamos `scatterplot` para observar la relación entre dos columnas numéricas. También usamos `regplot` para agregar una línea de tendencia.

Calculamos correlaciones para resumir numéricamente la relación lineal entre variables.

Además, vimos que podemos sumar una variable categórica con `hue` para enriquecer la lectura del gráfico.

Por último, construimos una matriz de correlación y la visualizamos con `heatmap`.

Lo más importante es recordar que una relación entre variables no debe interpretarse mirando una sola herramienta. Un gráfico muestra patrones visuales. La correlación resume una relación lineal. Y el contexto nos ayuda a interpretar qué puede significar esa relación dentro del dataset.

En el próximo capítulo vamos a seguir trabajando con visualizaciones, pero orientadas a comunicar resultados de manera más clara y ordenada.

Capítulo 37 — Comunicar resultados con visualizaciones claras

No alcanza con hacer un gráfico

En los capítulos anteriores aprendimos a construir distintos tipos de gráficos: histogramas, boxplots, gráficos de barras, gráficos de dispersión, líneas de tendencia y mapas de calor. Cada uno nos ayudó a responder una pregunta diferente sobre los datos.

Ahora vamos a trabajar sobre otro aspecto importante del análisis de datos: la comunicación visual. Un gráfico no solo debe existir; también debe poder leerse con claridad. Para eso necesitamos tomar decisiones sobre el título, los ejes, el orden de las categorías, los colores, el tamaño de la figura y el tipo de gráfico elegido.

En este capítulo vamos a revisar cómo mejorar visualizaciones para que comuniquen mejor una idea. No se trata de decorar los gráficos, sino de hacerlos más claros, más precisos y más útiles para quien los lee.

Vamos a seguir usando el dataset `tips`, porque ya conocemos sus columnas y eso nos permite concentrarnos en la forma de comunicar los resultados.

Cargar el dataset

Como en los capítulos anteriores, primero importamos las librerías necesarias y cargamos el dataset `tips`.

Después vamos a construir algunos gráficos y revisar cómo pequeñas decisiones pueden mejorar su lectura.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3

índice	total_bill	tip	sex	smoker	day	time	size
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset `tips`. Como ya vimos, cada fila representa una cuenta registrada en un restaurante, con información sobre el importe total, la propina, el día, el horario, el tamaño del grupo y algunas variables categóricas.

En este capítulo no vamos a buscar una técnica nueva de análisis, sino a mejorar la forma en que presentamos resultados. Para eso vamos a partir de una pregunta simple: ¿cuál es el promedio del importe total de la cuenta según el día?

Primero vamos a construir una tabla de resumen. Luego usaremos esa tabla para crear un gráfico y revisar qué decisiones ayudan a que sea más claro.

Preparar un resumen para graficar

Antes de hacer el gráfico, vamos a calcular el promedio de `total_bill` para cada día. También vamos a respetar el orden temporal básico de los días registrados en el dataset: `Thur`, `Fri`, `Sat` y `Sun`.

Este paso es importante porque un gráfico claro suele empezar con una tabla clara. Si los datos que alimentan al gráfico están bien organizados, después es más fácil controlar el orden, los nombres y la interpretación visual.

```
orden_dias = ["Thur", "Fri", "Sat", "Sun"]

promedio_dia = (
    df.groupby("day", observed=True)["total_bill"]
      .mean()
      .reindex(orden_dias)
      .reset_index()
)

promedio_dia["total_bill"] = promedio_dia["total_bill"].round(2)

promedio_dia
```

índice	day	total_bill
0	Thur	17.68
1	Fri	17.15

índice	day	total_bill
2	Sat	20.44
3	Sun	21.41

La tabla muestra el promedio del importe total de la cuenta según el día. Los valores están redondeados a dos decimales para que sean más fáciles de leer.

Podemos observar que **Sat** y **Sun** tienen promedios más altos que **Thur** y **Fri**. Sin embargo, por ahora no vamos a concentrarnos solamente en la interpretación del resultado, sino en cómo comunicarlo visualmente de manera clara.

A continuación vamos a construir un primer gráfico de barras. Será un gráfico válido, pero todavía podemos mejorarlo.

Primer gráfico: correcto, pero mejorable

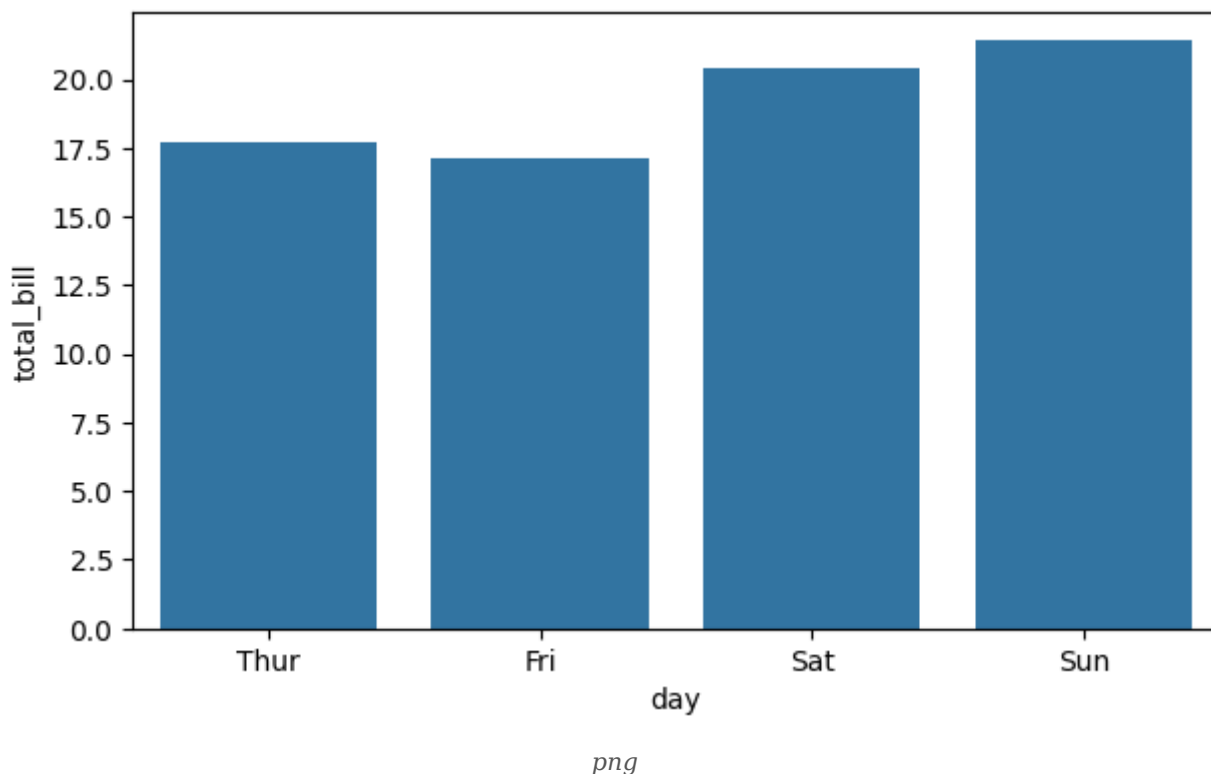
Vamos a empezar con un gráfico de barras simple. En el eje horizontal ubicaremos los días y en el eje vertical el promedio de **total_bill**.

Este primer gráfico ya comunica la comparación general, pero después vamos a revisar qué elementos pueden mejorarse para que la lectura sea más clara.

```
plt.figure(figsize=(7, 4))

sns.barplot(
    data=promedio_dia,
    x="day",
    y="total_bill"
)

plt.show()
```



El gráfico muestra el promedio de `total_bill` para cada día registrado en el dataset. La comparación general se entiende: `Sat` y `Sun` tienen promedios más altos, mientras que `Thur` y `Fri` tienen promedios más bajos.

Sin embargo, el gráfico todavía puede comunicar mejor. No tiene título, y los nombres de los ejes aparecen con los nombres originales de las columnas: `day` y `total_bill`. Esos nombres son útiles cuando trabajamos con código, pero pueden ser poco claros para una persona que solo está leyendo el gráfico.

Una primera mejora consiste en agregar un título descriptivo y etiquetas más claras para los ejes.

Mejorar título y etiquetas

Un gráfico claro debería responder rápidamente tres preguntas:

- qué se está comparando;
- qué representa cada eje;
- cuál es la unidad o el significado de los valores.

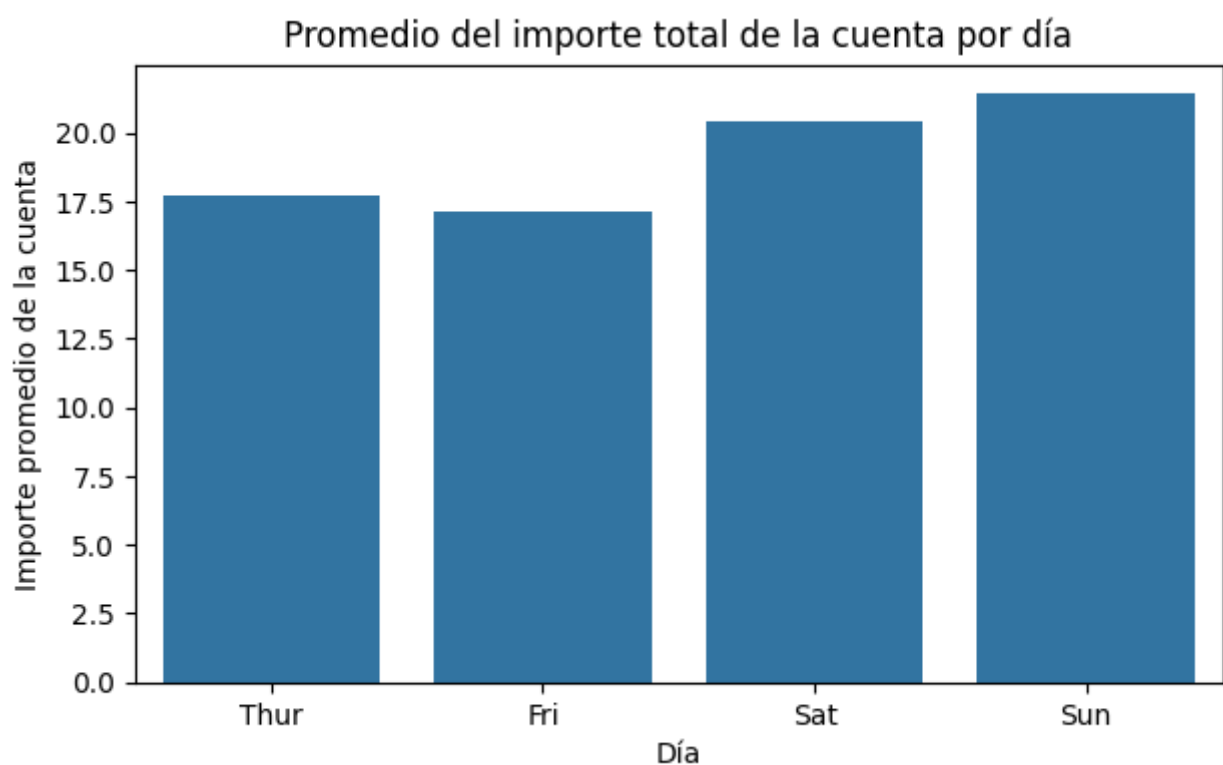
Para mejorar el gráfico anterior, vamos a agregar un título y cambiar las etiquetas de los ejes por textos más descriptivos.

```
plt.figure(figsize=(7, 4))

sns.barplot(
    data=promedio_dia,
    x="day",
    y="total_bill"
)

plt.title("Promedio del importe total de la cuenta por día")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")

plt.show()
```



png

Este segundo gráfico comunica la misma información que el anterior, pero lo hace con más claridad. El título indica qué se está comparando, el eje horizontal dice que las categorías son días y el eje vertical explica que los valores corresponden al importe promedio de la cuenta.

Este tipo de mejora parece simple, pero es muy importante. Cuando un gráfico se comparte en un informe, una presentación o un cuaderno, no siempre va acompañado de una explicación oral. Por eso, el gráfico debería ser lo suficientemente claro como para que otra persona pueda entenderlo sin tener que revisar el código que lo generó.

Ahora vamos a agregar otra mejora: mostrar el valor exacto encima de cada barra.

Agregar valores sobre las barras

A veces conviene mostrar los valores exactos directamente sobre el gráfico. Esto es útil cuando queremos facilitar la lectura sin obligar a la persona a estimar el valor mirando el eje vertical.

En este caso, como tenemos solo cuatro barras, podemos agregar el promedio encima de cada una sin sobrecargar demasiado la visualización.

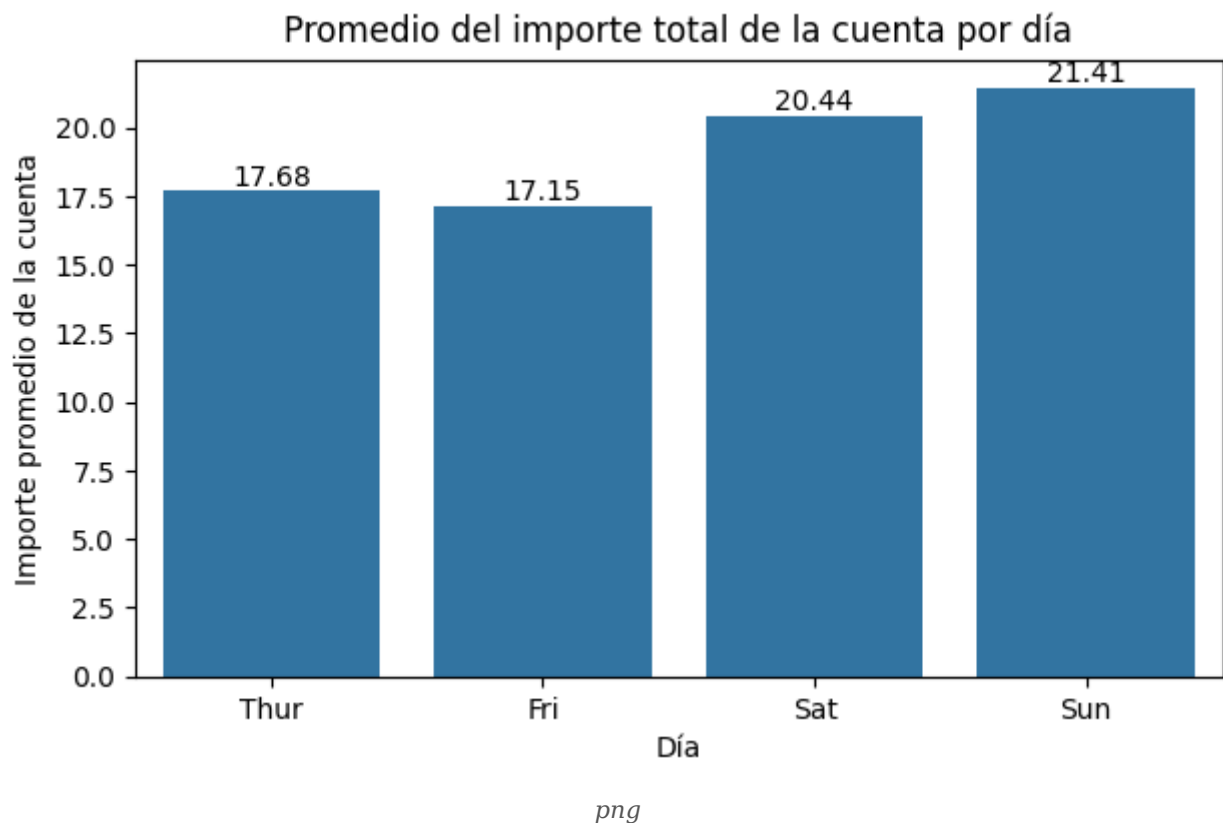
```
plt.figure(figsize=(7, 4))

grafico = sns.barplot(
    data=promedio_dia,
    x="day",
    y="total_bill"
)

plt.title("Promedio del importe total de la cuenta por día")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")

for barra in grafico.containers:
    grafico.bar_label(barra, fmt="%.2f")

plt.show()
```



El gráfico ahora incluye el valor exacto encima de cada barra. Esto facilita la lectura porque podemos ver directamente que los promedios son 17.68 para `Thur`, 17.15 para `Fri`, 20.44 para `Sat` y 21.41 para `Sun`.

Agregar valores puede ser útil cuando hay pocas barras y queremos comunicar cifras concretas. Sin embargo, no siempre conviene hacerlo. Si el gráfico tuviera muchas categorías, demasiadas etiquetas podrían hacerlo más difícil de leer.

En este caso, las etiquetas funcionan bien, aunque vemos que la etiqueta de `Sun` queda muy cerca del borde superior. Vamos a ajustar el límite del eje vertical para dejar un poco más de aire visual.

Dejar espacio para las etiquetas

Cuando agregamos etiquetas sobre las barras, a veces necesitamos ajustar el eje vertical para que el texto no quede pegado al borde del gráfico.

Para eso podemos usar `plt.ylim()`, que permite definir el límite inferior y superior del eje vertical.

```
plt.figure(figsize=(7, 4))

grafico = sns.barplot(
    data=promedio_dia,
```

```

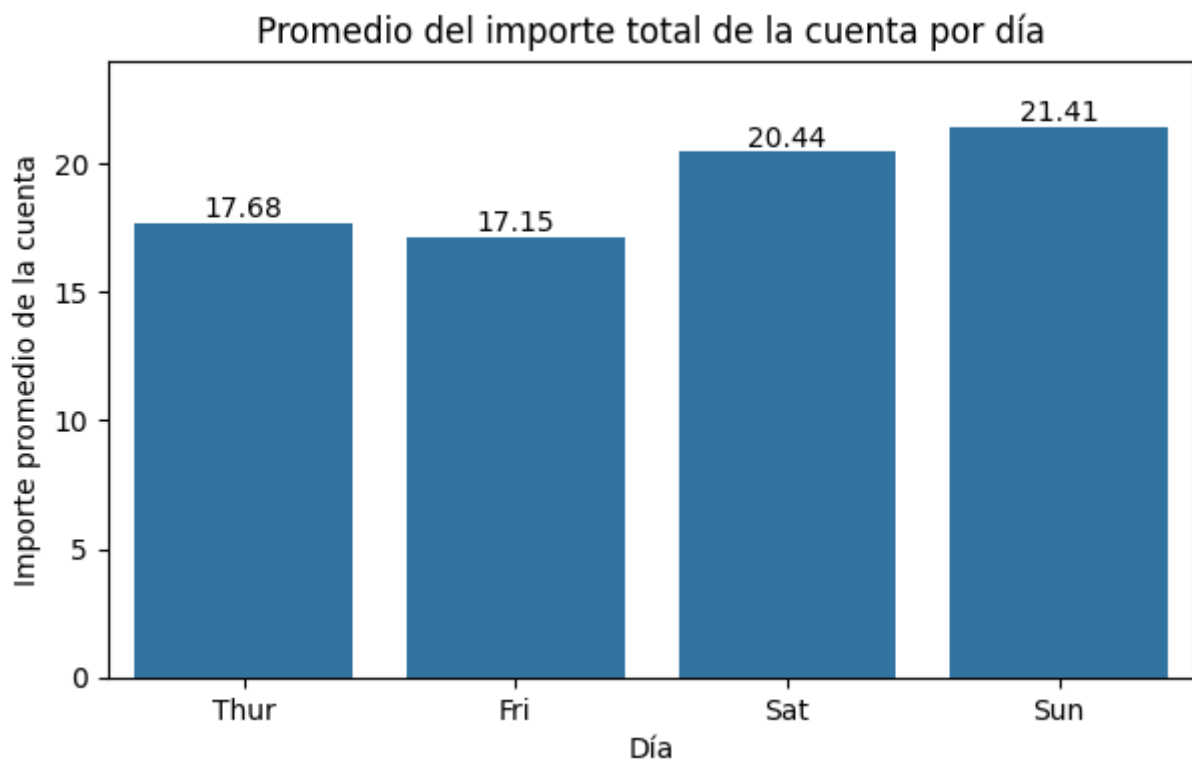
    x="day",
    y="total_bill"
)

plt.title("Promedio del importe total de la cuenta por día")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")
plt.ylim(0, 24)

for barra in grafico.containers:
    grafico.bar_label(barra, fmt="%.2f")

plt.show()

```



png

El ajuste del eje vertical mejora la presentación del gráfico. Al usar `plt.ylim(0, 24)`, dejamos un poco más de espacio por encima de la barra más alta, de modo que las etiquetas no quedan pegadas al borde superior.

La instrucción `plt.ylim()` permite definir los límites del eje vertical. En este caso usamos 0 como límite inferior y 24 como límite superior. Es importante que este tipo de ajuste no distorsione la lectura: el eje sigue empezando en 0, por lo que las alturas de las barras se comparan de manera clara.

También usamos `grafico.bar_label()` para agregar los valores sobre las barras. El argumento `fmt="%.2f"` indica que queremos mostrar los números con dos decimales.

Mejorar el estilo general

Además del título, las etiquetas y los valores, podemos mejorar la lectura usando un estilo visual más limpio. Por ejemplo, una grilla suave en el fondo puede ayudar a comparar las alturas de las barras sin recargar el gráfico.

También podemos definir un color único para las barras. En este caso no necesitamos muchos colores, porque todas las barras representan el mismo indicador: el promedio del importe total de la cuenta. Usar un color uniforme ayuda a que la atención se concentre en la comparación de valores, no en diferencias de color que no significan nada especial.

```
sns.set_theme(style="whitegrid")

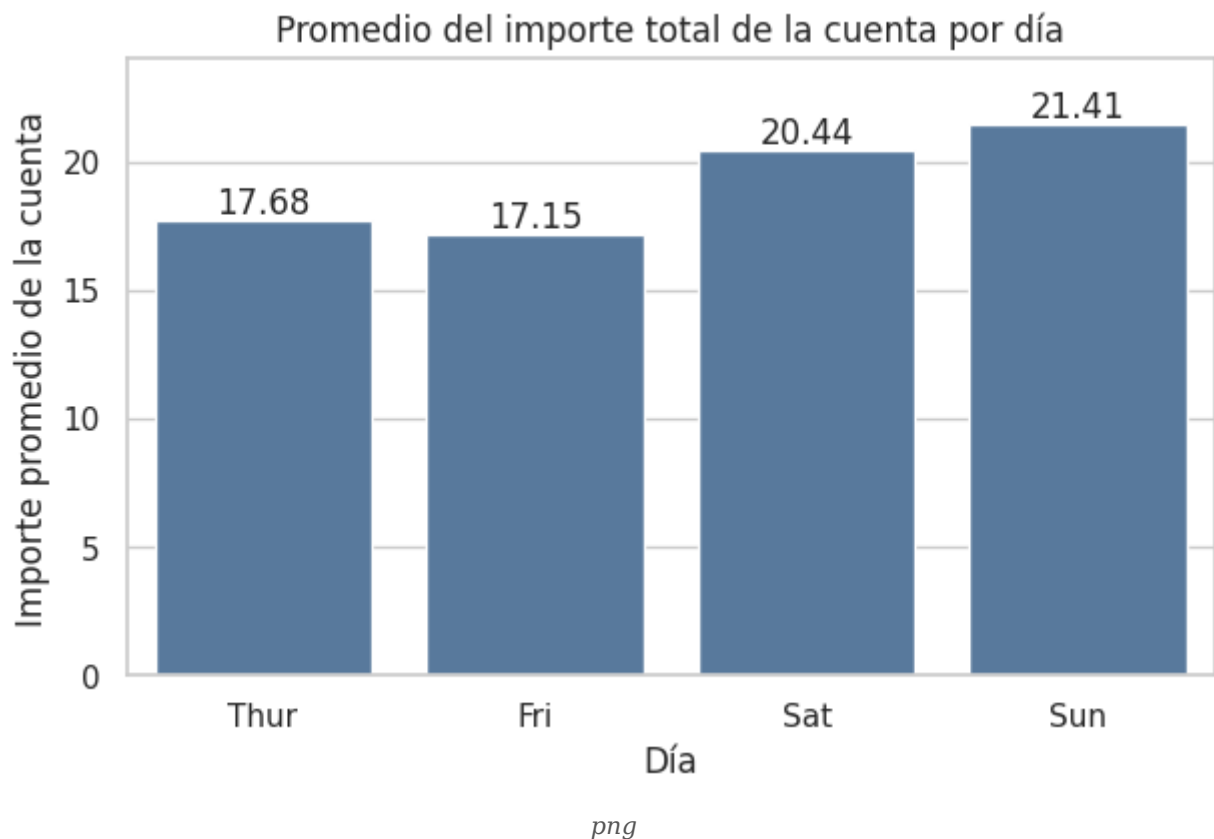
plt.figure(figsize=(7, 4))

grafico = sns.barplot(
    data=promedio_dia,
    x="day",
    y="total_bill",
    color="#4C78A8"
)

plt.title("Promedio del importe total de la cuenta por día")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")
plt.ylim(0, 24)

for barra in grafico.containers:
    grafico.bar_label(barra, fmt="%.2f")

plt.show()
```



El gráfico mejorado comunica la comparación con más claridad. La grilla horizontal ayuda a ubicar visualmente los valores, el color uniforme evita interpretar diferencias que no existen y las etiquetas sobre las barras permiten leer los promedios exactos.

También es importante notar que no usamos un color distinto para cada día. En este caso, todos los días forman parte de una misma comparación y representan el mismo indicador: el promedio de `total_bill`. Por eso, un solo color alcanza y evita sumar información visual innecesaria.

Este tipo de decisión forma parte de la comunicación de resultados. Un gráfico claro no necesariamente es el más llamativo, sino el que permite entender mejor la comparación que queremos mostrar.

Ordenar según el mensaje que queremos comunicar

Hasta ahora respetamos el orden temporal básico de los días registrados: `Thur`, `Fri`, `Sat`, `Sun`. Ese orden es útil cuando queremos leer los días como una secuencia.

Pero a veces queremos comunicar otra idea. Por ejemplo, si la pregunta principal fuera “¿qué día tiene el promedio más alto?”, podría ser más claro ordenar las barras de mayor a menor.

Vamos a crear una nueva versión de la tabla, ordenada según el promedio de `total_bill`.

```
promedio_dia_ordenado = promedio_dia.sort_values(
    "total_bill",
    ascending=False
)

promedio_dia_ordenado
```

índice	day	total_bill
3	Sun	21.41
2	Sat	20.44
0	Thur	17.68
1	Fri	17.15

La tabla ahora está ordenada según el valor de `total_bill`, de mayor a menor. Este orden cambia la lectura: ya no estamos mostrando los días como una secuencia temporal básica, sino como un ranking según el promedio del importe total de la cuenta.

Ambos criterios pueden ser válidos, pero responden a preguntas distintas. Si queremos observar los días en su orden habitual dentro del dataset, conviene usar `Thur`, `Fri`, `Sat`, `Sun`. Si queremos destacar qué día tiene el promedio más alto, puede ser más claro ordenar las barras de mayor a menor.

Ahora vamos a construir el gráfico usando esta tabla ordenada.

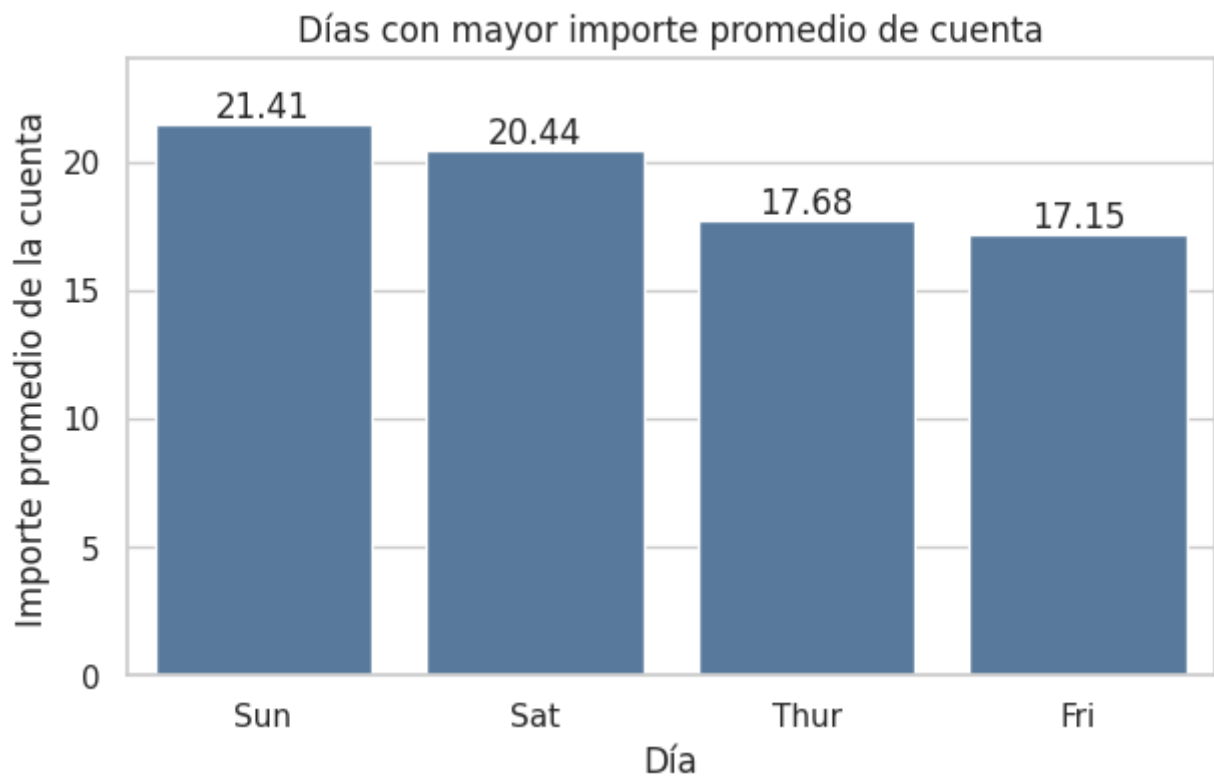
```
plt.figure(figsize=(7, 4))

grafico = sns.barplot(
    data=promedio_dia_ordenado,
    x="day",
    y="total_bill",
    color="#4C78A8"
)

plt.title("Días con mayor importe promedio de cuenta")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")
plt.ylim(0, 24)

for barra in grafico.containers:
    grafico.bar_label(barra, fmt="%.2f")

plt.show()
```



png

El gráfico ordenado de mayor a menor cambia la forma de leer la información. En lugar de seguir el orden temporal básico de los días registrados, ahora muestra un ranking según el promedio del importe total de la cuenta.

Esta versión es útil si queremos responder rápidamente cuál es el día con mayor promedio. La respuesta aparece al inicio del gráfico: **Sun**, seguido por **Sat**.

Sin embargo, este orden también tiene una desventaja: se pierde la lectura temporal básica de **Thur**, **Fri**, **Sat**, **Sun**. Por eso, el orden de las barras no es una decisión neutral. Depende de la pregunta que queremos comunicar.

Si la pregunta es “¿cómo se comportan los días en su secuencia?”, conviene mantener el orden temporal. Si la pregunta es “¿cuáles son los valores más altos?”, conviene ordenar como ranking.

Elegir títulos según la intención

El título de un gráfico también comunica una intención. No es lo mismo titular un gráfico “Promedio del importe total de la cuenta por día” que titularlo “Días con mayor importe promedio de cuenta”.

El primer título es más descriptivo y neutral. Indica qué se está mostrando, sin orientar demasiado la lectura. El segundo título destaca una comparación de ranking: invita a mirar qué días tienen valores más altos.

Ninguna de las dos opciones es siempre mejor. La elección depende del propósito del gráfico. En un análisis exploratorio, suele convenir usar títulos descriptivos. En una presentación de resultados, puede ser útil usar títulos más orientados al mensaje, siempre que no exageren ni afirmen más de lo que muestran los datos.

Revisar si el gráfico cuenta toda la historia

El gráfico de promedios por día es claro, pero todavía tiene una limitación: solo muestra el promedio. Como vimos en capítulos anteriores, un promedio puede ocultar diferencias importantes en la cantidad de registros, la dispersión o la presencia de valores extremos.

Por eso, cuando comunicamos resultados, también debemos preguntarnos si el gráfico elegido alcanza para sostener la interpretación. En este caso, conviene acompañar el promedio con otra información: la cantidad de cuentas registradas por día.

```
resumen_comunicacion = (
    df.groupby("day", observed=True)
    .agg(
        cantidad_cuentas=("total_bill", "count"),
        promedio_total_bill=("total_bill", "mean")
    )
    .reindex(orden_dias)
    .round(2)
    .reset_index()
)

resumen_comunicacion
```

índice	day	cantidad_cuentas	promedio_total_bill
0	Thur	62	17.68
1	Fri	19	17.15
2	Sat	87	20.44
3	Sun	76	21.41

La tabla combina dos indicadores por día: la cantidad de cuentas y el promedio de `total_bill`. Esta información ayuda a leer el gráfico anterior con más cuidado.

Por ejemplo, `Fri` tiene un promedio parecido al de `Thur`, pero tiene muchos menos registros: solo 19 cuentas. En cambio, `Sat` y `Sun` tienen más cuentas y también promedios más altos.

Cuando comunicamos resultados, este tipo de contexto es importante. Si mostramos solamente el promedio, podemos dar una imagen incompleta. Agregar la cantidad de registros permite entender mejor cuánta información sostiene cada valor.

Mostrar dos indicadores relacionados

Hay muchas formas de mostrar dos indicadores relacionados. Una opción simple es construir dos gráficos cercanos: uno para la cantidad de cuentas y otro para el promedio del importe total.

Esta estrategia suele ser más clara que intentar poner demasiada información dentro de un solo gráfico. Si usamos dos gráficos alineados y con el mismo orden de categorías, la persona que lee puede comparar ambos indicadores sin perderse.

```
fig, ejes = plt.subplots(1, 2, figsize=(11, 4))

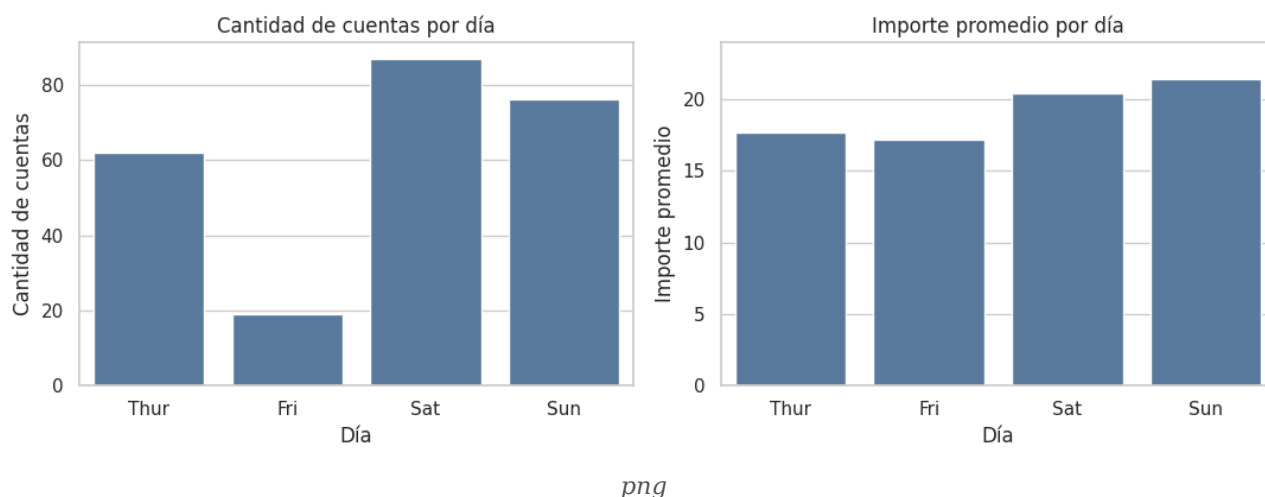
sns.barplot(
    data=resumen_comunicacion,
    x="day",
    y="cantidad_cuentas",
    color="#4C78A8",
    ax=ejes[0]
)

ejes[0].set_title("Cantidad de cuentas por día")
ejes[0].set_xlabel("Día")
ejes[0].set_ylabel("Cantidad de cuentas")

sns.barplot(
    data=resumen_comunicacion,
    x="day",
    y="promedio_total_bill",
    color="#4C78A8",
    ax=ejes[1]
)

ejes[1].set_title("Importe promedio por día")
ejes[1].set_xlabel("Día")
ejes[1].set_ylabel("Importe promedio")
ejes[1].set_ylim(0, 24)

plt.tight_layout()
plt.show()
```



Los dos gráficos muestran información complementaria. A la izquierda vemos la cantidad de cuentas registradas por día, y a la derecha vemos el importe promedio de la cuenta para esos mismos días.

Esta comparación ayuda a evitar una lectura incompleta. Por ejemplo, **Fri** tiene un promedio parecido al de **Thur**, pero tiene muchos menos registros. En cambio, **Sat** y **Sun** tienen más cuentas y también muestran promedios más altos.

La ventaja de usar dos gráficos separados es que cada uno conserva una escala clara. La cantidad de cuentas y el importe promedio son indicadores distintos, por lo que intentar mezclarlos en un solo eje podría confundir. En cambio, al ponerlos uno al lado del otro, podemos compararlos manteniendo la lectura ordenada.

Mejorar una figura con varios gráficos

Cuando usamos más de un gráfico en la misma figura, conviene cuidar algunos detalles adicionales. Por ejemplo, podemos agregar un título general para indicar cuál es la idea completa de la figura, mantener el mismo orden de categorías en ambos gráficos y agregar etiquetas de valores cuando no sobrecargan la visualización.

Vamos a construir una versión más pulida de la figura anterior, agregando un título general y valores sobre las barras.

```
fig, ejes = plt.subplots(1, 2, figsize=(11, 4))

fig.suptitle("Cuentas registradas e importe promedio por día", fontsize=14)

grafico_cantidad = sns.barplot(
    data=resumen_comunicacion,
    x="day",
    y="cantidad_cuentas",
```

```

        color="#4C78A8",
        ax=ejes[0]
    )

    ejes[0].set_title("Cantidad de cuentas")
    ejes[0].set_xlabel("Día")
    ejes[0].set_ylabel("Cantidad de cuentas")
    ejes[0].set_ylim(0, 100)

    for barras in grafico_cantidad.containers:
        grafico_cantidad.bar_label(barras, fmt="%i")

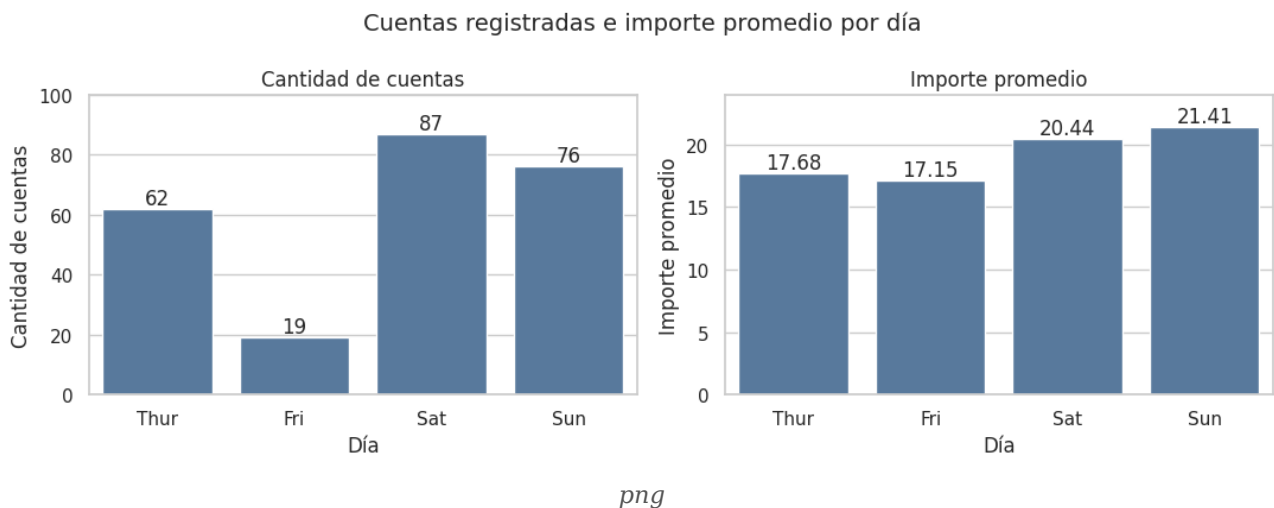
    grafico_promedio = sns.barplot(
        data=resumen_comunicacion,
        x="day",
        y="promedio_total_bill",
        color="#4C78A8",
        ax=ejes[1]
    )

    ejes[1].set_title("Importe promedio")
    ejes[1].set_xlabel("Día")
    ejes[1].set_ylabel("Importe promedio")
    ejes[1].set_ylim(0, 24)

    for barras in grafico_promedio.containers:
        grafico_promedio.bar_label(barras, fmt="%i")

plt.tight_layout()
plt.show()

```



La figura final integra dos indicadores relacionados sin mezclarlos en un único gráfico. A la izquierda vemos cuántas cuentas hay por día, y a la derecha vemos el importe promedio de esas cuentas.

Esta presentación ayuda a leer el promedio con más contexto. Por ejemplo, `Fri` tiene un promedio cercano al de `Thur`, pero está basado en muchos menos registros. En cambio, `Sat` y `Sun` tienen más cuentas y también importes promedio más altos.

También se puede observar que las etiquetas de valores funcionan bien porque hay pocas barras. Si tuviéramos muchas categorías, quizá sería mejor no mostrar todos los valores sobre el gráfico y dejar esa información en una tabla complementaria.

Usar color para destacar, no para decorar

El color puede ayudar mucho a comunicar una idea, pero conviene usarlo con intención. Si cada barra tiene un color diferente sin que eso represente una variable o un mensaje, el gráfico puede volverse más confuso.

Una estrategia útil es usar un color general para casi todos los elementos y destacar solo aquello que queremos señalar. Por ejemplo, si queremos resaltar el día con mayor importe promedio, podemos usar un color diferente para `Sun` y dejar el resto de las barras en un tono más neutro.

```
colores = [
    "#A0A7B0" if dia != "Sun" else "#4C78A8"
    for dia in promedio_dia["day"]
]

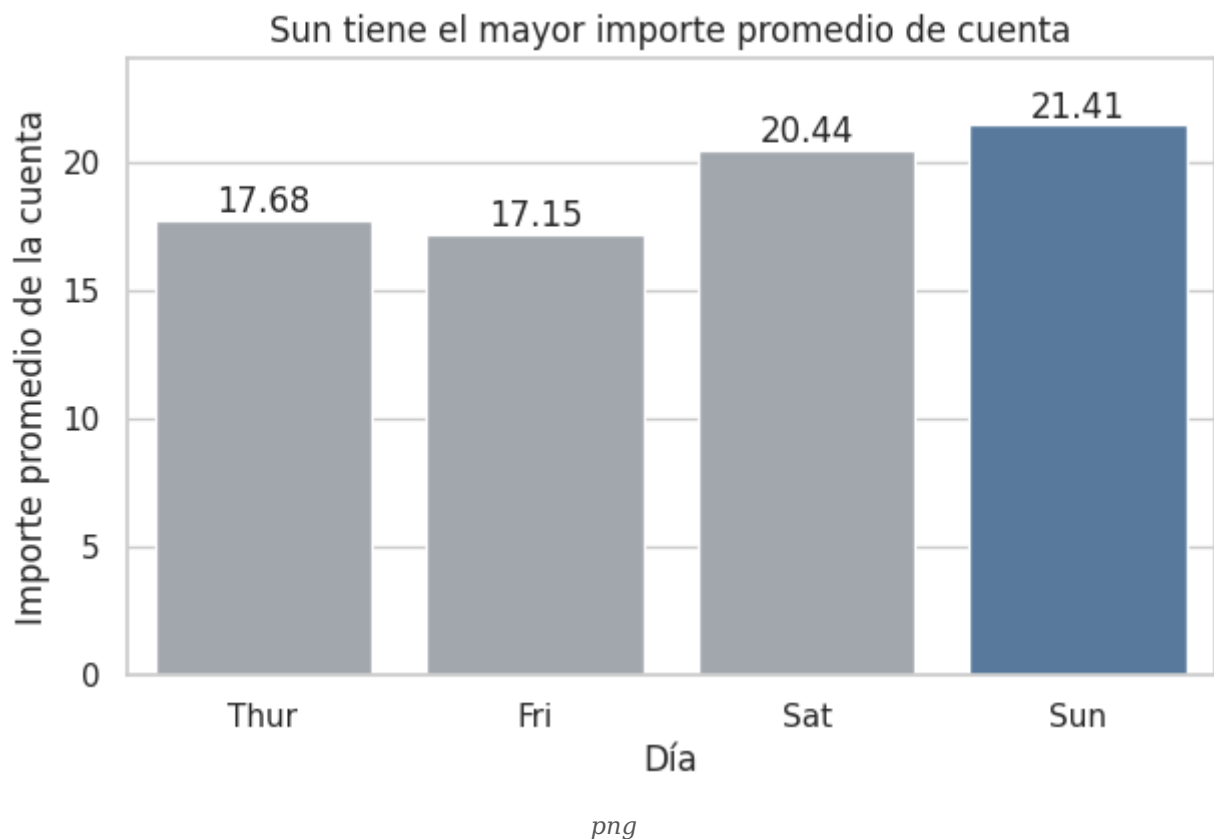
plt.figure(figsize=(7, 4))

grafico = sns.barplot(
    data=promedio_dia,
    x="day",
    y="total_bill",
    palette=colores,
    hue="day",
    legend=False
)

plt.title("Sun tiene el mayor importe promedio de cuenta")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")
plt.ylim(0, 24)

for barra in grafico.containers:
    grafico.bar_label(barra, fmt="%.2f")

plt.show()
```



En este gráfico usamos el color para destacar un mensaje específico: `Sun` tiene el mayor importe promedio de cuenta. Las barras de `Thur`, `Fri` y `Sat` aparecen en gris, mientras que `Sun` aparece en azul. Esto hace que la atención vaya rápidamente hacia el valor que queremos señalar.

Esta decisión funciona porque el color tiene una intención clara. No estamos usando colores diferentes para decorar, sino para marcar una comparación concreta. Aun así, el gráfico conserva los demás valores, por lo que la persona que lo lee puede ver que `Sat` también tiene un promedio alto y que la diferencia entre `Sat` y `Sun` no es enorme.

Este punto es importante: destacar un dato no debería ocultar el resto de la información ni exagerar la diferencia entre valores.

Cómo se definieron los colores

En la celda anterior creamos una lista de colores antes de construir el gráfico.

La parte principal fue:

```
colores = [  
    "#A0A7B0" if dia != "Sun" else "#4C78A8"
```

```
for dia in promedio_dia["day"]  
]
```

Esta instrucción recorre los días de la tabla `promedio_dia`. Si el día no es `Sun`, asigna un color gris. Si el día es `Sun`, asigna un color azul.

Después usamos esa lista en `sns.barplot()` mediante el argumento `palette=colores`.

También usamos `hue="day"` y `legend=False`. En versiones recientes de Seaborn, cuando queremos aplicar una paleta personalizada a categorías, conviene indicar explícitamente la variable categórica con `hue`. Como en este caso el color solo se usa para destacar las barras y no necesitamos una leyenda adicional, usamos `legend=False`.

No exagerar el mensaje

Cuando comunicamos resultados, es tentador escribir títulos muy fuertes. Por ejemplo, podríamos titular el gráfico: “El domingo es claramente el mejor día”. Pero ese título sería problemático, porque el gráfico solo muestra el importe promedio de las cuentas en este dataset.

Una opción más prudente es usar un título que diga exactamente lo que el gráfico permite sostener: `Sun tiene el mayor importe promedio de cuenta`. Ese título comunica el hallazgo principal, pero no agrega interpretaciones que no están en los datos.

En análisis de datos, una buena visualización no solo debe ser clara. También debe ser honesta con el alcance de la información disponible.

Cierre del capítulo

En este capítulo trabajamos sobre la comunicación visual de resultados. Partimos de un gráfico correcto pero básico, y fuimos agregando mejoras: título, etiquetas de ejes, valores sobre las barras, ajuste del eje vertical, estilo visual, orden de categorías, figuras con más de un gráfico y uso intencional del color.

También vimos que cada decisión debe responder a una intención. Ordenar las barras como ranking puede ser útil si queremos destacar los valores más altos, pero no siempre conviene si necesitamos respetar una secuencia temporal. Agregar etiquetas puede ayudar cuando hay pocas barras, pero puede recargar el gráfico si hay demasiadas categorías. Usar color puede guiar la lectura, siempre que no se use solo como decoración.

La idea central del capítulo es que comunicar con gráficos no consiste solamente en generar una imagen. Consiste en construir una visualización que ayude a entender mejor una pregunta, sin ocultar contexto ni exagerar conclusiones.

En el próximo capítulo vamos a avanzar hacia una etapa de cierre del bloque de análisis exploratorio, integrando varias herramientas trabajadas hasta ahora.

Capítulo 038 — Integrar un análisis exploratorio completo

Unir las piezas del análisis exploratorio

Hasta ahora trabajamos muchas herramientas por separado: inspección inicial de datos, resúmenes estadísticos, conteos, agrupaciones, tablas de resumen y distintos tipos de gráficos.

En este capítulo vamos a integrar esas herramientas en un recorrido más completo de análisis exploratorio. La idea no es aprender una función nueva, sino practicar cómo combinar lo que ya sabemos para responder preguntas sobre un dataset.

Vamos a seguir usando el dataset `tips`, porque ya conocemos sus columnas y eso nos permite concentrarnos en el proceso de análisis. El objetivo será construir una lectura ordenada: primero observar la estructura general, después plantear algunas preguntas, luego calcular resúmenes y visualizaciones, y finalmente escribir hallazgos prudentes.

Un análisis exploratorio no busca cerrar una verdad definitiva. Busca conocer mejor los datos, detectar patrones, encontrar diferencias interesantes y reconocer límites de interpretación.

Cargar el dataset

Comenzamos importando las librerías necesarias y cargando el dataset `tips`. Después de cargarlo, vamos a mirar las primeras filas para recordar qué información contiene cada registro.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

df.head()
```

índice	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3

índice	total_bill	tip	sex	smoker	day	time	size
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

La salida muestra las primeras filas del dataset `tips`. Cada fila representa una cuenta registrada en un restaurante e incluye información sobre el importe total de la cuenta, la propina, el día, el horario, el tamaño del grupo y algunas variables categóricas.

Como este capítulo busca integrar herramientas, no vamos a analizar una sola columna de manera aislada desde el comienzo. Primero vamos a hacer una revisión general del dataset para recordar cuántas filas y columnas tiene, qué tipos de datos aparecen y si hay valores faltantes.

Revisar la estructura general

Antes de calcular promedios o construir gráficos, conviene revisar la estructura del dataset. Esta etapa inicial nos ayuda a entender con qué datos estamos trabajando y a detectar posibles problemas básicos.

Vamos a consultar la cantidad de filas y columnas, y luego vamos a usar `info()` para ver los nombres de las columnas, los tipos de datos y la cantidad de valores no nulos.

```
df.shape
```

```
(244, 7)
```

El resultado `(244, 7)` indica que el dataset tiene 244 filas y 7 columnas.

En este contexto, cada fila representa una cuenta registrada en el restaurante, y cada columna representa una característica de esa cuenta. Esta primera revisión nos da una idea del tamaño general del dataset: no estamos trabajando con una tabla enorme, pero sí con suficientes registros como para explorar patrones, comparaciones y relaciones entre variables.

Ahora vamos a mirar información más detallada sobre las columnas.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
```

```
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null      float64
1   tip         244 non-null      float64
2   sex         244 non-null      category
3   smoker      244 non-null      category
4   day         244 non-null      category
5   time        244 non-null      category
6   size        244 non-null      int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

La salida de `info()` permite revisar varias cosas importantes. El dataset tiene 244 registros y todas las columnas tienen 244 valores no nulos, por lo que no aparecen valores faltantes en esta primera revisión.

También podemos distinguir dos tipos principales de variables. Por un lado, hay variables numéricas: `total_bill`, `tip` y `size`. Por otro lado, hay variables categóricas: `sex`, `smoker`, `day` y `time`.

Esta distinción es importante porque no analizamos todas las columnas de la misma manera. Para las variables numéricas podemos calcular medidas como promedio, mediana, mínimo y máximo. Para las variables categóricas, en cambio, suele ser más útil contar cuántas veces aparece cada categoría.

Con esta estructura general clara, podemos plantear algunas preguntas de análisis.

Plantear preguntas de análisis

Un análisis exploratorio suele empezar con preguntas. No hace falta que sean preguntas definitivas ni demasiado complejas; sirven como guía para decidir qué tablas y gráficos construir.

En este capítulo vamos a trabajar con algunas preguntas simples:

- ¿cómo se distribuyen los importes de las cuentas?
- ¿cómo se distribuyen las propinas?
- ¿qué días y horarios tienen más registros?
- ¿los importes cambian según el día o el horario?
- ¿hay relación entre el importe total de la cuenta y la propina?

Estas preguntas nos van a permitir recorrer variables numéricas, variables categóricas, comparaciones entre grupos y relaciones entre variables.

Resumir las variables numéricas

Vamos a empezar por las variables numéricas. Un resumen estadístico nos permite tener una primera idea de sus valores centrales, su dispersión y sus rangos.

En este dataset, las variables numéricas principales son `total_bill`, `tip` y `size`.

```
columnas_numericas = ["total_bill", "tip", "size"]  
  
df[columnas_numericas].describe().round(2)
```

índice	total_bill	tip	size
count	244.00	244.00	244.00
mean	19.79	3.00	2.57
std	8.90	1.38	0.95
min	3.07	1.00	1.00
25%	13.35	2.00	2.00
50%	17.80	2.90	2.00
75%	24.13	3.56	3.00
max	50.81	10.00	6.00

El resumen estadístico muestra que hay 244 valores para cada variable numérica, lo cual coincide con la revisión anterior: no hay valores faltantes en estas columnas.

En `total_bill`, el importe promedio de la cuenta es 19.79 y la mediana es 17.80. Como el promedio es mayor que la mediana, esto sugiere que algunos importes altos pueden estar elevando el promedio. El valor máximo es 50.81, bastante por encima del tercer cuartil, que es 24.13.

En `tip`, el promedio es 3.00 y la mediana es 2.90. Los valores centrales están más cerca entre sí, aunque también aparece una propina máxima de 10.00, bastante más alta que el tercer cuartil de 3.56.

La variable `size` indica el tamaño del grupo. La mediana es 2, lo que sugiere que muchas cuentas corresponden a grupos de dos personas. Sin embargo, el valor máximo llega a 6, por lo que también hay algunos grupos más grandes.

Este resumen no responde todas las preguntas, pero nos da una base para decidir qué conviene mirar con más detalle.

Contar las variables categóricas

Ahora vamos a revisar las variables categóricas. Para este tipo de columnas, una primera pregunta útil es cuántas veces aparece cada categoría.

Vamos a construir tablas de frecuencia para `day`, `time`, `sex` y `smoker`.

```
columnas_categoricas = ["day", "time", "sex", "smoker"]

for columna in columnas_categoricas:
    print(f"\nFrecuencias de {columna}:")
    print(df[columna].value_counts())
```

Frecuencias de day:

```
day
Sat      87
Sun      76
Thur     62
Fri      19
Name: count, dtype: int64
```

Frecuencias de time:

```
time
Dinner   176
Lunch    68
Name: count, dtype: int64
```

Frecuencias de sex:

```
sex
Male     157
Female    87
Name: count, dtype: int64
```

Frecuencias de smoker:

```
smoker
No       151
Yes       93
Name: count, dtype: int64
```

Las tablas de frecuencia muestran que las categorías no están distribuidas de manera uniforme. En la columna `day`, el día con más registros es `Sat`, seguido por `Sun` y `Thur`; `Fri` tiene muchos menos registros que los demás días. En la columna `time`, hay muchas más cuentas de `Dinner` que de `Lunch`.

También vemos que en este dataset hay más registros asociados a `Male` que a `Female`, y más cuentas de no fumadores que de fumadores. Estas diferencias no son un problema, pero son importantes para interpretar cualquier comparación posterior.

Por ejemplo, si comparamos promedios por día, debemos recordar que `Fri` tiene solo 19 registros. Del mismo modo, si comparamos `Lunch` y `Dinner`, tenemos que considerar que `Dinner` tiene muchos más datos. En análisis exploratorio, las frecuencias ayudan a darle contexto a los promedios, gráficos y relaciones que vamos a mirar después.

Visualizar la distribución de las cuentas

Después de revisar los resúmenes numéricos y las frecuencias, podemos mirar con más detalle una variable central del dataset: `total_bill`.

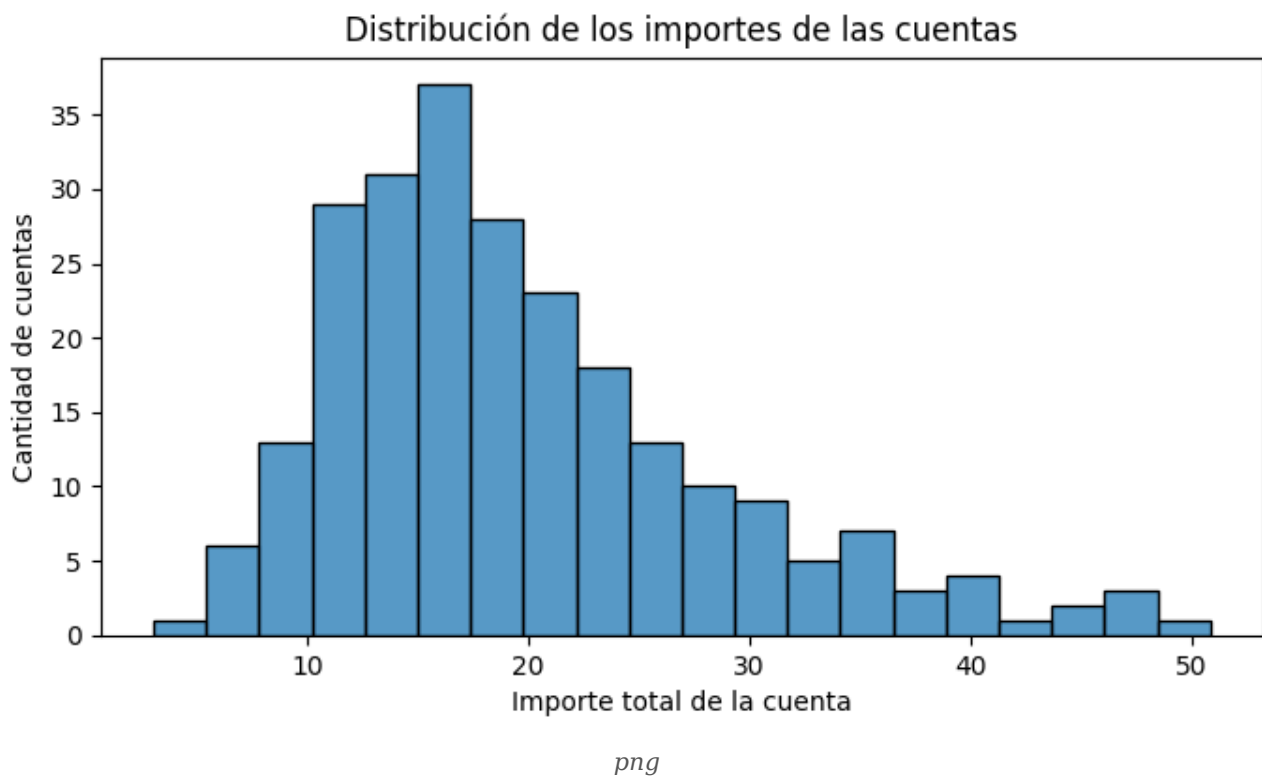
Vamos a construir un histograma para observar cómo se distribuyen los importes de las cuentas.

```
plt.figure(figsize=(8, 4))

sns.histplot(
    data=df,
    x="total_bill",
    bins=20
)

plt.title("Distribución de los importes de las cuentas")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El histograma muestra que la mayoría de las cuentas se concentra en importes bajos o medios, aproximadamente entre 10 y 25. A medida que los importes aumentan, la cantidad de cuentas disminuye.

También se observa una cola hacia la derecha: hay algunas cuentas con importes bastante más altos que el grupo principal, llegando a valores cercanos a 50. Esto ayuda a explicar por qué el promedio de `total_bill` era mayor que la mediana. Algunos valores altos empujan el promedio hacia arriba.

Esta visualización no reemplaza al resumen numérico, sino que lo complementa. La tabla nos dio valores concretos como promedio, mediana y máximo; el histograma nos permite ver la forma general de la distribución.

Visualizar la distribución de las propinas

Ahora vamos a mirar la distribución de `tip`. Esta variable está relacionada con `total_bill`, pero conviene analizarla también por separado para entender cómo se comportan las propinas registradas en el dataset.

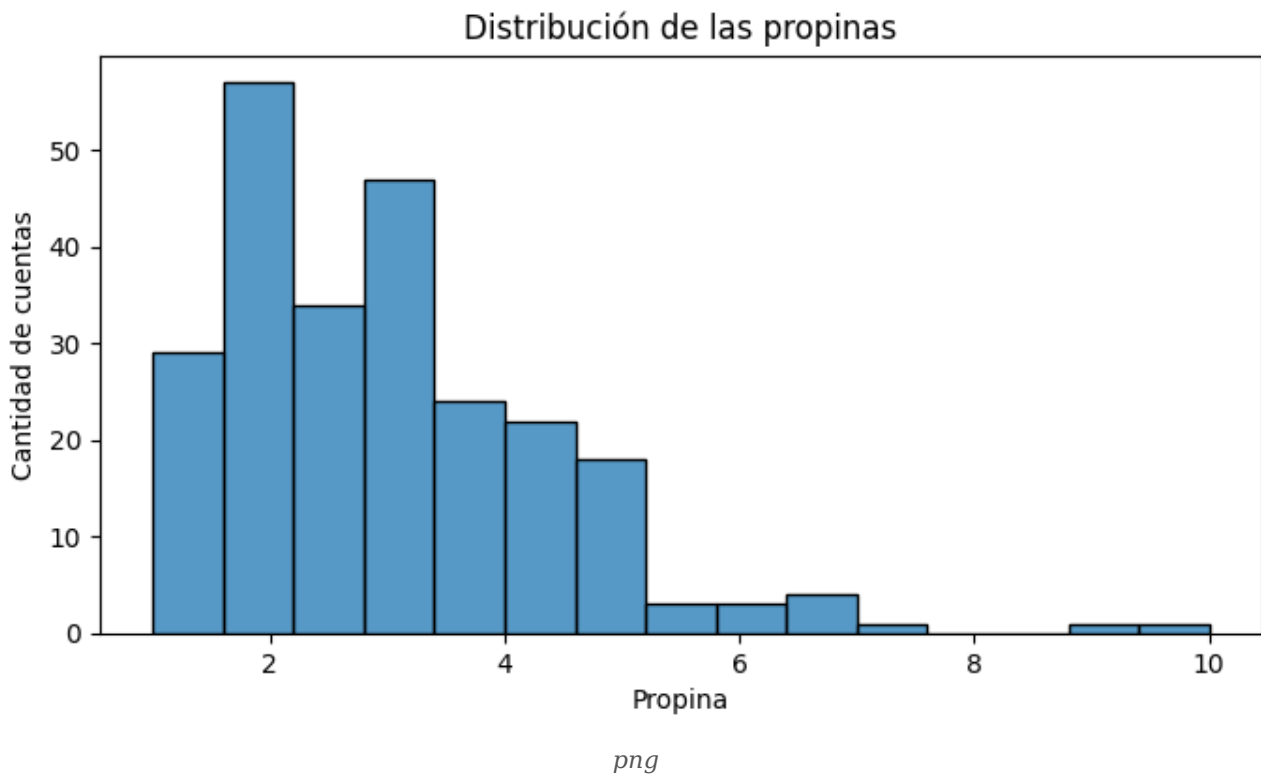
```
plt.figure(figsize=(8, 4))

sns.histplot(
    data=df,
    x="tip",
    bins=15
```

```
)

plt.title("Distribución de las propinas")
plt.xlabel("Propina")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El histograma de `tip` muestra que la mayoría de las propinas se concentra aproximadamente entre 1 y 5. Los valores más frecuentes aparecen alrededor de 2 y 3, mientras que las propinas más altas son mucho menos comunes.

También se observa una cola hacia la derecha. Hay algunos valores superiores a 6 y casos aislados cercanos a 9 o 10. Esto coincide con el resumen estadístico: la mediana era 2.90, el tercer cuartil era 3.56 y el valor máximo llegaba a 10.00.

Al integrar esta lectura con la distribución de `total_bill`, vemos una idea inicial: tanto los importes de las cuentas como las propinas se concentran en valores bajos o medios, con algunos valores altos menos frecuentes. Más adelante vamos a mirar si ambas variables se relacionan entre sí.

Comparar importes según el día

Después de mirar distribuciones generales, podemos comparar grupos. Una pregunta útil es si el importe total de la cuenta cambia según el día.

Para eso vamos a calcular una tabla de resumen por día. Incluiremos la cantidad de cuentas, el promedio y la mediana de `total_bill`, porque no conviene interpretar un promedio sin saber cuántos datos lo sostienen.

```
orden_dias = ["Thur", "Fri", "Sat", "Sun"]

resumen_dia = (
    df.groupby("day", observed=True)
      .agg(
        cantidad_cuentas=("total_bill", "count"),
        promedio_cuenta=("total_bill", "mean"),
        mediana_cuenta=("total_bill", "median")
      )
      .reindex(orden_dias)
      .round(2)
      .reset_index()
)

resumen_dia
```

índice	day	cantidad_cuentas	promedio_cuenta	mediana_cuenta
0	Thur	62	17.68	16.20
1	Fri	19	17.15	15.38
2	Sat	87	20.44	18.24
3	Sun	76	21.41	19.63

La tabla muestra que `Sat` y `Sun` tienen los promedios de cuenta más altos. `Sun` presenta el promedio más alto, con 21.41, y también la mediana más alta, con 19.63. En cambio, `Thur` y `Fri` tienen promedios y medianas más bajos.

También es importante mirar la cantidad de cuentas. `Fri` tiene solo 19 registros, bastante menos que los demás días, por lo que sus valores deben interpretarse con más cuidado. `Sat` y `Sun`, en cambio, tienen más registros y eso hace que la comparación sea más estable.

Esta tabla resume bien la comparación, pero un gráfico puede ayudarnos a ver las diferencias de forma más rápida.

Visualizar el promedio por día

Vamos a construir un gráfico de barras para comparar el promedio de `total_bill` según el día. Como este capítulo integra herramientas ya trabajadas, vamos a cuidar también la comunicación visual: usaremos un título claro, etiquetas descriptivas y valores sobre las barras.

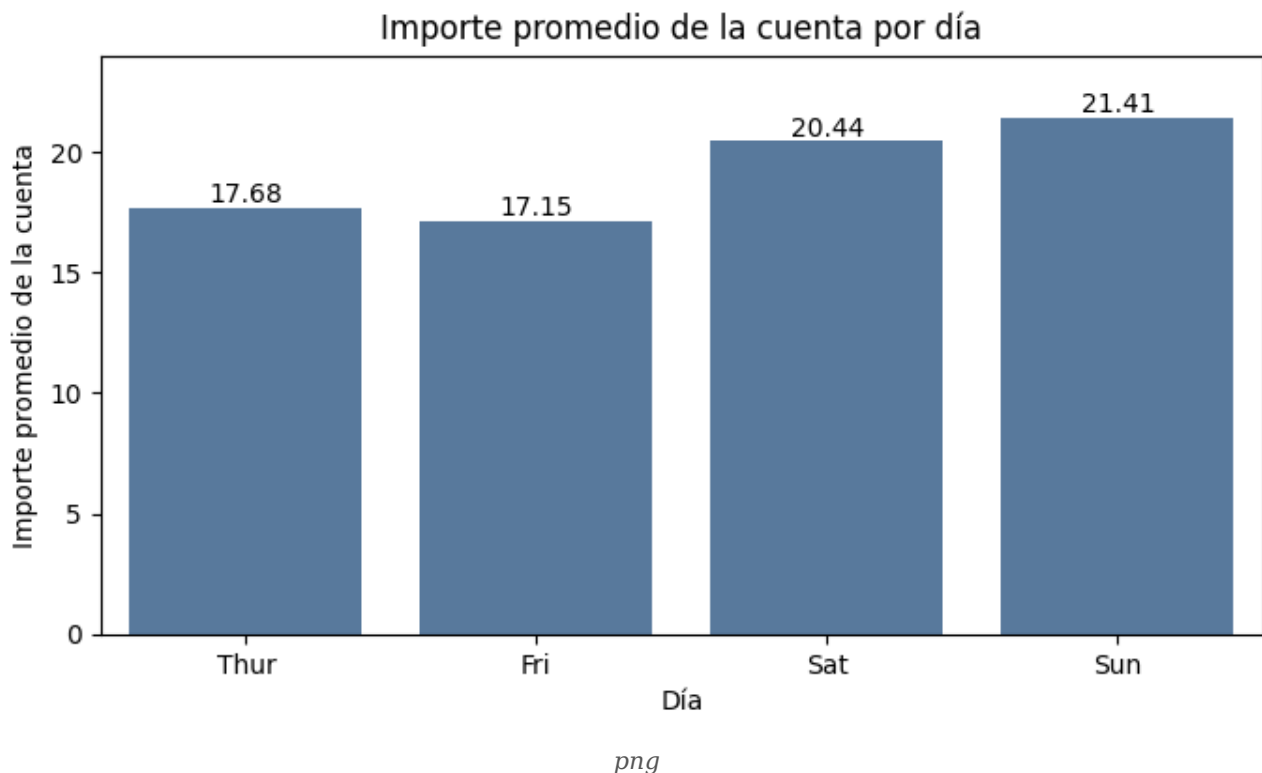
```
plt.figure(figsize=(8, 4))

grafico = sns.barplot(
    data=resumen_dia,
    x="day",
    y="promedio_cuenta",
    color="#4C78A8"
)

plt.title("Importe promedio de la cuenta por día")
plt.xlabel("Día")
plt.ylabel("Importe promedio de la cuenta")
plt.ylim(0, 24)

for barras in grafico.containers:
    grafico.bar_label(barras, fmt="%.2f")

plt.show()
```



El gráfico muestra que **Sat** y **Sun** tienen importes promedio más altos que **Thur** y **Fri**. La diferencia no es enorme, pero sí se observa un cambio entre los días de jueves y viernes, por un lado, y los días de fin de semana registrados, por otro.

Las etiquetas sobre las barras ayudan a leer los valores exactos sin depender solo del eje vertical. Aun así, este gráfico debe interpretarse junto con la tabla anterior, porque el promedio de cada día no está sostenido por la misma cantidad de registros. En particular, **Fri** tiene muchas menos cuentas que los demás días.

Hasta acá tenemos una primera comparación por día. Ahora vamos a sumar el horario para ver cómo se distribuyen las cuentas entre `Lunch` y `Dinner`.

Analizar día y horario juntos

El dataset también permite combinar dos variables categóricas: `day` y `time`. Esta combinación es útil porque no todos los días tienen registros en ambos horarios.

Antes de graficar o comparar importes, conviene revisar cuántas cuentas hay en cada combinación de día y horario. Esto nos ayuda a evitar interpretaciones débiles basadas en grupos con muy pocos datos.

```
cuentas_dia_horario = pd.crosstab(
    df["day"],
    df["time"]
).reindex(orden_dias)

cuentas_dia_horario
```

time	Lunch	Dinner
day		
Thur	61	1
Fri	7	12
Sat	0	87
Sun	0	76

La tabla muestra cuántas cuentas hay en cada combinación de día y horario. Esta información es clave para interpretar cualquier comparación posterior.

`Thur` aparece casi completamente asociado a `Lunch`: tiene 61 cuentas en almuerzo y solo 1 en cena. `Fri` tiene registros en ambos horarios, aunque son pocos en total. En cambio, `Sat` y `Sun` aparecen únicamente en `Dinner`, sin registros de `Lunch`.

Esto significa que no conviene comparar todas las combinaciones como si tuvieran la misma solidez. Por ejemplo, el único registro de `Dinner` en `Thur` no alcanza para hablar de una distribución o de un comportamiento típico. Del mismo modo, no podemos comparar almuerzos de `Sat` o `Sun` porque directamente no hay registros para esas combinaciones.

Ahora vamos a visualizar esta distribución de registros para que la diferencia entre días y horarios quede más clara.

Visualizar cuentas por día y horario

Vamos a construir un gráfico de barras para mostrar la cantidad de cuentas en cada día, separando por horario. Esta visualización no busca analizar importes todavía; su objetivo es mostrar cuántos datos hay detrás de cada combinación.

Este paso es importante porque, en un análisis exploratorio completo, antes de comparar promedios o distribuciones conviene saber si los grupos tienen suficientes registros.

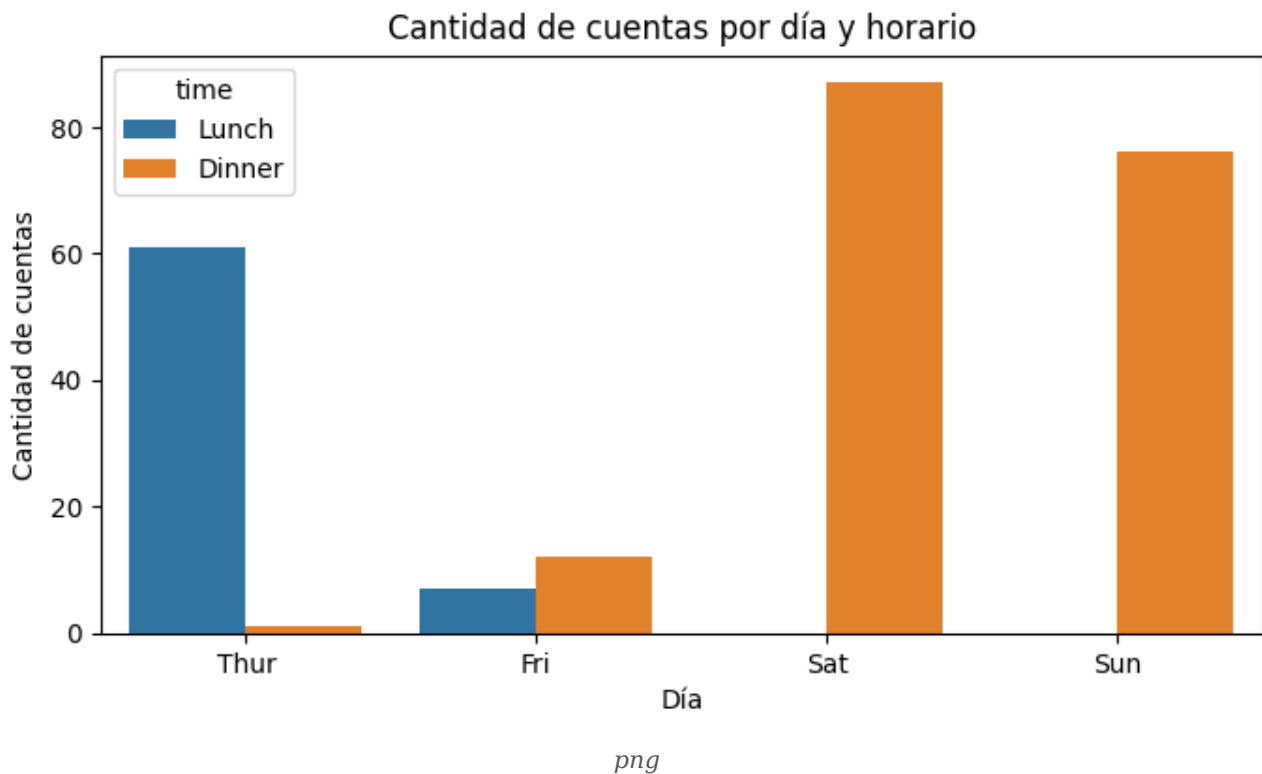
```
cuentas_dia_horario_grafico = (
    cuentas_dia_horario
    .reset_index()
    .melt(
        id_vars="day",
        var_name="time",
        value_name="cantidad_cuentas"
    )
)

plt.figure(figsize=(8, 4))

sns.barplot(
    data=cuentas_dia_horario_grafico,
    x="day",
    y="cantidad_cuentas",
    hue="time",
    order=orden_dias
)

plt.title("Cantidad de cuentas por día y horario")
plt.xlabel("Día")
plt.ylabel("Cantidad de cuentas")

plt.show()
```



El gráfico muestra la cantidad de cuentas por día y horario. La lectura visual confirma lo que ya aparecía en la tabla: `Thur` concentra casi todos sus registros en `Lunch`, `Sat` y `Sun` aparecen solo en `Dinner`, y `Fri` tiene pocos registros en ambos horarios.

Esta información es fundamental para el resto del análisis. Si más adelante observamos diferencias en importes promedio entre días u horarios, debemos recordar que no todos los grupos tienen la misma cantidad de datos. Algunas combinaciones, como `Thur` en `Dinner`, tienen tan pocos registros que no permiten una interpretación sólida.

En un análisis exploratorio completo, este tipo de revisión ayuda a evitar conclusiones apresuradas. No alcanza con ver que un grupo tiene un promedio alto o bajo: también necesitamos saber cuántas observaciones sostienen ese valor.

Comparar importes por horario

Ya revisamos los días y la distribución de registros por día y horario. Ahora podemos volver a una comparación más general: el importe total de la cuenta según el horario.

Vamos a calcular una tabla de resumen para `Lunch` y `Dinner`, incluyendo cantidad de cuentas, promedio, mediana, mínimo y máximo de `total_bill`.

```

resumen_horario = (
    df.groupby("time", observed=True)
    .agg(
        cantidad_cuentas=("total_bill", "count"),
        promedio_cuenta=("total_bill", "mean"),
        mediana_cuenta=("total_bill", "median"),
        minimo_cuenta=("total_bill", "min"),
        maximo_cuenta=("total_bill", "max")
    )
    .round(2)
    .reset_index()
)

resumen_horario

```

índice	time	cantidad_cuentas	promedio_cuenta	mediana_cuenta	minimo_cuenta	maximo_cuenta
0	Lunch	68	17.17	15.96	7.51	43.11
1	Dinner	176	20.80	18.39	3.07	50.81

La tabla muestra que **Dinner** tiene más cuentas que **Lunch**: 176 frente a 68. También tiene un promedio de cuenta más alto, 20.80 frente a 17.17, y una mediana más alta, 18.39 frente a 15.96.

Esto sugiere que, en este dataset, las cuentas de **Dinner** tienden a tener importes mayores que las cuentas de **Lunch**. Además, el valor máximo en **Dinner** llega a 50.81, mientras que en **Lunch** llega a 43.11.

De todos modos, la comparación debe leerse con cuidado porque los tamaños de los grupos son diferentes. **Dinner** tiene muchos más registros, y eso puede influir en la variedad de valores observados, incluidos los valores altos.

Visualizar la comparación por horario

Para complementar la tabla, vamos a usar un boxplot. Este gráfico permite comparar la distribución de **total_bill** entre **Lunch** y **Dinner**, mostrando mediana, dispersión y posibles valores extremos.

```

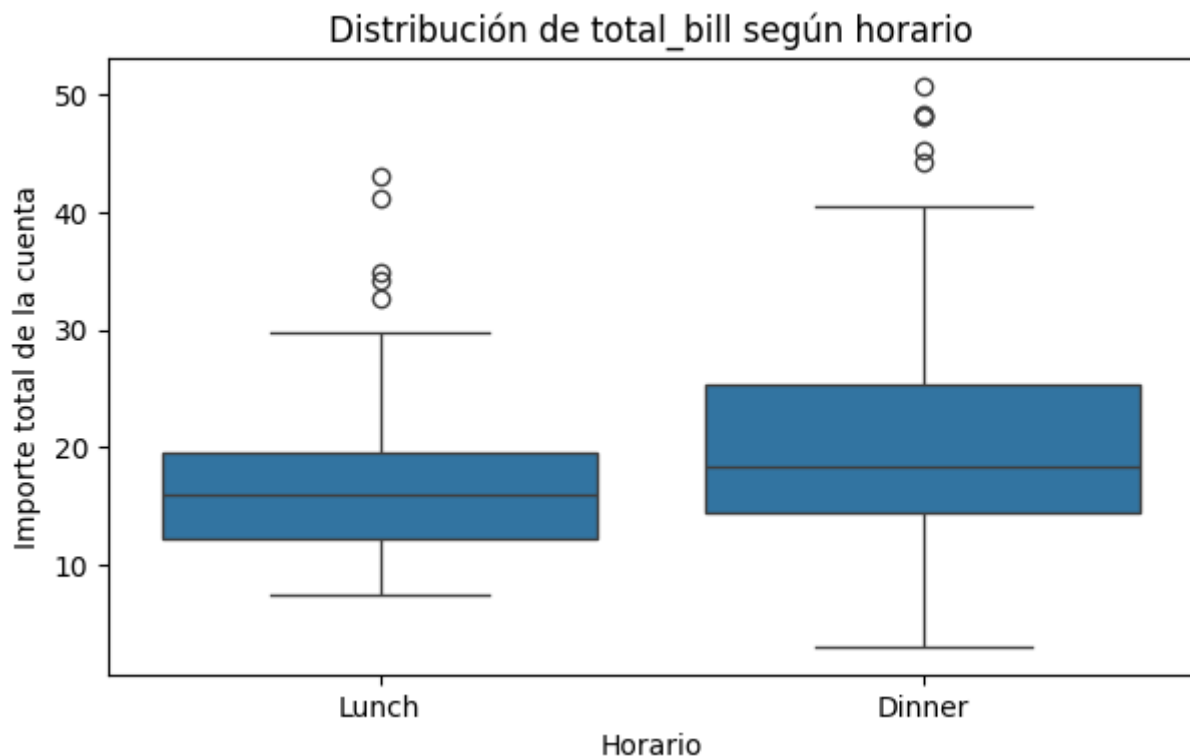
plt.figure(figsize=(7, 4))

sns.boxplot(
    data=df,
    x="time",
    y="total_bill"
)

plt.title("Distribución de total_bill según horario")
plt.xlabel("Horario")

```

```
plt.ylabel("Importe total de la cuenta")  
  
plt.show()
```



png

El boxplot muestra que la distribución de `total_bill` en `Dinner` está algo más desplazada hacia valores altos que la de `Lunch`. La mediana de `Dinner` es mayor, y la caja también aparece más extendida, lo que indica más dispersión en la mitad central de los datos.

En ambos horarios aparecen posibles valores extremos hacia importes altos. En `Lunch` hay algunas cuentas por encima de 30 y 40, mientras que en `Dinner` aparecen varios valores altos, incluso cercanos a 50.

Esta visualización complementa la tabla anterior: en este dataset, las cuentas de `Dinner` tienden a ser más altas que las de `Lunch`, pero también hay bastante variación dentro de cada horario. Además, como `Dinner` tiene muchos más registros, conviene no interpretar la cantidad de valores extremos sin recordar el tamaño de cada grupo.

Relacionar importe total y propina

Hasta ahora analizamos distribuciones y comparaciones entre grupos. Ahora vamos a mirar una relación entre dos variables numéricas: `total_bill` y `tip`.

La pregunta será: **¿las cuentas más altas suelen estar asociadas con propinas más altas?**

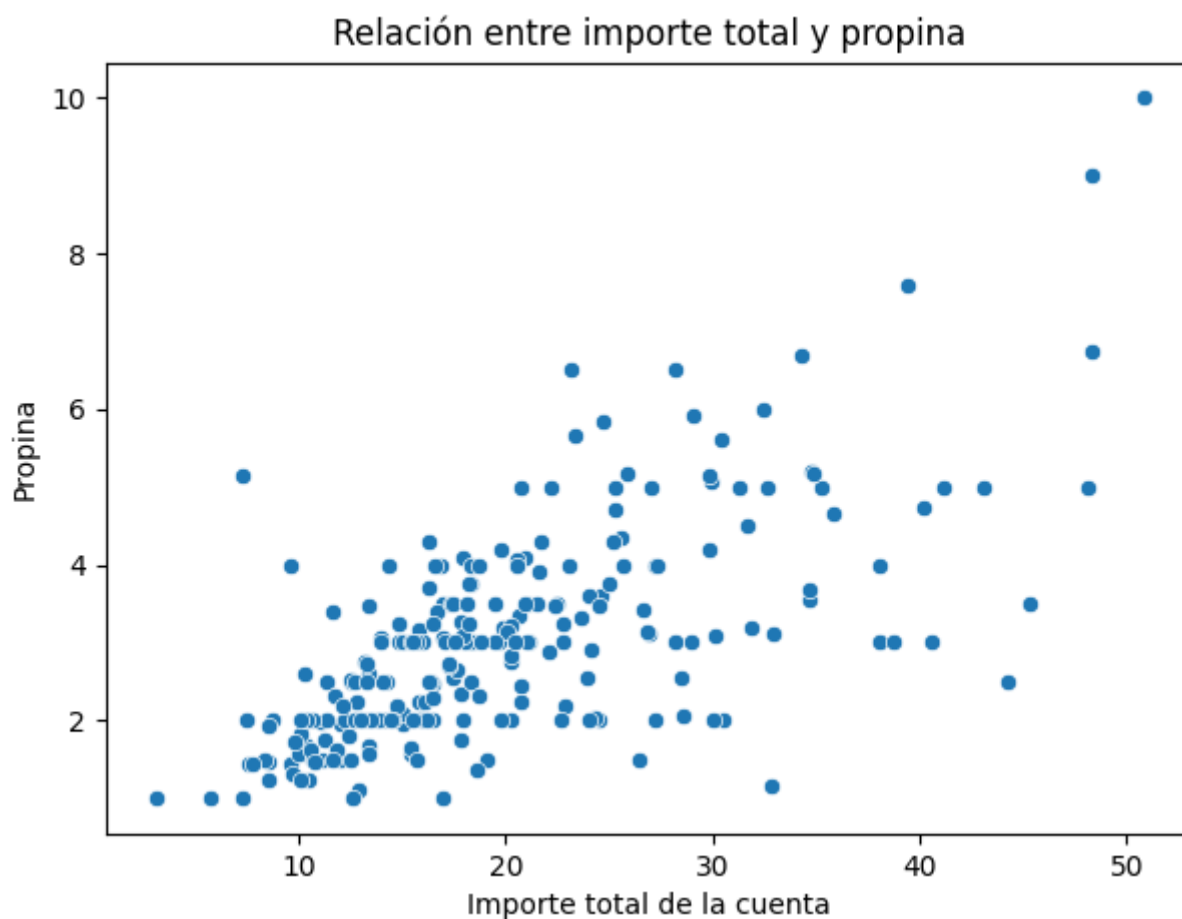
Para responderla, vamos a construir un gráfico de dispersión. Cada punto representará una cuenta del dataset.

```
plt.figure(figsize=(7, 5))

sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip"
)

plt.title("Relación entre importe total y propina")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Propina")

plt.show()
```



png

El gráfico de dispersión muestra la relación entre el importe total de la cuenta y la propina. Cada punto representa una cuenta del dataset.

La nube de puntos tiene una tendencia ascendente: en general, las cuentas con mayor `total_bill` tienden a estar asociadas con propinas más altas. Sin embargo, la relación no es perfecta. Para importes similares pueden aparecer propinas distintas, y también hay algunos puntos alejados del patrón general.

Esta visualización permite formular un hallazgo prudente: en este dataset, las cuentas más altas suelen estar asociadas con propinas más altas. No estamos diciendo que todos los casos sigan exactamente esa regla, ni que el importe total explique por sí solo el monto de la propina.

Medir la relación con correlación

Para complementar la lectura visual, vamos a calcular la correlación entre `total_bill` y `tip`.

La correlación resume la relación lineal entre dos variables numéricas. Sus valores van de -1 a 1: los valores positivos indican que ambas variables tienden a aumentar juntas, los valores negativos indican que una tiende a aumentar cuando la otra disminuye, y los valores cercanos a 0 indican poca relación lineal.

```
correlacion_total_tip = df["total_bill"].corr(df["tip"])  
  
round(float(correlacion_total_tip), 2)
```

0.68

La correlación entre `total_bill` y `tip` es 0.68. Es un valor positivo, por lo que coincide con la lectura visual del gráfico de dispersión: cuando el importe total de la cuenta aumenta, la propina tiende a aumentar también.

No se trata de una correlación perfecta, porque no vale 1. Esto también coincide con el gráfico: los puntos muestran una tendencia ascendente, pero no están ubicados exactamente sobre una línea.

En el contexto de este análisis exploratorio, podemos decir que existe una relación positiva moderada entre el importe total de la cuenta y la propina. Como siempre, la correlación ayuda a resumir una relación lineal, pero no reemplaza la visualización ni explica por sí sola las causas de esa relación.

Reunir los principales hallazgos

Después de revisar la estructura del dataset, resumir variables, contar categorías, visualizar distribuciones, comparar grupos y observar relaciones entre variables, podemos reunir algunos hallazgos principales.

En un análisis exploratorio, esta etapa es importante porque nos obliga a pasar de tablas y gráficos aislados a una lectura integrada. La idea no es escribir conclusiones exageradas, sino organizar lo que los datos muestran de manera prudente.

Algunos hallazgos posibles de este análisis son:

- El dataset tiene 244 cuentas registradas y no presenta valores faltantes en sus columnas.
- La mayoría de las cuentas tiene importes bajos o medios, con algunos valores altos menos frecuentes.
- Las propinas también se concentran en valores bajos o medios, aunque aparecen algunos valores altos aislados.
- `Sat` y `Sun` tienen importes promedio de cuenta más altos que `Thur` y `Fri`.
- `Fri` tiene pocos registros, por lo que sus valores deben interpretarse con cuidado.
- `Dinner` tiene más registros que `Lunch` y también presenta importes de cuenta más altos en promedio.
- La distribución de registros por día y horario no es uniforme: `Thur` aparece principalmente en `Lunch`, mientras que `Sat` y `Sun` aparecen solo en `Dinner`.
- Existe una relación positiva entre `total_bill` y `tip`: las cuentas más altas tienden a asociarse con propinas más altas.

Reconocer límites del análisis

Todo análisis exploratorio tiene límites. Reconocerlos no debilita el trabajo; al contrario, ayuda a interpretar los resultados con más cuidado.

En este caso, estamos trabajando con un dataset pequeño y conocido, usado frecuentemente para practicar análisis de datos. No tenemos fechas completas, información del restaurante, contexto económico, ubicación, menú, precios reales ni detalles sobre cómo se recolectaron los datos.

Por eso, los hallazgos deben leerse como observaciones dentro de este dataset, no como conclusiones generales sobre todos los restaurantes o todos los comportamientos de consumo.

Cierre del capítulo

En este capítulo integramos varias herramientas de análisis exploratorio. Revisamos la estructura del dataset, diferenciamos variables numéricas y categóricas, calculamos resúmenes, contamos categorías, construimos gráficos de distribución, comparamos grupos y analizamos la relación entre dos variables numéricas.

El recorrido mostró que un análisis exploratorio no es una lista de comandos sueltos. Es una forma ordenada de hacer preguntas, mirar los datos desde distintos ángulos y construir interpretaciones prudentes.

También vimos que las tablas y los gráficos se complementan. Las tablas nos dan valores precisos; los gráficos nos ayudan a ver formas, comparaciones y relaciones. Las conclusiones aparecen cuando conectamos ambas cosas con una pregunta clara.

En el próximo capítulo vamos a trabajar sobre cómo escribir conclusiones a partir de los datos, diferenciando observaciones, interpretaciones y límites.

Capítulo 039 — Escribir conclusiones a partir de los datos

Pasar del resultado a la conclusión

En el capítulo anterior integramos varias herramientas de análisis exploratorio. Revisamos la estructura del dataset, resumimos variables, contamos categorías, construimos gráficos, comparamos grupos y observamos relaciones entre variables.

Ahora vamos a trabajar sobre una parte distinta del análisis: la escritura de conclusiones.

Una conclusión no es simplemente copiar un número o describir un gráfico. Es una frase que interpreta un resultado, lo conecta con una pregunta y reconoce hasta dónde podemos llegar con los datos disponibles.

Por ejemplo, no es lo mismo escribir:

```
Dinner tiene un promedio de 20.80
```

que escribir:

```
En este dataset, las cuentas de Dinner tienden a tener importes más altos que las de Lunch, aunque la comparación debe leerse considerando que Dinner tiene muchos más registros.
```

La primera frase solo informa un valor. La segunda convierte ese valor en una conclusión más útil, porque lo compara, lo interpreta y marca una precaución.

Qué vamos a trabajar en este capítulo

En este capítulo vamos a usar algunos resultados obtenidos en el análisis exploratorio anterior, pero el foco no estará en volver a calcularlos desde cero. El objetivo será aprender a redactar conclusiones a partir de esos resultados.

Vamos a distinguir entre:

- observaciones;
- interpretaciones;
- conclusiones;
- límites del análisis.

También vamos a practicar una idea importante: una buena conclusión debe decir algo relevante, pero no debe afirmar más de lo que los datos permiten sostener.

Para eso vamos a trabajar con ejemplos basados en el dataset `tips`.

Resultados de partida

Tomaremos como punto de partida algunos resultados ya obtenidos:

- el dataset tiene 244 cuentas registradas y no presenta valores faltantes;
- la mayoría de las cuentas se concentra en importes bajos o medios;
- las propinas también se concentran en valores bajos o medios;
- `Sat` y `Sun` tienen importes promedio más altos que `Thur` y `Fri`;
- `Fri` tiene pocos registros en comparación con los demás días;
- `Dinner` tiene más registros que `Lunch` y también un importe promedio más alto;
- la distribución entre día y horario no es uniforme;
- la correlación entre `total_bill` y `tip` es aproximadamente 0.68.

A partir de estos resultados vamos a construir frases de análisis más claras y cuidadas.

Observación, interpretación y conclusión

Antes de escribir conclusiones, conviene separar tres niveles de escritura.

Una **observación** describe algo que aparece directamente en una tabla o gráfico. Por ejemplo: `Dinner` tiene 176 registros y `Lunch` tiene 68.

Una **interpretación** explica qué puede significar esa observación dentro del análisis. Por ejemplo: `la comparación entre horarios no está basada en grupos del mismo tamaño`.

Una **conclusión** integra la observación y la interpretación para responder una pregunta. Por ejemplo: `En este dataset, Dinner concentra más registros y presenta importes promedio más altos que Lunch, pero la diferencia debe leerse considerando el tamaño desigual de los grupos`.

Esta separación ayuda a escribir con más precisión. No todas las frases de un análisis tienen que ser conclusiones finales; muchas veces primero necesitamos describir, luego interpretar y recién después concluir.

Mejorar frases de análisis

Una misma información puede escribirse de maneras muy distintas. Algunas frases son demasiado pobres porque solo repiten un número. Otras son problemáticas porque exageran lo que los datos permiten afirmar.

La escritura de conclusiones requiere equilibrio: necesitamos decir algo claro, pero también respetar los límites del dataset.

Veamos algunos ejemplos.

Ejemplos de mejora de frases

Ejemplo 1: cantidad de registros por horario

Resultado observado:

`Dinner` tiene 176 registros y `Lunch` tiene 68.

Frase débil:

Hay más cenas que almuerzos.

Frase mejorada:

En este dataset, `Dinner` concentra muchos más registros que `Lunch`, por lo que las comparaciones entre horarios deben leerse considerando que los grupos tienen tamaños diferentes.

Ejemplo 2: promedio de cuenta por horario

Resultado observado:

El promedio de `total_bill` es 20.80 en `Dinner` y 17.17 en `Lunch`.

Frase débil:

Dinner es más caro.

Frase mejorada:

En este dataset, las cuentas de `Dinner` tienen un importe promedio más alto que las de `Lunch`. Sin embargo, esto describe los datos disponibles y no permite afirmar que todas las cenas sean más caras que todos los almuerzos.

Ejemplo 3: pocos registros en un grupo

Resultado observado:

`Fri` tiene solo 19 registros.

Frase débil:

Friday no sirve.

Frase mejorada:

`Fri` tiene pocos registros en comparación con los demás días, por lo que sus promedios y comparaciones deben interpretarse con más cautela.

Ejemplo 4: correlación entre variables**Resultado observado:**

La correlación entre `total_bill` y `tip` es 0.68.

Frase débil:

El total causa la propina.

Frase mejorada:

Existe una relación positiva moderada entre `total_bill` y `tip`: en general, las cuentas más altas tienden a asociarse con propinas más altas. Esto no implica causalidad ni explica por sí solo el comportamiento de las propinas.

La tabla muestra que una conclusión no debería limitarse a repetir un resultado numérico. También debe explicar qué significa ese resultado dentro del análisis.

Por ejemplo, decir que `Dinner` tiene 176 registros y `Lunch` tiene 68 es correcto, pero todavía no alcanza. La conclusión mejora cuando conectamos ese dato con una consecuencia analítica: las comparaciones entre horarios no están sostenidas por la misma cantidad de observaciones.

También aparece otro cuidado importante: evitar afirmaciones demasiado generales. Si en este dataset el promedio de `Dinner` es más alto que el de `Lunch`, podemos decir eso mismo, pero no deberíamos convertirlo en una regla universal sobre todos los restaurantes, todas las cenas o todos los almuerzos.

Una buena conclusión suele combinar tres elementos: el resultado observado, una interpretación razonable y una advertencia sobre el alcance de esa interpretación.

Evitar conclusiones exageradas

Uno de los errores más comunes al escribir conclusiones es exagerar el resultado. Esto puede ocurrir cuando usamos palabras demasiado fuertes, cuando generalizamos más allá del dataset o cuando confundimos una asociación con una causa.

Por ejemplo, a partir del análisis de `tips`, sería exagerado escribir:

Las personas siempre dejan más propina cuando la cuenta es más alta.

El gráfico de dispersión y la correlación sugieren una relación positiva entre `total_bill` y `tip`, pero no muestran una regla perfecta. Hay cuentas con importes parecidos y propinas diferentes. Además, el dataset no contiene toda la información necesaria para explicar por qué una persona dejó una propina determinada.

Una versión más prudente sería:

En este dataset, las cuentas con importes más altos tienden a estar asociadas con propinas más altas, aunque la relación no es perfecta y no permite afirmar causalidad.

Algunas expresiones útiles

Al escribir conclusiones exploratorias, pueden ser útiles algunas expresiones que ayudan a mantener la prudencia.

En este dataset...

Sirve para recordar que la conclusión se refiere a los datos analizados, no a una regla general.

Ejemplo:

En este dataset, Dinner concentra más registros que Lunch.

Los datos sugieren...

Sirve para presentar una lectura posible sin convertirla en una afirmación definitiva.

Ejemplo:

Los datos sugieren una relación positiva entre `total_bill` y `tip`.

Tiende a...

Sirve para describir patrones generales, especialmente cuando no todos los casos siguen exactamente la misma regla.

Ejemplo:

La propina tiende a aumentar cuando aumenta el importe total de la cuenta.

Debe interpretarse con cuidado...

Sirve para señalar límites, como pocos registros o grupos desbalanceados.

Ejemplo:

El promedio de Fri debe interpretarse con cuidado porque hay pocos registros.

No alcanza para afirmar...

Sirve para evitar conclusiones causales o demasiado fuertes.

Ejemplo:

Esta relación no alcanza para afirmar causalidad.

Dentro de los datos analizados...

Funciona de manera parecida a “en este dataset” y ayuda a ubicar el alcance de la conclusión.

Ejemplo:

Dentro de los datos analizados, Sun presenta el mayor promedio de cuenta.

Estas expresiones no son fórmulas obligatorias, pero ayudan a construir un tono analítico más preciso. En análisis exploratorio, muchas veces no buscamos demostrar una afirmación definitiva, sino describir patrones, diferencias y relaciones que aparecen en los datos.

La clave está en evitar dos extremos. Por un lado, no queremos escribir frases tan débiles que solo repitan números sin interpretación. Por otro lado, tampoco queremos conclusiones tan fuertes que los datos no puedan sostener.

El objetivo es escribir frases claras, útiles y honestas con la información disponible.

Describir, comparar y concluir

Cuando escribimos sobre datos, no todas las frases cumplen la misma función. Algunas frases describen un resultado, otras comparan valores y otras construyen una conclusión más completa.

Una **descripción** dice qué aparece en una tabla o gráfico. Por ejemplo: Sun tiene un promedio de cuenta de 21.41.

Una **comparación** pone ese valor en relación con otros. Por ejemplo: Sun tiene un promedio más alto que Thur, Fri y Sat.

Una **conclusión** interpreta esa comparación dentro del análisis. Por ejemplo: Dentro de los datos analizados, Sun presenta el mayor importe promedio de cuenta, aunque esta lectura debe considerarse junto con la cantidad de registros disponibles para cada día.

Las tres formas son útiles. El problema aparece cuando un análisis se queda solo en la descripción y no avanza hacia una interpretación, o cuando salta demasiado rápido a una conclusión sin mostrar en qué resultado se apoya.

Ejemplo con el promedio por día

Tomemos uno de los resultados del análisis anterior: los importes promedio de cuenta por día.

Los valores eran:

- **Thur** : 17.68
- **Fri** : 17.15
- **Sat** : 20.44
- **Sun** : 21.41

Una descripción posible sería:

Sun tiene un importe promedio de cuenta de 21.41.

Una comparación agregaría más contexto:

Sun tiene el importe promedio de cuenta más alto, seguido por Sat.

Una conclusión más completa podría ser:

Dentro de los datos analizados, Sun y Sat presentan importes promedio de cuenta más altos que Thur y Fri. Esta diferencia sugiere un cambio entre los días registrados de fin de semana y los otros días, aunque la interpretación debe considerar que la cantidad de registros no es igual para todos los días.

En el ejemplo anterior, la conclusión no agrega una técnica nueva. Lo que hace es organizar mejor la lectura del resultado.

Primero identifica el patrón principal: **Sun** y **Sat** tienen promedios más altos. Después evita una afirmación demasiado fuerte: no dice que “los fines de semana siempre se gasta más”, porque el dataset no permite sostener una conclusión general de ese tipo. Finalmente, incorpora una precaución metodológica: los días no tienen la misma cantidad de registros.

Esa combinación es una buena práctica al escribir conclusiones exploratorias: decir algo relevante, pero mantener claro el alcance de la afirmación.

Usar límites como parte de la conclusión

A veces se piensa que mencionar límites debilita una conclusión. En realidad, suele mejorarla. Una conclusión con límites claros es más confiable que una afirmación demasiado amplia.

Por ejemplo, podríamos escribir:

Dinner tiene cuentas más altas que Lunch.

Esa frase apunta en una dirección correcta, pero queda incompleta. Una versión mejor sería:

En este dataset, Dinner presenta un importe promedio de cuenta más alto que Lunch. Sin embargo, Dinner también tiene muchos más registros, por lo que la comparación debe leerse considerando el tamaño desigual de los grupos.

La segunda versión no niega el resultado. Lo hace más preciso.

Tipos de límites frecuentes

En análisis exploratorio, algunos límites aparecen con mucha frecuencia. Conviene aprender a reconocerlos para incorporarlos a la escritura cuando corresponda.

Un primer límite es el **tamaño del dataset**. Si tenemos pocos registros, las conclusiones deben ser más prudentes.

Otro límite es el **desbalance entre grupos**. Si un grupo tiene muchos datos y otro tiene muy pocos, comparar promedios puede ser útil, pero requiere cuidado.

También está el límite del **contexto disponible**. Si no sabemos cómo se recolectaron los datos, en qué período, en qué lugar o bajo qué condiciones, no conviene generalizar demasiado.

Finalmente, existe el límite de la **causalidad**. Que dos variables se relacionen no significa automáticamente que una cause la otra.

Convertir hallazgos en un párrafo

Hasta ahora vimos frases individuales. Pero en un informe o en un cuaderno de análisis, muchas veces necesitamos escribir párrafos completos.

Un buen párrafo de conclusión no debería ser una lista desordenada de resultados. Conviene que tenga una idea principal y que los resultados funcionen como apoyo de esa idea.

Por ejemplo, si queremos escribir sobre diferencias entre días, no alcanza con acumular datos sueltos. Necesitamos conectar los valores, compararlos y marcar alguna precaución.

Supongamos que queremos escribir una conclusión sobre los importes por día.

Una versión débil podría ser:

Thur tiene 17.68. Fri tiene 17.15. Sat tiene 20.44. Sun tiene 21.41. Sun es el mayor.

Esa frase no es incorrecta, pero se parece más a una repetición de la tabla que a una conclusión.

Una versión más clara podría ser:

Dentro de los datos analizados, los días con mayor importe promedio de cuenta son Sun y Sat. Ambos superan a Thur y Fri, lo que sugiere una diferencia entre los días registrados de fin de semana y los demás días del dataset. Sin embargo, esta comparación debe leerse con cautela porque la cantidad de registros no es igual para todos los días, especialmente en Fri, que tiene pocos casos.

La segunda versión organiza mejor la lectura. No repite todos los valores, identifica el patrón principal y agrega un límite importante para interpretar la comparación.

Una estructura posible para escribir conclusiones

Una forma simple de escribir un párrafo de conclusión es seguir esta estructura:

1. Presentar el patrón principal.
2. Mencionar el resultado que lo sostiene.
3. Agregar una interpretación prudente.
4. Señalar un límite si corresponde.

Por ejemplo:

En este dataset, Dinner presenta importes de cuenta más altos que Lunch. Esta lectura se apoya en que Dinner tiene un promedio de total_bill de 20.80, frente a 17.17 en Lunch. La diferencia sugiere que las cuentas de cena tienden a ser más altas dentro de los datos analizados. Sin embargo, Dinner también tiene muchos más registros que Lunch, por lo que la comparación debe interpretarse considerando el tamaño desigual de los grupos.

Esta estructura no es una fórmula obligatoria, pero ayuda a evitar dos problemas frecuentes: quedarse solo en números sueltos o escribir conclusiones demasiado generales.

Conclusiones sobre relaciones entre variables

Cuando escribimos sobre relaciones entre variables numéricas, también debemos ser cuidadosos. En el análisis anterior vimos que la correlación entre `total_bill` y `tip` era aproximadamente 0.68, y que el gráfico de dispersión mostraba una tendencia ascendente.

Una conclusión débil sería:

`total_bill` y `tip` están relacionados.

Una conclusión exagerada sería:

Si aumenta `total_bill`, aumenta automáticamente `tip`.

Una versión más adecuada sería:

En este dataset, `total_bill` y `tip` presentan una relación positiva moderada. Las cuentas con importes más altos tienden a estar asociadas con propinas más altas, aunque la relación no es perfecta y no permite afirmar causalidad.

Esta última frase es mejor porque indica la dirección de la relación, reconoce que hay variabilidad y evita transformar una correlación en una causa.

Evitar tres errores comunes

Al redactar conclusiones a partir de datos, conviene evitar tres errores frecuentes.

El primero es **copiar números sin interpretarlos**. Los valores son importantes, pero una conclusión debe explicar qué significan dentro del análisis.

El segundo es **generalizar más allá del dataset**. Si los datos corresponden a una muestra específica, no podemos convertirlos automáticamente en una regla universal.

El tercero es **confundir relación con causalidad**. Una correlación o una tendencia visual puede mostrar asociación, pero no demuestra por sí sola que una variable cause a la otra.

Evitar estos errores ayuda a escribir conclusiones más claras, más útiles y más honestas con los datos disponibles.

De varios hallazgos a un cierre general

Al final de un análisis, podemos reunir varios hallazgos en un cierre general. Ese cierre no debería intentar repetir todo lo que se hizo, sino seleccionar las ideas más importantes.

Por ejemplo, a partir del análisis exploratorio del dataset `tips`, podríamos escribir un cierre como este:

El análisis exploratorio muestra que la mayoría de las cuentas y propinas se concentra en valores bajos o medios, con algunos valores altos menos frecuentes. También se observan diferencias entre días y horarios: Sat y Sun presentan importes promedio de cuenta más altos, mientras que Dinner concentra más registros y tiene un promedio mayor que Lunch. Además, `total_bill` y `tip` muestran una relación positiva moderada. Estos hallazgos deben interpretarse dentro del alcance del dataset, considerando que algunos grupos tienen pocos registros y que no contamos con información contextual adicional sobre la recolección de los datos.

Este párrafo integra distribución, comparación entre grupos, relación entre variables y límites del análisis.

Cierre del capítulo

En este capítulo trabajamos sobre la escritura de conclusiones a partir de datos. El foco no estuvo en calcular nuevos resultados, sino en aprender a comunicar mejor los resultados de un análisis exploratorio.

Vimos que una buena conclusión no se limita a copiar números ni a describir un gráfico. Una conclusión útil conecta un resultado con una interpretación, responde una pregunta y reconoce los límites de lo que puede afirmarse.

También diferenciamos entre observación, interpretación y conclusión. Esta separación ayuda a escribir con más precisión: primero identificamos qué muestran los datos, luego pensamos qué significa ese resultado dentro del análisis y finalmente redactamos una conclusión prudente.

Además, revisamos errores frecuentes, como generalizar más allá del dataset, usar frases demasiado fuertes o confundir correlación con causalidad. En análisis exploratorio, muchas veces trabajamos con patrones y señales, no con demostraciones definitivas.

La idea central del capítulo es que escribir conclusiones también forma parte del análisis de datos. No alcanza con producir tablas y gráficos: necesitamos convertir esos resultados en explicaciones claras, honestas y útiles para quien lee.

En el próximo capítulo vamos a usar esta idea para armar un informe breve de análisis exploratorio, organizando preguntas, resultados, visualizaciones y conclusiones dentro de una estructura más completa.

Capítulo 040 — Armar un informe breve de análisis exploratorio

Del análisis al informe

En los capítulos anteriores analizamos el dataset `tips` desde distintas perspectivas: revisamos su estructura, resumimos variables numéricas, contamos categorías, comparamos grupos, construimos visualizaciones y escribimos conclusiones prudentes a partir de los resultados.

En este capítulo vamos a dar un paso más: organizar esos resultados en un informe breve de análisis exploratorio.

Un informe no tiene que incluir todo lo que hicimos durante el análisis. Al contrario, una parte importante del trabajo consiste en seleccionar qué resultados son realmente útiles para responder una pregunta o comunicar una idea principal. Por eso, el foco de este capítulo no estará en aplicar muchas herramientas nuevas, sino en ordenar, elegir y presentar mejor la información.

Vamos a trabajar con una estructura simple:

- propósito del informe;
- breve descripción del dataset;
- resultados principales;
- visualizaciones relevantes;
- conclusiones;
- límites del análisis.

La idea es construir un informe claro, breve y honesto con los datos disponibles.

Datos de referencia

Vamos a seguir trabajando con el dataset `tips`, ya analizado en capítulos anteriores. Sabemos que contiene 244 registros y 7 columnas, con información sobre cuentas, propinas, día, horario, tamaño de mesa y algunas variables categóricas asociadas.

En este capítulo no vamos a volver a explorar el dataset desde cero. Vamos a usar esos resultados conocidos como base para armar un informe breve de análisis exploratorio.

La siguiente celda solo carga las librerías y el dataset; no produce salida visible:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")
```

Definir el propósito del informe

Antes de elegir tablas, gráficos o conclusiones, conviene definir qué quiere comunicar el informe. Un análisis exploratorio puede producir muchos resultados, pero un informe breve necesita seleccionar solo los más importantes.

En este caso, vamos a armar un informe con este propósito:

Describir los patrones generales de cuentas y propinas en el dataset `tips`, prestando atención a las diferencias por día y horario, y a la relación entre el importe total de la cuenta y la propina.

Este propósito nos ayuda a decidir qué incluir y qué dejar afuera. Por ejemplo, no necesitamos mostrar todas las tablas posibles ni repetir cada gráfico trabajado en capítulos anteriores. Vamos a seleccionar algunos resultados que permitan responder estas preguntas:

- ¿cómo se distribuyen los importes de las cuentas y las propinas?
- ¿hay diferencias relevantes entre días u horarios?
- ¿qué relación aparece entre el importe total de la cuenta y la propina?
- ¿qué límites debemos tener en cuenta al interpretar estos resultados?

Con esta orientación, el informe será breve, pero no será una simple acumulación de números.

Seleccionar resultados relevantes

Una vez definido el propósito, el siguiente paso es elegir qué resultados vamos a usar. En un informe breve, seleccionar es tan importante como calcular.

Para este informe vamos a recuperar algunos resultados ya conocidos y organizarlos en tablas pequeñas:

- un resumen general de las variables numéricas principales;
- una comparación por horario;
- una comparación por día.

Estas tablas no serán el informe completo. No buscamos rehacer todo el análisis, sino recuperar una base mínima para elegir qué comunicar.

```
resumen_general = df[["total_bill", "tip", "size"]].agg(["mean", "median", "min",
"max"]).round(2)

resumen_por_horario = (
    df.groupby("time", observed=True)
    .agg(
        cantidad_cuentas=("total_bill", "count"),
        promedio_cuenta=("total_bill", "mean"),
        mediana_cuenta=("total_bill", "median"),
        promedio_propina=("tip", "mean")
    )
    .round(2)
)

resumen_por_dia = (
    df.groupby("day", observed=True)
    .agg(
        cantidad_cuentas=("total_bill", "count"),
        promedio_cuenta=("total_bill", "mean"),
        mediana_cuenta=("total_bill", "median"),
        promedio_propina=("tip", "mean")
    )
    .round(2)
)

display(resumen_general)
display(resumen_por_horario)
display(resumen_por_dia)
```

índice	total_bill	tip	size
mean	19.79	3.0	2.57
median	17.80	2.9	2.00
min	3.07	1.0	1.00
max	50.81	10.0	6.00

time	cantidad_cuentas	promedio_cuenta	mediana_cuenta	promedio_propina
Lunch	68	17.17	15.96	2.73
Dinner	176	20.80	18.39	3.10

day	cantidad_cuentas	promedio_cuenta	mediana_cuenta	promedio_propina
Thur	62	17.68	16.20	2.77
Fri	19	17.15	15.38	2.73

day	cantidad_cuentas	promedio_cuenta	mediana_cuenta	promedio_propina
Sat	87	20.44	18.24	2.99
Sun	76	21.41	19.63	3.26

Interpretar qué resultados conviene usar

Las tablas anteriores resumen varios resultados útiles, pero un informe breve no debería limitarse a pegarlas una detrás de otra. Primero conviene identificar qué información aporta cada una.

El resumen general muestra que las cuentas tienen una media de 19.79 y una mediana de 17.80. Como la media es algo mayor que la mediana y el valor máximo llega a 50.81, los datos sugieren que hay algunas cuentas altas que empujan el promedio hacia arriba. Algo parecido ocurre con las propinas: la mayoría se ubica en valores bajos o medios, aunque aparecen algunos valores más altos.

La comparación por horario muestra que `Dinner` concentra más registros que `Lunch` y también presenta un promedio de cuenta más alto. Esta diferencia puede ser relevante para el informe, pero debe interpretarse considerando que los grupos no tienen el mismo tamaño.

La comparación por día muestra que `Sun` y `Sat` tienen los promedios de cuenta más altos, mientras que `Thur` y `Fri` tienen valores más bajos. En particular, `Fri` debe leerse con cuidado porque tiene pocos registros en comparación con los otros días.

A partir de esto, para el informe breve conviene seleccionar tres ideas principales:

- la distribución general de cuentas y propinas;
- las diferencias entre días y horarios;
- la relación entre el importe total de la cuenta y la propina.

Elegir visualizaciones para el informe

Un informe breve no necesita incluir todos los gráficos posibles. Es mejor elegir pocas visualizaciones, pero que acompañen directamente las ideas principales.

Primero vamos a incluir una visualización sobre la distribución de `total_bill` y `tip`. Esto permite mostrar cómo se concentran los valores y si aparecen algunos casos altos menos frecuentes.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

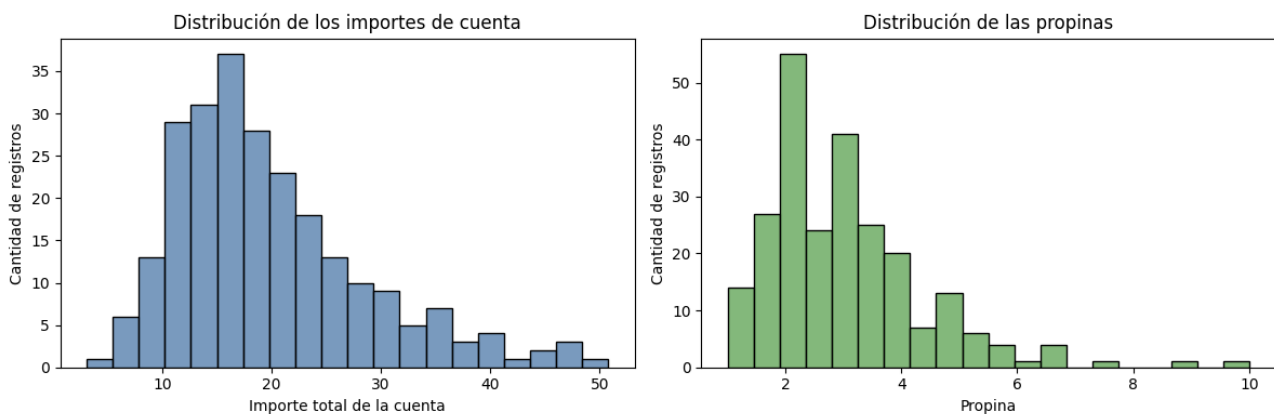
```

sns.histplot(data=df, x="total_bill", bins=20, ax=axes[0], color="#4C78A8")
axes[0].set_title("Distribución de los importes de cuenta")
axes[0].set_xlabel("Importe total de la cuenta")
axes[0].set_ylabel("Cantidad de registros")

sns.histplot(data=df, x="tip", bins=20, ax=axes[1], color="#59A14F")
axes[1].set_title("Distribución de las propinas")
axes[1].set_xlabel("Propina")
axes[1].set_ylabel("Cantidad de registros")

plt.tight_layout()
plt.show()

```



png

Los histogramas muestran que `total_bill` y `tip` no se distribuyen de manera uniforme. En ambos casos, la mayor parte de los registros se concentra en valores bajos o medios, mientras que los valores más altos aparecen con menor frecuencia.

En `total_bill`, la concentración principal se observa aproximadamente entre importes de 10 y 25, con una cola hacia valores más altos. En `tip`, la mayor concentración aparece entre propinas cercanas a 2 y 4, aunque existen algunos casos aislados con propinas más altas.

Para el informe breve, esta visualización puede servir para comunicar una primera idea general: dentro de los datos analizados, las cuentas y propinas más frecuentes se ubican en rangos moderados, pero hay algunos valores altos que conviene tener presentes al interpretar promedios.

Visualizar diferencias por día y horario

Después de observar la distribución general, podemos elegir una visualización que ayude a comparar grupos. Como el propósito del informe incluye diferencias por día y horario, vamos a construir gráficos de barras con el promedio de cuenta según estas variables.

Estos gráficos no reemplazan a las tablas anteriores, pero ayudan a comunicar el patrón principal de manera más rápida.

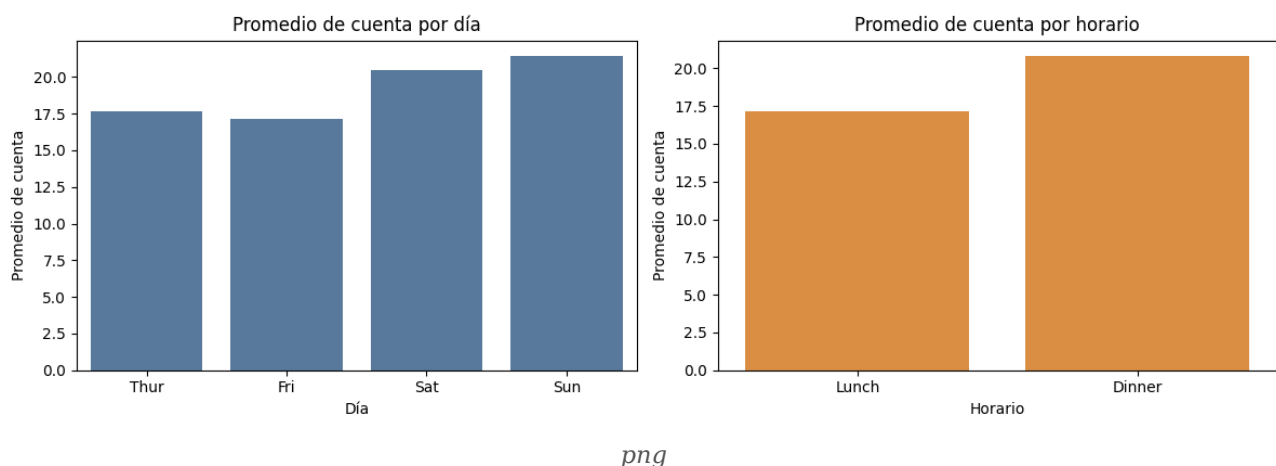
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

orden_dias = ["Thur", "Fri", "Sat", "Sun"]

sns.barplot(data=df, x="day", y="total_bill", order=orden_dias, errorbar=None,
ax=axes[0], color="#4C78A8")
axes[0].set_title("Promedio de cuenta por día")
axes[0].set_xlabel("Día")
axes[0].set_ylabel("Promedio de cuenta")

sns.barplot(data=df, x="time", y="total_bill", errorbar=None, ax=axes[1],
color="#F28E2B")
axes[1].set_title("Promedio de cuenta por horario")
axes[1].set_xlabel("Horario")
axes[1].set_ylabel("Promedio de cuenta")

plt.tight_layout()
plt.show()
```



Los gráficos de barras muestran que el promedio de cuenta es más alto en **Sun** y **Sat** que en **Thur** y **Fri**. También se observa que **Dinner** tiene un promedio de cuenta más alto que **Lunch**.

Estos resultados son útiles para el informe porque resumen diferencias entre grupos de manera visual. Sin embargo, deben interpretarse junto con las cantidades de registros vistas antes: **Fri** tiene pocos casos y **Dinner** tiene muchos más registros que **Lunch**.

Por eso, en el informe no alcanza con decir que un grupo tiene mayor promedio. También conviene aclarar si la comparación está apoyada en grupos de tamaños similares o si requiere cautela.

Visualizar la relación entre cuenta y propina

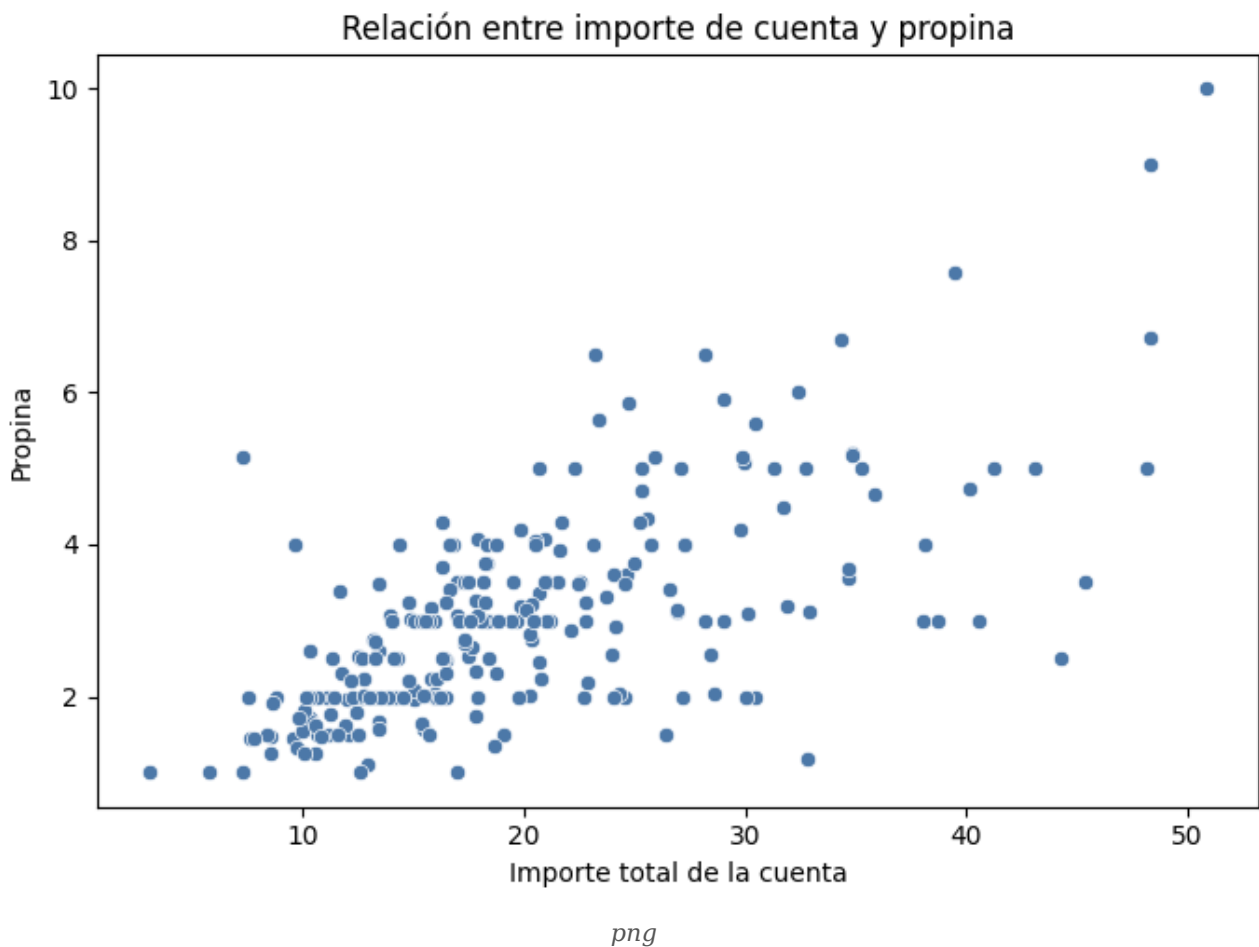
El tercer resultado importante para el informe es la relación entre el importe total de la cuenta y la propina. Para comunicarlo, podemos usar un gráfico de dispersión.

Este gráfico permite observar si, dentro del dataset, las cuentas más altas tienden a estar asociadas con propinas más altas.

```
plt.figure(figsize=(7, 5))

sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    color="#4C78A8"
)

plt.title("Relación entre importe de cuenta y propina")
plt.xlabel("Importe total de la cuenta")
plt.ylabel("Propina")
plt.tight_layout()
plt.show()
```



El gráfico de dispersión muestra una relación positiva entre `total_bill` y `tip`: en general, las cuentas de mayor importe tienden a estar asociadas con propinas más altas.

De todos modos, la relación no es perfecta. Para importes similares aparecen propinas diferentes, y también hay algunos casos alejados del conjunto principal de puntos. Por eso, esta visualización permite comunicar una tendencia general, pero no alcanza para explicar por sí sola por qué se deja una determinada propina.

En el informe breve, este gráfico puede acompañar una conclusión prudente: dentro de los datos analizados, existe una asociación positiva entre el importe total de la cuenta y la propina, sin afirmar causalidad.

Organizar el informe

Ahora que seleccionamos los resultados principales, podemos ordenar el informe. Una estructura breve y clara podría ser:

- 1. Propósito del análisis**

Explicar qué se quiere observar en el dataset.

- 2. Descripción general de los datos**

Presentar el dataset de manera breve, sin repetir toda la exploración inicial.

- 3. Resultados principales**

Incluir los hallazgos más importantes sobre distribuciones, diferencias entre grupos y relación entre variables.

- 4. Conclusiones**

Integrar los resultados en una lectura general.

- 5. Límites del análisis**

Aclarar qué aspectos deben interpretarse con cuidado.

Esta estructura ayuda a que el informe no sea una sucesión de tablas y gráficos sin conexión. Cada parte cumple una función dentro del texto.

Decidir qué dejar afuera

Armar un informe también implica descartar información. Durante un análisis exploratorio podemos generar muchas tablas, gráficos y comparaciones, pero no todas tienen la misma importancia para comunicar el resultado final.

En este informe no vamos a incluir todas las salidas posibles del análisis de `tips`. Por ejemplo, no hace falta repetir cada conteo de categorías ni todos los gráficos contruidos en capítulos anteriores. Tampoco necesitamos mostrar todas las combinaciones entre variables si no aportan directamente al propósito definido.

Para mantener el informe breve, vamos a priorizar:

- la distribución general de cuentas y propinas;
- las diferencias principales por día y horario;
- la relación entre importe de cuenta y propina;
- las precauciones necesarias para interpretar esos resultados.

Esta selección permite construir un informe más claro y enfocado.

Informe breve de análisis exploratorio

Propósito del análisis

El propósito de este informe es describir algunos patrones generales del dataset `tips`, prestando atención a los importes de las cuentas, las propinas, las diferencias por día y horario, y la relación entre el importe total de la cuenta y la propina.

No se busca explicar causalmente el comportamiento de las propinas, sino resumir los principales resultados observados durante el análisis exploratorio.

Descripción general del dataset

El dataset contiene 244 registros y 7 columnas. Cada registro representa una cuenta e incluye información como el importe total (`total_bill`), la propina (`tip`), el día (`day`), el horario (`time`) y el tamaño de la mesa (`size`), entre otras variables.

Como se trata de un dataset acotado y ya preparado para el análisis, los resultados deben interpretarse dentro de ese alcance. No contamos, por ejemplo, con información detallada sobre el período de recolección, el tipo de restaurante o el contexto en que se registraron los datos.

Resultados principales

En primer lugar, los importes de las cuentas se concentran principalmente en valores bajos o medios. La media de `total_bill` es 19.79 y la mediana es 17.80, mientras que el valor máximo llega a 50.81. Esto sugiere que hay algunas cuentas altas que pueden influir sobre el promedio.

Algo similar ocurre con las propinas. La media de `tip` es 3.00 y la mediana es 2.90, con un valor máximo de 10.00. Los histogramas muestran que la mayoría de las propinas se ubica en valores moderados, aunque existen algunos casos más altos.

Al comparar por horario, `Dinner` concentra más registros que `Lunch` y también presenta un promedio de cuenta más alto. En los datos analizados, el promedio de `total_bill` es 20.80 para `Dinner` y 17.17 para `Lunch`. Esta diferencia debe interpretarse considerando que los grupos no tienen el mismo tamaño: `Dinner` tiene 176 registros, mientras que `Lunch` tiene 68.

También aparecen diferencias entre días. `Sun` y `Sat` presentan los promedios de cuenta más altos, con valores de 21.41 y 20.44 respectivamente. `Thur` y `Fri` tienen promedios menores, aunque `Fri` debe leerse con especial cuidado porque solo cuenta con 19 registros.

Por último, el gráfico de dispersión entre `total_bill` y `tip` muestra una relación positiva: en general, las cuentas de mayor importe tienden a estar asociadas con propinas más altas. Sin embargo, la relación no es perfecta y no permite afirmar causalidad.

Conclusiones

Dentro de los datos analizados, las cuentas y propinas se concentran principalmente en valores bajos o medios, con algunos valores altos menos frecuentes. También se observan diferencias por día y horario: `Dinner` presenta un promedio de cuenta más alto que `Lunch`, y `Sun` y `Sat` muestran los promedios más altos entre los días registrados.

Además, existe una asociación positiva entre el importe total de la cuenta y la propina. Esto significa que, en general, las cuentas más altas tienden a estar acompañadas por propinas más altas, aunque la variabilidad observada indica que esa relación no es exacta.

Límites del análisis

Estos resultados deben interpretarse con prudencia. El dataset tiene un tamaño limitado y algunos grupos, como `Fri`, cuentan con pocos registros. Además, no todos los días y horarios están representados de la misma manera, por lo que algunas comparaciones pueden estar influidas por el desbalance entre grupos.

También es importante recordar que el análisis exploratorio permite identificar patrones y relaciones, pero no demuestra causas. Para explicar por qué algunas cuentas tienen propinas más altas que otras, haría falta contar con más información contextual y aplicar otros tipos de análisis.

Revisar si el informe cumple su propósito

Después de escribir una primera versión del informe, conviene revisarlo con algunas preguntas simples.

La primera pregunta es si el informe responde al propósito definido al comienzo. En este caso, el objetivo era describir patrones generales de cuentas y propinas, observar diferencias por día y horario, y comentar la relación entre `total_bill` y `tip`. El informe se mantiene dentro de ese eje y no intenta explicar temas que no fueron analizados.

La segunda pregunta es si los resultados seleccionados son suficientes, pero no excesivos. El informe no incluye todas las tablas ni todos los gráficos posibles, sino solo los resultados que ayudan a comunicar las ideas principales. Esto lo hace más claro para quien lee.

La tercera pregunta es si las conclusiones son prudentes. En el texto se usan expresiones como “dentro de los datos analizados”, “tienden a” y “debe interpretarse con prudencia”. Estas formulaciones ayudan a evitar afirmaciones demasiado fuertes.

Por último, conviene revisar si los límites del análisis están explícitos. En este caso, se mencionan el tamaño limitado del dataset, la baja cantidad de registros en algunos grupos, el desbalance entre días y horarios, y la imposibilidad de afirmar causalidad a partir del análisis exploratorio.

Una idea importante: el informe no es el análisis completo

Durante un análisis exploratorio podemos probar muchas tablas, gráficos y comparaciones para entender mejor los datos. Sin embargo, el informe no tiene que mostrar todo ese recorrido, sino presentar una selección clara de los resultados que mejor responden al propósito definido.

En este ejemplo, el informe se concentra en tres ejes: la distribución general de cuentas y propinas, las diferencias principales por día y horario, y la relación entre el importe de la cuenta y la propina. Esa selección ayuda a que cada resultado tenga una función dentro de la explicación y evita que el informe se transforme en una acumulación de salidas.

Cierre del capítulo

En este capítulo trabajamos sobre cómo armar un informe breve de análisis exploratorio. El foco no estuvo en aplicar nuevas herramientas técnicas, sino en seleccionar, ordenar y comunicar resultados de manera clara.

Vimos que un informe necesita un propósito definido. Ese propósito ayuda a decidir qué tablas, gráficos y conclusiones incluir, y también qué información dejar afuera. Un buen informe no es una acumulación de todas las salidas generadas durante el análisis, sino una selección organizada de los resultados más relevantes.

También construimos una estructura posible para presentar el análisis: propósito, descripción general del dataset, resultados principales, conclusiones y límites. Esta estructura permite que quien lee entienda no solo qué muestran los datos, sino también cómo interpretar esos resultados y con qué precauciones.

La idea central es que comunicar un análisis también forma parte del trabajo con datos. Después de explorar, calcular y visualizar, necesitamos convertir los resultados en una explicación breve, coherente y honesta con la información disponible.

En el próximo capítulo vamos a hacer un repaso integrador del bloque de análisis exploratorio, recuperando las principales herramientas trabajadas y revisando cómo se conectan entre sí dentro de un proceso completo de análisis.

Capítulo 041 — Cerrar el análisis exploratorio

En este bloque trabajamos muchas herramientas para explorar datos con Python. Aprendimos a inspeccionar un dataset, reconocer tipos de variables, resumir valores numéricos, contar categorías, comparar grupos, construir visualizaciones, observar relaciones entre variables y escribir conclusiones prudentes.

Este capítulo funciona como cierre de esa etapa.

No vamos a incorporar una técnica nueva ni repetir todos los cálculos realizados en capítulos anteriores. La idea es mirar el recorrido completo y entender por qué el análisis exploratorio es una base importante para seguir avanzando.

Hasta ahora trabajamos principalmente con preguntas como estas: qué datos tenemos, cómo están organizados, qué valores son frecuentes, qué diferencias aparecen entre grupos, qué relaciones se observan entre variables y qué límites debemos considerar antes de sacar conclusiones.

En la próxima etapa, ese trabajo previo va a ser fundamental para abordar nuevos problemas y nuevas herramientas. La exploración de datos seguirá siendo una base necesaria antes de avanzar hacia formas más complejas de análisis.

Qué aprendimos en esta etapa

El análisis exploratorio de datos no es una única técnica. Es un proceso compuesto por varias decisiones pequeñas.

Primero necesitamos conocer la estructura general del dataset: cuántas filas y columnas tiene, qué variables incluye, qué tipos de datos aparecen y si existen valores faltantes. Después podemos avanzar hacia resúmenes numéricos, conteos de categorías, comparaciones entre grupos y visualizaciones.

Cada herramienta cumple una función distinta. `describe()` ayuda a resumir variables numéricas. `value_counts()` permite contar categorías. `groupby()` sirve para comparar grupos. Los histogramas muestran distribuciones. Los boxplots ayudan a ver dispersión y posibles valores extremos. Los gráficos de barras comunican comparaciones, y los gráficos de dispersión permiten observar relaciones entre variables numéricas.

Pero lo más importante no es memorizar instrucciones aisladas. Lo importante es entender qué pregunta responde cada herramienta y cómo interpretar la salida obtenida.

Del resultado a la interpretación

A lo largo del bloque también vimos que analizar datos no consiste solamente en ejecutar código. Una tabla o un gráfico son puntos de partida, no conclusiones automáticas.

Por ejemplo, si un grupo tiene un promedio más alto que otro, todavía necesitamos preguntarnos cuántos registros hay en cada grupo, si la diferencia parece relevante, si existen valores extremos y si el dataset permite sostener esa lectura.

También vimos que las relaciones entre variables deben interpretarse con cuidado. Una correlación o una tendencia visual puede sugerir una asociación, pero no demuestra por sí sola que una variable cause cambios en otra.

Esta forma de pensar va a seguir siendo necesaria en cualquier etapa posterior. A medida que avancemos hacia herramientas más complejas, también vamos a necesitar interpretar resultados, revisar supuestos y ubicar cada salida dentro del contexto de los datos.

Por qué este recorrido prepara el camino

El análisis exploratorio no es una etapa aislada. Antes de aplicar técnicas más avanzadas, necesitamos conocer los datos con los que vamos a trabajar.

En este bloque aprendimos a mirar un dataset desde varias perspectivas: su estructura, sus variables, sus valores frecuentes, sus distribuciones, sus diferencias entre grupos y sus relaciones internas. También aprendimos a escribir conclusiones prudentes y a reconocer límites.

Todo ese recorrido construye una base importante para seguir avanzando. Cuando más adelante trabajemos con modelos, no vamos a partir solamente del código o de los algoritmos, sino de una comprensión previa de los datos.

Por eso, cerrar esta etapa no significa dejar atrás el análisis exploratorio. Significa reconocerlo como una herramienta que vamos a seguir usando cada vez que necesitemos entender un conjunto de datos antes de tomar decisiones.

El análisis exploratorio como hábito de trabajo

Una de las ideas más importantes de este tomo es que analizar datos requiere método. No alcanza con ejecutar instrucciones sueltas ni con mirar una sola tabla o un solo gráfico.

Explorar datos implica hacer preguntas, elegir herramientas adecuadas, observar resultados, interpretarlos y revisar sus límites. A veces una salida confirma lo que esperábamos; otras veces muestra algo inesperado y nos obliga a mirar mejor.

Ese hábito de trabajo será útil en cualquier etapa posterior. Ya sea que estemos preparando un informe, estudiando un nuevo dataset o avanzando hacia modelos predictivos, la primera tarea seguirá siendo la misma: entender los datos.

Síntesis final del Tomo II

A lo largo de este tomo trabajamos sobre una idea central: los datos necesitan ser explorados antes de ser interpretados.

Comenzamos con preguntas simples sobre la estructura del dataset y avanzamos progresivamente hacia resúmenes, conteos, agrupaciones, visualizaciones, comparaciones, relaciones entre variables, conclusiones e informes breves.

Ese recorrido permitió construir una forma de trabajo. Primero observamos los datos, después elegimos herramientas para resumirlos o visualizarlos, luego interpretamos los resultados y finalmente comunicamos las conclusiones con prudencia.

También aprendimos que no todas las salidas tienen el mismo peso. Algunas sirven para inspeccionar, otras para comparar, otras para detectar patrones y otras para comunicar resultados. Parte del análisis consiste justamente en decidir qué mirar, qué destacar y qué límites mencionar.

El análisis exploratorio no termina en una tabla ni en un gráfico. Termina cuando logramos transformar los resultados en una explicación clara, honesta y útil.

Cierre del capítulo

Este capítulo cerró el bloque de análisis exploratorio de datos. No incorporamos nuevas herramientas técnicas, sino que organizamos el recorrido realizado y recuperamos las ideas principales.

Vimos que explorar datos implica mucho más que ejecutar código. Implica formular preguntas, revisar la estructura del dataset, resumir variables, comparar grupos, visualizar patrones, interpretar resultados y comunicar conclusiones de manera responsable.

También dejamos planteada una continuidad: las herramientas de análisis exploratorio seguirán siendo necesarias en etapas posteriores. Cada vez que

trabajemos con un nuevo dataset, necesitaremos volver a mirar sus variables, sus distribuciones, sus relaciones y sus límites.

Con esto cerramos el Tomo II. El próximo paso será abrir una nueva etapa de trabajo, usando la base construida hasta aquí para avanzar hacia nuevos problemas y nuevas herramientas.

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>